

Table of Contents

1. [Building pipelines • Overview • Palantir](#)
2. [Building pipelines • Overview • Palantir](#)
3. [Building pipelines • Types of pipelines • Palantir](#)
4. [Building pipelines • Supported languages • Palantir](#)
5. [Building pipelines • Considerations: Pipeline Builder and Code Repositories • Palantir](#)
6. [Building pipelines • Getting started • Create a dataset batch pipeline with Pipeline Builder](#)
7. [Building pipelines • Getting started • Create a media set batch pipeline with Pipeline Build](#)
8. [Building pipelines • Getting started • Create a dataset batch pipeline with Code Repositorie](#)
9. [Building pipelines • Getting started • Create a media set batch pipeline with Code Repositor](#)
10. [Building pipelines • Getting started • Create an incremental pipeline with Pipeline Builder](#)
11. [Building pipelines • Getting started • Create a streaming pipeline with Pipeline Builder •](#)
12. [Building pipelines • Getting started • Faster pipelines with Pipeline Builder • Palantir](#)
13. [Building pipelines • Getting started • External pipelines with Pipeline Builder • Palantir](#)
14. [Building pipelines • Incremental pipelines • Overview • Palantir](#)
15. [Building pipelines • Incremental pipelines • Creating incremental syncs • Palantir](#)
16. [Building pipelines • Incremental pipelines • Maintaining high performance • Palantir](#)
17. [Building pipelines • Streaming pipelines • Overview • Palantir](#)
18. [Building pipelines • Streaming pipelines • Comparison: Streaming vs. batch • Palantir](#)
19. [Building pipelines • Streaming pipelines • Performance considerations • Palantir](#)
20. [Building pipelines • Streaming pipelines • Streaming compute usage • Palantir](#)
21. [Building pipelines • Streaming pipelines • Keys • Palantir](#)
22. [Building pipelines • Streaming pipelines • Streaming stateful transforms • Palantir](#)
23. [Building pipelines • Scheduling • Overview • Palantir](#)
24. [Building pipelines • Scheduling • Create a schedule • Palantir](#)
25. [Building pipelines • Scheduling • View and modify schedules • Palantir](#)
26. [Building pipelines • Scheduling • Find and manage schedules • Palantir](#)
27. [Building pipelines • Scheduling • Common scheduling configurations • Palantir](#)
28. [Building pipelines • Scheduling • Trigger types reference • Palantir](#)
29. [Building pipelines • Scheduling • Troubleshooting reference • Palantir](#)
30. [Building pipelines • Scheduling • Add schedule to a Marketplace product • Palantir](#)
31. [Building pipelines • Scheduling • Parameterization • Palantir](#)
32. [Building pipelines • Logic Flows \[Sunset\] • Overview • Palantir](#)
33. [Building pipelines • Logic Flows \[Sunset\] • Create a connected flow • Palantir](#)
34. [Building pipelines • Logic Flows \[Sunset\] • Compass File Lister • Palantir](#)
35. [Building pipelines • Best practices • Recommended Project and team structure • Palantir](#)
36. [Building pipelines • Best practices • Development best practices • Palantir](#)
37. [Building pipelines • Best practices • Branching and release process • Palantir](#)
38. [Building pipelines • Best practices • Scheduling best practices • Palantir](#)
39. [Building pipelines • Best practices • Building a production pipeline • Palantir](#)

Building pipelines

The very first steps to creating a [data pipeline](#) are to connect organizational data sources to Foundry and get data flowing through the system. Initially, the emphasis should be on validating that data is high-quality and can serve as a reliable foundation for use case development, model development, and analytics.

This section of documentation focuses on the initial stages of creating a pipeline, when business requirements may still be in flux and changes to pipeline logic are occurring frequently. In this phase, the emphasis is on laying a solid foundation—both to support target use cases and to enable pipeline maintenance in the future.

Palantir Learning portal

You can find a deep dive course on building your first pipeline at learn.palantir.com.

Initial steps

In most cases, these are the initial steps you should follow in pipeline development:

- Set up the [recommended Project structure](#) so that data security and governance are organized from the very beginning of your development process.
- Create a batch pipeline in [Pipeline Builder](#) or [Code Repositories](#) to process input datasets, perform data cleaning and filtering, and join with other datasets to create high-

API Reference ↗

that can feed into the [Ontology](#) to support workflow development.

Map your initial datasets into [object types](#) and [link types](#) in the Ontology.

  [Schedules](#) so that data begins to flow through regularly.

Send feedback

Beyond these steps, there are a number of steps you can take to make your pipeline more robust and scalable, including adding unit tests, setting up a branching and release process, and defining health checks. [Learn about best practices for pipeline development.](#)

Incremental pipelines

If the scale of changes to the input data flowing into your pipeline is high, it may be best to create an [incremental pipeline](#) to process the changed data in a performant way. In most cases, you can begin with a batch pipeline and put an incremental pipeline into place afterwards to improve performance and reduce latency.

In some cases, it is preferable to design your pipeline to be incremental from the start, especially when you know that the scale of new data flowing into your pipeline will be very high. However, writing and maintaining incremental pipelines comes with much more complexity than batch pipelines. [Learn more about the different types of pipelines in Foundry.](#)

Streaming pipelines

If the requirements for the latency of your data are very low, it may be best to create a [streaming pipeline](#) to process input data in a performant way. Given streaming pipelines are only as fast as their slowest component, pipelines should be designed from the start to ensure the pipeline will hit the target latency and throughput. Review our [comparison of streaming versus batch processes](#) for a more nuanced analysis.

← PREVIOUS
Resource guides / Streaming profiles

NEXT →
Types of pipelines

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[API Reference ↗](#)

[Terms of Use ↗](#)

[Send feedback](#)

Building pipelines

The very first steps to creating a [data pipeline](#) are to connect organizational data sources to Foundry and get data flowing through the system. Initially, the emphasis should be on validating that data is high-quality and can serve as a reliable foundation for use case development, model development, and analytics.

This section of documentation focuses on the initial stages of creating a pipeline, when business requirements may still be in flux and changes to pipeline logic are occurring frequently. In this phase, the emphasis is on laying a solid foundation—both to support target use cases and to enable pipeline maintenance in the future.

Palantir Learning portal

You can find a deep dive course on building your first pipeline at learn.palantir.com.

Initial steps

In most cases, these are the initial steps you should follow in pipeline development:

- Set up the [recommended Project structure](#) so that data security and governance are organized from the very beginning of your development process.
- Create a batch pipeline in [Pipeline Builder](#) or [Code Repositories](#) to process input datasets, perform data cleaning and filtering, and join with other datasets to create high-

API Reference ▸ that can feed into the [Ontology](#) to support workflow development.

Map your initial datasets into [object types](#) and [link types](#) in the Ontology.

  [Schedules](#) so that data begins to flow through regularly.

Send feedback

Beyond these steps, there are a number of steps you can take to make your pipeline more robust and scalable, including adding unit tests, setting up a branching and release process, and defining health checks. [Learn about best practices for pipeline development.](#)

Incremental pipelines

If the scale of changes to the input data flowing into your pipeline is high, it may be best to create an [incremental pipeline](#) to process the changed data in a performant way. In most cases, you can begin with a batch pipeline and put an incremental pipeline into place afterwards to improve performance and reduce latency.

In some cases, it is preferable to design your pipeline to be incremental from the start, especially when you know that the scale of new data flowing into your pipeline will be very high. However, writing and maintaining incremental pipelines comes with much more complexity than batch pipelines. [Learn more about the different types of pipelines in Foundry.](#)

Streaming pipelines

If the requirements for the latency of your data are very low, it may be best to create a [streaming pipeline](#) to process input data in a performant way. Given streaming pipelines are only as fast as their slowest component, pipelines should be designed from the start to ensure the pipeline will hit the target latency and throughput. Review our [comparison of streaming versus batch processes](#) for a more nuanced analysis.

← PREVIOUS
Resource guides / Streaming profiles

NEXT →
Types of pipelines

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[API Reference ↗](#)

[Terms of Use ↗](#)

[Send feedback](#)

Types of pipelines

There are three main types of pipelines you can create in Foundry, and each provides different tradeoffs according to a few criteria:

- **Latency.** How long does the pipeline take to run end-to-end?
- **Complexity.** How difficult is it to author the pipeline and maintain it over time?
- **Compute cost.** How much does the pipeline actually cost in terms of compute hours?
- **Resilience to change in data scale.** How much additional data will flow into your pipeline over time?

The three types of pipelines are:

- **Batch** pipelines fully recompute every dataset that has changed on each run.
- **Incremental** pipelines only process new data that has changed since the last run.
- **Streaming** pipelines run continuously and process new data as it arrives in the platform.

Below, we discuss each type of pipeline, its tradeoffs, and how to get started authoring this type of pipeline. For convenience, here is a summary table of the types of pipelines according to the tradeoffs mentioned above.

Criterion	Batch	Incremental	Streaming
Latency	High	Low	Very low
Complexity	Low	Medium	High
Compute cost	Medium	Low	High
Resilience to change in data scale	Low	High	High

Foundry offers [faster](#) versions of batch and incremental pipelines for improved performance at the cost of not supporting all expressions and transformations available in

Send feedback

standard pipelines.

Batch

In a **batch pipeline**, all datasets in the pipeline are fully recomputed whenever upstream data changes. Because everything is recomputed, the end-to-end performance of the pipeline is very consistent over time, and the code and maintenance complexity of the pipeline is minimal. To enable more users to contribute to batch pipelines, a broad set of languages and tools are available for batch pipeline authoring, including SQL.

Examining batch pipelines according to the criteria above:

- *Latency* of batch pipelines can be very high, as all data must be processed whenever it lands into the platform, even if it has not changed since the last sync. However, latency may be low if the overall data scale is small.
- *Complexity* of batch pipelines is very low. Although it is usually still necessary to understand table manipulation concepts such as filters, joins, and aggregations, minimal further knowledge is necessary.
- *Compute costs* for batch pipelines can be high, as a large amount of repeated computation occurs on every build of the pipeline. Again, this factor can be ignored if overall data scale is small.
- *Resilience to change in data scale* is low. If large amounts of new data flow into the pipeline, such as when input datasets represents high-volume events, batch pipelines will become unmanageably expensive and take too long to run.

In most cases, you should begin pipeline development in Foundry by creating a batch pipeline and extending it to support incremental computation as the use case for the pipeline is validated. In many cases, you can keep using a batch pipeline indefinitely, especially if your data scale is low (e.g., less than tens of millions of rows).

If you expect that you will need to make your pipeline incremental in the future, we recommend using either Python or Java for batch pipeline development, as these languages support incremental computation.

Get API Reference [or learn how to create a batch pipeline in Pipeline Builder](#), or by following the tutorials for other languages:

Send feedback

- [Create a batch pipeline in Python](#)

- [Create a batch pipeline in Java](#)
- [Create a batch pipeline in SQL](#)

Faster

- i** Faster pipelines were previously known as *lightweight pipelines*, as the term "lightweight" referred to the reduced time and compute resources required to execute these pipelines as opposed to the size of the data they handle. The name change reflects that faster pipelines reduce both execution time and compute resource usage, even for large-scale datasets.

Pipeline Builder now supports faster pipelines, which can provide faster execution in certain conditions. It uses a backend powered by [DataFusion](#), an open-source query engine written in [Rust](#). Compared to traditional Spark-based pipelines, faster pipelines can substantially accelerate compute processes for small to medium-sized datasets.

You can create a faster pipeline from scratch or convert an existing batch or incremental pipeline to a faster pipeline. Learn more about [faster pipelines in Pipeline Builder](#)

Incremental

In an **incremental pipeline**, only the rows or files of data that have changed since the last build are computed. This is suitable for processing event data and other datasets with large amounts of data changing over time. In addition to reducing the overall amount of computation, the end-to-end latency of the pipeline can be reduced significantly as compared to batch pipelines. Only Python and Java APIs are available for incremental computation.

Examining incremental pipelines according to the criteria above:

- **Latency** of incremental pipelines can be made very low—on the order of minutes. In our [API Reference](#), sufficiently low latency to meet the vast majority of organizational requirements.
- **Complexity** of incremental pipelines is medium-to-high. Rather than the high-level concept of a tabular dataframe, writing and maintaining an incremental

[Send feedback](#)

pipeline requires understanding how data flows into Foundry [dataset transactions](#), how to handle cases when input datasets are updated non-incrementally, and how to maintain high performance over time.

- *Compute costs* for incremental pipelines can be lower than for batch pipelines on high-scale datasets, as the amount of actual computation occurring can be minimized.
- *Resilience to change in data scale* is high. Because only new data is processed, incremental pipelines avoid needing to redo computation on large datasets where most of the data is not changing.

To learn more about incremental pipelines, refer to these resources:

- [Incremental pipelines overview](#)
- [Create an incremental pipeline in Pipeline Builder](#)
- [Incremental pipelines in Python](#)
- [Maintaining high incremental performance](#)

Streaming

In a **streaming pipeline**, your code runs continuously to process any new data that streams into Foundry, enabling the lowest levels of latency but incurring the highest amounts of complexity and compute costs. In general, it is helpful to think about streaming pipelines as closer to managing a microservice than managing a compute job—you need to be very thoughtful about uptime, resiliency, and stateful operations in order to run a streaming pipeline successfully.

Examining streaming pipelines according to the criteria above:

- *Latency* of streaming pipelines can be made very low. If you have a strong requirement for data to be available for a use case in less than one minute, then streaming pipelines may be a good fit.
- *Complexity* of streaming pipelines is very high. Writing these pipelines requires avoiding a broad set of patterns, such as stateful operations, which can unexpectedly cause instability in the future. In addition, streaming pipelines tend to have lower tolerance to downtime than batch or incremental pipelines, as downtime can result in use cases that require constantly fresh data to consider any interruption as an outage.

API Reference ↗

- *Compute costs* for streaming pipelines can be very high, as the name implies. Streaming pipelines requires compute resources to always be available to process new input data.

Send feedback

- *Resilience to change in data scale* is high. Streaming pipelines are designed to support high throughput and can typically process higher changes in data scale than batch or even incremental pipelines.

In most cases, it is best to avoid creating a streaming pipeline unless your use case has very low latency requirements. Incremental pipelines can often be made performant down to minute-level end-to-end latencies to meet most needs without incurring the added complexity and compute costs of streaming pipelines.

To learn more about streaming pipelines, refer to the following resources:

- [Create a streaming pipeline in Pipeline Builder](#)

Additional documentation for streaming pipelines will be available soon. If you are interested in building a streaming pipeline, contact your Palantir representative.

PREVIOUS
← Overview

NEXT
Supported languages →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Supported languages

Before getting started with your data transformation, it's important to consider the benefits as well as the limitations of each language. This table includes a summary of the key differences between the supported languages:

Description	SQL	Python	Java
<i>Non-proprietary language:</i> documentation available online	✓	✓	✓
<i>Support for file access:</i> read and write files in Foundry datasets—this means your data transformation can operate on unstructured data		✓	✓
<i>Transform Level Logic Versioning (TLLV):</i> more info in the TLLV section	✓	✓	
<i>Incremental computation:</i> more info in the incremental computation section		✓	✓
<i>Support for removing inherited markings</i>	✓	✓	✓
<i>Multiple output datasets allowed per file</i>		✓	✓
<i>Support for dataset previews</i>	✓	✓	✓
<i>Custom Transforms profiles</i>	✓	✓	✓

SQL

SQL is a language that has plenty of external documentation available online. Here are some key links to get started with data transformations in SQL:

API Reference ↗



the most performant language (including most Spark optimi

Send feedback

- Transforms SQL gives you access to a SQL scratchpad that allows you to run sample SQL queries to check your SQL syntax.

[Learn more about SQL Transforms.](#)

Python

Python is a language with plenty of external documentation available online. You may want to write data transformations in Python so that you can take advantage of the language-specific capabilities and libraries of Python. The Python API is lower-level than other languages like SQL. Here are some key benefits of using Python:

- The `transforms` `Python` library is an API that exposes functionalities such as file reads and writes. File-based data transformations can be useful early on in data transformation pipelines when you want to parse and clean data.
- There is first-class support for using external libraries such as pandas, NumPy, and other machine learning libraries.
- You get access to the full Spark Python (PySpark) API, which includes additional features of Spark that aren't supported in other languages.

[Learn more about Python Transforms.](#)

Java

Java is a language with plenty of external documentation available online. You may want to write data transformations in Java so that you can take advantage of the language-specific capabilities in Java. Java is a lower-level API than other languages like SQL. Here are some key benefits of using Java:

- The `transforms` `Java` library is an API that exposes functionalities such as file reads and writes. File-based data transformations can be useful early on in data transformation pipelines when you want to parse and clean data.

[API Reference ↗ Java Transforms.](#)

Send feedback

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Building pipelines / Considerations: Pipeline Builder and Code Repositories

Considerations: Pipeline Builder and Code Repositories

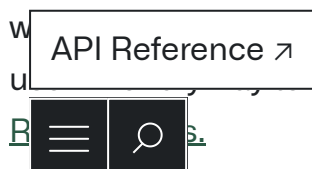
Foundry has two products available for writing and managing data pipelines: **Pipeline Builder** and **Code Repositories**. These tools are complementary and are built to work together to provide solutions for all pipelining needs. The guide below is intended to help you determine which tool is best suited to your use case and how to use them in conjunction with each other.

Pipeline Builder

Pipeline Builder is Foundry's primary application for fast, flexible, and scalable delivery of data pipelines while providing robustness and security. With Pipeline Builder, end users and data engineers can collaborate in a graph and form-based environment to integrate data, create business logic transformation, and define a rigorous release process for production pipelines. Users can write pipelines that provide real time feedback, with no need to use code. Additionally, Pipeline Builder uses health checks that guarantee only fully compliant data will be deployed to production. [Learn more about Pipeline Builder.](#)

Code Repositories

Code Repositories provides a web-based integrated development environment (IDE) for writing on production-ready code in Foundry. The application provides a way to interact with the underlying Git repository. [Learn more about Code](#)



Send feedback

Best practices

We recommend building your pipeline design in Pipeline Builder. Doing so will:

- Enable collaboration between different user groups through an easily understandable point-and-click interface.
- Safeguard pipeline health by utilizing Pipeline Builder’s rails for safe and usage-efficient data transformations and pipeline management.

In cases where users require specialized code-based logic not available in Pipeline Builder, Code Repositories should be used to create those stages to add to the main pipeline. Some examples of these specialized cases include:

- Making API calls
- Using custom libraries
- Adding code-based logical concepts

Since both Pipeline Builder and Code Repositories use Foundry datasets as inputs and outputs, a pipeline input built in Code Repositories can be added before, after, and in the middle of a pipeline in Pipeline Builder. Schedules and health checks can be configured for the full pipeline in Data Lineage, regardless of the application used to create the pipeline.

[Learn more about Data Lineage.](#)

Feature summary

The following table describes the features and support available in Pipeline Builder and Code Repositories. As explained above, using both tools together allows you to create robust, type-safe, reusable pipelines with specialized, code-based logic.

	Pipeline Builder	Code Repositories
Recommended API Reference ↗	Build and maintain production pipelines for organizations and specialized pipelines for cross-organization collaboration.	Create specialized, code-based data transformations to add to a pipeline.
Build interface		Send feedback

	Pipeline Builder	Code Repositories
Pipeline interface	Graph and form-based	Web-based integrated development environment (IDE)
Supported languages	No code required	Python, SQL, Java, Mesa
Reusability	Copy and paste complete pipelines or pipeline stages.	Reuse utility functions and libraries, and copy code between files.
Type-safe functions	Strongly typed; errors are flagged immediately instead of at build time.	Code-based; errors surfaced at build time.
Parameters	User-defined persistent parameters that can be used across a pipeline.	Code-defined constant can be used in a repository.
Supported pipelines		
Batch pipeline	Yes	Yes
Streaming pipeline	Yes	Yes (for advanced users)
File Based transformation	Yes	Yes
Incremental computation	Yes	Yes
Filesystem and API access	No	Yes
Pipeline testing		
Data preview scope	Preview based on full dataset.	Preview data sample.
Data preview timeline	Preview updates in real time.	Preview upon request.
API Reference ↗ checkpoints	Preview each transformation step.	Preview intermediary dataframes

Send feedback

Pipeline Builder		Code Repositories
		selected checkpoints in debug mode.
Debug	Type-safe; errors surface while creating the pipeline and do not require checks or builds to debug.	Debugger and Read-Eval-Print Loop (REPL) support.
Unit testing	Yes	Yes (for advanced users)
Pipeline management		
Data expectations	Yes	Yes
Schedules	Yes	Yes
Publish custom libraries	In development	Yes
Versioning	Full versioning workflow on rails for no-code/high-code user collaboration.	Full Git workflow.
Build memory management	Users can set an approved or custom compute profile.	Code-based configuration is available.
Manage security markings	Yes	Yes

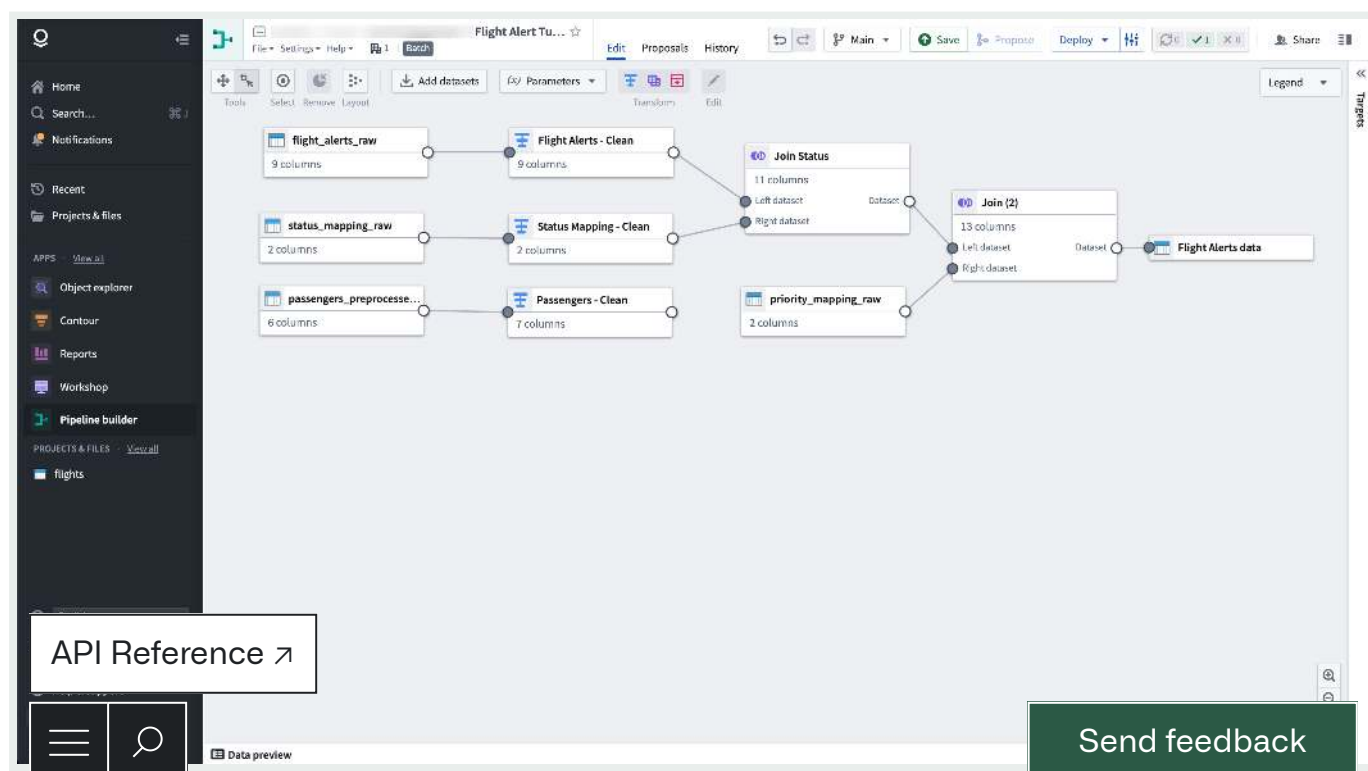
Data connectivity & integration / Building pipelines / Getting started / Create a dataset batch pipeline with Pipeline Builder

Create a dataset batch pipeline with Pipeline Builder

In this tutorial, you will use Pipeline Builder to create a simple pipeline with an output of a single dataset with information on flight alerts. You can then analyze this output dataset with tools like Contour or Code Workbook to answer questions such as which flight paths have the greatest risk of disruption.

The datasets used below are searchable by name in the dataset import step; you can find them in your Foundry filesystem.

At the end of this tutorial, you will have a pipeline that looks like the following:



The pipeline will produce a new dataset output of **Flight Alerts Data** that you can use for further exploration in the platform.

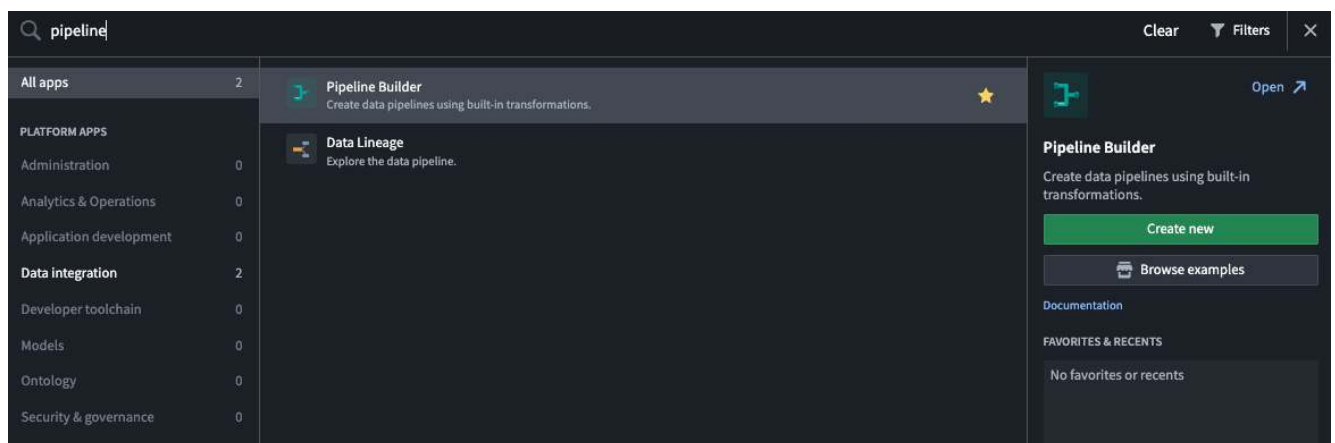
Palantir Learning portal

You can find a deep dive course on building your first pipeline at learn.palantir.com.

Part 1: Initial setup

First, create a new pipeline.

1. When logged into Foundry, select **Applications** from the left navigation sidebar to search for and open **Pipeline Builder**



2. Next, on the Pipeline Builder landing page, create a new pipeline by selecting **New pipeline**, then choose **Batch pipeline**. Under **Select batch compute**, choose **Standard**.

[API Reference ↗](#)

[Send feedback](#)

Configuration

**Batch pipeline**
Builds and transforms entire datasets on each deploy. For data that is ingested periodically.

**Streaming pipeline**
Transforms data continuously as new data is made available. For high frequency ingestion.

Select batch compute

**Standard**Default
Build your pipelines using advanced expressions. Backed by Spark.

**Faster**New
Speed up your pipelines and save compute. Backed by DataFusion. [Learn More](#)

**External**
Build your pipelines using external compute platforms. [Learn More](#)

[Compare types](#)

[← Back](#)

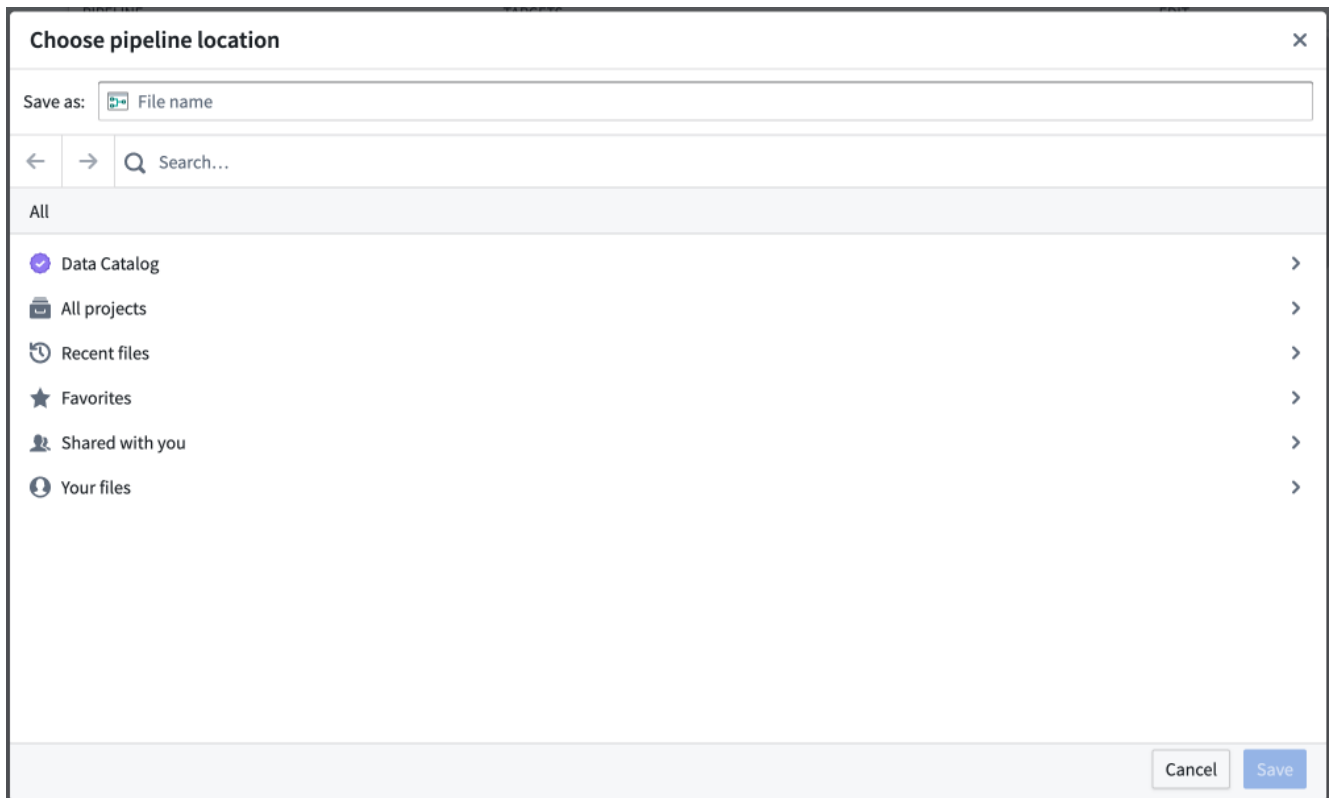
Create pipeline

i The ability to create a streaming pipeline is not available on all Foundry environments. Contact Palantir Support for more information if your use case requires it.

3. Select a location to save your pipeline. Note that pipelines cannot be saved in personal folders.

API Reference ↗

Send feedback



4. Select **Create pipeline**.

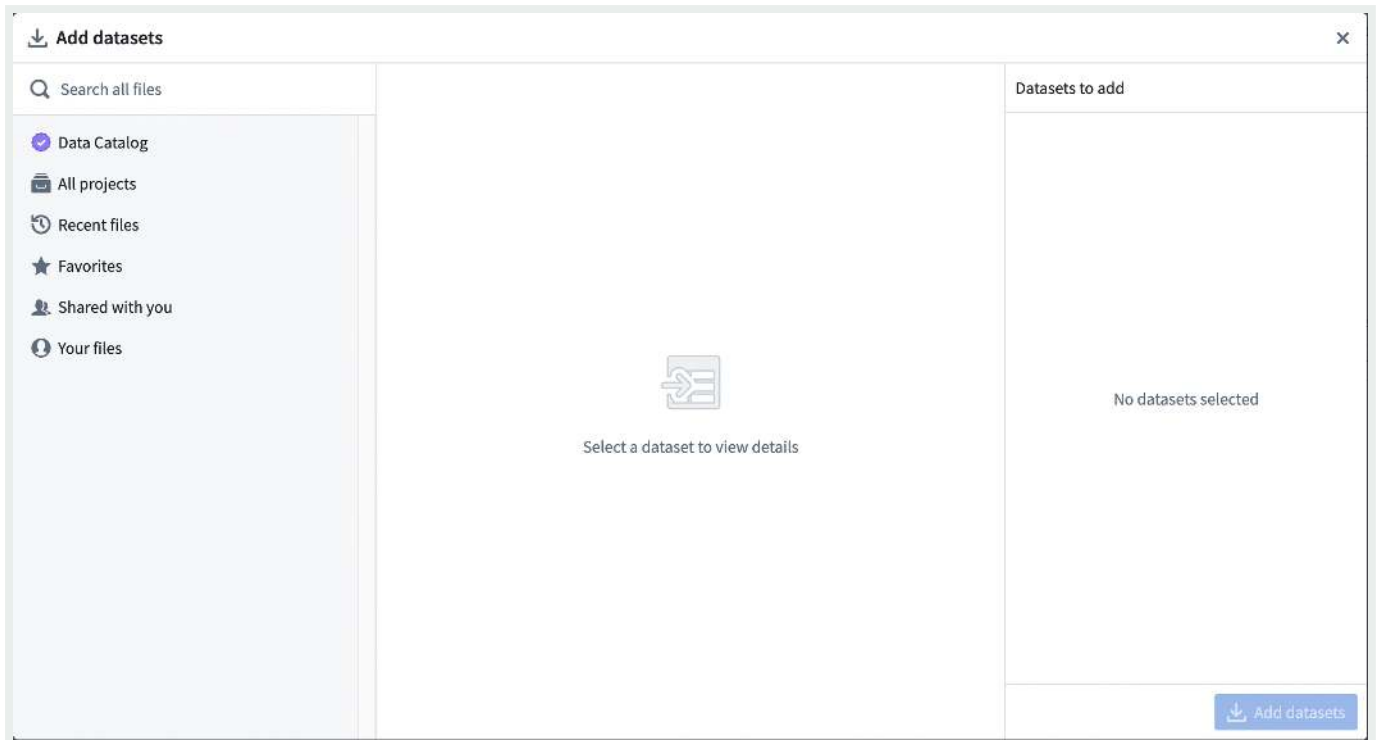
Part 2: Add datasets

Now, you can add datasets to your pipeline workflow. Use sample datasets of notional or open-source data.

From the Pipeline Builder page, select **Add datasets** from Foundry.

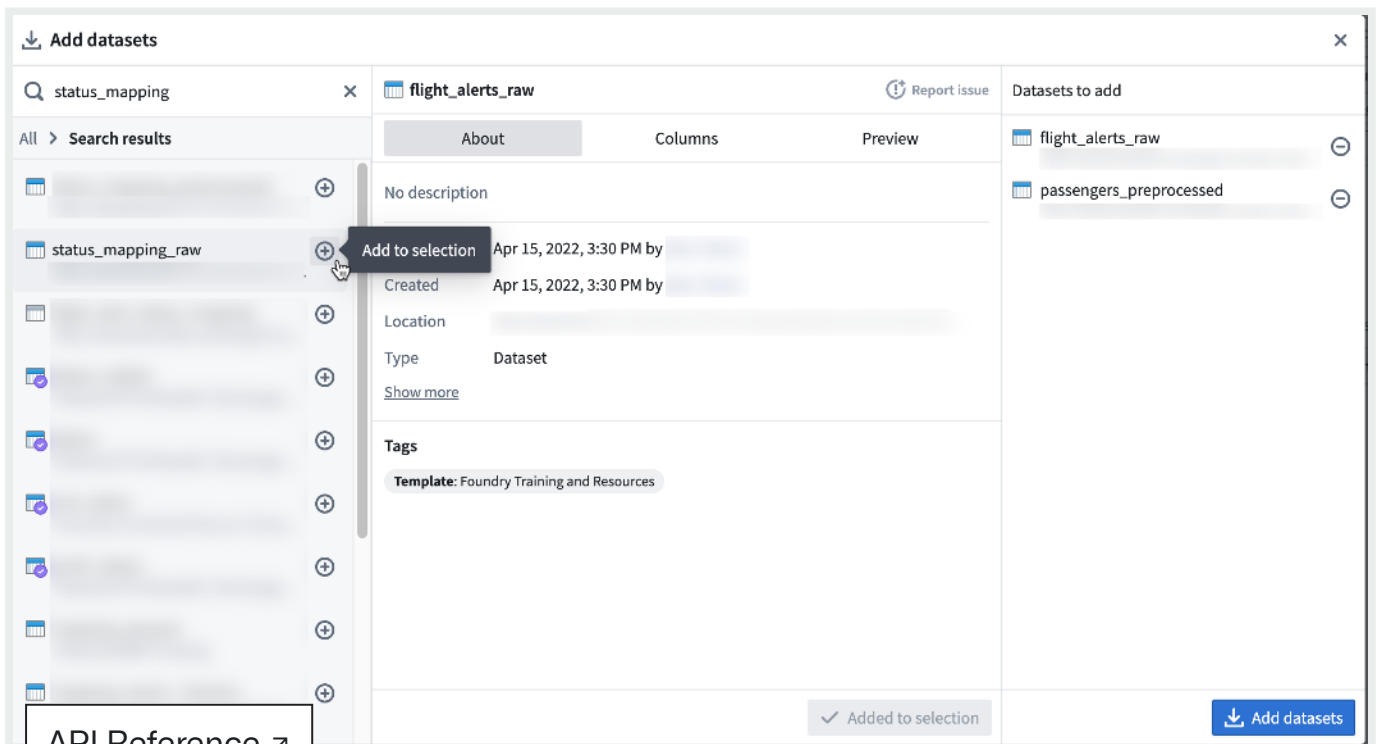
[API Reference ↗](#)

[Send feedback](#)



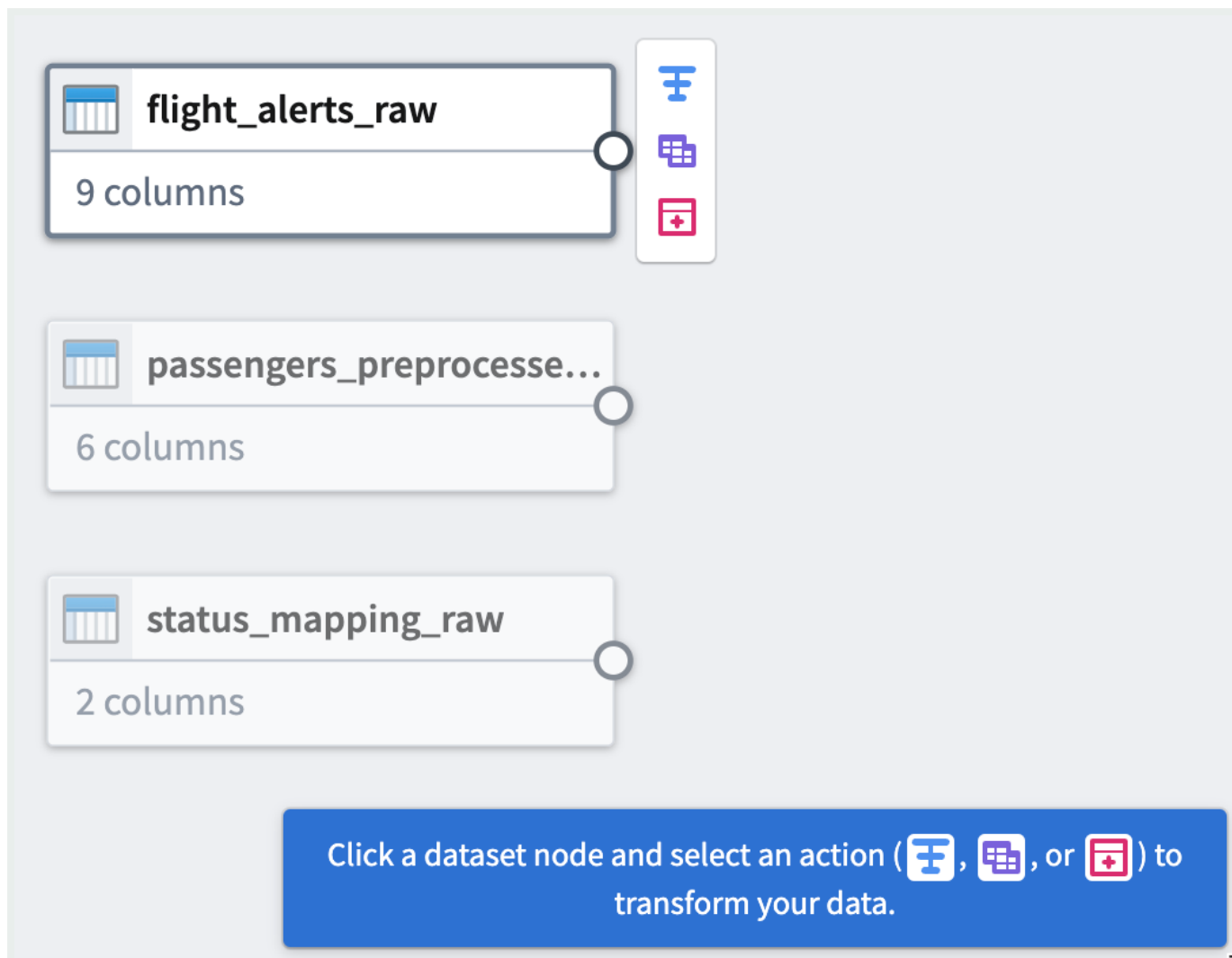
Alternatively, you can drag and drop a file from your computer to use as your dataset.

In this walkthrough example, you can add the `passengers_preprocessed`, `flight_alerts_raw`, and `status_mapping_raw` datasets. To add a selection of datasets, select the dataset. Then, choose the inline **+** icon, or select **Add to Selection**.



After selecting all required datasets, choose **Add datasets**.

Send feedback



flight_alerts_raw

9 columns

passengers_preprocesse...

6 columns

status_mapping_raw

2 columns

Click a dataset node and select an action (filter, join, or add) to transform your data.

Part 3: Clean data

After adding raw datasets, you can perform some basic cleaning transforms to continue defining your pipeline. You will transform three of your raw datasets.

Dataset 1

The first step is to clean the `passengers_preprocessed` dataset. Start by setting up a cast transform to change the `dob` column name into `dob_date` while converting the values to the `MM/dd/yy` format.

[API Reference ↗](#)

1. Select the `passengers_preprocessed` node in your graph.

[Send feedback](#)

2. Select **Transform**.



3. Search for and select the **cast** transform from the dropdown menu to open the cast configuration board.

	country	dob	first_name	flyer_status	last_name	passenger_id
1	Mexico	7/16/73	Benjamin	None	Payne	07fa349b080426ca05b0b6d1
2	Germany	3/6/85	Liam	gold	Snyder	a5d7acfc367b46cd8ba9269
3	Germany	12/22/77	Luna	silver	Austin	7df9d0cc7b314f9abb4cc0a
4	France	8/31/96	Olivia	silver	Rice	b10330dfabe34e0390e18153
5	Netherlands	6/4/91	Jack	platinum	Payne	1ca721c0131d54016bfab0c941
6	UK	3/15/78	Mia	silver	Matthews	f6dde90173c14ac9a971950cf
7	US	10/4/77	Camila	None	Hernandez	e18c5085d5ec469286682575
8	Germany	4/17/98	Olivia	platinum	James	a93cdeb7372c64502adb34ae3
9	UK	2/15/61	Daniel	gold	Morris	8ed994dc01f748f9692568ca
10	Brazil	7/22/78	Charlotte	platinum	Fisher	6bcab72a5cd84c10b4d885f3
11	Brazil	10/8/75	Emma	gold	Smith	e05052df166c4f51aa2d9c99c
12	Canada	2/16/79	Liam	silver	Tucker	56f2c4a1b3d5436daab67a18
13	Mexico	2/14/71	Sebastian	None	Hudson	efcc4171d4e54b0d8ef493f8f
14	Mexico	10/26/92	Olivia	silver	Kennedy	59eebbc3e9f14c3586661c56c
15	Brazil	8/29/50	Mason	platinum	Anderson	9b1f6aa52744573a715801ae
16	US	2/2/87	Liam	None	Vargas	a07c320796d3d0713b485cb4c
17	France	2/11/87	Sophia	gold	Watson	dd139d53b4d64c151b49b224
18	France	1/15/72	Alexander	platinum	Robinson	d8dfb037d654040e136e97a
19	Canada	12/23/77	Logan	gold	Gray	d52a18a7895242e0bf0ef1ee
20	Canada	5/3/83	Michael	None	Kim	1ed0ffdc205ee47d699d02236
21	Canada	9/2/87	Jackson	gold	Evans	b41b886609e24c3ba0e08f2b6
22	France	5/28/98	Mia	silver	Bennett	dca351c9e7704dca0e46e5e3
23	UK	7/16/85	Jack	None	Miller	3402557c7a84459980eaf7dd1

4. From the **Expression** field, select `dob` ; for **Type**, select `Date` .

5. Enter `MM/dd/yy` for the **Format** type. Be sure to use uppercase `MM` to ensure a successful cast transform. Change the output column name to `dob_date` .

Your cast board should look like this:

API Reference ↗

Send feedback

6. Select **Apply** to add the transform to your pipeline.

Title case transform

Now, you can format the `flyer_status` column values to start with an uppercase letter.

1. In the transform search field, search for and select the **Title case** transform to open the title case configuration board.
2. In the **Expression** field, select the `flyer_status` column from the dropdown.
Your title case board should look like this:

3. Select **Apply** to add the transform to your pipeline.
4. In the top left corner of the transform configuration window, rename the transform `Passengers_Clean`.

[API Reference ↗](#)

[Send feedback](#)

passengers_preprocessed
6 columns

Cast →

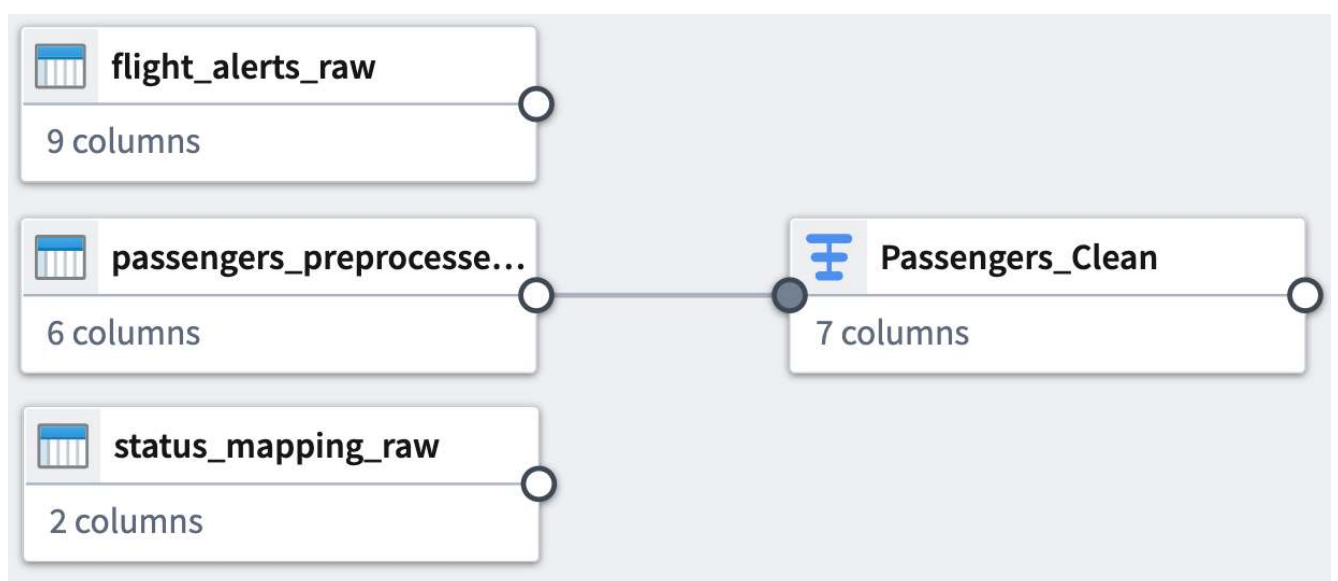
dob_date

Title case →

flyer_status

Search transforms and columns...

5. Select **Back to graph** at the top right of the screen to return to your pipeline graph.



Dataset 2

Now, clean the `flight_alerts_raw` dataset. The first step is to set up another cast transform to convert the `flight-date` column values into a `MM/dd/yy` format.

Cast transform

1. Select the `flight_alerts_raw` dataset node in your graph.

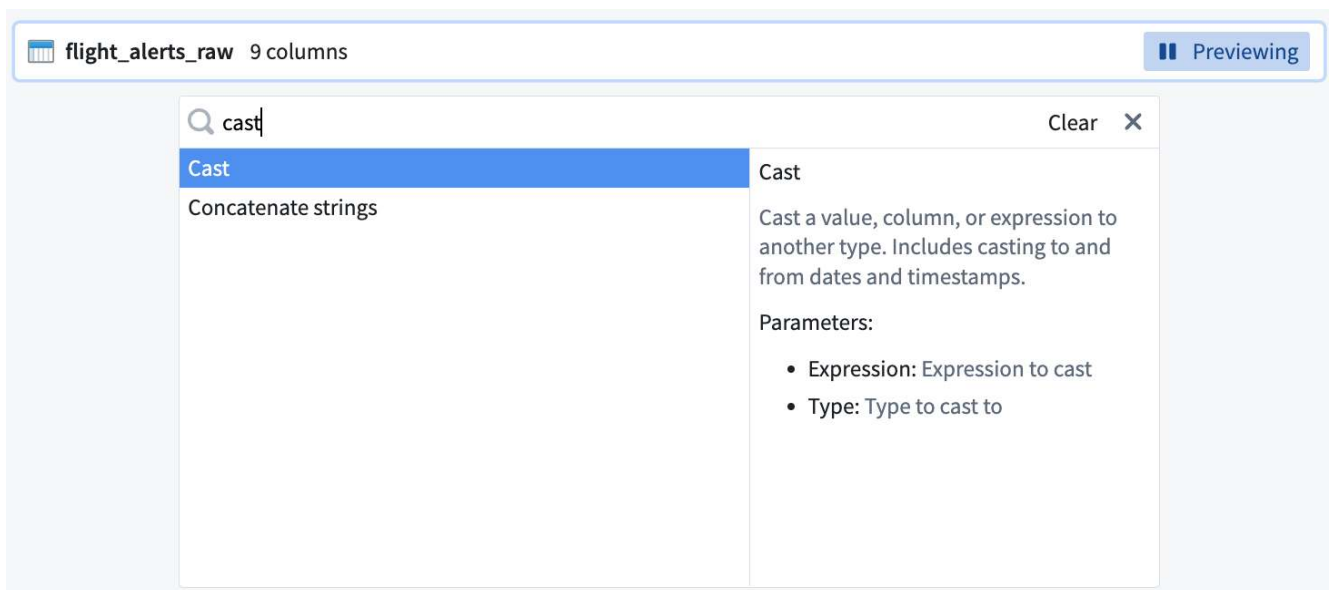
2. Select **Transform**.

API Reference ↗

Send feedback

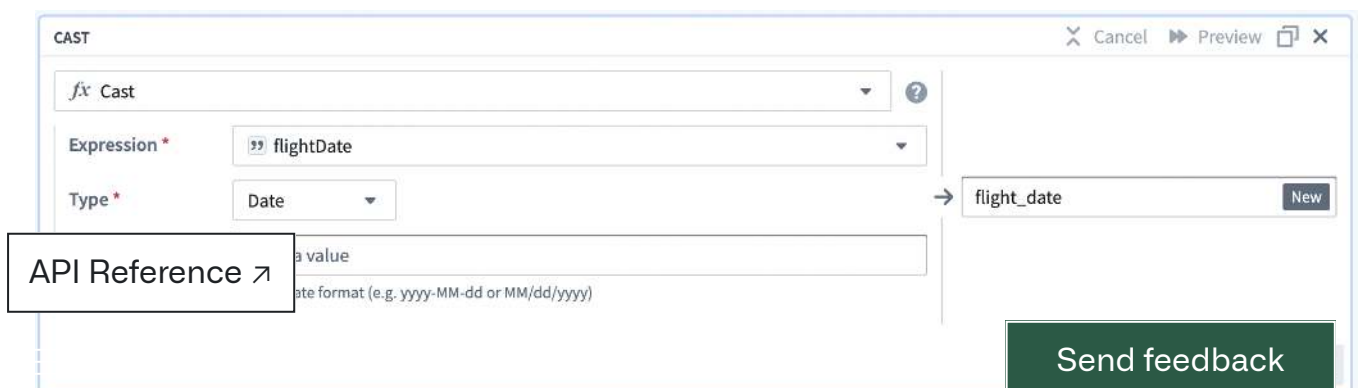


3. Search for and select the **cast** transform from the dropdown menu to open the cast configuration board. You can read the function definition listed on the right side of the selection box to learn more about the function.



4. In the **Expression** field, select the `flight_date` column from the dropdown.
5. Choose `Date` from the **Type** field dropdown menu.
6. Enter `MM/dd/yy` for the **Format** type. Be sure to use uppercase `MM` to ensure a successful cast transform.

Your cast board should look like this:



7. Select **Apply** to add the transform to your pipeline.

Clean string transform

Now, add a **Clean string** transform that will remove whitespace from `category` column values. For example, the transform will convert `delay...` string values to `delay`.

1. Search for and select the clean string transform from the dropdown to open the clean string configuration board.
2. In the **Expression** field, select the `category` column from the dropdown menu.
3. Check the boxes for all three of the **Clean actions** options:
 - Converts empty strings to null
 - Reduce sequences of multiple whitespace characters to a single whitespace
 - Trims whitespace at beginning and end of string

Your clean string board should look like this:

CLEAN STRING

fx Clean string

Expression * category

Clean actions *

- ☒ Convert empty strings to null
- ☒ Reduce sequences of multiple whitespace characters to a single whitespace
- ☒ Trims whitespace at beginning and end of string

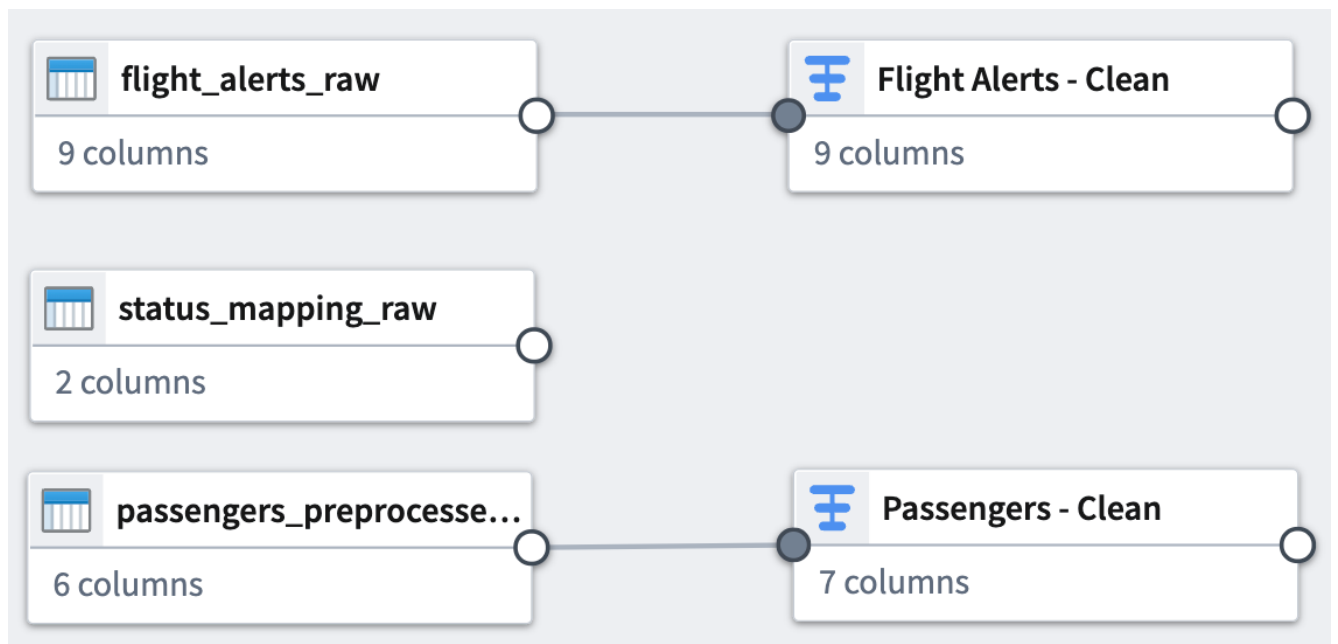
category Replace

Apply Cancel

4. Select **Apply** to add the transform to your pipeline.
5. In the top left corner of the transform configuration window, rename the transform `Flight Alerts - Clean`.
6. Select **Back to graph** at the top right to return to your pipeline graph.

API Reference ↗

Send feedback



Dataset 3

Finally, clean the `status_mapping_raw` dataset.

Clean string transform

You will only apply a **Clean string** transform to this dataset.

1. Select the `status_mapping_raw` dataset node in your graph.
2. Select **Transform**.



API Reference ↗ **Transforms and columns...** field, select the `mapped_value` column from the dropdown menu.

Send feedback

status_mapping_raw 2 columns

Previewing

APPLY EXPRESSION

Cancel Preview

mapped_value → mapped_value

Replace

Apply Cancel

4. In the same field, search and select the clean string transform from the dropdown.

5. Check the boxes for all three of the **Clean actions** options:

- Converts empty strings to null
- Reduce sequences of multiple whitespace characters to a single whitespace
- Trims whitespace at beginning and end of string

Your clean string board should look like this:

status_mapping_raw 2 columns

Previewing

CLEAN STRING

Cancel Preview

fx Clean string

Expression * mapped_value

Clean actions *

☒ Convert empty strings to null

☒ Reduce sequences of multiple whitespace characters to a single whitespace

☒ Trims whitespace at beginning and end of string

mapped_value

Replace

Apply Cancel

Search transforms and columns...

6. Select **Apply** to add the transform to your pipeline.

7. In the top left corner of the transform configuration window, rename the transform

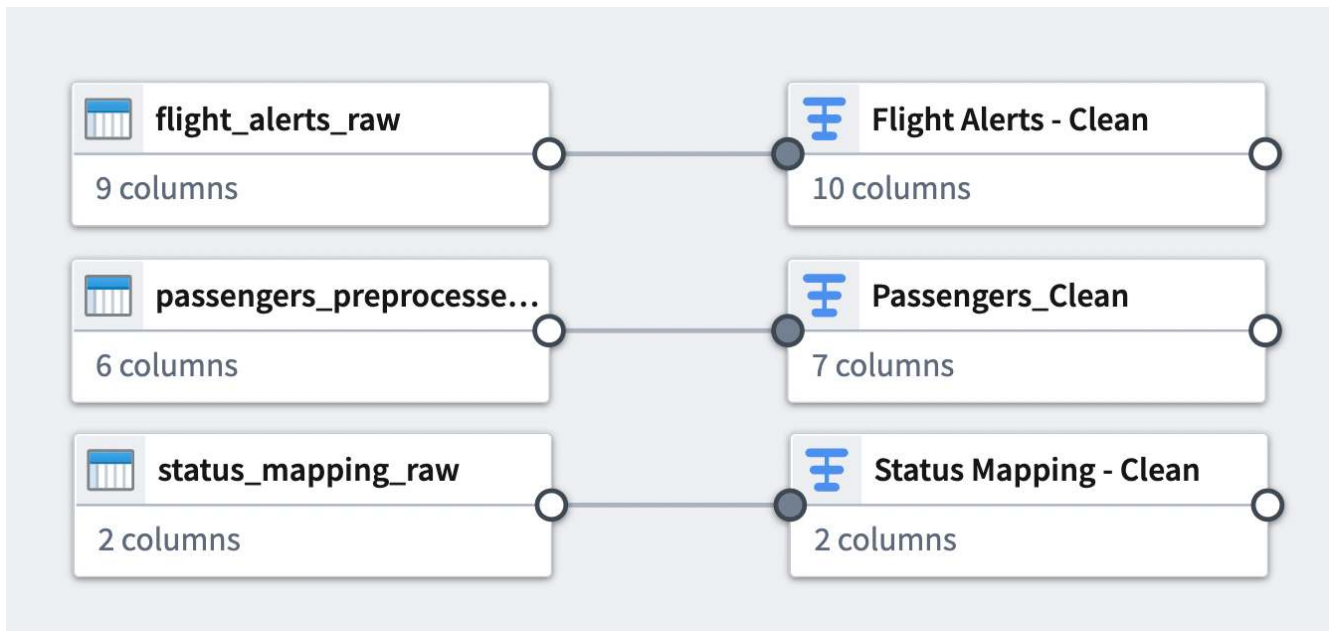
Status Mapping – Clean.

8. Select **Back to graph** at the top right to return to your pipeline graph.

You can see the connection between the transforms you just added and the datasets to which you applied them.

API Reference ↗

Send feedback



Part 4: Join datasets

Now, you can combine some of the cleaned datasets with **joins**. A join allows you to combine datasets with at least one matching column. You will add two joins to your pipeline workflow.

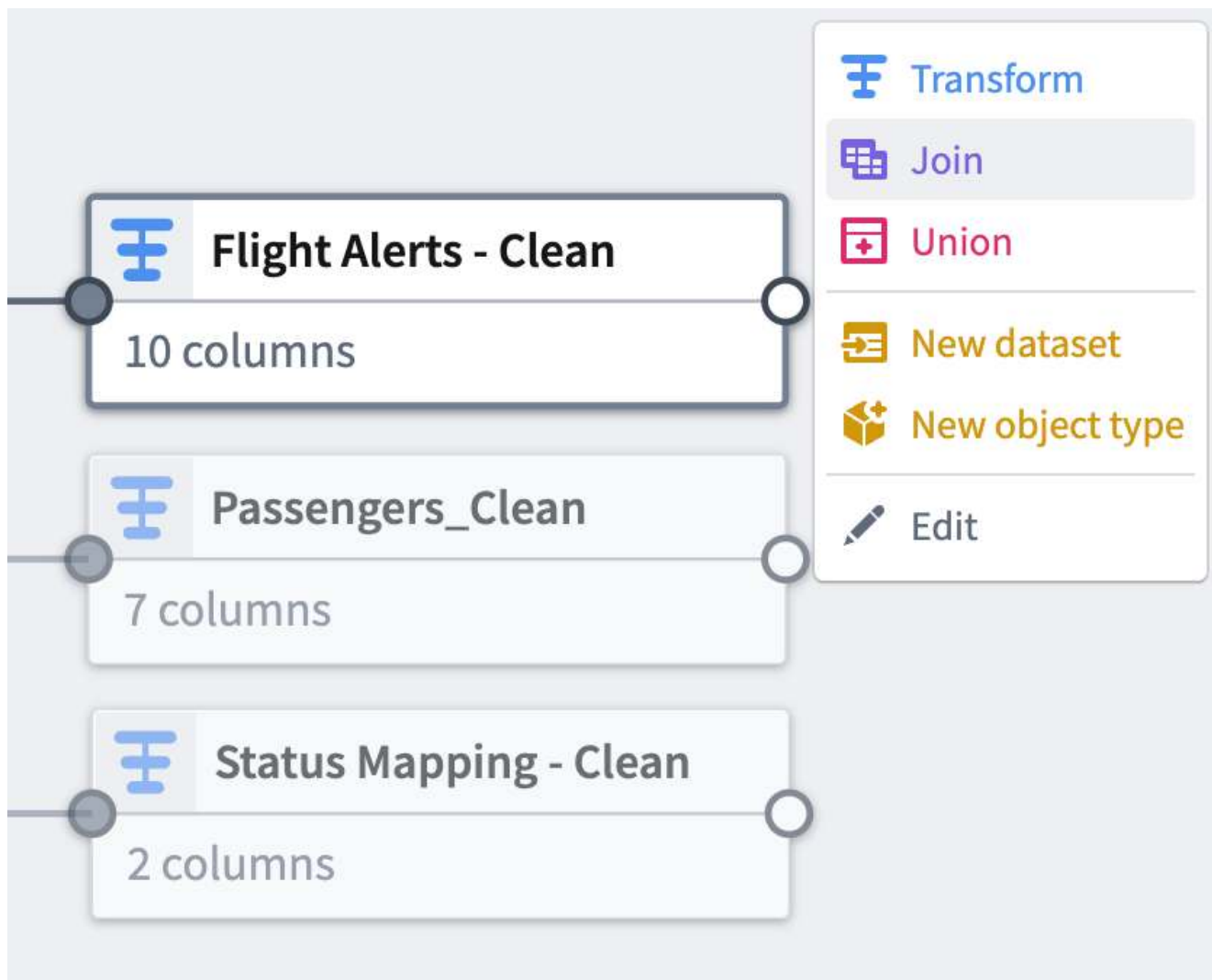
Join 1

Your first join will combine two of the cleaned datasets.

1. Select the `Flight Alerts - Clean` transform node. This will be the left side of the join.
2. Select **Join**.

[API Reference ↗](#)

[Send feedback](#)



3. Choose the **Status Mapping - Clean** node to add it as the right side of the join.
4. Select **Start** to open the join configuration board.

Select another table on the graph to start configuring the join.

Left: Flight Alerts - Clean Right: Status Mapping - Clean

[Start](#) [×](#)

5. Verify that the **Join type** is set to **Left join**.
 6. Set the **Match condition** columns to **status** is equal to **value**.
 7. Select **Show advanced** to view additional configuration options.
 8. Set the **Prefix** of the right **Status Mapping - Clean** dataset to **status**.
- Your join configuration board should look like this:

[API Reference ↗](#)

[Send feedback](#)

Join Status

Applied Back to graph

Join type

Left join

Keep all rows from the left table and matching rows from the right table

Input tables

Flight Alerts - Clean

Swap

Status Mapping - Clean

Match condition

status

is equal to

value

Add match condition

Hide advanced

Flight Alerts - Clean

Status Mapping - Clean

Prefix

Select columns

Search for columns...

Select types

9 of 9 columns selected

Select all

Deselect all

category

String

✓

flight_date

Date

✓

alert_display_name

String

✓

flight_id

String

✓

flight_display_name

String

✓

rule_id

String

✓

rule_name

String

✓

priority

Integer

✓

status

Search for columns...

Select types

2 of 2 columns selected

Select all

Deselect all

status_mapped_value

String

✓

status_value

Integer

✓

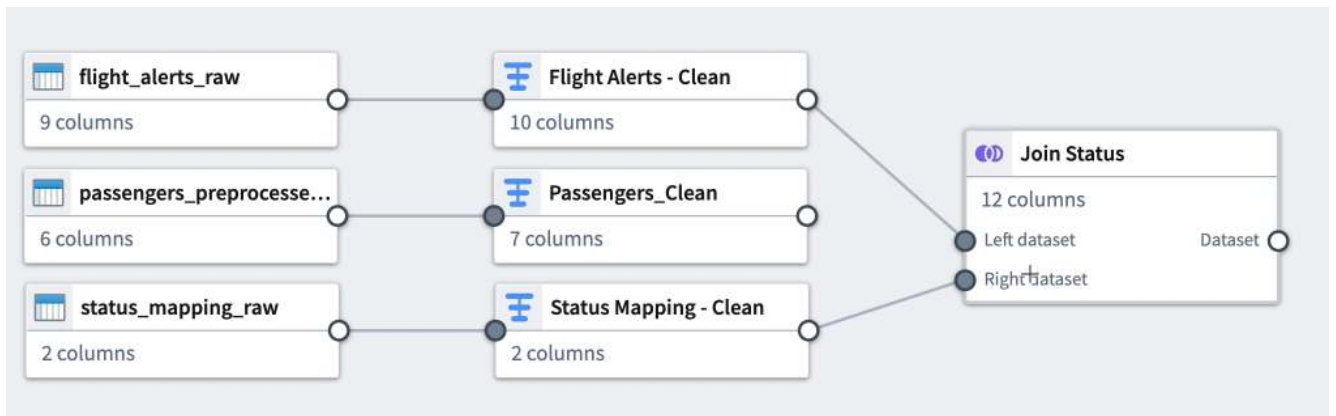
- Select **Apply** to add the join to your pipeline.
- View a preview of the join output table in the **Preview** pane at the bottom of the configuration window.

Preview	Left input	Right input	Output table	Errors
Search 11 columns...			rule_name String priority Integer status Integer status_mapped_valu... String status_value Integer	
category	1	33a2-40c0-b1c6- Extreme Delays	2	0
flight_date	2	33a2-40c0-b1c6- Extreme Delays	2	0
alert_display_name	3	33a2-40c0-b1c6- Extreme Delays	2	0
flight_id	4	33a2-40c0-b1c6- Extreme Delays	3	0
flight_display_name	5	33a2-40c0-b1c6- Extreme Delays	3	0
rule_id	6	33a2-40c0-b1c6- Extreme Delays	2	0
rule_name	7	33a2-40c0-b1c6- Extreme Delays	3	0
priority	8	d022-4c34-b19f- Security Delays over 1 Hou	3	0
status	9	33a2-40c0-b1c6- Extreme Delays	3	0
status_mapped_value	10	33a2-40c0-b1c6- Extreme Delays	2	0
status_value	11	33a2-40c0-b1c6- Extreme Delays	3	0
	12	33a2-40c0-b1c6- Extreme Delays	2	0
	13	33a2-40c0-b1c6- Extreme Delays	3	0
	14	33a2-40c0-b1c6- Extreme Delays	2	0
	15	33a2-40c0-b1c6- Extreme Delays	3	0
	16	33a2-40c0-b1c6- Extreme Delays	2	0
	17	d022-4c34-b19f- Securitv Delays over 1 Hou	3	0

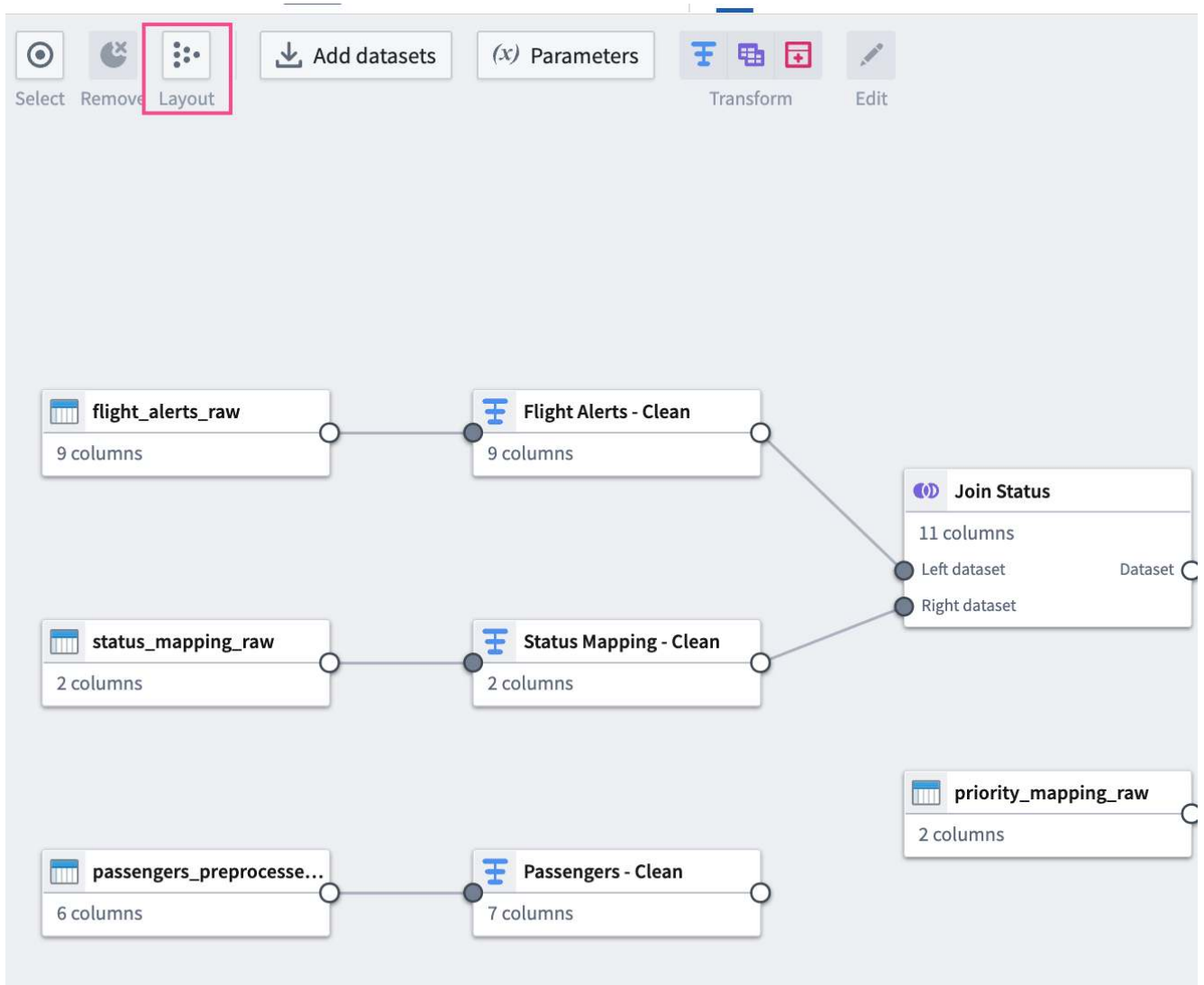
- In the top left corner of the join configuration window, rename the join **Join Status**.
- Select **Back to graph** at the top right to return to your pipeline graph.

API Reference ↗

Send feedback



13. To make the graph easier to read, select the **Layout** icon to automatically arrange the datasets, or manually drag the two connected datasets next to each other.



[API Reference ↗](#)

Join 2

[Send feedback](#)

For your second join, combine your first join output table with another raw dataset.

1. Add the `priority_mapping_raw` dataset to the graph by selecting **Add datasets**.
2. Select the `Join Status` node you just added to the graph. This will be the left side of the join.
3. Select **Join**.
4. Select the `priority_mapping_raw` dataset node to add it as the right side of your join.
5. Select **Start** to open the configuration board.

Select another table on the graph to start configuring the join.

Left: Join Status

Right: priority_mapping_raw

Start

X

6. Verify that the **Join type** is set to `Left join`.
 7. Set the **Match condition** columns to `priority` is equal to `value`.
 8. Select **Show advanced** to view additional configuration options.
 9. Set the **Prefix** of the right `priority_mapping_raw` dataset to `priority`.
- Your join configuration board should look like this:

Join (1)

Apply

Back to graph

Input tables

Join Status

Swap

priority_mapping_raw

Match condition

priority

is equal to

value

Add match condition

Hide advanced

Join Status

priority_mapping_raw

Prefix

Select columns

priority

Search for columns...

Select types

12 of 12 columns selected

Select all

Deselect all

category String ✓

flight_date Date ✓

alert_display_name String ✓

flight_id String ✓

flight_display_name String ✓

flightDate String ✓

Search for columns...

Select types

2 of 2 columns selected

Select all

Deselect all

priority_value Integer ✓

priority_mapped_value String ✓

10. Select **Apply** to add the join to your pipeline.
11. View a preview of the join output table in the **Preview** pane at the bottom of the configuration window.

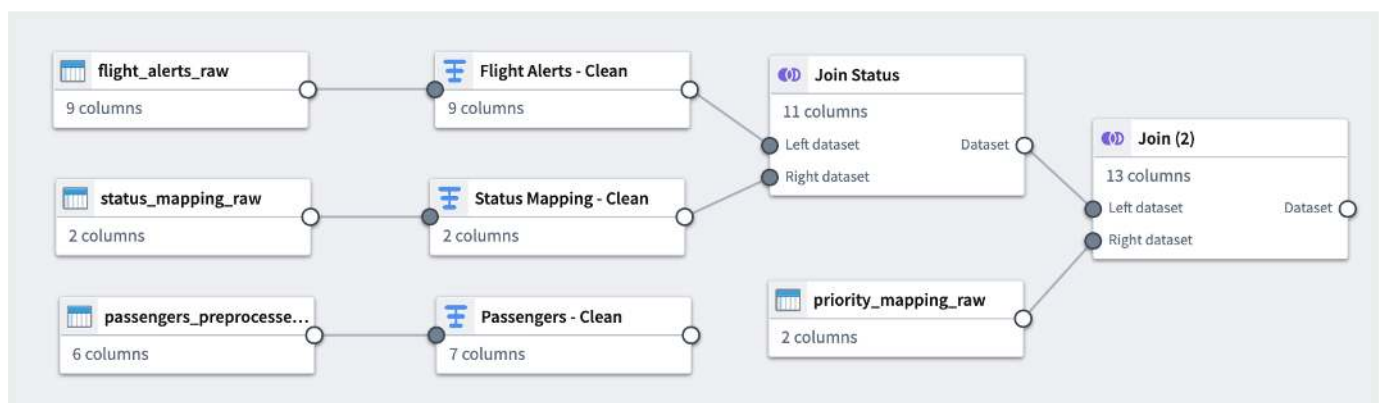
[API Reference ↗](#)

[Send feedback](#)

Preview	Left input	Right input	Output table	Errors
Search 13 columns...				
category	1	1	0 pending_triage	0
flight_date	2	1	0 pending_triage	0
alert_display_name	3	1	0 pending_triage	0
flight_id	4	1	0 pending_triage	0
flight_display_name	5	1	0 pending_triage	0
rule_id	6	1	0 pending_triage	0
rule_name	7	1	0 pending_triage	0
priority	8	1	0 pending_triage	0
status	9	1	0 pending_triage	0
status_mapped_value	10	1	0 pending_triage	0
status_value	11	1	0 pending_triage	0
priority_value	12	1	0 pending_triage	0
	13	1	0 pending_triage	0
	14	1	0 pending_triage	0
	15	1	0 pending_triage	0
	16	1	0 pending_triage	0
	17	1	0 pending_triage	0
	18	1	0 pending_triage	0

12. In the upper left corner of the join configuration window, rename the join **Join (2)**.
13. Select **Back to graph** at the top right to return to your pipeline graph.

You can now see the connection between the joins you just added and the datasets to which you applied them.



Part 5: Add an output

Now that you have finished transforming and structuring your data, you can add an output. For this tutorial, you will add a dataset output.

1. In the **Pipeline outputs** sidebar to the right of the Pipeline Builder graph, name the output **Flight Alerts data**. Then select **Add dataset output**.
2. Link **Join (2)** to the output by selecting the white circle to the right of the join node and **API Reference** the **Flight Alerts data** dataset.
3. Select **Use input schema** to use existing schema.
4. From here, select the columns of data to keep. In this case, keep a

Send feedback

[» Datasets](#)
[Object types](#)
[Link types](#)

[Back to dataset outputs](#)

[Flight Alerts data](#)

Dataset will be created after first build

[Configure expectations](#)

☐ Show errors only

...	category	✓	✕
...	flight_date	✓	✕
...	alert_display_name	✓	✕
...	flight_id	✓	✕
...	flight_display_name	✓	✕
...	rule_id	✓	✕
...	rule_name	✓	✕
123	priority	✓	✕
123	status	✓	✕
...	status_mapped_value	✓	✕
123	status_value	✓	✕
123	priority_value	✓	✕
...	priority_mapped_value	✓	✕

[+ Add column](#)
[Use input schema](#)

Part 6: Build the pipeline

To build your pipeline, select **Deploy** in the top right of your graph. Then, choose **Save and deploy**.

API Reference ↗

Send feedback

Deploy this pipeline

Update pipeline logic and build target outputs.

Last deployment
None

Deploy settings
Errors 0

Compute profile

Default
Extra small - warm pool

Select outputs to build

Deploying the pipeline will update logic for all outputs, but only the selected outputs will be built.

All

Single jobs

☒
☐
☐
Flight Alerts data

View changes
Save and deploy

You should see a small alert indicating the deploy was successful. Select **View** in the alert box to open the **Build progress** page.

Builds > Build of: Flight Alerts data

Explore lineage
Actions
Cancel build
1 0 0

Build info

Status Running
Duration 2m 33s
Estimated --
Started Today at 10:52 AM
Ended --
Started by --
Progress 0 of 1 job succeeded
Build ID

Build progress

Gantt chart
Progress details
Job status
Dataset path...

Queued
Running
Succeeded
Failed
Canceled

Datasets	Start time	Duration	Status	Logs	Actions
Flight Alerts data	Today at 10:52 AM	2m 28s	Waiting for resources		

Started job
Today at 10:52 AM (3s)

Waiting for resources
Today at 10:52 AM (2m 28s)

From this page, you can monitor the progress of your build until the dataset output is ready.

API Reference ↗

Send feedback

Builds > Build of: Flight Alerts data

Build info

Status

Succeeded

Duration

19m 45s

Estimated

--

Started

Today at 10:52 AM

Ended

Today at 11:12 AM

Started by

Progress

1 of 1 job succeeded

Build ID

You can now access your dataset by selecting **Actions > Open**.

Flight Alerts data

File Help 1 1

Actions

Share

Preview History Details Health Compare

Analyze Build

Flight Alerts data

Report issue

About Columns Schedules

Enter description...

Updated 53 minutes ago by

Created 1 hour ago by

Location

Type Dataset

Size 13 columns • 2.5k rows • 2 files • 443KB

Updated via Flight Alert

Tutorial Pipeline

Show more

Tags

Add tags

Health Checks

View details

There are no health checks on this dataset.

Inputs

Explore data lineage

Flight Alert Tutorial Pipeline

flight_alerts_raw

/Public/Foundry Training and Resources/Foundry Referen...

status_mapping_raw

/Public/Foundry Training and Resources/Foundry Referen...

priority_mapping_raw

/Public/Foundry Training and Resources/Foundry Referen...

Load more...

Custom metadata

Flight Alerts data

Showing 300 of 2.5k rows

13 columns

Search columns...

	category	flight_date	alert_display_name	flight_id	flight_display_name	rule_id	rule_name	priority
	String	Date	String	String	String	String	String	Integer
1	delay	2018-06-23	Extreme Delays: UA2042 DC 06c94752f639f89de59091d	UA2042 DCA->SFO 2018-06	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
2	delay	2018-02-12	Extreme Delays: O05447 DI a4d95b805947b3c43faabe	O05447 DRO->DEN 2018-0	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
3	delay	2018-06-10	Extreme Delays: UA262 BN 4855994b9e8c836a0058dc	UA262 BNA->ORD 2018-06	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
4	delay	2018-06-28	Extreme Delays: EV4046 JA cee8337621aef950b51814	EV4046 JAX->IAH 2018-06-2	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
5	delay	2018-01-12	Extreme Delays: EV4040 SY a925da0df8a9574cb4e448	EV4040 SYR->EWR 2018-01	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
6	delay	2018-02-28	Extreme Delays: NK852 MY 7e43630031de47c72281fcf	NK852 MYR->DTW 2018-02	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
7	delay	2018-06-03	Extreme Delays: O05200 LJ 4bee58e34ee37c38264cf8e	O05200 LAX->SBA 2018-06	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
8	delay	2018-06-01	Extreme Delays: DL1465 AT 6b0c1f0860de1c94fca1d5c	DL1465 ATL->SAV 2018-06	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
9	delay	2018-02-25	Extreme Delays: YX3719 EY d4abca603e5665eff5b27a0	YX3719 EYW->EWR 2018-02	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
10	delay	2018-01-06	Extreme Delays: O04638 CL ebf17a17194309075bd48e	O04638 CLE->MSP 2018-01	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
11	delay	2018-07-26	Extreme Delays: DL2044 IAH 8ccf2fc7ed640f3a82f9efc81	DL2044 IAH->DTW 2018-07	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
12	delay	2018-05-22	Extreme Delays: YV6341 IAF flc847f660cbb3883fa3f627	YV6341 IAD->OKC 2018-05	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
13	delay	2018-07-04	Extreme Delays: YV6031 IAF cc1505a76170b47517fb1bf	YV6031 IAD->PVD 2018-07	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
14	delay	2018-07-27	Extreme Delays: O03836 RI fbsa8e423076712a2b3484	O03836 RDU->BOL 2018-07	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
15	delay	2018-01-01	Extreme Delays: O05921 AI 365324dc84d3fa3df281900	O05921 ATL->ORD 2018-01	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
16	delay	2018-05-01	Extreme Delays: O05273 DI 4696e2cd2be1cc170d841bf	O05273 DEN->COS 2018-0	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
17	delay	2018-06-27	Extreme Delays: DL318 PBI 4480a4b7c2981fab6f8601c	DL318 PBI->JFK 2018-06-21	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
18	delay	2018-07-22	Extreme Delays: DL2173 MC 4498d6cfc4f396b99cccd1ea	DL2173 MCO->LGA 2018-07	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
19	delay	2018-03-28	Extreme Delays: O03018 A2 ed0a2ed98b2fbf88829282e	O03018 ASE->LAX 2018-03	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
20	delay	2018-07-16	Extreme Delays: YX5964 BO 6d8e1b97af4fb65d3c9badb	YX5964 BOS->LGA 2018-07	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
21	delay	2018-05-08	Extreme Delays: UA1749 GF ea0f21c7fe9cb34f032ae5e7	UA1749 GEG->SFO 2018-05	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
22	delay	2018-02-02	Extreme Delays: EV5575 AT 4e114c69eba89919955fd7	EV5575 ATL->BTR 2018-02	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
23	delay	2018-01-14	Extreme Delays: O03122 OI f82281d64a74cd27d025ad	O03122 ORD->CHO 2018-0	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
24	delay	2018-05-02	Extreme Delays: UA2000 OF 8c155ae7c10b207e77f6cb	UA2000 ORD->IAD 2018-05	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
25	delay	2018-06-18	Extreme Delays: O05433 OI 34b9229832ae52fe5b2af08	O05433 ORD->SLC 2018-06	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
26	delay	2018-07-22	Extreme Delays: DL2430 EV 696ca85d9af673c649f54c0	DL2430 EWR->DTW 2018-07	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
27	delay	2018-03-31	Extreme Delays: UA2163 SA b8e0fa1da83325fde30186f	UA2163 SAN->EWR 2018-03	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
28	delay	2018-01-14	Extreme Delays: EV4138 OH a78e28ac9deb6063eaz22ff	EV4138 OMA->EWR 2018-01	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
29	delay	2018-05-14	Extreme Delays: O03049 SL 7fac516784206c948aca554	O03049 SLC->ORD 2018-05	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
30	delay	2018-05-20	Extreme Delays: UA768 SF ee980fcd53f72af078808054	UA768 SFO->PDX 2018-05-2	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
31	delay	2018-02-09	Extreme Delays: AA1014 DT 6024f421660d274ee043e8	AA1014 DTW->DFW 2018-0	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
32	delay	2018-02-11	Extreme Delays: 9E3755 AT 07551454377557648a0300	9E3755 ATW->DTW 2018-02	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
33	delay	2018-05-14	Extreme Delays: UA2400 BC 814f97b652de72460bd88a	UA2400 BOS->SFO 2018-05	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
34	delay	2018-04-02	Extreme Delays: O05295 DI 301d053b93c02b7c51b155f	O05295 DEN->DRO 2018-0	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
35	delay	2018-03-17	Extreme Delays: EV4250 SD eb2b9e67a777a6f264d1c6f	EV4250 SDF->IAH 2018-03	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
36	delay	2018-05-16	Extreme Delays: AA1209 DF 4990c2e8969e63626ba08f	AA1209 DFW->MIA 2018-05	14ea29e1-33a2-40c0-b1c6	Extreme Delays		
37	delay	2018-01-14	Extreme Delays: O05143 OI 5bbcf56fbaca117f4fa3909f	O05143 ORD->EAU 2018-01	14ea29e1-33a2-40c0-b1c6	Extreme Delays		

With this last step, you generated your pipeline output. This output is a dataset that can be used in other applications in Foundry, such as [Contour](#) or [Code Workbook](#).

Send feedback

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

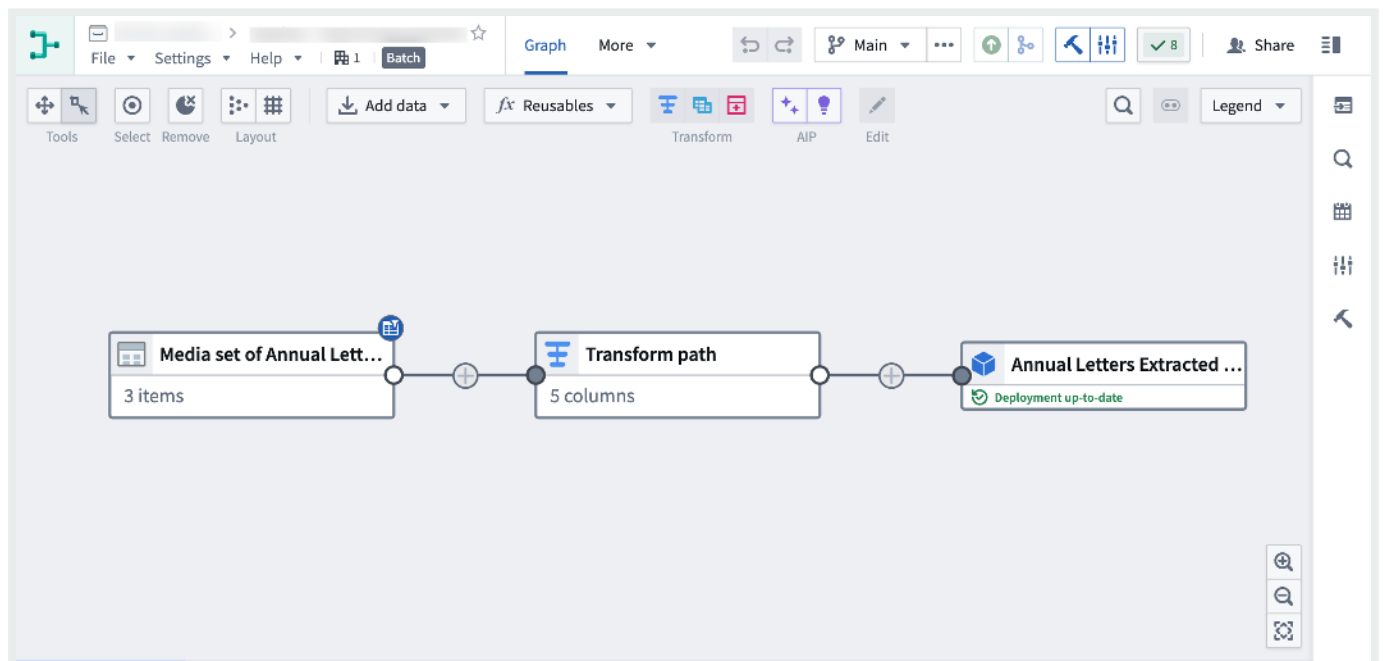
Data connectivity & integration / Building pipelines / Getting started / Create a media set batch pipeline with Pipeline Builder

Create a media set batch pipeline with Pipeline Builder

In this tutorial, you will use Pipeline Builder to create a standard batch pipeline with media sets to extract text from PDF.

This example uses PDFs of publicly available documents published by Palantir.

At the end of this tutorial, you will have a pipeline that looks like the following:



The pipeline will produce a new Object output of the extracted PDF text, which can be used

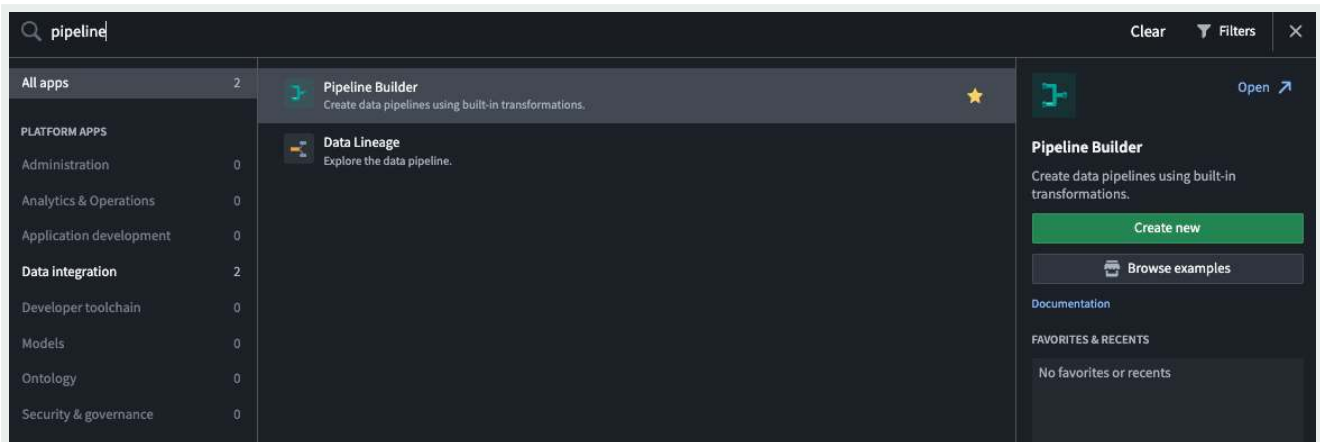
for [API Reference](#).

Send feedback

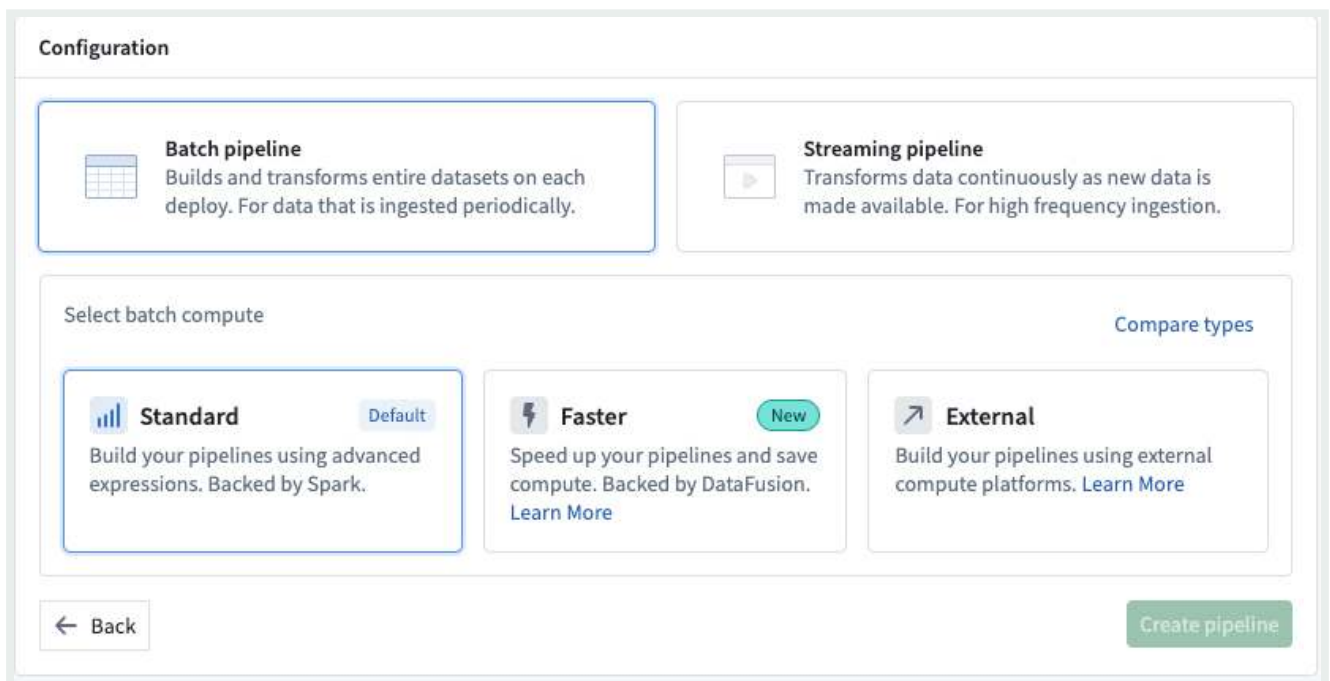
Part 1: Initial setup

First, create a new pipeline.

1. When logged into Foundry, access **Pipeline Builder** from the left navigation bar. If Pipeline Builder is not in the list of applications, select **View all** and find **Pipeline Builder** under the **Build & Monitor Pipelines** section.



2. Next, on the top right of the Pipeline Builder landing page, create a new pipeline by selecting **New pipeline**, then choose **Batch pipeline**. Under **Select batch compute**, choose **Standard** or **Faster**. For this example, choose **Standard**.

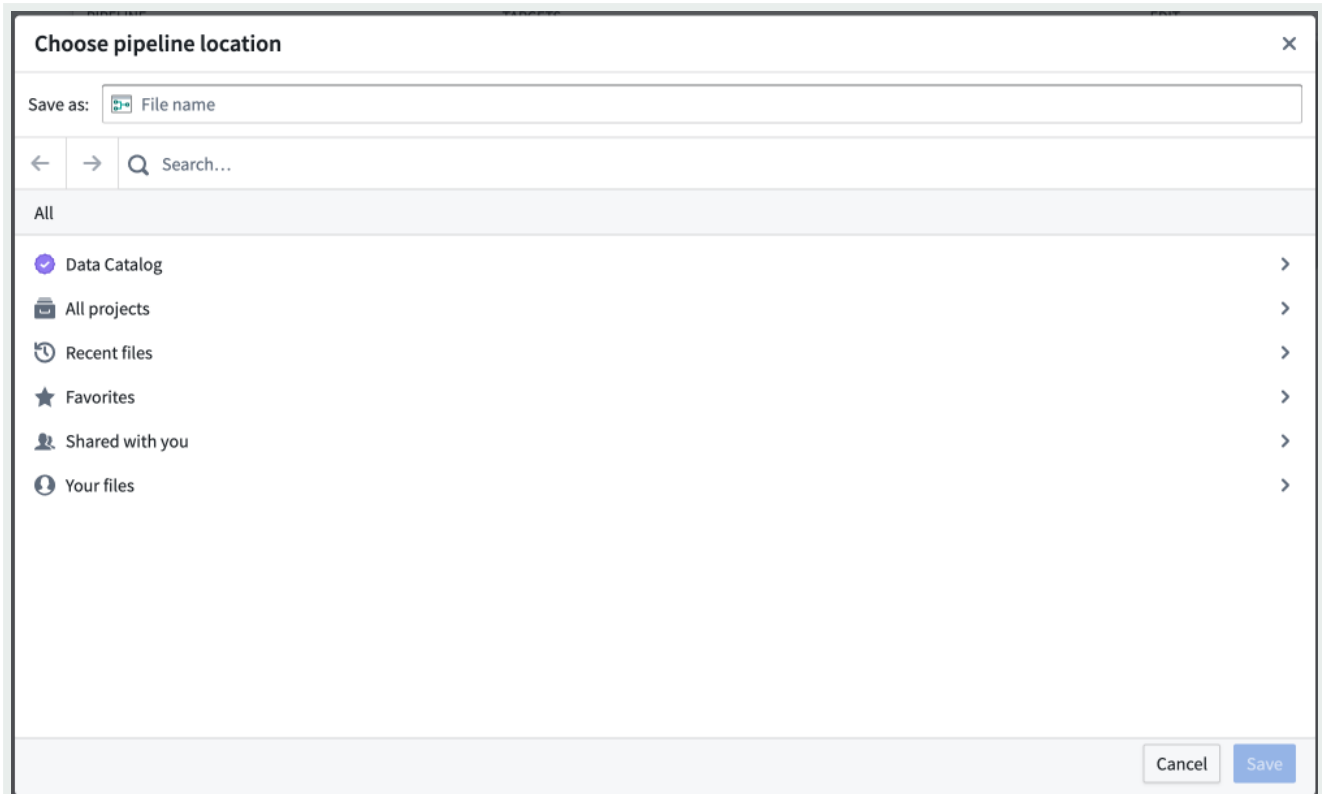


i The ability to create a streaming pipeline is not available on all Foundry environments. Contact Palantir Support for more information if your use case requires it.

[API Reference ↗](#)

3. Select a location to save your pipeline. Note that pipelines cannot be saved to folders.

[Send feedback](#)



4. Choose **Create pipeline**.

Part 2: Add media sets

Now, you can add datasets to your pipeline workflow. For this tutorial, you will use PDFs of publicly available documents from Palantir.

1. From the Pipeline Builder page, select **Add Foundry data** on the home page.

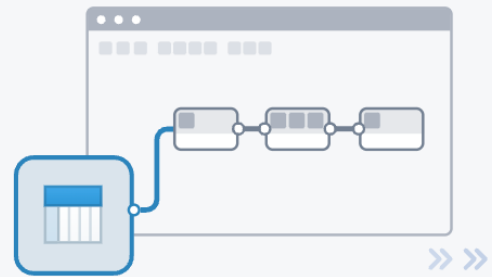
[API Reference ↗](#)

[Send feedback](#)

Welcome to Pipeline Builder

Get started by adding datasets, then define transform logic to derive target outputs.

 [Take a tour](#)



Add Foundry data

Recommended if you have already ingested data into Foundry.

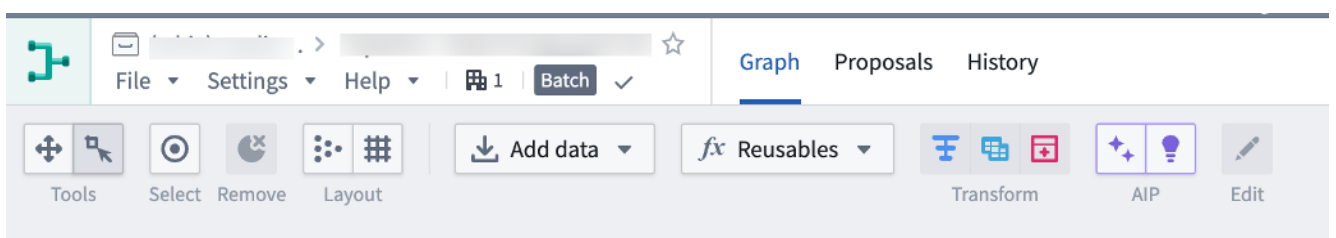
Upload from your computer

Recommended if you have sample data available locally.

Manually enter data

Recommended if you do not have data available to import.

You can also select the **Add data** action on the top panel.



Alternatively, you can drag and drop a file from your computer to use as your media set.

2. If you selected **Add data** or **Add Foundry data**, you will have the option to select your desired media sets.

[API Reference ↗](#)

[Send feedback](#)

⬇️

Add data

✕

🔍

Search all files

All > All projects >

Media set of Annual Letters

Other Dataset

Enter description...

⊕

Add to selection

Updated

36 minutes ago by

Created

36 minutes ago by

Location

.

Type

Media set

Media item count

1

Updated via

File imports

Show more

Tags

Add tags

⊕

Add all to selection

⬇️

Add data

⊕

Add to selection

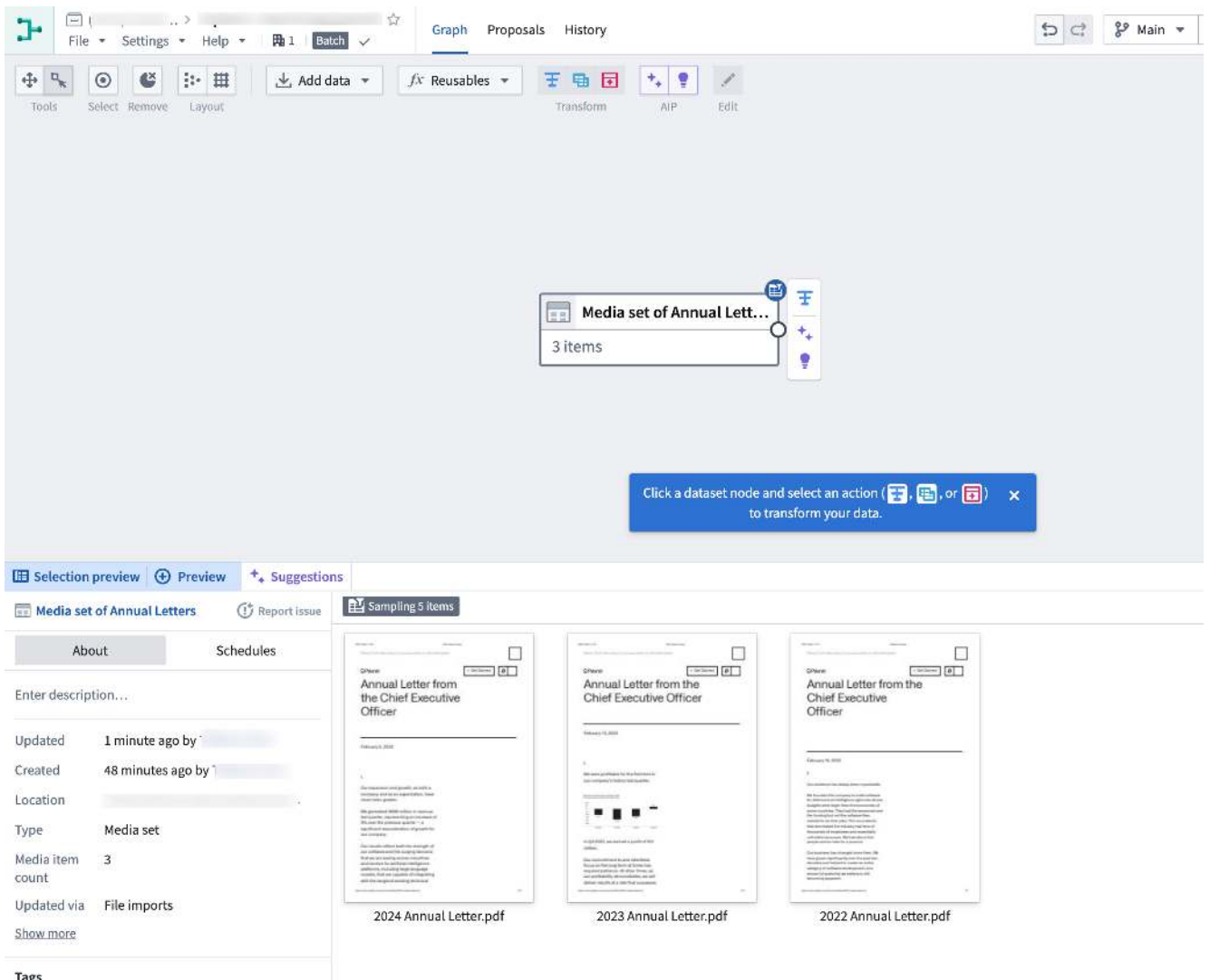
⬇️

Add data

Datasets to add (0)

No datasets selected

- When all media sets are selected, choose **Add data**.
- When you have imported your media set you will be able to see the media set with thumbnail preview.



Part 3: Media set transformations

After adding raw media sets, you can perform some basic transformations. For this workflow, you will extract the text from these PDF files.

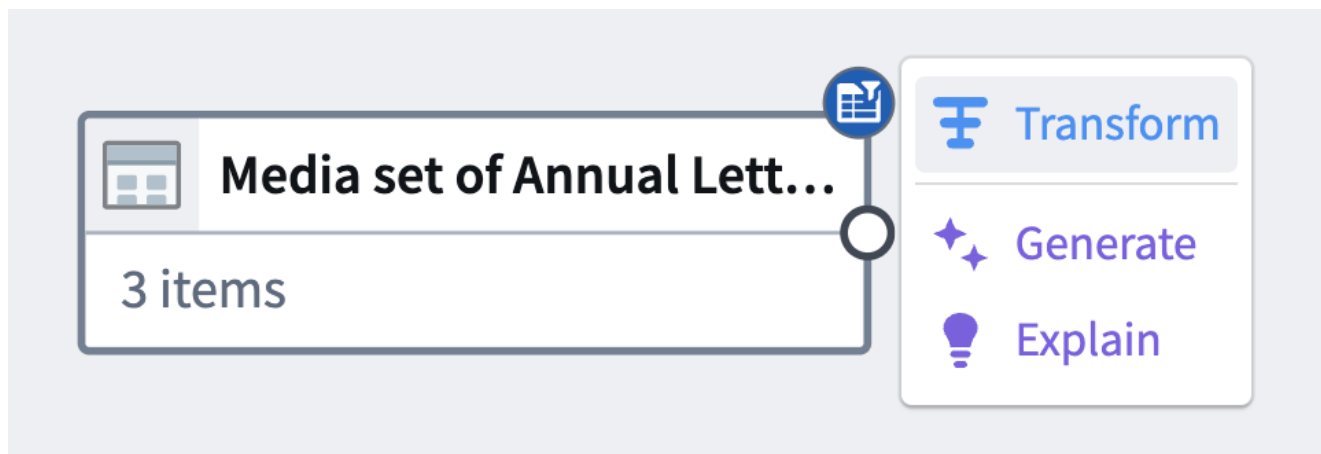
Extract text from PDF

You can directly transform media or extract information from media using [media references](#). In this example, you will extract text from the **Media set of Annual Letters** media set.

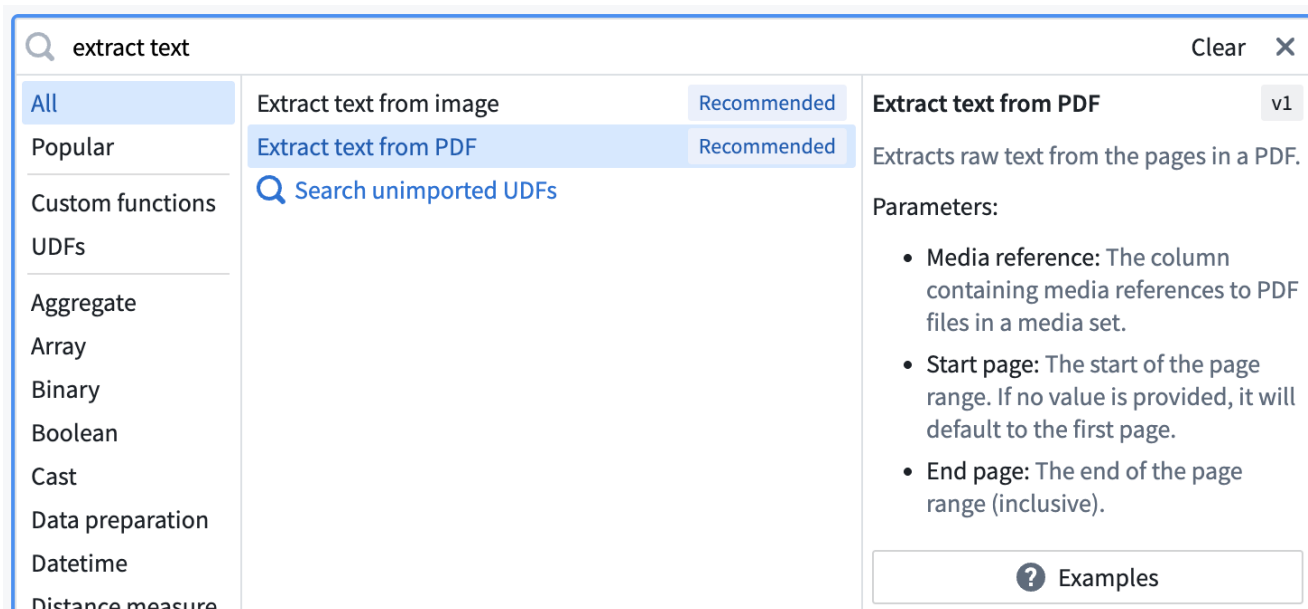
1. API Reference  a set of Annual Letters node in your graph.

2. Select **Transform**.

Send feedback



3. Search for and select the **Extract text from PDF** transform from the dropdown to open the board.



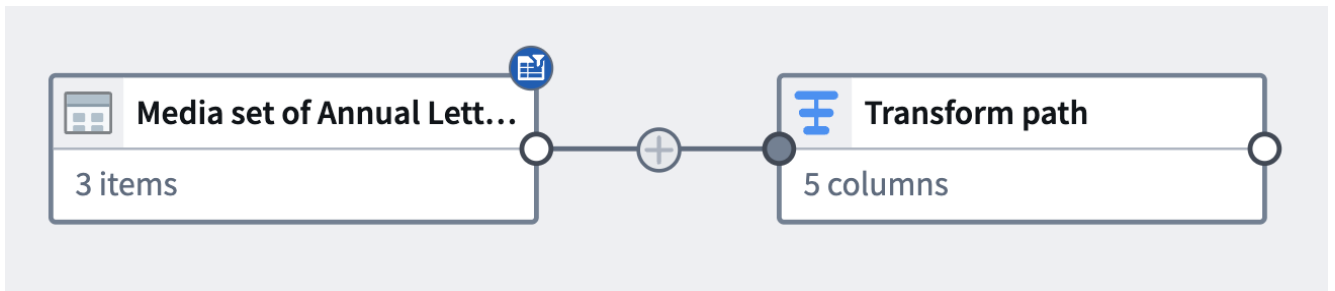
4. Select the extract method according to your needs and fill out the rest of the parameters.

- **Raw text** : Computer-generated PDFs
- **OCR** : Photocopies
- **Layout aware** : Text and bounding boxes

API Reference ↗

Send feedback

7. Select **Close** at the top right to return to your pipeline graph.



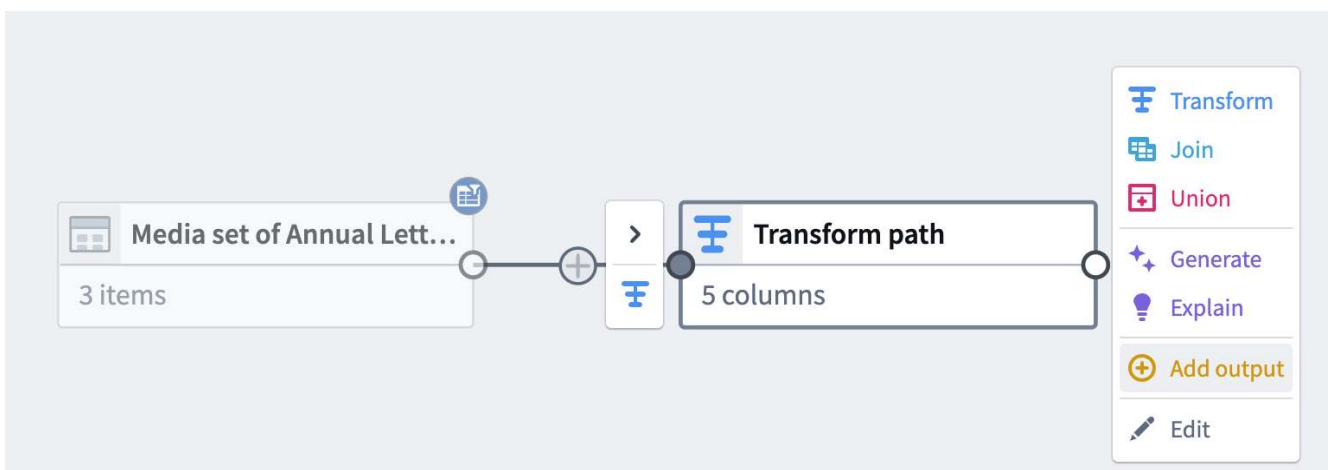
(Optional) Semantic search workflow

If desired, you can continue with a [semantic search workflow](#) with your extracted text.

Part 4: Add an output

Now that you have finished extracting text from your PDFs and potentially running extra string transformations, you can add an output. For this tutorial, you will add an object output.

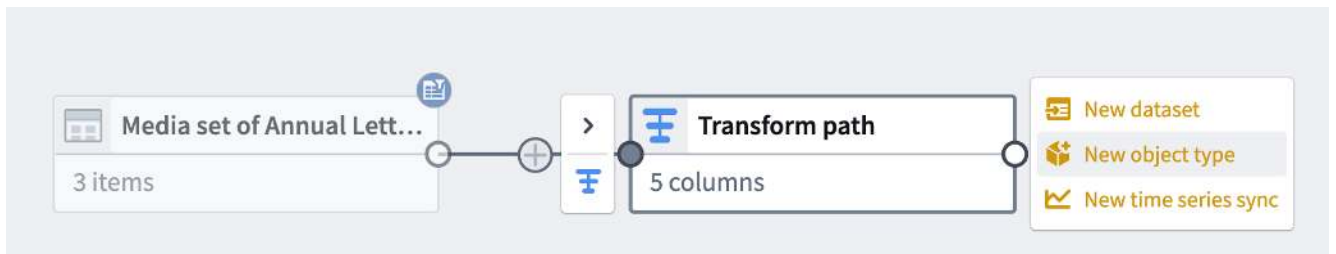
1. In the **Transforms** node where you have completed your transformations, select **Add output**.



2. Select **New object type**.

API Reference ↗

Send feedback



3. Name your object type and set the Ontology by choosing **Please select an ontology**.

Pipeline outputs   [+ Add](#)

 Annual Letters Extracted PDF 

✓ 5/5 columns mapped

 **Pipeline not yet deployed**
Deploy the pipeline to run the transforms and build all outputs.

Target ontology  **Please select an ontology**

Output folder *No location selected*







4. Select **Edit** and edit any column mapping. Ensure that you choose a valid column for the primary key.

[API Reference ↗](#)

[Send feedback](#)

←

Back to outputs

✎

Enable edits...

⚙️

...

📄

Name and icon

📄

Annual Letters Extracted PDF

Plural name

Annual Letters Extracted PDFS

Object type ID

annual-letters-extracted-pdf

✓

5/5 columns mapped

○

Not yet deployed

🔍

Search properties...

⚙️

Configure output

⋮

Pdf Ocr

✓

...

🕒

Timestamp

✓

...

”

Path

✓

...

🖼️

Media Reference

✓

...

”

Media Item Rid

🔑

📌

✓

...

Part 5: Build the pipeline

1. To build your pipeline, make sure to select **Save**, then **Deploy > Deploy pipeline**.

[API Reference ↗](#)

[Send feedback](#)

 Main ▼

 Save

 Propose

Deploy ▼

Deploy this pipeline

Update pipeline logic and build target outputs.

 Deploy pipeline

2. You should see `Initializing deployment` under the `Deploy Pipeline` sidebar option.

Close

Deploying 



 8

Share 



 Initializing deployment...

Start time	Feb 8, 2024, 5:23 PM
Duration	6s

View deployment history

Cancel deployment

API Reference ↗

Send feedback

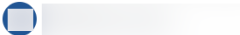


3. Select **View deployment history** to track the progress of your deployment. You should be led to the **History** tab in your pipeline where you can view the statuses and history of your deployments:

 **Deployment** 

19 seconds ago 

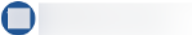
- ✓ Deployed logic for 1 target
 - Building output (0/1) [Build report](#)
- ✓ Published 1 ontology type output.



 **Deployment** 

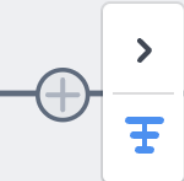
1 minute ago

- ✓ Deployed logic for 1 target
- ✓ Finished building output (1/1) [Build report](#)
- ✓ Published 1 ontology type output.



(Optional) Part 6: North of the Ontology

Once deployment has completed and your object is initialized, you should be able to directly action on your object output. Select **Create Workshop module** to generate a Workshop module with your pipeline output.



Annual Letters Extracted ...
 Deployment up-to-date

API Reference ↗

 Open in Object Explorer

 Analyze in Quiver

 Create Workshop module

 New link type

 Edit

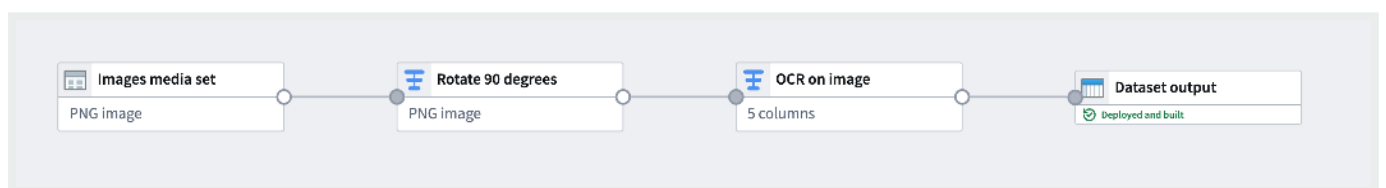
Send feedback

With this last step, you have generated your pipeline output and a Workshop module.

FAQ

Can I transform media before extracting data from it?

Yes, you can transform media before extracting data from it, and this works with many extraction operations including extracting text and using large language models (LLMs). As an example, consider a media set containing landscape-oriented images that need to be rotated clockwise 90 degrees before performing OCR-based text extraction. You can add an expression to rotate images and subsequently add an expression to extract text with OCR.



⚠ To safely write the pipeline results to a dataset output, you must remove the media reference column from the output schema as transformed media references are not valid column outputs.

To do this, remove the column using the output dialog.

PREVIOUS

← Create a dataset batch pipeline with
Pipeline Builder

NEXT

Create a dataset batch pipeline with →
Code Repositories

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Building pipelines / Getting started / Create a dataset batch pipeline with Code Repositories

Create a dataset batch pipeline with Code Repositories

This guide will step you through a simple data transformation example using the **Code Repositories** application. You will learn how to write and edit SQL code as well as build your datasets. You will also learn how to work on branches to allow you to collaborate with colleagues.

1. Create a repository

Get started by creating a new repository. To do so, navigate to a [Project](#) in Foundry, select **+** **New** in the top-right, then select **Code repository**.

For this guide, we will write a SQL transform. Give your repository a name, then select **SQL** in the dropdown under **Language template**. Then, select **Initialize repository**.

[API Reference ↗](#)[Send feedback](#)

<>

Building pipe... > Simple batch pipeline ☆
File ▾ Help ▾ 1

Share

☰

Initialize Repository

1

Name your repository

Simple batch pipeline

Repository type Select the type of repo you want to initiate.

Data Transforms
Transform and integrate datasets using your preferred language.

2

Language template

SQL ▾

3

Initialize repository

2. Import your data

If you've already imported a raw dataset that you will be working with, you can move on to the next step. Otherwise, you can download this sample dataset:

[Download titanic.csv](#)

Refer to the guide on [manual data uploads](#) to learn how to upload this dataset into your Project alongside your repository.

Data Catalog	Projects	Your files	Shared with you
Building pipelines ☆ 1			
Files	Autosaved	Project Catalog	Trash Inference
NAME ^		LAST UPDATED	
Simple batch pipeline		Fri, Apr 22, 2022, 5:55:36 PM	
API Reference ↗		Fri, Apr 22, 2022, 5:55:51 PM	

Send feedback

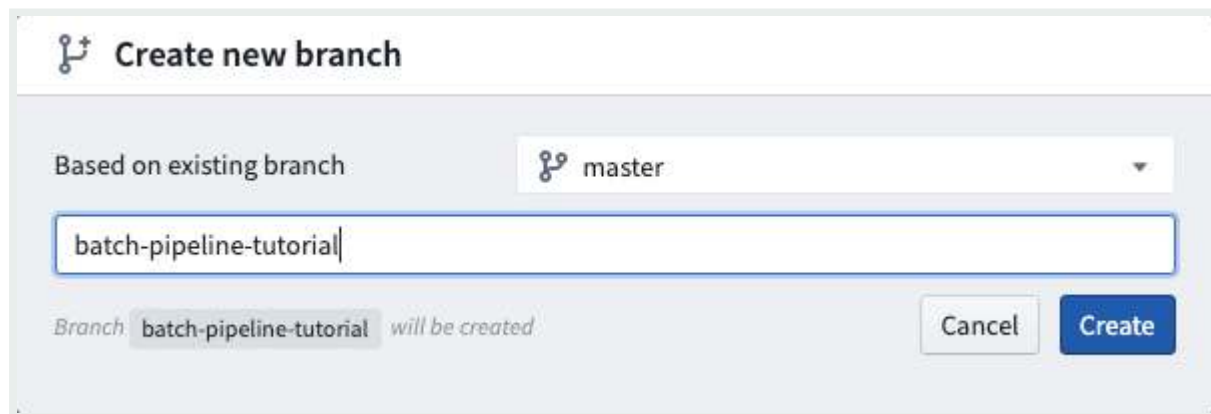
3. Create a branch

By creating personal **branches** to make changes, instead of directly editing the master version of the code, you can safely collaborate on the same Code Repository with your colleagues. You can track and undo changes as well as merge changes into the master code. Code can be reviewed on a line-by-line basis meaning that changes to production pipelines can easily be discussed amongst teammates. To learn more about branching in Foundry, see [this page](#).

When you navigate to a Code Repository, you will be on the `master` branch by default. It is best practice for the `master` branch to be protected, meaning that it is not possible to directly edit code on that branch. Note that you can read files on protected branches, but you cannot edit or create files.

Before you can add changes to your Code Repository, you must first create your own branch which contains a copy of the code on the `master` branch. To create your branch, click the  icon next to the current branch name.

This opens a dialog for selecting an existing branch and choosing a custom name for the new branch:

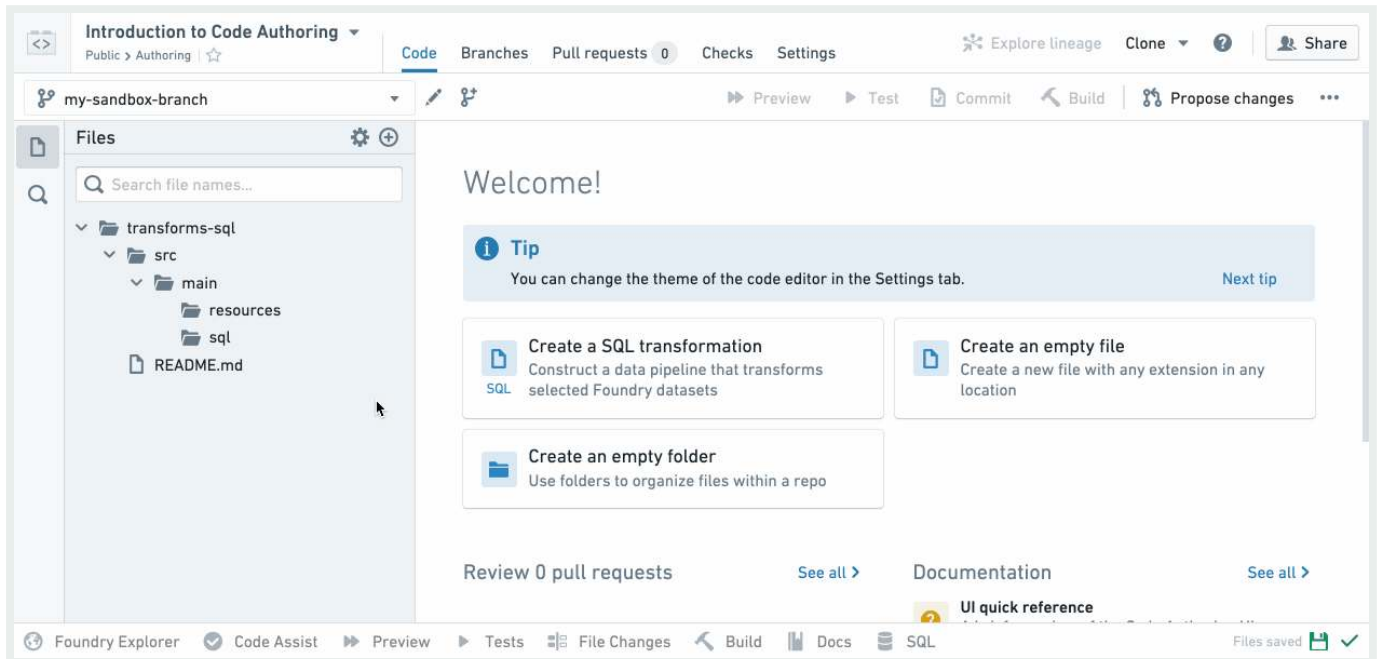


The image shows a 'Create new branch' dialog box. At the top, it says 'Create new branch' with a branch icon. Below that, there's a section 'Based on existing branch' with a dropdown menu showing 'master'. A text input field contains 'batch-pipeline-tutorial'. At the bottom, it says 'Branch batch-pipeline-tutorial will be created'. There are 'Cancel' and 'Create' buttons.

After you create a new branch, you will see an identical file tree on the left-hand side. You've simply created a copy of the code on the `master` branch you started on. You can now edit files in your branch.

Creating a new file

Now that you're working in your own branch, create a new SQL file by clicking the ellipses icon when you hover over a folder and then selecting **New file**. Once you select **New file**, you will be prompted to select the type of file and give it a name. For this example, select **SQL Transformation** and pick a filename (without spaces or special characters):



Notice that your new SQL file is highlighted in the file tree at the position where the resulting dataset will exist when you build it.

i If a filename is green in the file tree, it means it's a newly created file in your branch that doesn't exist on the master branch where you started. If a filename is orange, it means it's a file that exists on the `master` but has been modified in your branch.

Your newly created `.sql` file will declare an output dataset based on the filename you provide. For instance, if your repository is inside `/Public/Authoring` and you create `titanicAnalysis.sql`, your new file will automatically declare an output dataset `/Public/Authoring/titanicAnalysis`.

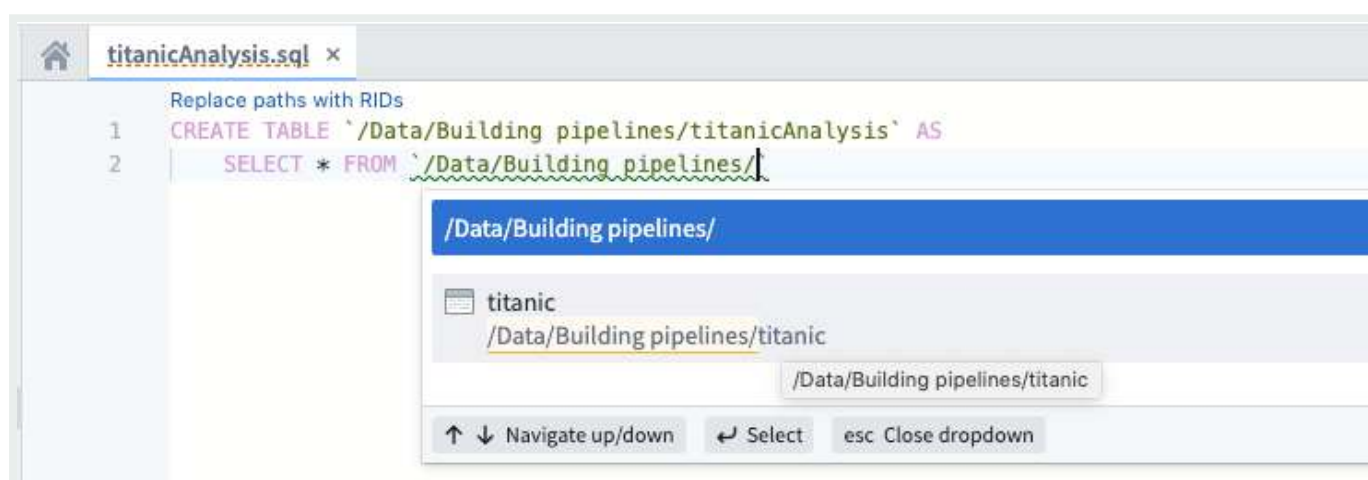
E [API Reference ↗](#)

[Send feedback](#)

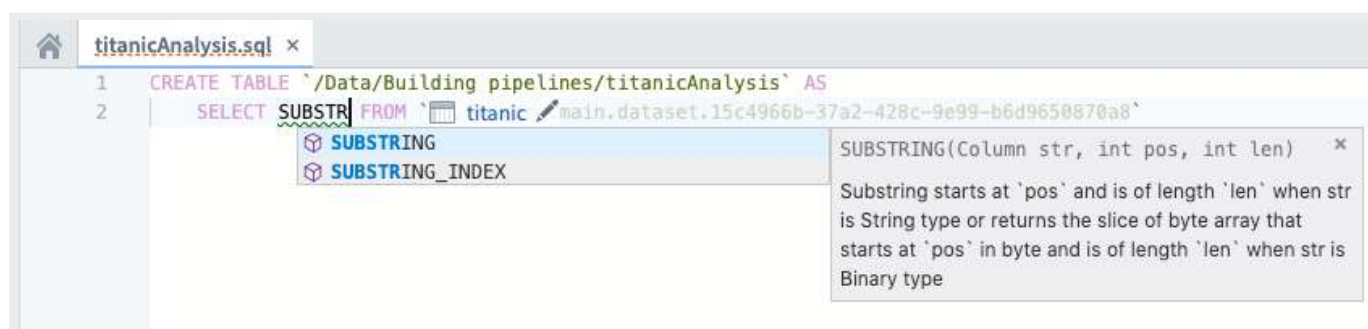
Next, replace the placeholder text with your actual data transformation code. First, replace ``SOURCE_DATASET_PATH`` with the path to an actual input dataset. In this example, you will use the `titanic` dataset you imported in step (2) of this tutorial.

Notice that when you start typing a backtick, auto-complete will show you an interactive menu listing datasets you can use. When you select a dataset from the list, the dataset reference will be replaced by the unique ID of the dataset. This makes it so that your transformation code will continue to work even if the dataset is moved later.

Type the name of your project in the backticks, find the `titanic` dataset, and select it from the menu.



Continue writing SQL code to perform transformations on your data. You will see various help dialogs appear as you type SQL functions. For instance, say you want to create a new column with a one-letter abbreviated gender of passengers in your “titanic” dataset. You can view information about how to use the `SUBSTRING` function:



Before moving on, finish writing your data transformation code to select the “Name”, “Age”, “Sex”, and “Survived” columns as well as a derived column called “Gender”. The “Gender” column represents the abbreviated gender of passengers; to create it, use the `SUBSTRING` function on the “Sex” column.

Send feedback



Note that you must define an alias for any derived columns you create in SQL. For more information about writing SQL data transformations, refer to the [Spark SQL language reference](#).

5. Test your changes

i In Foundry, datasets can be branched (similar to code). This is useful for testing the design of multi-step data pipelines. For instance, you can test changes to pieces of data pipelines in isolation without breaking downstream dependencies for anyone who doesn't rely on your branch.

Now that you've written your data transformation code, you should test the changes you've made in your branch. It's important to test your changes to be sure your code is working as expected before merging your changes into the `master` branch.

Use preview to iterate on your changes

As you write your code, you can use the **Preview** feature to accelerate the development cycle and iterate on changes quickly. Preview runs your code on sampled inputs and provides a sample output without the need to commit your changes, run checks, or materialize a dataset in Foundry. Sample outputs may not be representative of the result of a build, but they can provide a way to confirm your code is working and producing the expected results.

API Reference ↗

To access the **Preview** feature, click the **Preview** button in the top bar of the Code Editor. It's possible to preview file-based datasets or ones with schema.

Send feedback

For datasets with a schema, you can customize the inputs used when previewing your changes by clicking the **settings icon** for the input you wish to edit. The options are:

- **Original input:** Use a sample of the original input.
- **Previous preview:** For datasets produced in the same repository, you can chain previewed changes and use a preview of the dataset as an input to your preview.
- **Apply custom filters:** Apply filters on supported columns to test your changes on a specific subset of the input. For example: "all rows with a timestamp within a given window", or "all rows with a given string value".
- **Select a different dataset:** Select another dataset with all the columns that your code requires, to test specific cases or to provide any other custom sample as your input.

When running Preview for the first time on a dataset containing files, you must configure the files that will be used within the sample. Once the sample files have been selected, they can be reconfigured by selecting the relevant input. After saving the configuration, Preview will execute the code on the chosen sample of files.

When running Preview again, there will be no need to reconfigure input files. Once Preview has executed, you can view the sample output as rows or files. If you have the required permissions, you can also choose to download the output files.

Commit your changes

After writing new code, you can commit your changes. In Code Repositories, you commit changes when you want to label the work you've performed. Even before you make a commit, your work is auto-saved by default. A commit specifically labels your set of changes when you reach a stopping point.

Tip

Clicking the **Commit** button commits your changes and runs automatic checks on your code. Clicking the **Build** button also commits your changes. Specifically, clicking **Build** runs automatic code checks and starts building your output dataset.

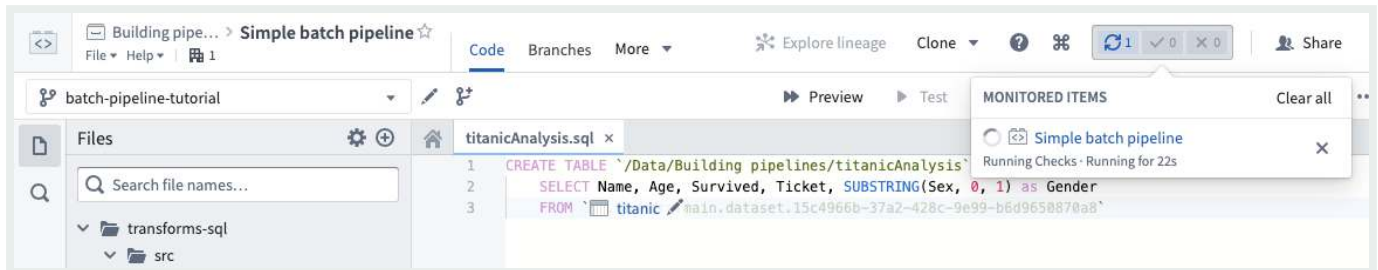
If you want to quickly test your changes without building your dataset to ensure your

API Reference ↗

the code checks, click **Commit**. Otherwise, you can skip ahead to [build your dataset](#).

Send feedback

To commit the changes you've made, click the  **Commit** button at the top right corner and enter a summary of the changes you've made. Committing changes triggers automatic checks to run on your code. An icon in the top-right corner indicates the status of these checks; hover over it to see more details.



Build your dataset on your branch

To test your changes, click the  **Build** button at the top of the screen.

Once you click the build button, two things happen: automatic checks run on your code and your output dataset starts to build. During this time, either a new output dataset will be created from the code in your branch or an existing output dataset will get updated to reflect your changes. You can view the progress of the running tasks in the **Build** helper. Once the tasks complete, the  icon indicates that each task has successfully completed. If you see the  icon instead, click the **details** button in the **Build** helper to learn more about what went wrong. This will take you to the **Checks** tab where you can look for error messages and also re-trigger your build.

Here is some important information about testing your changes and building your datasets:

- Clicking the **Build** button at the top right corner is equivalent to clicking the **Build** button in the **Build** helper at the bottom of the Code Repositories interface.
- You must select the file containing your data transformation logic before you can actually click the build button to run checks and start building your output dataset.
- Each time you trigger a build on your branch, it will queue up behind existing builds that are already running.

[API Reference ↗](#)

Preview your dataset

[Send feedback](#)

Once your tasks successfully complete and your dataset gets built, you can preview your built dataset in the **Build** helper:

The screenshot shows the Foundry Explorer interface. At the top, there's a breadcrumb 'Building pipe... > Simple batch pipeline' and tabs for 'Code', 'Branches', and 'More'. Below this is a toolbar with 'Preview', 'Test', 'Commit', 'Build', and 'Propose' buttons. The left sidebar shows a file tree with 'transforms-sql', 'src', 'main', 'resources', 'sql', and 'titanicAnalysis.sql'. The main editor displays the SQL code for 'titanicAnalysis.sql':

```
1 CREATE TABLE `Data/Building pipelines/titanicAnalysis` AS
2 SELECT Name, Age, Survived, Ticket, SUBSTRING(Sex, 0, 1) as Gender
3 FROM `titanic`
```

Below the editor, the 'Build' section shows a 'Build finished' status. It includes a table preview for 'titanicAnalysis' with 300 of 891 rows shown. The table has 5 columns: Name (String), Age (Double), Survived (Integer), Ticket (String), and Gender (String). The bottom status bar indicates 'Code Assist running', 'Project scoped', and 'Files saved'.

	Name String	Age Double	Survived Integer	Ticket String	Gender String
1	Braund, Mr. Owen Harris	22	0	A/5 21171	m
2	Cumings, Mrs. John Bradley (Flo	38	1	PC 17599	f
3	Heikkinen, Miss. Laina	26	1	STON/O2. 3101282	f
4	Futrelle, Mrs. Jacques Heath (Lil	35	1	113803	f
5	Allen, Mr. William Henry	35	0	373450	m
6	Moran, Mr. James	null	0	330877	m
7	McCarthy, Mr. Timothy J	54	0	17463	m
8	Palsson, Master. Gosta Leonard	2	0	349909	m
9	Johnson, Mrs. Oscar W (Elisabet	27	1	347742	f
10	Nasser, Mrs. Nicholas (Adele Ach	14	1	237736	f
11	Sandstrom, Miss. Marguerite Rui	4	1	PP 9549	f
12	Bonnell, Miss. Elizabeth	58	1	113783	f
13	Saundercock, Mr. William Henry	20	0	A/5. 2151	m
14	Andersson, Mr. Anders Johan	39	0	347082	m

Click on the link to your dataset in the **Build** helper to open your full dataset.

Now, you have built the dataset on your own branch. Continue reading through the rest of this guide to learn how to merge your data transformation code into the master branch.

6. Propose your changes for review

By this point, you have:

- Created your own branch to make your changes on,
- Created a new SQL file with your data transformation code,
- [API Reference](#) ↗ es and built your dataset in your branch, and
- Previewed your built output dataset.

[Send feedback](#)

Now, you will propose your changes for review by your teammates. After you've tested your changes and are happy with your resulting output dataset, you can propose your changes for review by your teammates.

i Tip

Users with **Owner** permissions will be able to enable the option to “Automatically merge changes” when creating a *pull request*. This option is only available if at least one required check is configured and passing for your repository’s branch. If you enable the option to “Automatically merge changes”, your *pull request* will automatically get merged into the main code after you create it. Once the changes from your branch are merged into the main code, your branch will also get automatically deleted.

To create a new **Pull Request**, click the  **Propose changes** button at the top right corner. This will open the "New pull request" page where you can write a description of your changes and click the **Create pull request** button.

Building pipe... > Simple batch pipeline

Code Branches More

Explore lineage Clone

0 2 0

Share

< Titanic Analysis

Open

Add description

wants to merge into master from batch-pipeline-tutorial a few seconds ago

Overview

Files changed 2

Impact analysis

Pipeline review

Security changes

Commits 2

Conversation

This pull request is open.

batch-pipeline-tutorial is up-to-date with master.

No code approval required.

Checks passed 6 minutes ago.

Add/remove reviewers

View details

merged ☐ Merge when ready

Close

Squash and merge

API Reference ↗

This creates a new pull request with your proposed changes. The pull request page is a wide range of tools you can use to review how the proposed changes will affect your data

Send feedback

pipeline:

- The **Files changed** tab allows teammates to review your code on a line-by-line basis
- The **Impact analysis** tab shows how affected datasets have changed on your branch, including changes to dataset schemas and health checks
- The **Pipeline review** tab shows a graph of your data pipeline and highlights how changes in this pull request affect the pipeline visually.

[Learn more about understanding the impact of changes in a Pull Request.](#)

In general, it's important to invite others to review your changes before they're merged into the master branch. Users can approve or reject on a file-by-file basis to keep track of which files still need to be adjusted before the pull request can be merged. To see which users have already approved or rejected a particular file, hover over the corresponding indicator icon.

Using the individual file review buttons is optional, but when you reject one file, this automatically rejects the pull request. Similarly, when you approve the pull request, all individual files get approved.

7. Merge your changes into the **master** branch

Finalizing your changes

Once your changes have been reviewed, a user with the appropriate permissions (by default, Owners and Editors) can *merge* the changes on your pull request to combine them into **master**.

For the purposes of this tutorial, proceed by selecting the **Squash and merge** button at the bottom-right of your screen, then select **Squash and merge** again in the confirmation dialog.

API Reference ↗ may have different policies that must be met before changes are merged. Policies are defined by the repository owner in the [branch settings page](#) and presented to code authors in the Pull Request page.

Send feedback

Validating that your changes are on master

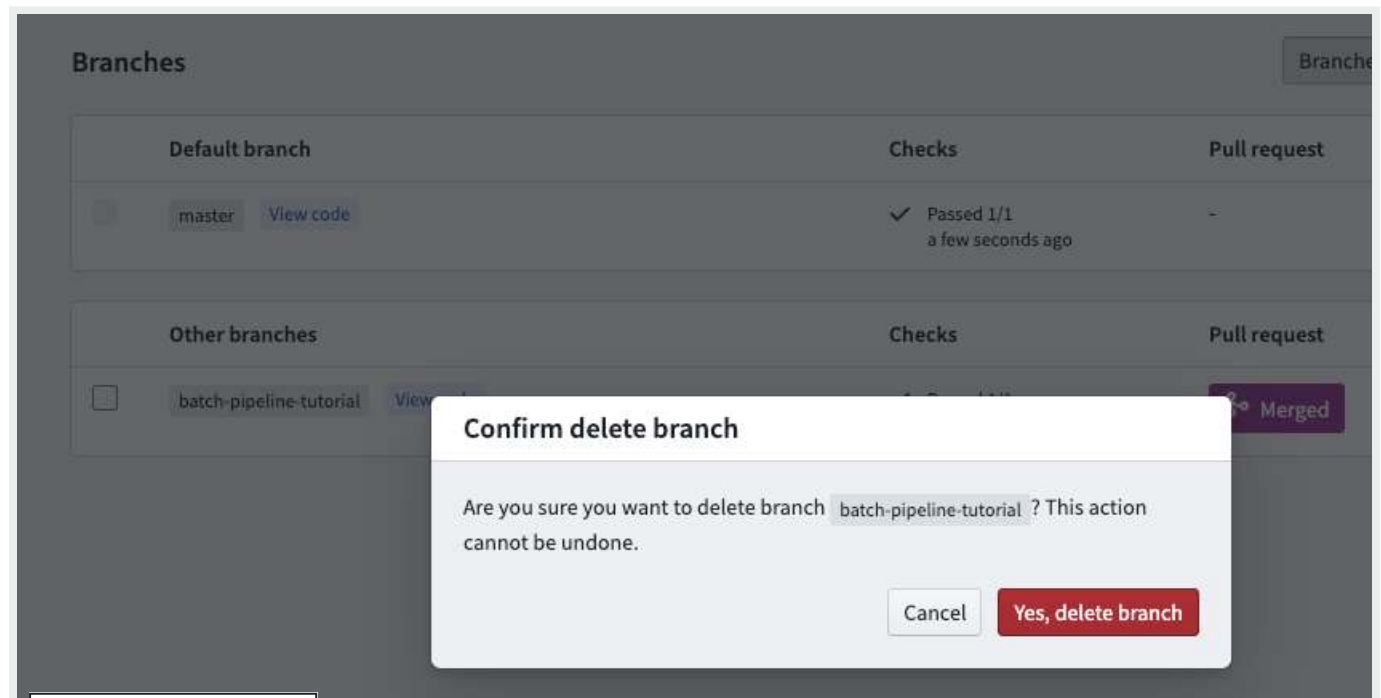
Once your proposed changes have been accepted, you should validate that your the changes you made in your branch are reflected on the `master` branch. To do so, click the **Code** tab, select the `master` branch, and browse the files. Make sure you see the changes that you made in your branch.

Deleting your branch

⚠ Warning

Do not delete any branches that you did not create. For others working in the same Code Repository, this could result in lost work!

Now you can delete the branch you created at the start of this tutorial to reduce clutter. Since your changes have been merged into the `master` branch, there is no need to keep your branch. Navigate to the **Branches** tab, and look under “Personal branches”. Delete the branch you created by clicking the  icon:



[API Reference ↗](#)

 The pull request page offers a "Delete branch after merge" option

[Send feedback](#)

clean-up of branches. This option is unavailable for protected branches.

To delete a protected branch, you first need to [unprotect](#) it and then follow the steps above.

8. Build your dataset on the `master` branch

The final step is to build your new dataset on the `master` branch. Similar to [build the dataset in your branch](#), click the  **Build** button at the top of the screen when your SQL code file is selected.

Once your tasks successfully complete and your dataset gets built, you can click on the link to your dataset in the **Build** helper to open your full dataset.

Congratulations! You have successfully created a new data transformation and published your changes using Code Repositories. Here are some possible next steps to continue learning:

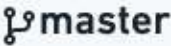
- Learn [how to create an incremental pipeline](#).
- Learn how to create more complex pipelines using [Python Transforms](#).
- Explore details about core concepts like [datasets](#), [branching](#), and [builds](#).

9. Revert changes

If you notice problems with a code change you have already merged into the master branch, there is an easy way to undo those changes. You can revert a specific commit by locating it in the commit history of the master branch. On the **Branches** tab of your repository, click on `master` to see the chronological list of all commits.

[API Reference ↗](#)

[Send feedback](#)


[View checks](#)
[View code](#)

✓ Checks passed a few seconds ago.
[View details](#)

Commits

↻
f9773faa948
Revert
a minute ago

Change input dataset
admin admin <example@localhost>

↻
b79f1fbd6e8
Revert
a minute ago

Add another test case
admin admin <example@localhost>

↻
55da8bcdf85
Revert
2 minutes ago

Add documentation
admin admin <example@localhost>

↻
c888493f689
Revert
2 minutes ago

Rewrite logic of main pipeline
admin admin <example@localhost>

↻
dc64d8efa0f
Revert
Jul 2, 2019 4:26 PM

Generated shrinkwrap file transforms-shrinkwrap.yml

You can view a certain commit's code changes by clicking on the commit hash. Once you have located the commit you want to revert, click **Revert**. This will open a pull request into the master branch which you can review and merge.

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Building pipelines / Getting started / Create a media set batch pipeline with Code Repositories

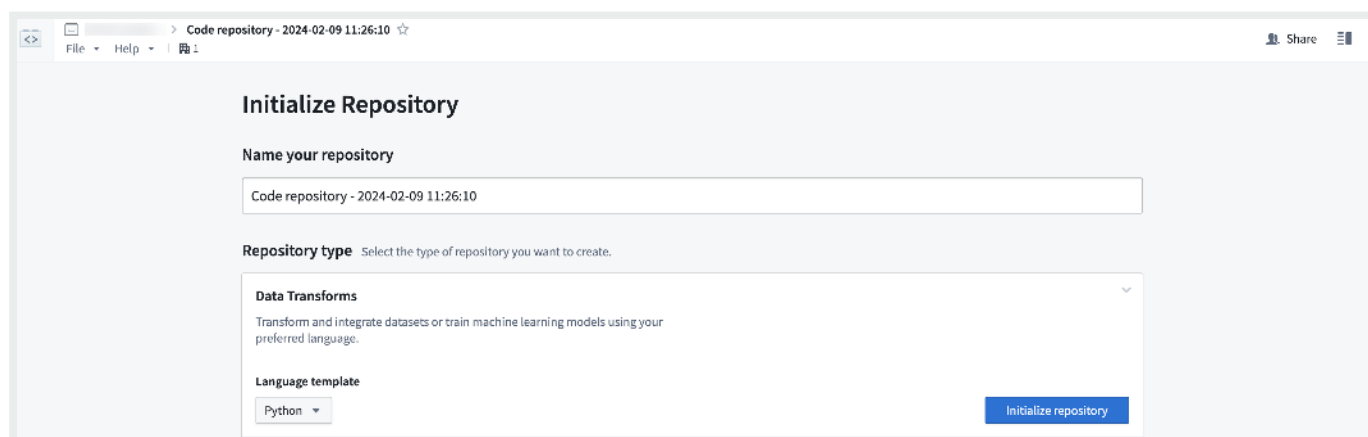
Create a media set batch pipeline with Code Repositories

This guide will walk you through a simple transformation example using the **Code Repositories** application. You will learn how to write and edit Python code with media sets.

1. Create a repository

Get started by creating a new repository. To do so, navigate to a [Project](#) in Foundry, select **+** **New** in the upper right, then select **Code repository**.

For this guide, we will write a Python transform. Give your repository a name and select **Python** in the dropdown menu under **Language template**. Then, select **Initialize repository**.



The screenshot shows the 'Initialize Repository' dialog box. At the top, it says 'Code repository - 2024-02-09 11:26:10'. Below this, there's a section 'Name your repository' with a text input field containing 'Code repository - 2024-02-09 11:26:10'. Underneath is 'Repository type' with a note 'Select the type of repository you want to create.' and a dropdown menu. The dropdown is open, showing 'Data Transforms' as the selected option, with a description: 'Transform and integrate datasets or train machine learning models using your preferred language.' Below the dropdown is 'Language template' with a dropdown menu showing 'Python'. At the bottom right is a blue button labeled 'Initialize repository'.

2. Import your data

[API Reference ↗](#)

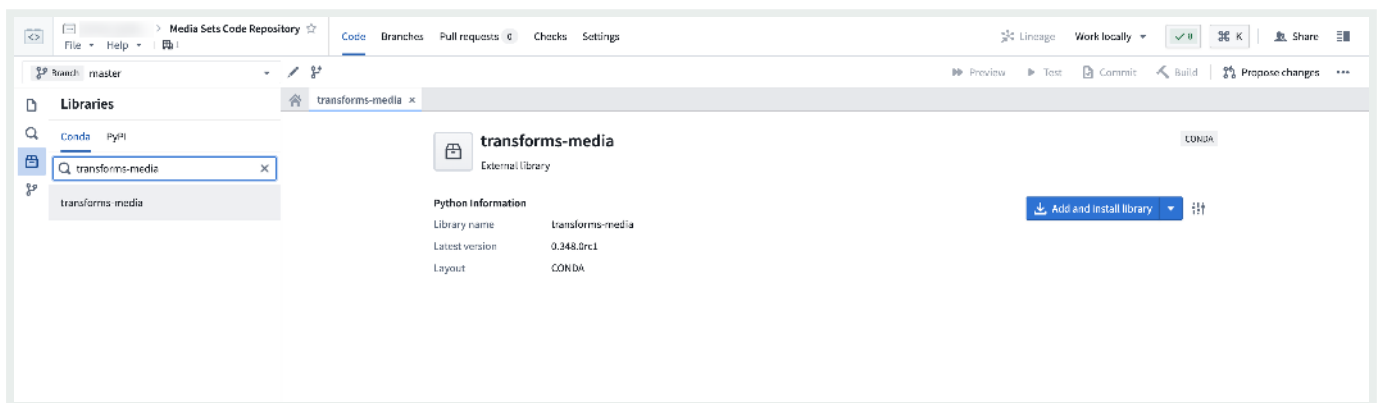
Send feedback

If you already imported a raw media set to work with, you can move on to the next step. Otherwise, refer to the media sets guide on [how to import media into Foundry](#).

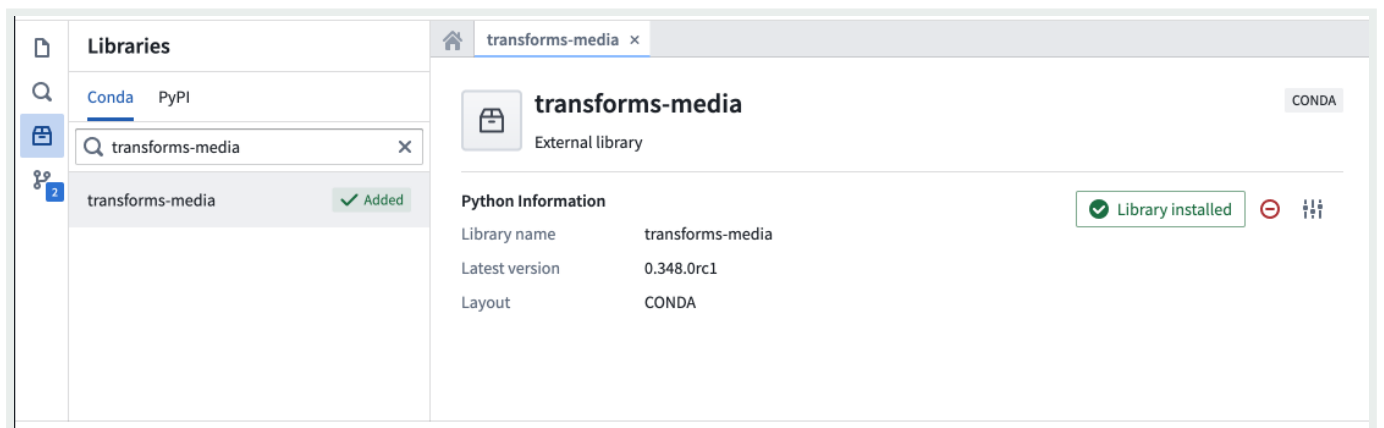
3. Import the `transforms-media` library

To use media set transformations, you must import the `transforms-media` library.

Navigate to the **Libraries** option from the left sidebar, then search for and select the `transforms-media` library. Then, choose to **Add and install library**.



The status should show as **Library installed** once the install is complete.

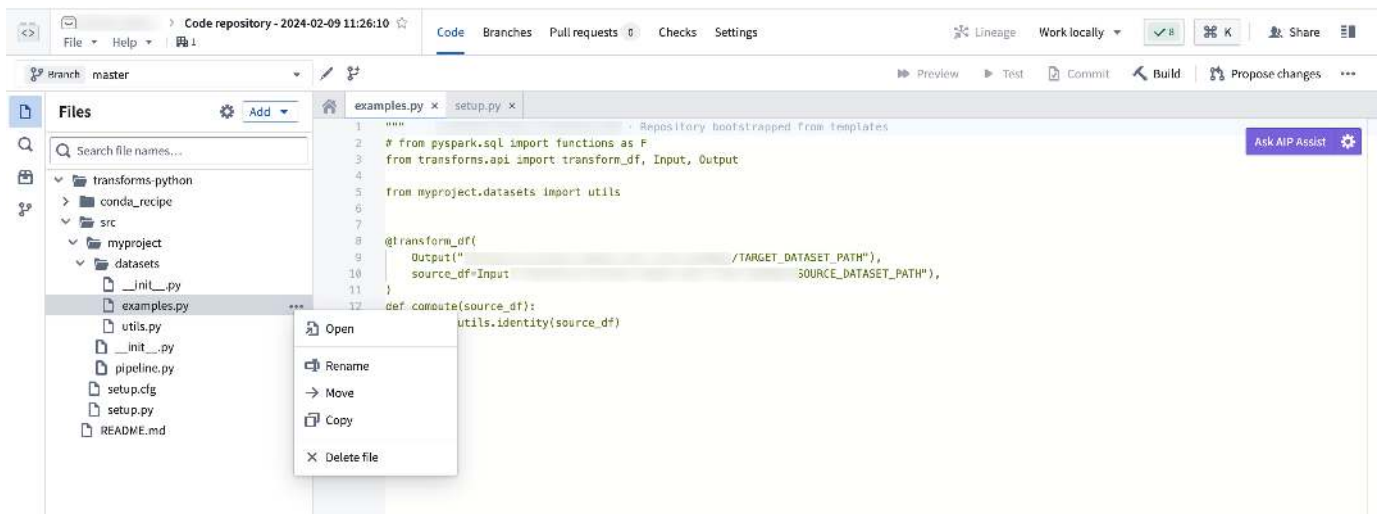


4. Edit code

API Reference ↗ file

Send feedback

Creating a Python transforms repository should auto-generate an `examples.py` file which you can use as your base. You can rename the file by selecting the ellipses icon, choosing **Rename**, and naming your file accordingly.



If you want to create a new file, select the ellipses icon and choose **New file** under the **Datasets** directory. Ensure you are creating a Python file for the purposes of this workflow.

Import the correct libraries into your file

When working with media sets, you must use the `@transform` decorator. Media set inputs and outputs must be passed in using `transforms.mediasets.MediaSetInput` and `transforms.mediasets.MediaSetOutput` specifications:

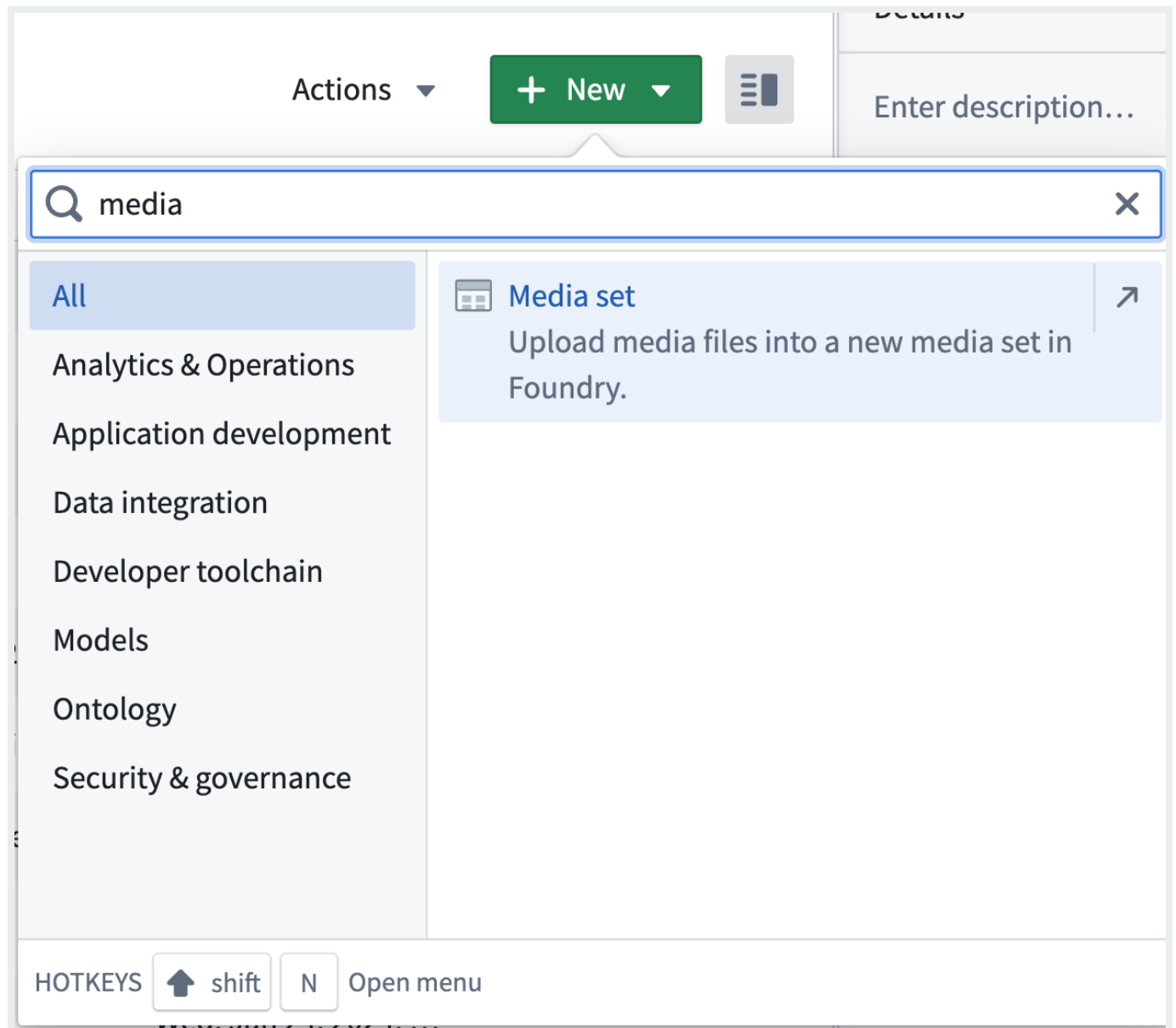
```
1 from transforms.api import transform
2 from transforms.mediasets import MediaSetInput, MediaSetOutput
3 @transform(
4     images=MediaSetInput('/examples/images'),
5     ...
6 )
```

Ensure output file exists

If [API Reference ↗](#) media set output file, be sure to create it before referencing it in the transform.

Send feedback

You can create this file by navigating to your Project folder (where you first created the Python transforms code repository), and selecting **New > Media set** from the dropdown menu.



Use `get_media_item()`, `get_media_item_by_path()`, or `list_media_items_by_path_with_media_reference()`

To transform each media item in your media set, you must reference these media items in your transformations with `get_media_item()` or `get_media_item_by_path()`.

API Reference ↗

Note that calls to `get_media_item()`, `get_media_item_by_path()`, return a Python file-like stream object. All options accepted by `io.open()` are also supported. streams, meaning that random access is not supported.

Send feedback

You can reference single media items as follows:

```
1 from transforms.api import transform
2 from transforms.mediasets import MediaSetInput, MediaSetOutput
3 @transform(
4     images=MediaSetInput('/examples/images'),
5     output_images=MediaSetOutput('/examples/output_images')
6 )
7 def translate_images(images, output_images):
8     image1 = images.get_media_item_by_path("image1") # using path
9     image2 = images.get_media_item("ri.mio.main.media-item.123") # using R
```

However, you will likely want to list all media items in your media set by putting them into a dataframe. You must list the media items with

```
list_media_items_by_path_with_media_reference():
```

```
1 from transforms.api import transform, Output
2 from transforms.mediasets import MediaSetInput
3 @transform(
4     images=MediaSetInput('/examples/images'),
5     listing_output=Output('/examples/listed_images')
6 )
7 def translate_images(ctx, images, listing_output):
8     media_items_listing = images.list_media_items_by_path_with_media_refe
9
10     # You can perform any PySpark function with the listed output
```

The listing will have the following schema:

mediaItemRid	path	mediaReference
ri.mio.main.media-item.1	item1.jpg	{{reference1}}
ri.mio.main.media-item.2	item2.jpg	{{reference2}}
ri.mio.main.media-item.3	item3.jpg	{{reference3}}

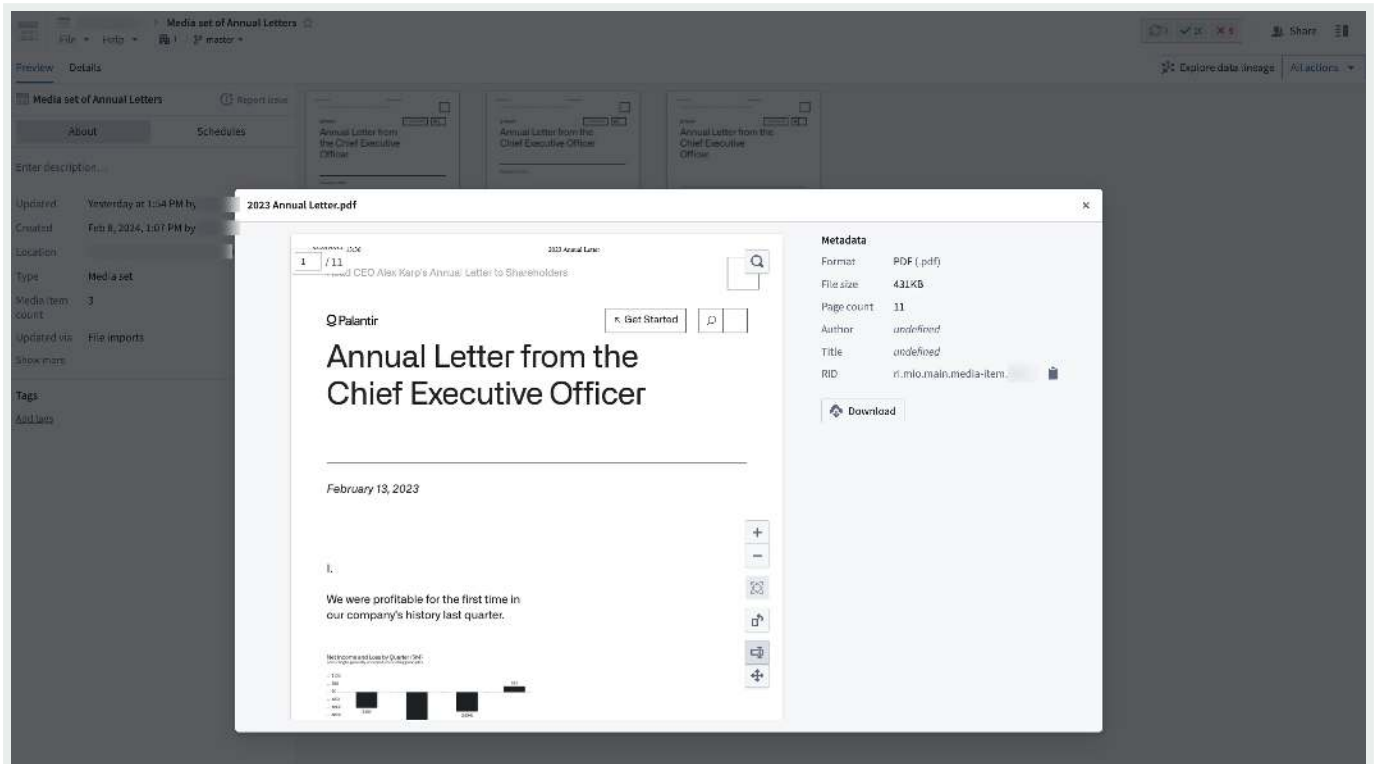
≡

Send feedback

Example: Turn a PDF media set into a new PNG media set

For this example, we will loop through all the media items in our PDF media set and save them as PNG images.

Input example:



Notice that `list_media_items_by_path_with_media_reference()` returns a dataframe, which allows you to perform PySpark functions on the output. For this example, we are collecting the `mediaItemRid` column values to loop through as an array:

```
1 from transforms.api import transform
2 from transforms.mediasets import MediaSetInput, MediaSetOutput
3
4
5 @transform(
6     pdfs=MediaSetInput('INPUT_MEDIA_SET_PATH'),
7     output_pngs=MediaSetOutput('OUTPUT_MEDIA_SET_PATH')
8 )
```

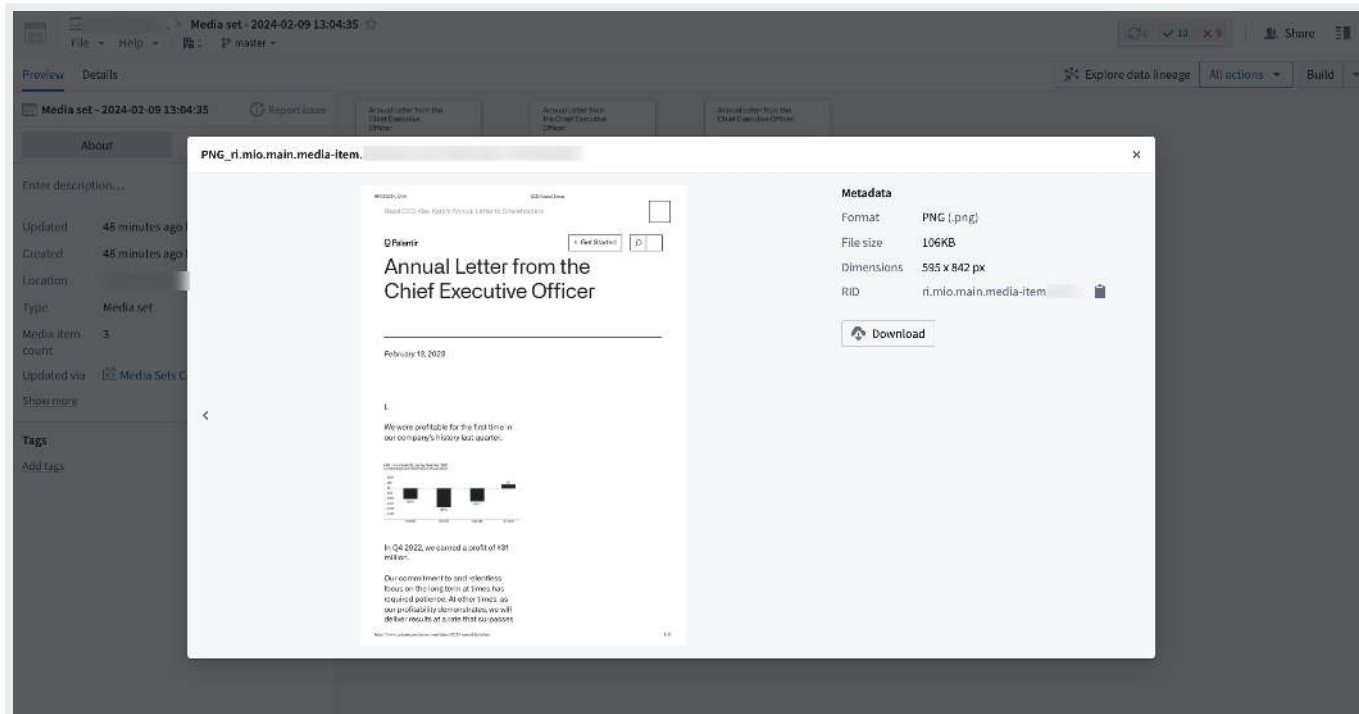
API Reference ↗

```
    images(ctx, pdfs, output_images):
11     items_listing = pdfs.list_media_items_by_path_with_media_refer
12     rid_list = media_items_listing.rdd.map(lambda x: x
13     for rid in rid_list:
```

Send feedback

```
14 output_png = pdfs.transform_document_to_png(rid, 0)
15 output_images.put_media_item(output_png, "PNG_"+rid)
```

This will produce an output media set with PNG files (as noted from the metadata property):



Review a list of available out-of-the-box transformations in our [documentation](#).

4. Test your changes

- i** In Foundry, media sets can be branched similarly to code repositories. Branching is useful for testing the design of multi-step data pipelines. For instance, you can test changes to parts of data pipelines in isolation without breaking downstream dependencies for users who do not rely on your branch.

After writing your data transformation code, you should test the changes you made. Testing ensures your code is working as expected before merging changes.

[API Reference ↗](#)

5. Commit your changes

[Send feedback](#)

Now, you can commit your changes in Code Repositories to label the work you performed. Before you make a commit, your work is auto-saved by default. A commit specifically labels your set of changes when you reach a stopping point.

i Select the **Commit** button to commit your changes and run automatic checks on your code. Choosing the **Build** button also commits your changes, runs automatic code checks, and starts building your output dataset. To quickly test your changes without building your dataset to ensure your code passes the code checks, select **Commit**. Otherwise, you can skip ahead to [merge and build your changes](#).

6. Merge and build your changes

Once you are satisfied with the new media set output, work with any collaborators to merge, build, and view the changes.

PREVIOUS

← [Create a dataset batch pipeline with Code Repositories](#)

NEXT

[Create an incremental pipeline with Pipeline Builder](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Building pipelines / Getting started / Create an incremental pipeline with Pipeline Builder

Create an incremental pipeline with Pipeline Builder

In this tutorial, we will use Pipeline Builder to create a simple incremental pipeline with an output of a single dataset.

The datasets used below are hypothetical examples to illustrate how incremental computation would be applicable.

i For incremental pipelines, you have the option to [force incremental behavior for outputs](#) and fail the build if it cannot run incrementally.

Part 1: Problem statement

Suppose we have an input dataset of `flights` that appends new data every week. We want to filter down to only the flights departing `JFK` airport, then append only those flights to the output `filtered_flights`.

Let's say that the `flights` dataset is 20 million rows, but only 1 million rows are added each week. With incremental computation, the pipeline only needs to consider the latest unprocessed transactions in `flights` instead of all rows as in snapshot computation.

API Reference ↗



When a pipeline runs regularly, incremental processing can significantly reduce the data of each run, saving time and resources.

Send feedback

Now, let's walk through how to set up an incremental pipeline.

Part 2: Validate incremental requirements

 A pipeline will only run with incremental computation if all the considerations in this section are satisfied. For example, your input must update through `APPEND` or `UPDATE` transactions that do not modify existing files. Otherwise, marking your input as incremental will have no effect.

First, check that all incremental constraints are satisfied:

- The input `flights` is updating through `APPEND` transactions or `UPDATE` transactions that do not modify existing files.
- The logic for computing `filtered_flights` from `flights` does not require changing any previously written data in `filtered_flights` during later builds.
 - If you wish to change your pipeline logic (for example, to also include flights departing `LGA` airport), you can update the pipeline. If you want to apply that logic to previously-processed flights, you may want to [replay](#) your pipeline.
- If the pipeline includes window functions, aggregations, or pivots, ensure that these are meant to operate on the current transaction only.

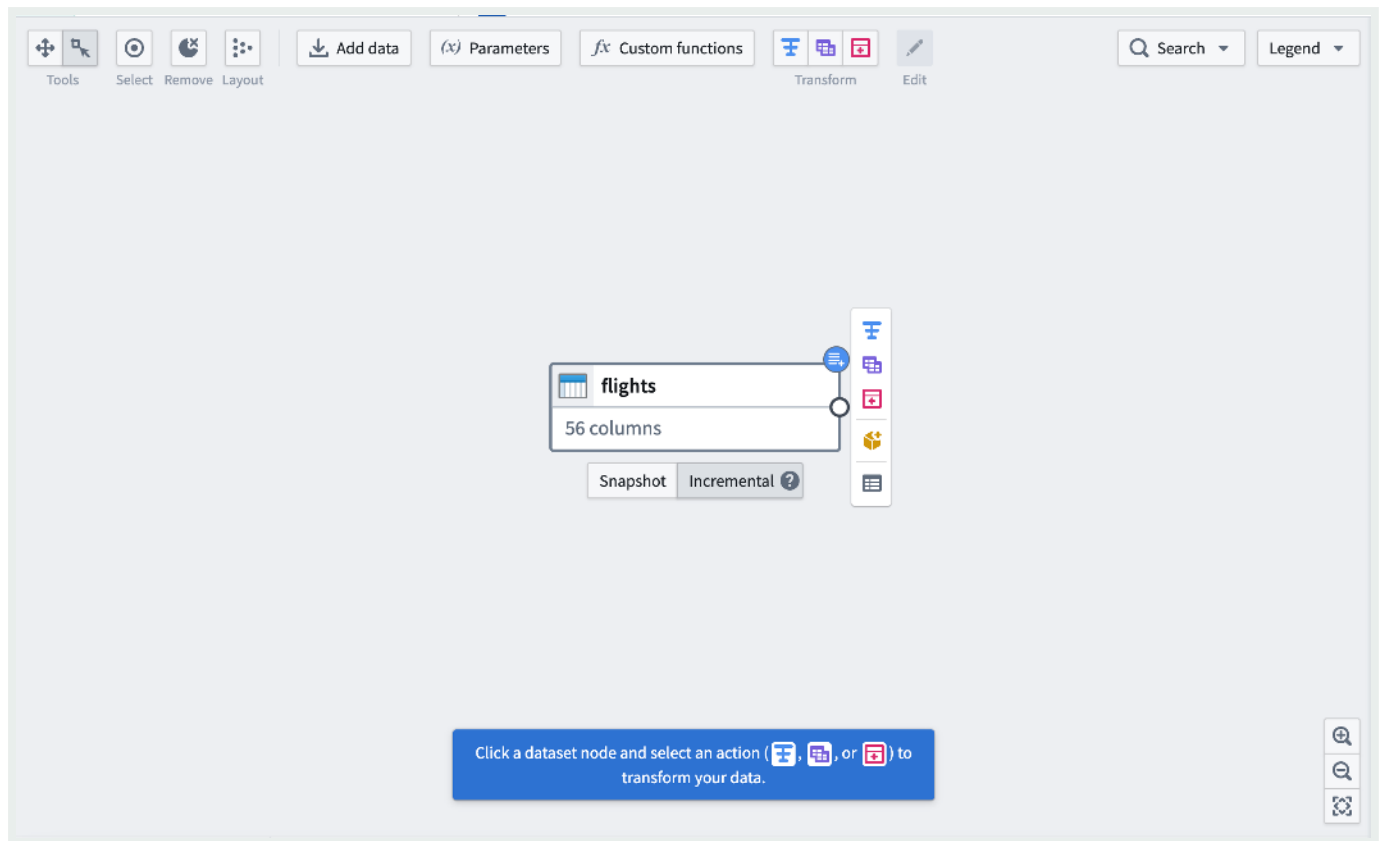
For a full list of considerations, reference these important [restrictions](#) for incremental computation in Pipeline Builder.

Part 3: Create your pipeline

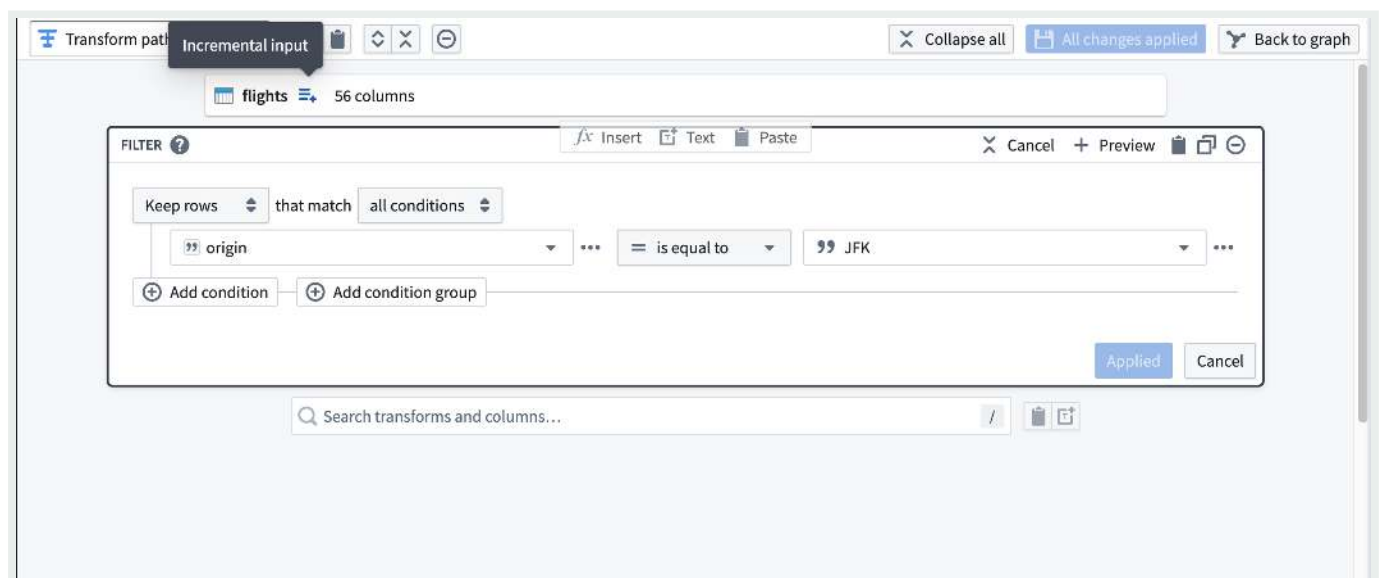
Now, we can initialize a new pipeline (for a step-by-step walkthrough, reference [creating a batch pipeline in Pipeline Builder](#)). Assume that we have imported `flights` as an input dataset.

First, mark your input dataset as **Incremental** using the buttons below the dataset. You will see [API Reference ↗](#) appear in the top right corner to indicate the change.

Send feedback



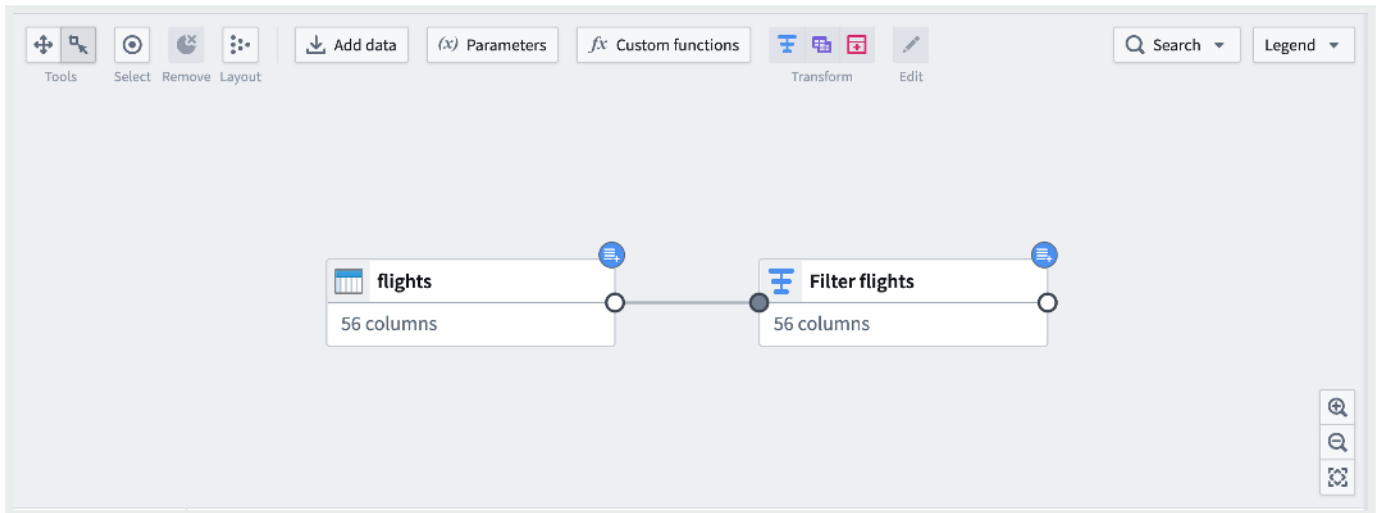
Next, add a transform to filter `flights` to those departing `JFK` airport. Notice the icon to the right of the dataset input labeled with the tooltip **Incremental input**. Downstream transformations will have this icon to indicate that they are being processed incrementally.



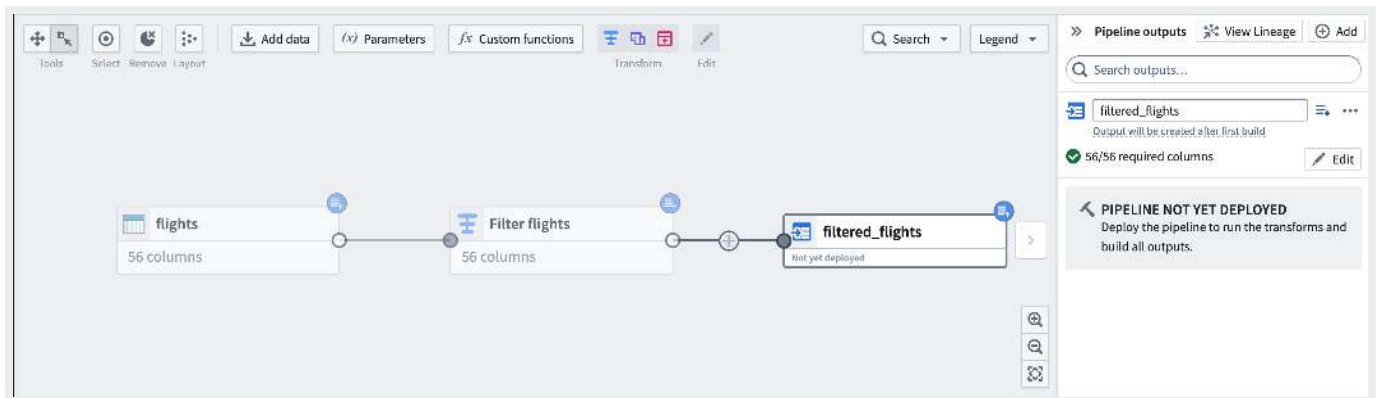
On the graph, downstream nodes will be marked with the same blue badge as the input.

[API Reference ↗](#)

[Send feedback](#)



Finally, add an output dataset `filtered_flights`.

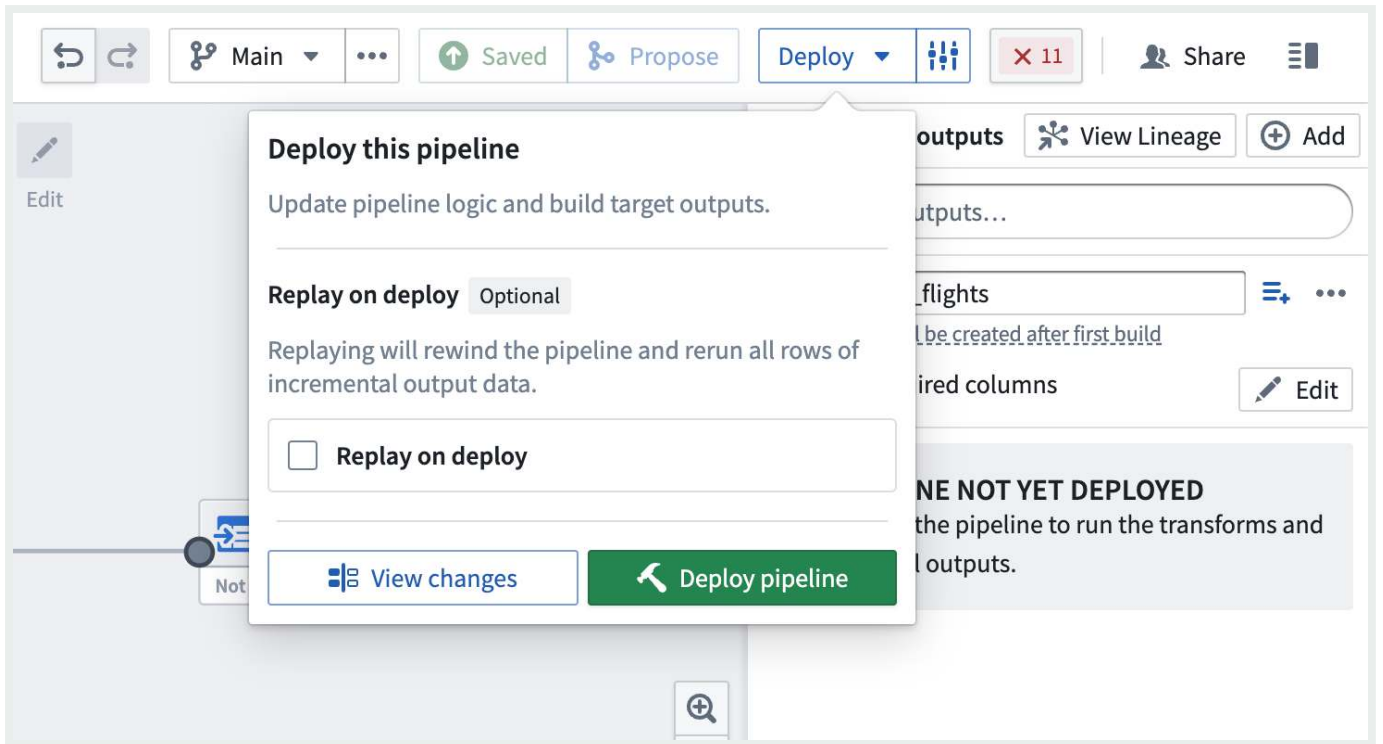


Part 4: Deploy output dataset

You are now ready to [deploy your pipeline](#).

[API Reference ↗](#)

[Send feedback](#)



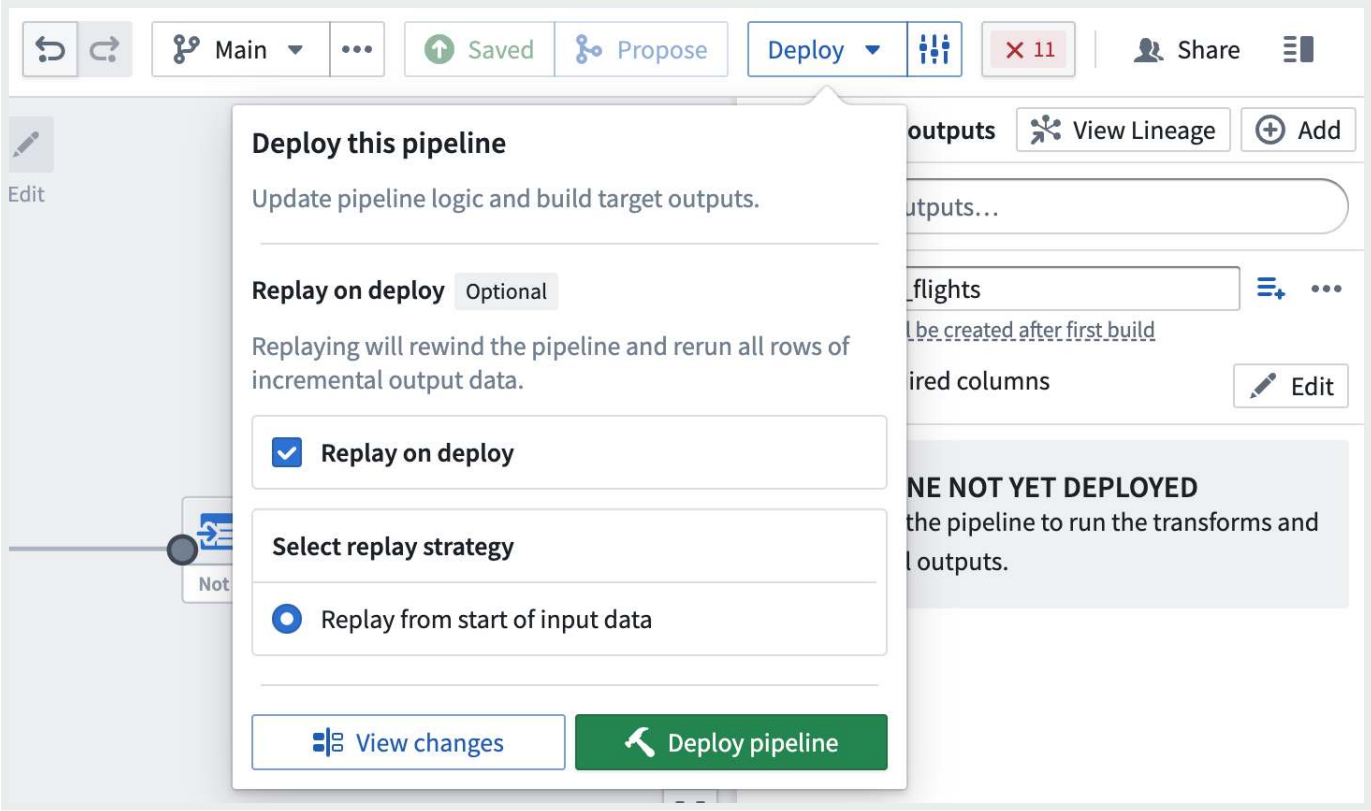
Replay on deploy

Sometimes, it may be necessary to reprocess previous input transactions (for example, if the logic changed and the previous version of your output data is now outdated). In these instances, you can select **Replay on deploy** to run the entire input through the pipeline logic. After replaying, your pipeline should continue with incremental computation as new append transactions are added to the input.

⚠ Replaying on deploy will produce a `SNAPSHOT` transaction on the output dataset.

[API Reference ↗](#)

[Send feedback](#)



Data connectivity & integration / Building pipelines / Getting started / Create a streaming pipeline with Pipeline Builder

Create a streaming pipeline with Pipeline Builder

In this tutorial, we will use Foundry Streaming and Pipeline Builder to create a simple pipeline with an output of a single dataset with information on sensor temperatures. You will learn how to create a stream in Foundry, push records into that stream, and transform them in Pipeline Builder.

Part 1. Initial setup

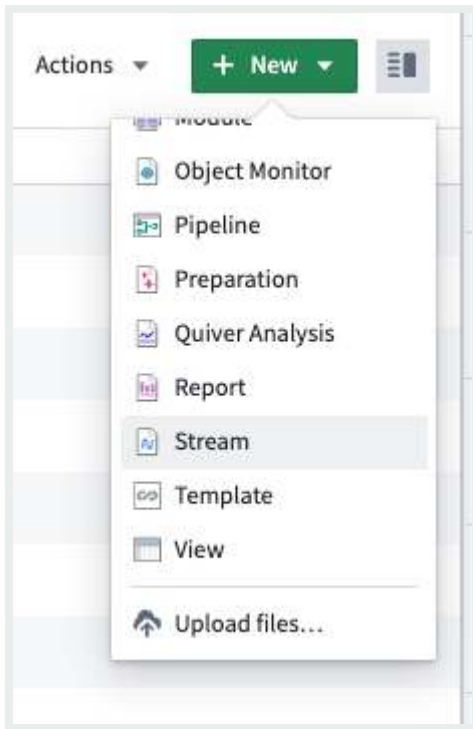
First, we need to create a new stream.

1. When logged into Foundry, navigate to a [Project](#) in Foundry, select **+ New** in the top right corner, then select **Stream**.

API Reference ↗



Send feedback



2. Next, you will need to define your stream. For this guide, we will create a simple one partition stream and manually push records to it.

On the **Define** page, select **Normal** for the throughput and define a basic schema as: **sensor_id:** String, **temperature:** Double.

Temperature Stream

Define | Connect | Push

Throughput | Partitions: 1

Normal
Suitable for up to 5MB/s of throughput

- Row order guaranteed on arrival
- Lowest compute cost
- Suitable for up to 5MB/s of throughput

Recommended

Very high
Best for data over 5MB/s of throughput

- Order not guaranteed
- Higher compute cost
- Best for data over 5MB/s of throughput

Schema | Generate from JSON sample...

COLUMNS

Name	Type	Nullable?
sensor_id	99 String	☐
temperature	123 Double	☐
Add a column...	99 String	☐

SCHEMA JSON

```
{
  "type": "string",
  "nullable": false,
  "name": "sensor_id",
  "customMetadata": {}
},
{
  "type": "DOUBLE",
  "nullable": false,
  "name": "temperature",
  "customMetadata": {}
}
```

Keys | Optional

Optionally select a set of key columns in order to guarantee ordering between unique IDs. Rows will not be guaranteed to be stored in order unless the value of their keys are the same or the stream is configured with a single partition only.

API Reference ➔

Send feedback

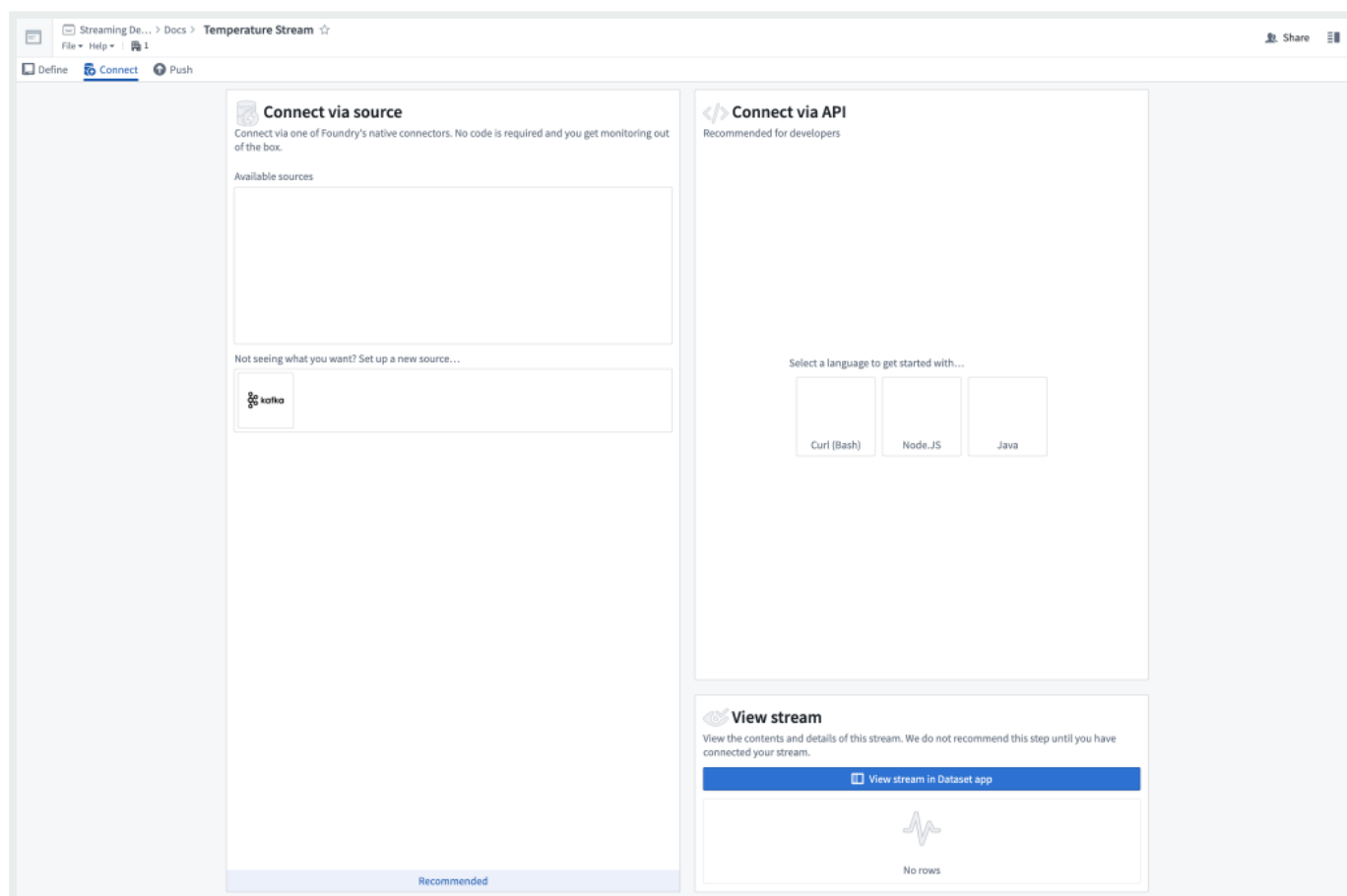
✓ Partitions: 1 | ✓ 2 columns | ✓ 0 keys

3. Select **Create stream**. This will take you to the **Connect** page where you can specify how to connect to the streaming data.

Part 2. Push records into the stream

We are now ready to connect our stream. At this point, we could set up a streaming data ingestion task with a [source](#). For this tutorial, we will instead manually push records to the stream with **Curl**.

1. First, select **Curl (Bash)** under the **Connect via API** section to set up authentication for your stream. We will use a personal token to submit records.

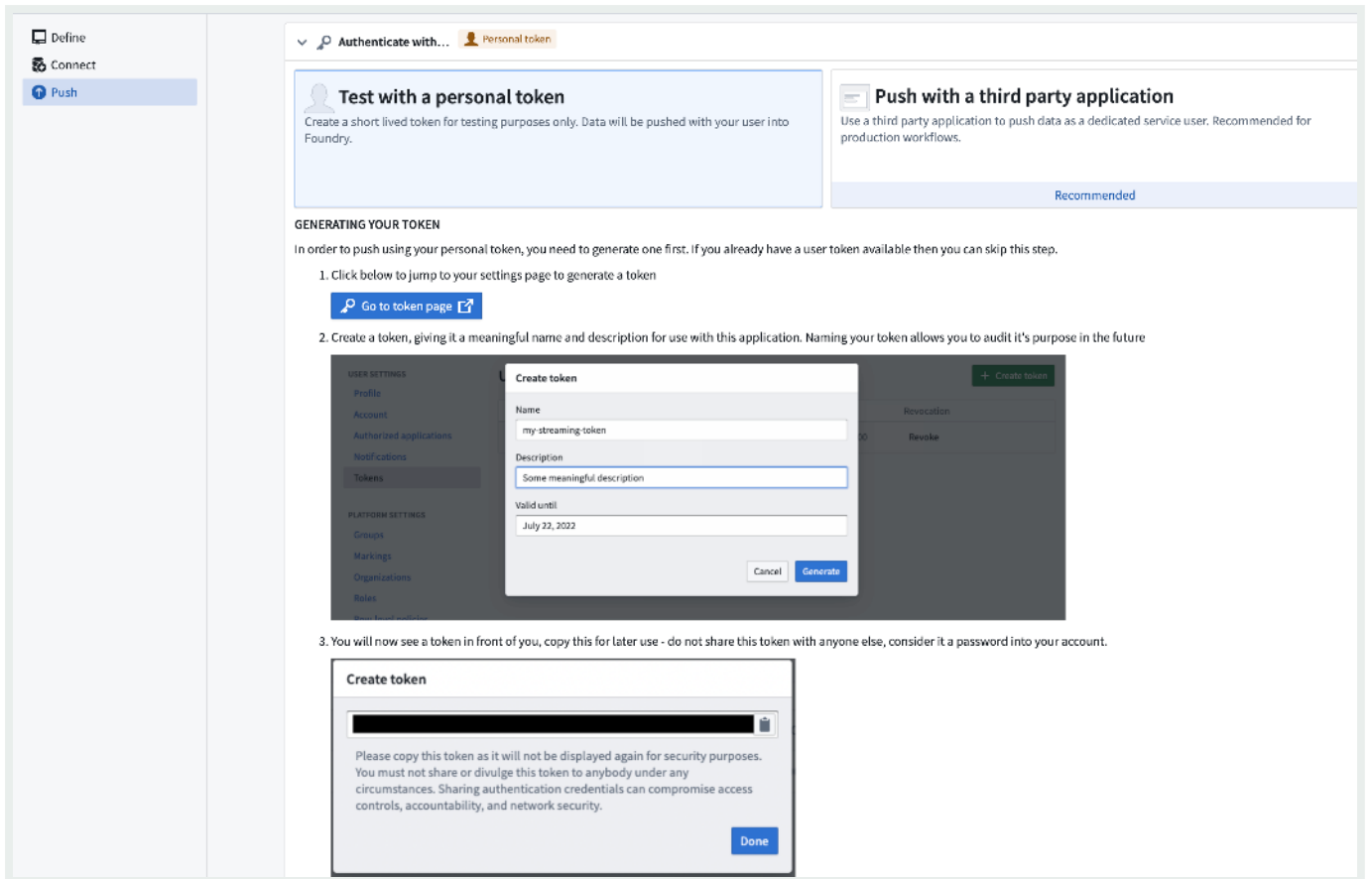


2. Select **Test with a personal token** and follow the on-screen prompts for generating a short-lived personal token.

API Reference ↗

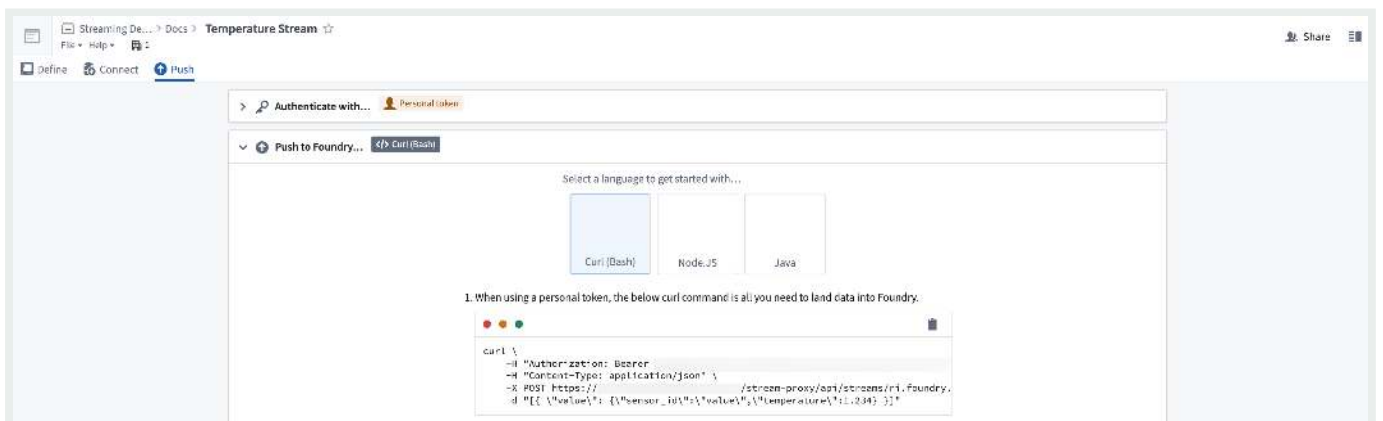
Personal tokens should not be used for production pipelines. Production pipelines should use an [OAuth token workflow](#).

Send feedback



3. Paste your generated token into the text box, then click **Next Step**.

4. Copy the **Curl** command. Open a terminal on your computer that can execute Bash and paste the command. Run the command in your terminal.



Within seconds, you will see a record appear in the stream viewer on the page:

API Reference ↗

Send feedback

 Start pipelining...		 View this dataset...	
	sensor_id String	temperature Double	
1	sensor3		3.142
2	sensor2		2.132
3	sensor1		1.234

We have now ingested streaming data in real time. Let's transform that data now.

Part 3. Transform a stream

1. Select the **Start pipelining** button to begin writing a basic streaming transform in Pipeline Builder.

2. If the request has been successful, your new rows will appear below.			
 Start pipelining...		 View this dataset...	
	sesnor_id String	temperature Double	

2. In the **Create new pipeline** modal, select the **Streaming pipeline** type, and click **Create Pipeline**.

[API Reference ↗](#)

[Send feedback](#)

Create new pipeline

Pipeline name and location

 New pipeline (Thu, Nov 3, 2022, 5:28:10 PM)
/Palantir/Streaming Demos/Docs

 Edit location

Initial pipeline datasets

1 initial dataset will be added to the pipeline

Pipeline type

☐ **Batch pipeline**
Builds and transforms entire datasets on each deploy. Use for data that is ingested periodically.

☒ **Streaming pipeline**
Transforms data continuously as new data is made available. Use for data that is ingested at a high frequency.

Create pipeline

This will create a pipeline for the input stream, displayed on a graph.

Selecting the input stream node will display a preview of the data. Note that the preview runs on a cold storage view of the stream; records from the stream will be delayed before they appear.

[API Reference ↗](#)

[Send feedback](#)

The screenshot shows the Palantir Pipeline Builder interface. At the top, there's a navigation bar with 'File', 'Settings', and 'Help' menus. Below it, a toolbar includes 'Add data', 'Parameters', 'Transform', and 'Edit' buttons. The main canvas displays a 'Temperature Stream' node with 2 columns. A blue tooltip提示 says 'Click a dataset node and select an action (Add, Transform, or Edit) to transform your data.' The bottom panel shows a 'Selection preview' for the 'Temperature Stream' node, displaying a table with 3 rows and 2 columns: 'sensor_id' (String) and 'temperature' (Double). The table data is as follows:

	sensor_id	temperature
1	sensor1	1.234
2	sensor2	2.132
3	sensor3	3.142

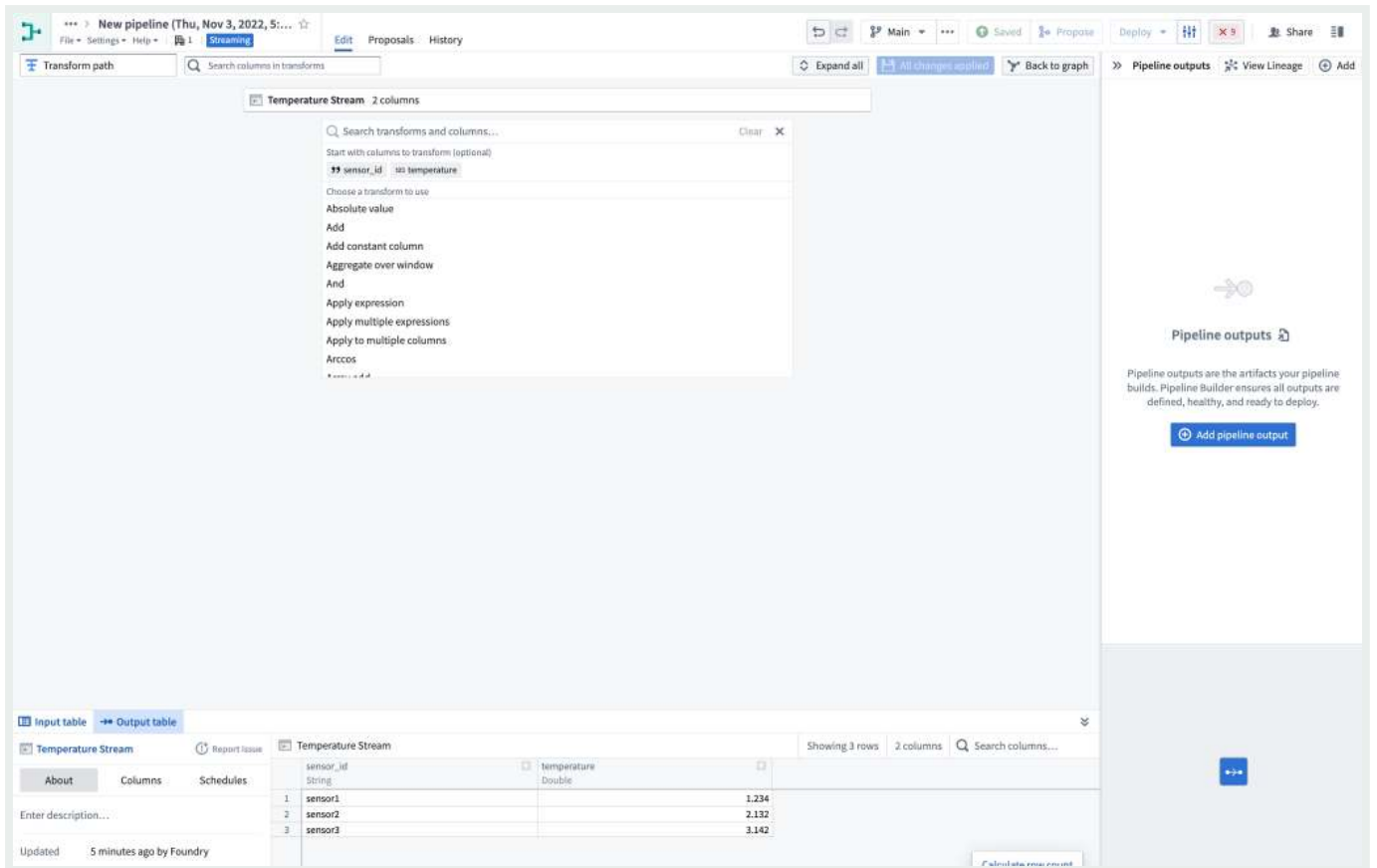
The right sidebar shows 'Pipeline outputs' information and an 'Add pipeline output' button.

3. Click on the input stream node on the graph and select the **Transform** action (the blue **T** icon next to the input node).

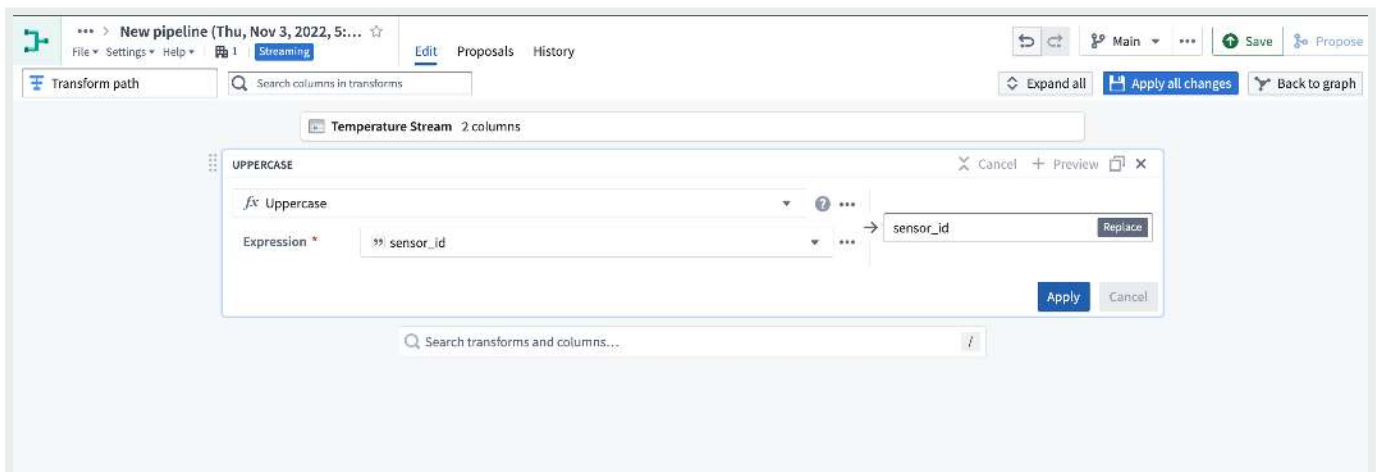
This will open a list of all transforms currently supported for streams based on the input types of the columns in the stream. For this tutorial, we will convert all `sensor_ids` to uppercase, remove any whitespace on them, and filter by temperatures exceeding three degrees.

[API Reference ↗](#)

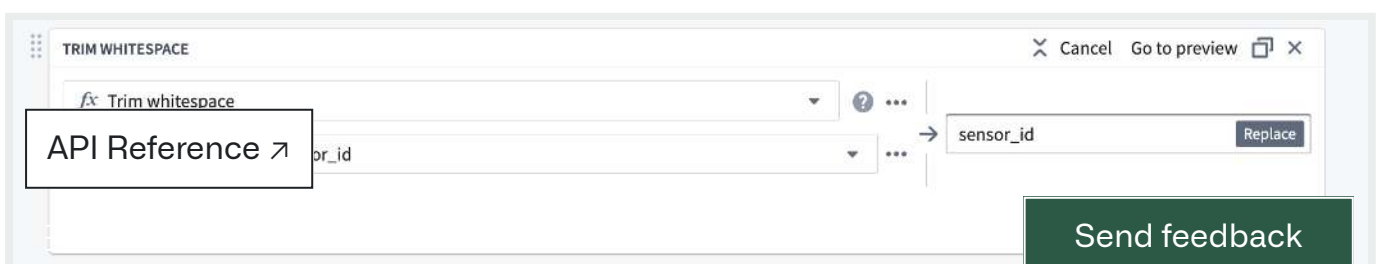
[Send feedback](#)



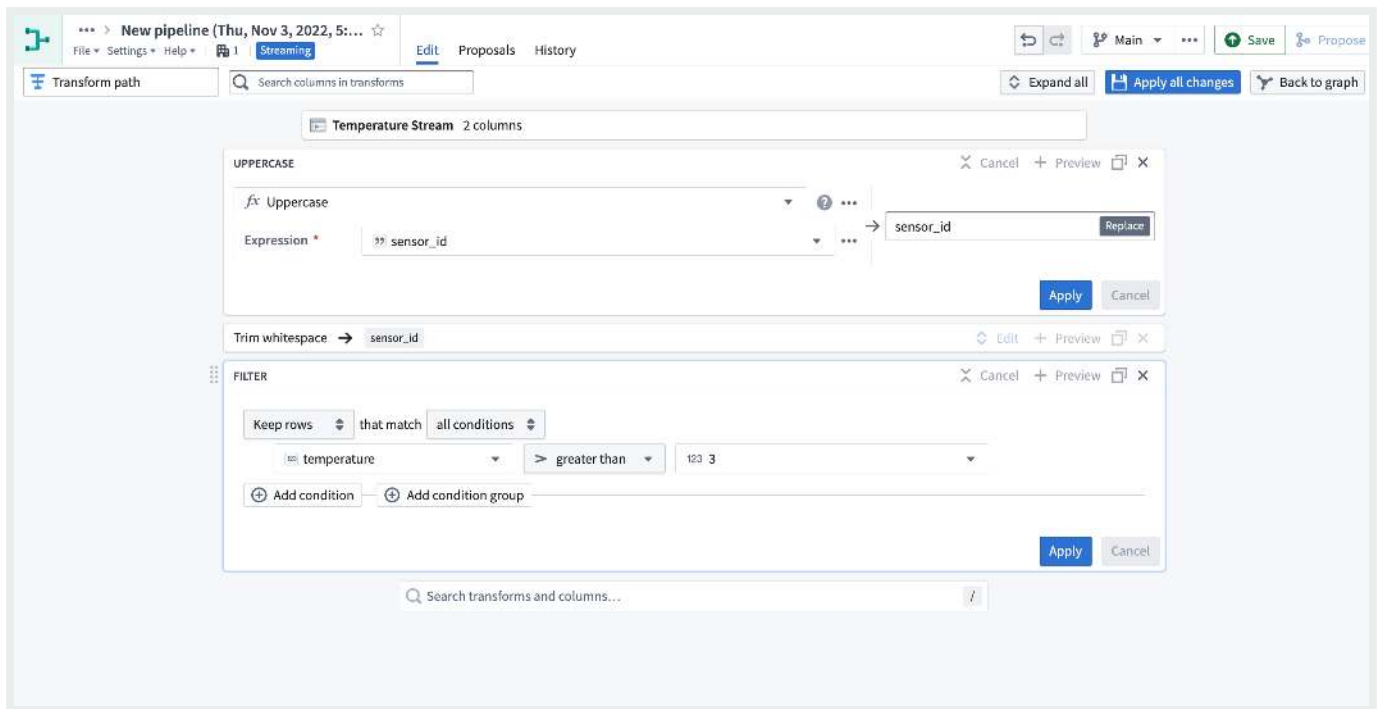
4. Select the **Uppercase** transform, choose the `sensor_id` column, and click **Apply**.



5. Then, search for the **Trim whitespace** transform and select it. Choose the `sensor_id` column again, and click **Apply**.



6. For the final transform, first search for the **Filter** transform and choose **Keep rows**. Then, select the `temperature` column, set the filter to **greater than 3**, and select **Apply**.



7. Click **Apply all changes** to the top right of your screen. Then, select **Back to graph** to return to your pipeline.

[API Reference ↗](#)

[Send feedback](#)

*** > New pipeline (Thu, Nov 3, 2022, 5:...) ☆

File Settings Help | 1 Streaming Edit Proposals History

Tools Select Remove Layout Add data Parameters Transform Edit Search Legend

Temperature Stream 2 columns

Transform path 2 columns

Selection preview Transformations

Transform path

Search 2 columns...

sensor_id String temperature Double

1 SENSOR3 3.142

8. Select the **Transform path** node we just created, then click **New dataset**.

*** > New pipeline (Thu, Nov 3, 2022, 5:...) ☆

File Settings Help | 1 Streaming Edit Proposals History

Tools Select Remove Layout Add data Parameters Transform Edit Search Legend

Temperature Stream 2 columns

Transform path 2 columns

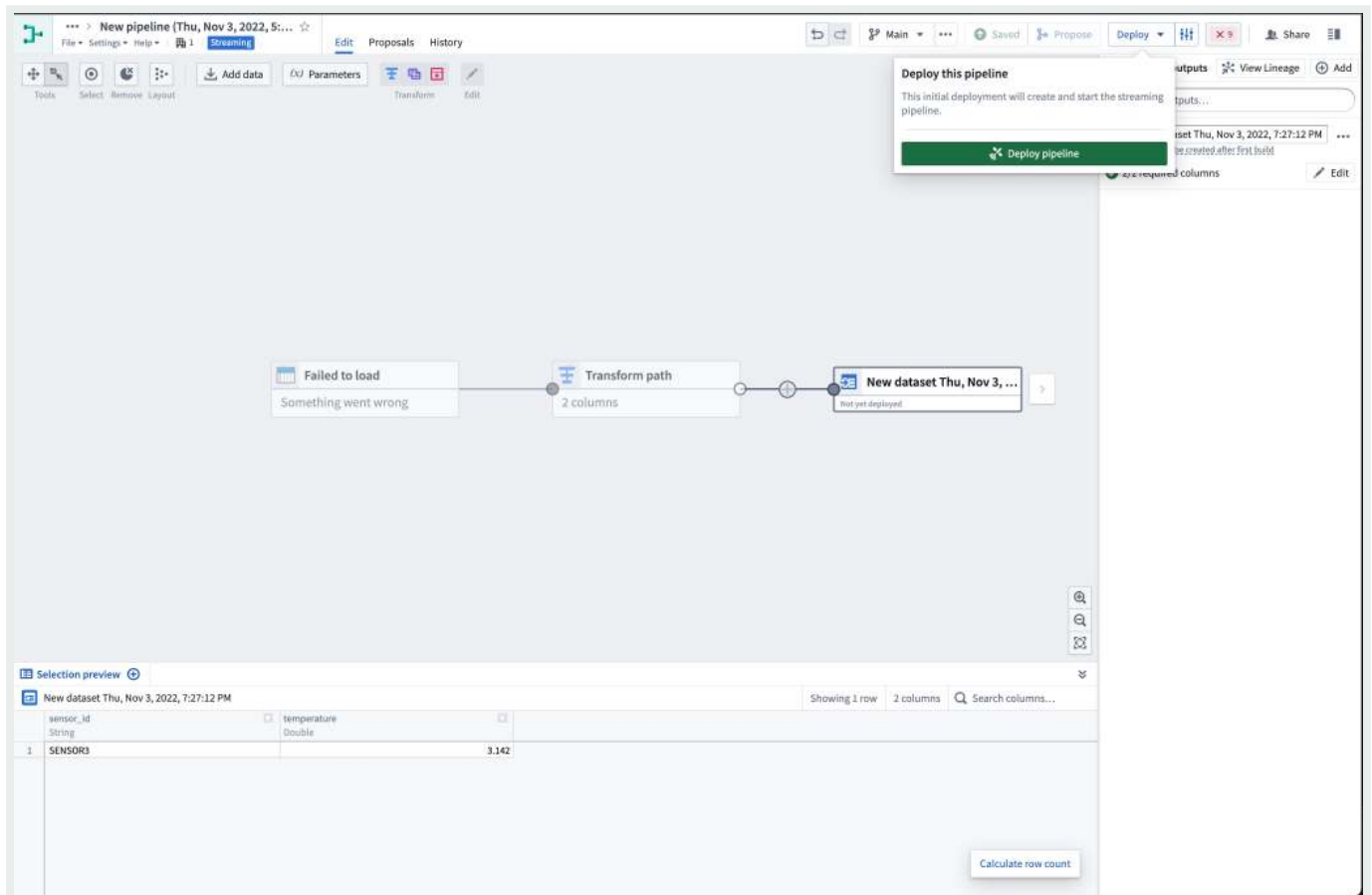
Transform Join Union New dataset Edit

API Reference ↗

Send feedback

9. In the top right corner of the application, first click **Save** to apply all new changes to your pipeline. Then, click **Deploy** and **Deploy pipeline**.

! If you save your changes without deploying them, your pipeline logic will **not** update to the latest changes. You must deploy the pipeline to capture changes to transform logic.



10. Select the output stream node you just created, then click on the stream name above the **Data preview** section at the bottom of your graph.

[API Reference ↗](#)

[Send feedback](#)

The screenshot shows a data pipeline with three components: 'Temperature Stream' (2 columns), 'Transform path' (2 columns), and 'New dataset Thu, Nov 3, ...' (Deployed). Below the pipeline is a 'Selection preview' table with the following data:

	sensor_id String	temperature Double
1	SENSOR3	3.142

Buttons for 'Calculate row count' and a URL are visible at the bottom.

This will take you to the stream preview page with the output stream from your transform.

i The streaming cluster takes about one minute to start, so you may not see records immediately. Once running, however, the cluster will process all new records in real time.

The screenshot shows the 'New dataset Thu, Nov 3, 2022, 7:27:12 PM' stream preview page. The left sidebar contains metadata:

- About**
 - Location: /Palantir/Streaming Demos/Docs/New data...
 - Type: Streaming dataset (Streaming)
 - Table size: 2 columns
 - Last archived: 2 minutes ago
 - Updated via: New pipeline (Thu, Nov 3, 2022, 5:28:10 PM)
- Stream**
 - Status: Running
 - Up since: 2 minutes ago
- Tags**
 - Add tags
- Health Checks**
 - View details
- Custom metadata**
 - Add custom metadata

The main table shows the same data as the previous screenshot. A 'Send feedback' button is located at the bottom right.

Next steps

Now that you know how to create a simple streaming pipeline, learn more about managing streams by exploring how to [debug a failing stream](#). For more advanced transform functionality, learn more about [Pipeline Builder](#).

PREVIOUS

← [Create an incremental pipeline with Pipeline Builder](#)

NEXT

[Faster pipelines with Pipeline Builder](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Data connectivity & integration / Building pipelines / Getting started / Faster pipelines with Pipeline Builder

Faster pipelines in Pipeline Builder

i Faster pipelines were previously known as *lightweight pipelines*, as the term "lightweight" referred to the reduced time and compute resources required to execute these pipelines as opposed to the size of the data they handle. The name change reflects that faster pipelines reduce both execution time and compute resource usage, even for large-scale datasets.

⚠ If you are unfamiliar with creating pipelines in Pipeline Builder, review the documentation on [how to create a batch pipeline in Pipeline Builder](#) before proceeding.

Pipeline Builder now supports faster pipelines, which can reduce execution times for batch and incremental pipelines. This pipeline uses a backend powered by [DataFusion](#), an open-source query engine written in [Rust](#). Compared to traditional Spark-based pipelines, faster pipelines can substantially accelerate compute processes.

Faster pipelines are specifically engineered to optimize build times and execute low-latency operations efficiently. In particular, pipelines that run in under 15 minutes will benefit most from faster pipeline configuration.

We encourage you to experiment with different pipeline configurations to improve performance. You can explore the capabilities by testing them on a branch or making a copy of [API Reference](#) to compare the new performance with your original configuration.



Create a new faster pipeline

Send feedback

1. Open Pipeline Builder and select **Create new pipeline**.
2. After entering a name for your pipeline and the desired location, choose **Faster pipeline** under **Pipeline type**.
3. Select **Create pipeline**.

Create new pipeline

Get started on transforming your data with a new pipeline

Name and location

New Faster Pipeline

Select location

Create project

No location selected

Configuration

**Batch pipeline**
Builds and transforms entire datasets on each deploy. For data that is ingested periodically.

**Streaming pipeline**
Transforms data continuously as new data is made available. For high frequency ingestion.

Select batch compute

**Standard** Default
Build your pipelines using advanced expressions. Backed by Spark.

**Faster** New
Speed up your pipelines and save compute. Backed by DataFusion. [Learn More](#)

**External** Beta
Build your pipelines using external compute platforms. [Learn More](#)

← Back

Create pipeline

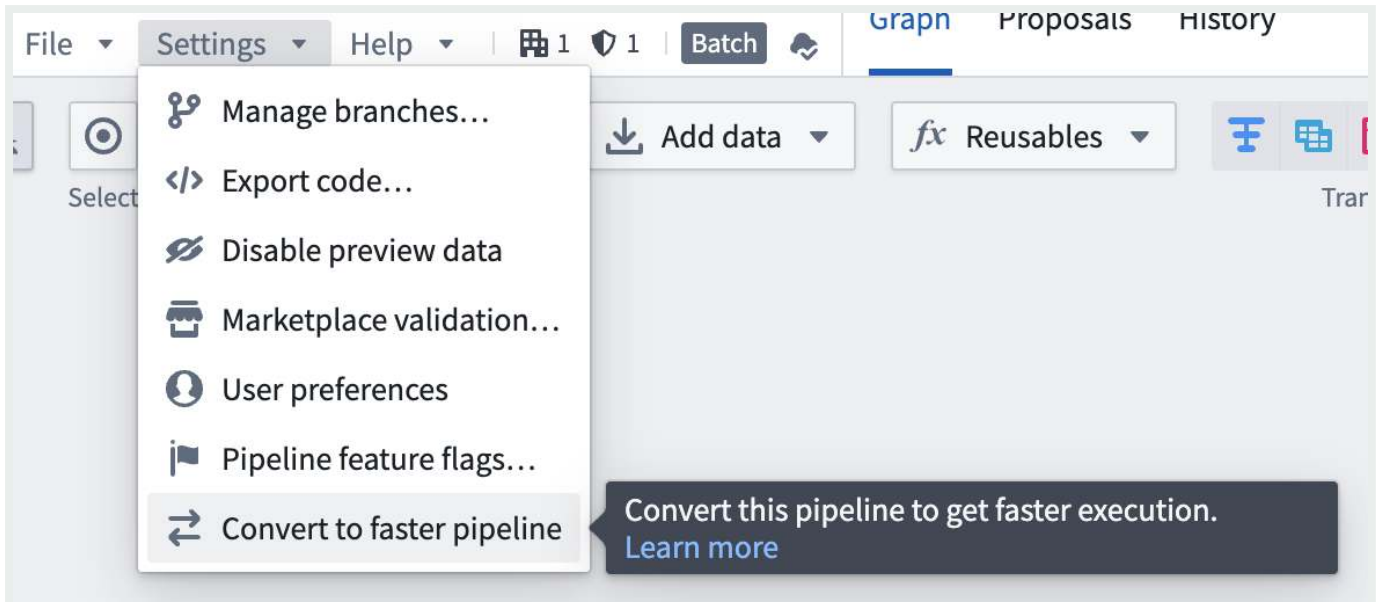
Convert between faster and standard batch pipelines

You can convert between faster and standard batch pipelines, and vice versa, by following the steps below. This conversion can be reversed at any time by repeating the process and selecting the desired options.

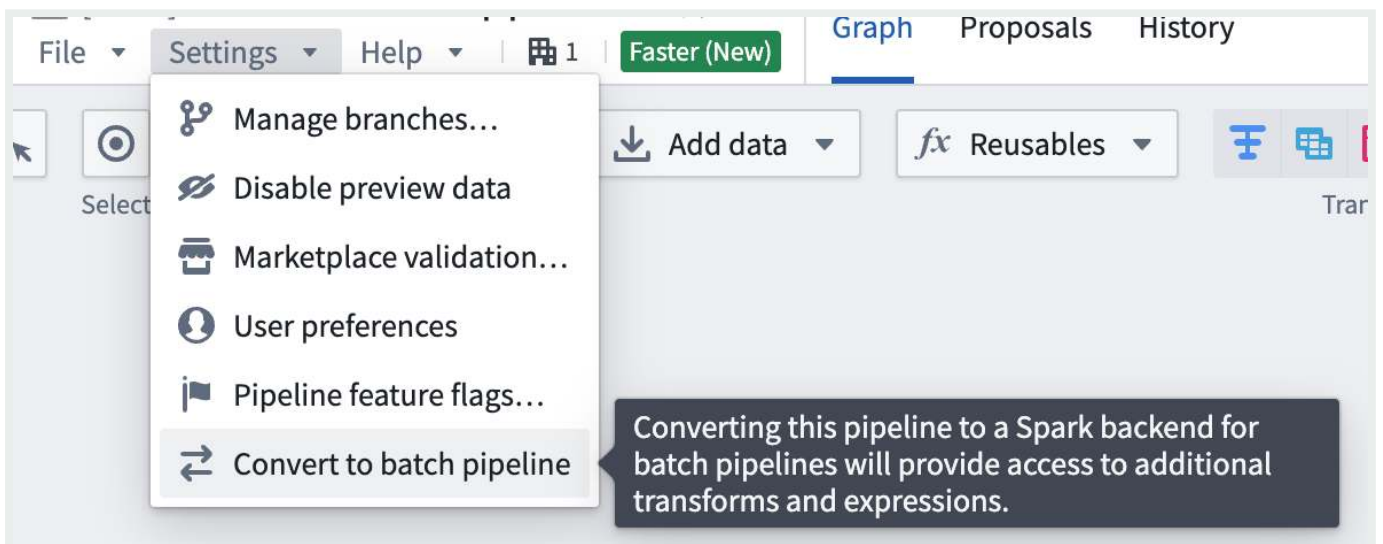
1. To convert a batch pipeline, go to **Settings** and select **Convert to Faster pipeline**.

[API Reference ↗](#)

[Send feedback](#)



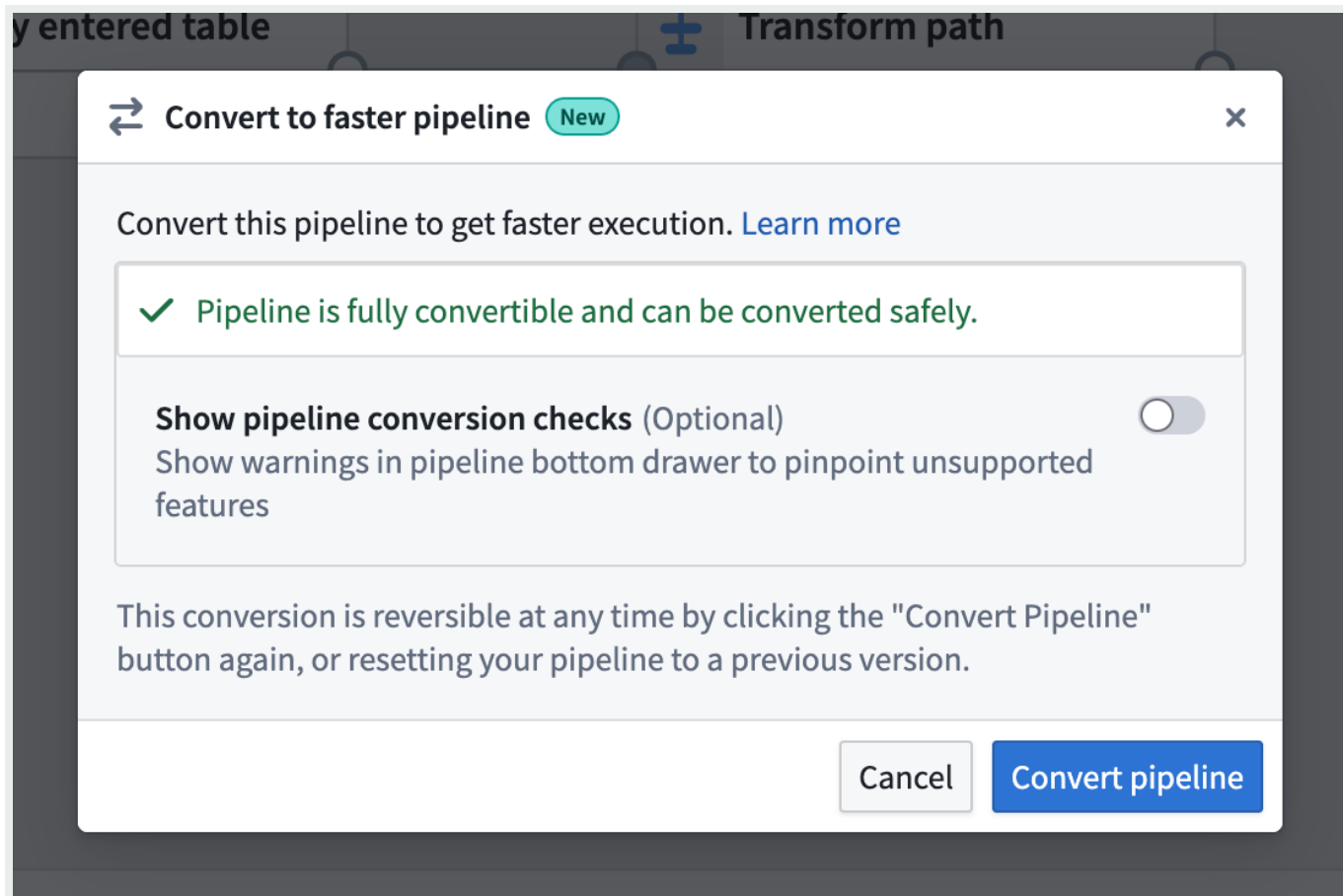
To convert a pipeline back to a batch pipeline, go to **Settings** and select **Convert to Batch pipeline**.



2. If the pipeline is compatible with the new pipeline type, you will see a dialog box where you can confirm the conversion.

[API Reference ↗](#)

[Send feedback](#)



3. If the pipeline is not compatible with the new pipeline type, a warning will appear when you try to convert your pipeline. The warning will list any expressions or transforms that are incompatible with faster pipelines.

 **Convert to faster pipeline** New ×

Convert this pipeline to get faster execution. [Learn more](#)

 **Conversion is not supported in this pipeline**

The following errors must be resolved to convert the pipeline:

- Transform of type "histogramChart" not found

Show pipeline conversion checks (Optional) ☐

Show warnings in pipeline bottom drawer to pinpoint unsupported features

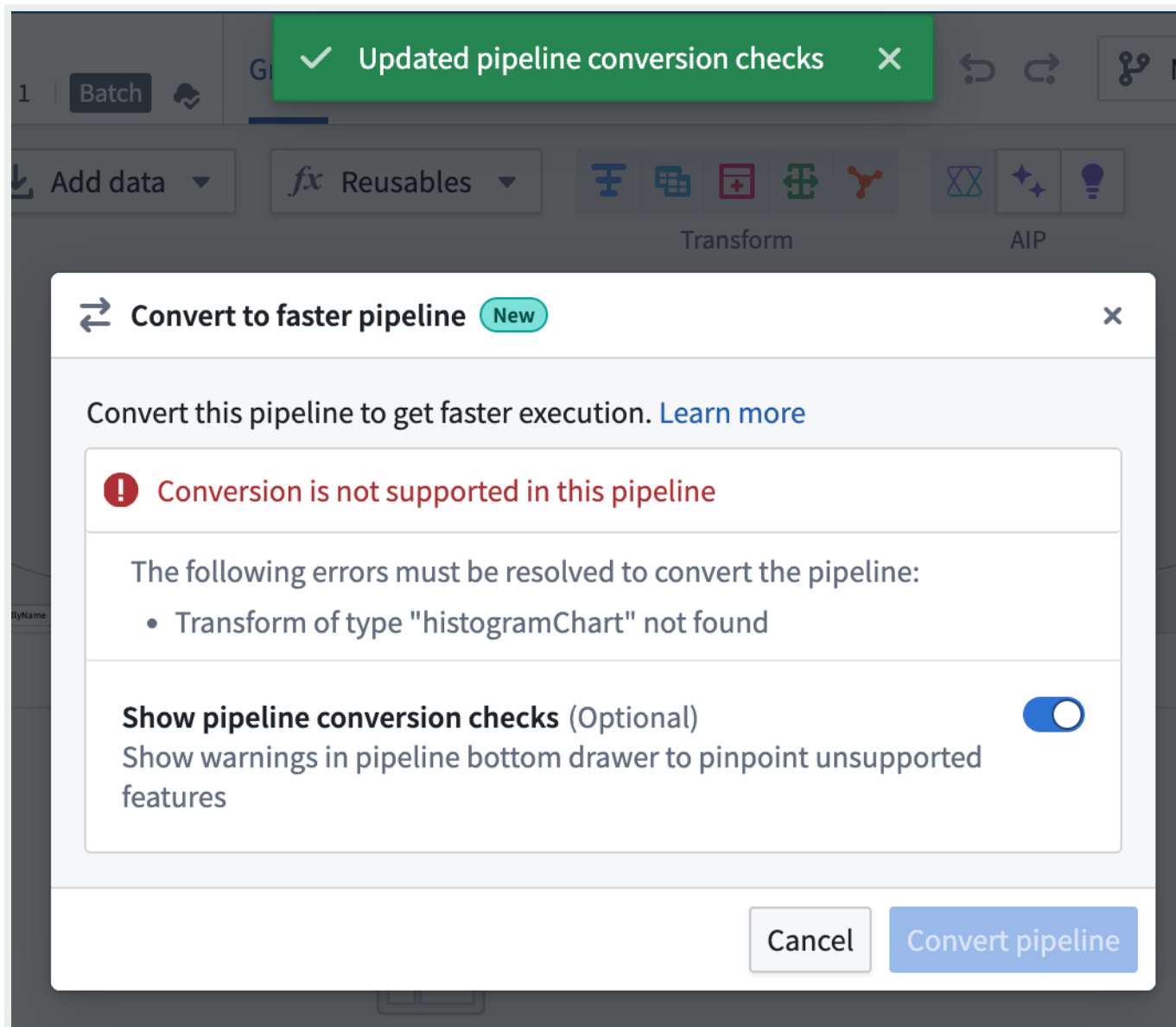
Resolve errors to convert the pipeline

Convert pipeline

You can toggle on **Show pipeline conversion checks** at any point to see anything that's not compatible with the faster pipeline option.

API Reference ↗

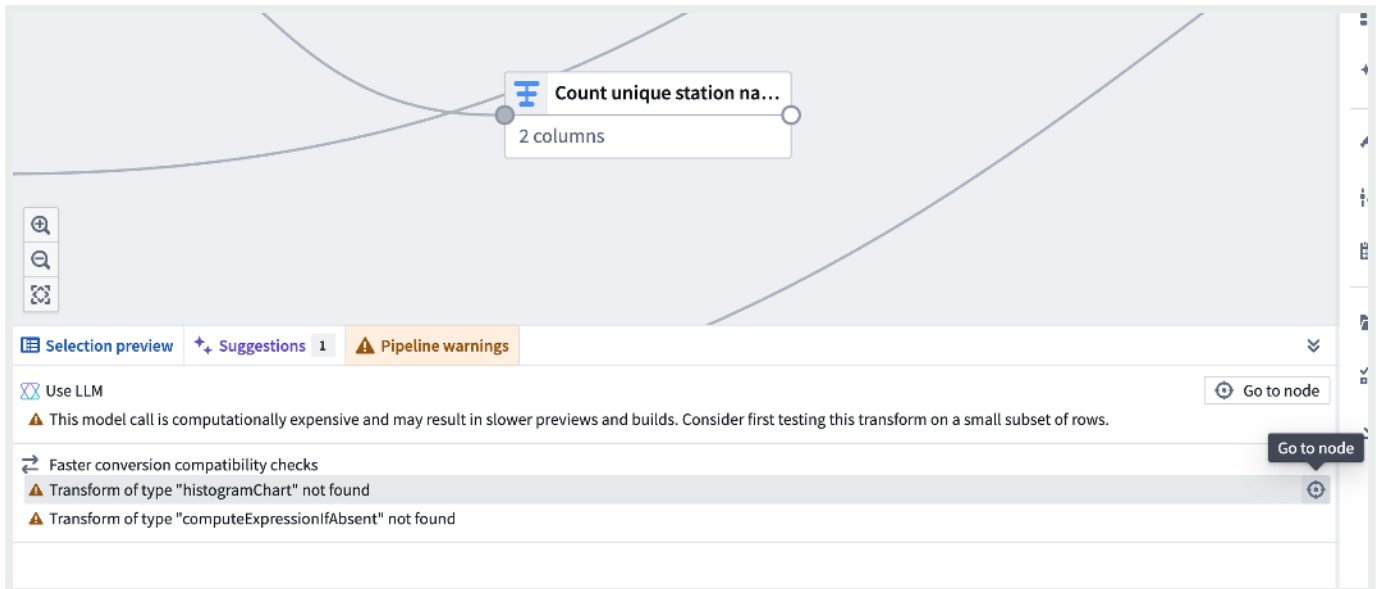
Send feedback



After you toggle on **Show pipeline conversion checks**, a **Faster conversion compatibility checks** section will appear in the **Pipeline warnings** panel at the bottom of the screen. This section lists any transforms and expressions that are not supported with faster pipelines. To quickly locate the node with an unsupported transform, click the **Go to node** icon on the right side of the corresponding row.

[API Reference ↗](#)

[Send feedback](#)



Known limitations

Faster pipelines do not currently support the same set of transforms and expressions as standard batch pipelines. Most notably, unsupported transforms and expressions include LLM features, geospatial functionality, and media set operations.

Due to the differences between faster and batch pipelines, you should always verify results using **Preview** or by examining build outputs.

Most supported expressions in faster pipelines will behave as their batch equivalents.

Known limitations include:

- Floating point results may vary in the last digits.
- Decimal overflow will throw an error instead of outputting a `NULL` value.
- Structs cannot be compared with `<`, `>`, `==`, etc.
- `pow` overflow returns `NULL` instead of `inf`.
- Cast functionality may have differences for complex types, such as structs, arrays, maps, and their conversions into strings. For example, nulls may be rendered differently when these types are converted to strings.
- Limited format support for `TimestampToString`, `DateToString`, `StringToTimestamp`, and `StringToDate`.

[API Reference](#) ⌵ not supported for complex types, such as structs, arrays, and maps.

- Empty outputs will result in 0 files rather than an empty file.
- Stats, other than row count, are not supported on build.
- Creating time series reference values is not supported.

[Send feedback](#)

- Trained model nodes are not supported.

Pipeline type support comparison

i The size recommendations below are intended as a general rule and do not apply to all queries. For the right types of transforms, the faster lightweight engine can process even terabyte-scale inputs on a single node.

If your pipeline does not run into any of the limitations in the section above, we recommend using the faster option first. You can switch to the standard Spark option if your data scale demands it or if there are unsupported transforms and expressions you want to use. You can [convert between the different types of pipelines](#).

Characteristic	Standard	Faster
Optimal (uncompressed) data size	> 50GB	1-50GB
Optimal number of rows [*]	> 200 million	1-200 million
Startup overhead	Significant	Minimal
Memory efficiency	Good	Excellent
Processing speed (small data)	Slow	Excellent
Processing speed (medium data)	Fast	Excellent
Processing speed (large data)	Excellent	Variable
Parallel execution	Distributed	Single-node
Memory spilling	Automatic	Automatic

^{*} The number of rows tolerable to each query engine will vary depending on the schema. These numbers are given as a general guide for common cases.

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Building pipelines / Getting started / External pipelines with Pipeline Builder

External pipelines in Pipeline Builder

i If you're new to Pipeline Builder, review [how to create a batch pipeline in Pipeline Builder](#) before proceeding.

Pipeline Builder now offers **external pipelines**, which push down compute to external compute engines. This functions in a similar manner as [compute pushdown in Python transforms](#), and allows Foundry's pipeline management, data lineage, and security functionality to be used on top of external data warehouse compute.

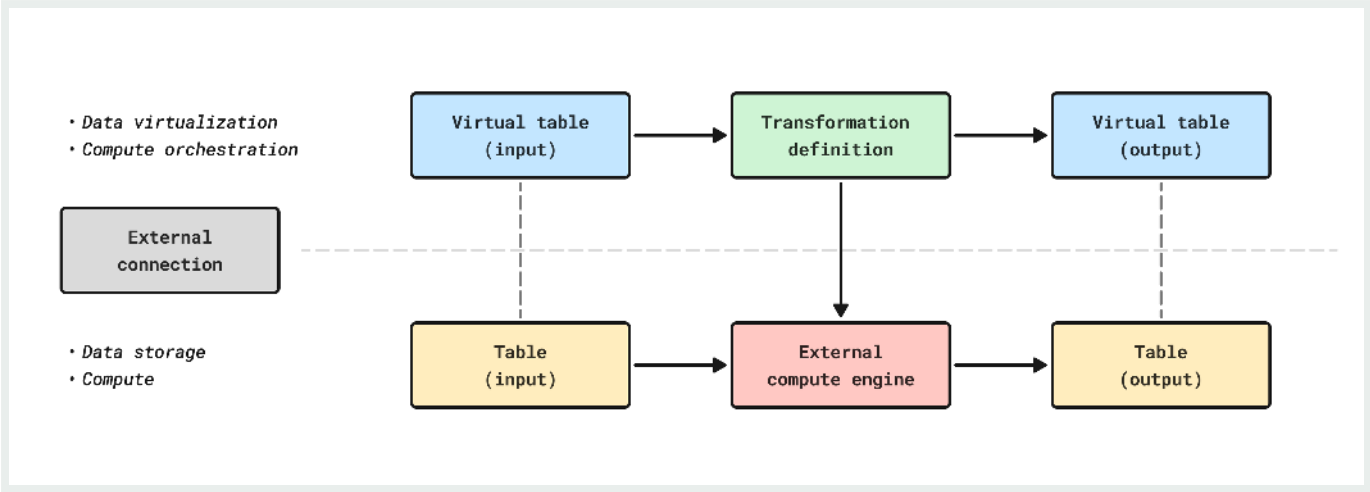
As with compute pushdown in Python transforms, all inputs and outputs from external pipelines must be [virtual tables](#).

Tables built with external compute can be composed together with datasets and tables built with Foundry-native compute using Foundry's scheduling tools, allowing you to orchestrate complex multi-technology pipelines using the exact right compute at every step along the way.

API Reference ↗



Send feedback



Supported external compute engines for Pipeline Builder

Currently, Databricks and Snowflake are supported external compute engines in Pipeline Builder. To use other external compute engines, such as BigQuery, use [transforms with compute pushdown](#).

Source type	Status
BigQuery	Not available
Databricks	Generally available
Snowflake	Generally available

Create a new external pipeline

1. Open Pipeline Builder and select **Create new pipeline**.
2. After entering a name for your pipeline and the desired location, choose **Batch pipeline** > **External** in the configuration settings and select **Next**.
3. Search for and select your [supported external source](#) and import it into the pipeline.
4. Now you can add virtual tables from that source to the graph and create your pipeline as usual.
5. [API Reference ↗](#) its will be [virtual table outputs](#) in the source.
6. When ready to build, save and deploy the pipeline. The pipeline will compute and then output the result as a virtual table with storage.

Send feedback

- i** All input and output tables must be virtual tables from the same source you selected as part of the pipeline setup. For Databricks external pipelines, only Databricks tables are supported. For Snowflake external pipelines, only Snowflake virtual tables from the same external volume are supported. Foundry datasets are not supported as inputs or outputs when using external pipelines.

Create new pipeline

Get started on transforming your data with a new pipeline

Name and location

 Pipeline Builder - 2026-04-17 15:47:20

 Edit location

/Path/folder

Configuration

**Batch pipeline**
Builds and transforms entire datasets on each deploy. For data that is ingested periodically.

**Streaming pipeline**
Transforms data continuously as new data is made available. For high frequency ingestion.

Select batch compute

Compare types

**Standard** Default
Build your pipelines using advanced expressions. Backed by Spark.

**Faster** New
Speed up your pipelines and save compute. Backed by DataFusion. [Learn More](#)

**External**
Build your pipelines using external compute platforms. [Learn More](#)

← Back

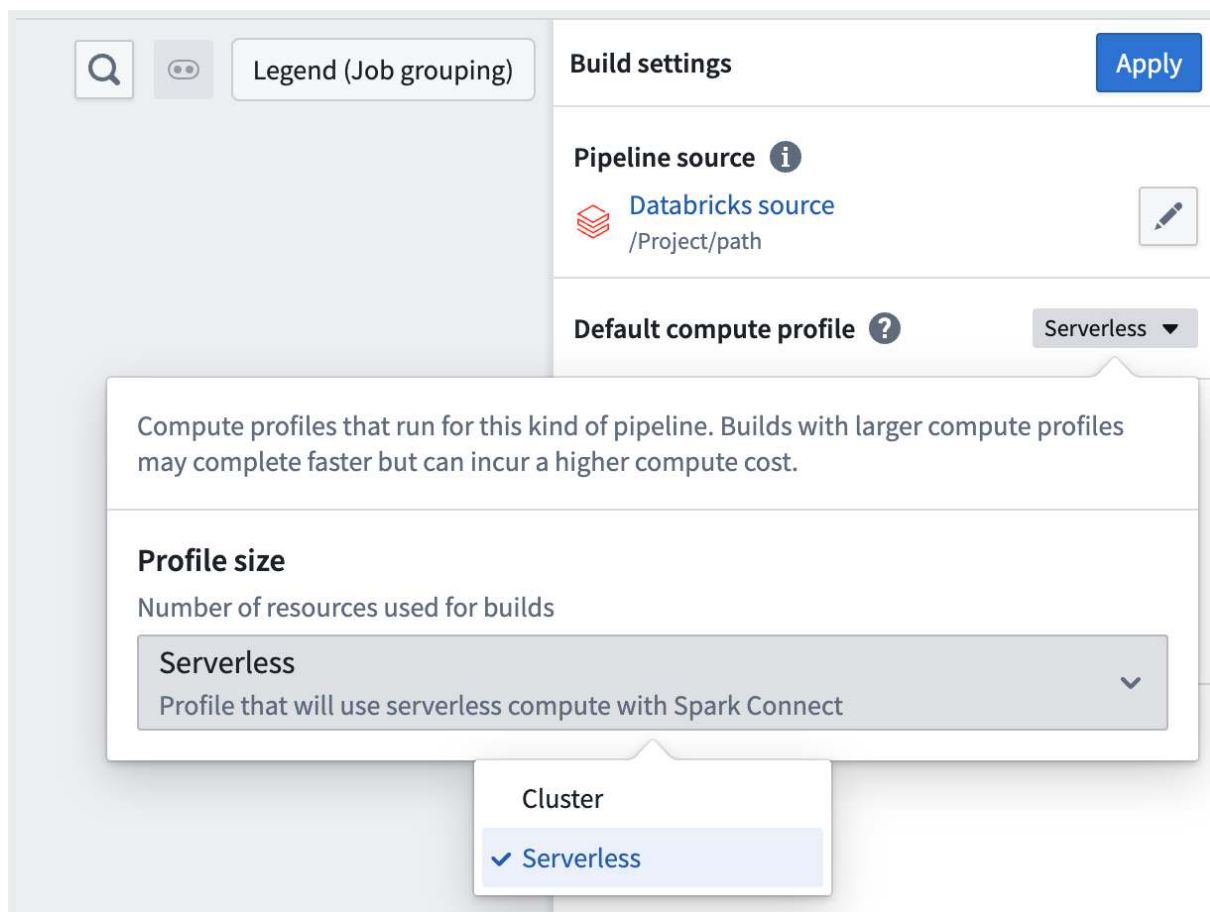
Next

Configuring build settings

You can edit your pipeline source and configure source-specific compute options in the build settings panel.

[API Reference ↗](#)

[Send feedback](#)



The configuration for external compute is available under the **Default compute profile** build setting.

Source type	Compute profile	Description
Databricks	Serverless	Use serverless compute with Databricks Connect (default). Refer to the official Databricks documentation for more information on compute options in Databricks Connect.
Databricks	Classic compute	Specify the cluster ID of a classic compute cluster. If unspecified, the Spark cluster ID will be derived from the source configuration. Must be provided if the source is configured to use a SQL warehouse.
API Reference	Compute warehouse	Specify the Snowflake warehouse to use for compute. If unspecified, the compute warehouse configured on the source will be used.

[Send feedback](#)

Known limitations

External pipelines do not currently support the same set of transforms and expressions as standard batch pipelines.

Due to the differences between external and batch pipelines, you should always verify results using **Preview** or by examining build outputs.

Currently unsupported features and expressions include:

- Incremental computation
- LLM features
- Media set operations
- Union
- User-defined functions
- Geospatial operations

PREVIOUS

← [Faster pipelines with Pipeline Builder](#)

NEXT

[Incremental pipelines / Overview](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Building pipelines / Incremental pipelines / Overview

Incremental pipelines

[Incremental pipelines](#) are often used to process input datasets that change significantly over time. By avoiding unnecessarily computing on all the rows or files of data that have not changed, incremental pipelines enable lower end-to-end latency while minimizing compute costs.

However, incremental pipelines carry additional development and maintenance complexity that you should be aware of before getting started.

Background

Here are some of the factors related to incremental pipelines you may want to consider:

- Developing an incremental pipeline requires a thorough understanding of how datasets change over time in Foundry using [transactions](#). You will need to interact with the concepts of dataset transactions in Data Connection syncs and transformation logic to effectively create and manage an incremental pipeline over time.
- Once you understand how transactions work in Foundry, you will need to design your pipeline to be resilient to unexpected transactions in your input datasets. Although incremental pipelines generally only process changed data that arrives in the form of **APPEND** transactions, your logic must be resilient to input datasets occasionally being recomputed, which results in a **SNAPSHOT** transaction. Ideally, your transformation logic should be written with thorough [unit tests](#) to validate behavior before this happens in practice.

API Reference ▸ Incremental pipelines remain performant in the long run, you will need to

understand how datasets change over time when many **APPEND** transactions are

causing datasets to consist of a large volume of small files

Send feedback

understanding how [Spark](#) handles large numbers of files and how this affects Spark partitioning. [Read more about maintaining high performance for incremental pipelines.](#)

Getting started

Get started with incremental pipelines by reviewing the following recommended resources:

- Learn how to [create incremental syncs](#) to bring data into Foundry incrementally.
- See an example of how to [create an incremental pipeline](#) with Pipeline Builder.
- Refer to the [Python incremental overview](#) to learn about developing incremental transform logic.

PREVIOUS

← [Getting started / External pipelines with Pipeline Builder](#)

NEXT

[Creating incremental syncs](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Data connectivity & integration / Building pipelines / Incremental pipelines / Creating incremental syncs

Creating incremental syncs

Although it is possible to derive `APPEND`-only datasets in a pipeline from Data Connection syncs that are configured as `SNAPSHOT` transactions, most of the benefits of incremental pipelines come from applying incremental end-to-end. This means that data syncs into Foundry should consist of `APPEND` transactions that only bring new data into the system. An added benefit of configuring incremental syncs is that they minimize load on the source system and can reduce data storage requirements.

Most datasets synced from source systems consist of files synced from a file system, or extracts from a database or data warehouse configured using a JDBC source type. The following guides walk you through how to configure incremental syncs for these source types:

- [Optimize file-based append syncs](#)
- [Incremental JDBC syncs](#)

PREVIOUS
← Overview

NEXT
Maintaining high performance →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[API Reference ↗](#)



Send feedback

Data connectivity & integration / Building pipelines / Incremental pipelines / Maintaining high performance

Maintaining high performance

Incremental transforms can help minimize resource use by ensuring that only new rows are processed whenever a pipeline updates. In Foundry, incremental transforms can only add new rows to the dataset and cannot replace or remove rows. This may be the case for data ingests as well. Since each run typically processes a small amount of data, each of these additions are small, but all of them must be considered together when rendering the active view.

Typically this is fine, but when tens or hundreds of thousands of small updates have accumulated (which can happen after weeks or months of uninterrupted building, depending on frequency), read and write performance may begin to degrade. This may be caused by several related factors, as discussed below.

Possible causes

Large file counts

Having too many files in a dataset causes slowness in backend requests that access the current view's data. This can cause preview unavailability issues, failures in Contour, and severe slowness in Transforms, especially Python Transforms, due to special overhead associated with loading files in that language environment.

Over time, any operation involving the active dataset view will be dominated by inefficient data that transforms that initially run in the order of minutes will end up taking several hours or days. In extreme cases, backend requests may even time out and fail.

API Reference ↗

Send feedback

Additionally, as part of writing a Spark DataFrame to an output in an APPEND or UPDATE transaction, an incremental transform needs to construct an in-memory list of all of the existing files in the output's current view. For outputs with many files, this operation can take a long time and may require the use of a high driver memory [Spark profile](#) to avoid out-of-memory errors.

Large transaction counts

Similarly to having too many files, too many [transactions](#) in the view can cause slowness. However, this slowness occurs not only when files are requested, but whenever the view range is resolved, which can affect dataset functionality such as computing stats, loading history, and so on. In transforms, this slowness will manifest in the pre-compute stage, when the environment is being set up and before any Spark details are available, making the issue difficult to debug.

Usually transactions accumulate in tandem with files, but in some cases empty transactions can cause issues even with a relatively low file count. Conversely, even a relatively low number of transactions may result in a large number of files if each transaction contains many files.

Possible solutions

Regular snapshotting

The simplest and, in some situations, most effective solution to keep incremental pipelines performant is to run regular snapshots. Snapshots will re-process the full data and replace the output with a fresh view, which can be partitioned efficiently. Snapshotting is a compute-intensive operation, which is why incremental processing is often preferred over regular processing, but occasional snapshotting can help prevent an excess of files or transactions from building up in the same view. To determine snapshotting frequency, administrators should balance performance for reading and writing data.

Snapshotting has cascading effects, in which each incremental transform that uses an input that has been snapshotted will snapshot itself. Rather than directly forcing an incremental transform to run a snapshot, we just need to snapshot our inputs and let the Transforms application determine that the incremental transform needs

See [API Reference](#) ↗

Send feedback

snapshotting as well. Thus, a useful way to set up an incremental pipeline to regularly snapshot is to set up a dummy input that will build on the intended snapshot schedule, and make the dummy input always run in snapshot mode.

Regular snapshotting has several drawbacks, as described below:

- Snapshotting can complicate the lineage, since dummy inputs will appear in searches and in the folder structure, which may be confusing.
- Snapshotting can be very expensive, even if done infrequently.
- While a snapshot is running, no updates can propagate through the pipeline, which can interfere with SLAs (service-level agreements).
- Snapshotting does not work for raw datasets, such as those backed by a Data Connection sync.
- Not every transform is written in a way that supports snapshotting (though we recommend that they always should).

Dataset projections

The most recommended way of dealing with incremental buildup is to add a [dataset projection](#) to the datasets involved. Dataset projections offer an alternate mechanism for querying for data in a dataset, and are stored and updated independently from the canonical dataset. Because of these traits, dataset projections can break out of the append-only model of incremental computation, and reorganize their internal data representation automatically as the volume of data grows. This is called compaction—compaction ensures reads are always performant from the projection, no matter how many files or transactions are in the canonical dataset.

This is particularly useful for incremental pipelines because dataset projections do not need to be completely in sync with the canonical dataset for readers to benefit from improved performance. All Foundry products know how to combine data coming from the projection and from the canonical dataset to reconstruct the view the same as if the projection wasn't there. For example, if a dataset has 100 incremental transactions, and has a projection that was built with the first 99, then 99 will be read from the projection and only one from the dataset. Because of this, it is usually sufficient to update dataset projections during them very computationally cheap to maintain.

Note that because the dataset projection is a separate resource from the dataset, it can be built at any time, even if the canonical dataset is itself building (and the other way

[API Reference](#) ↗

[Send feedback](#)

around)). Readers will just use whatever state is current according to the valid transactions. For example, if a dataset is building transaction 10 and the projection starts building at the same time, it will read from transaction 9. A reader that queries data in this scenario will read transactions 1-8 from the projection, and 9 from the dataset, effectively seeing the same data as if reading from the dataset directly.

Drawbacks of dataset projections include:

- Dataset projections use more storage than just the dataset alone would.
- Dataset projections do not work out of the box with Foundry Retention.
- Dataset projections do not modify the canonical dataset in any way (so if the projection is removed, reads will become inefficient again).

Retention policies

Sometimes a pipeline's use case does not require keeping historical data in-platform forever, and it's fine to retain only the most recent transactions, which can be done automatically using Foundry Retention. In this case incremental pipelines can be built without special consideration, as long as the transforms logic doesn't include any cross-transactional dependencies (such as aggregations or differential computation). A special `allow_retention` flag must be set in Python Transforms to the incremental decorator (otherwise `DELETE` transactions will trigger a snapshot run).

Drawbacks of retention policy changes include:

- Loss of historical data.
- If transactions aren't self-contained units from a data perspective, retention policies may lead to inconsistent state (e.g. an end event without a matching start).

Additional options to consider

Aborting transactions

[API Reference ↗](#)

Under some circumstances, transactions will be committed with zero-length or empty files. These transactions have no impact on the view, but they

[Send feedback](#)

updates and will trigger schedules and all related side effects, which may result in wasted computation. Empty transactions can also greatly increase the file and transaction counts.

Empty transactions are best avoided at the source, which is generally a Data Connection [sync](#). Data Connection will always abort transactions with no files, but empty files can still be generated. Empty transactions can be a particular issue with custom plugins; at times it may not be possible to modify the plugin to avoid empty transactions (for instance, if a non-empty transaction is required to update the Data Connection incremental metadata). In other cases, no-file transactions can be committed by transforms or other means.

To minimize the impact of these empty or no-file transactions, we can explicitly abort transactions at the most upstream transform in the pipeline. When we detect that we receive an empty input, or end up with an empty output, we can call `.abort()` on the output object; this will cause the job to be aborted, along with its transaction. Aborted transactions are effectively cancelled and will not trigger schedules or cause any side effects. Aborting the transaction will break the chain and stop propagating empty transactions down the pipeline. Aborting the transaction will also not contribute failure statistics (whereas purposefully failing the build would contribute to failure statistics).

Note that aborted builds are considered successful, and will advance the input transaction pointers. Therefore, aborting with non-empty inputs will discard data, which may not be desired if you want to stop the build for another reason, like a failed precondition.

Changelog datasets

Changelog logic enables you to implement edit semantics on append-only transactions, making it possible to perform joins and aggregations reliably in incremental pipelines. However, besides the previously-mentioned file and transaction count problems, implementing edit semantics on append-only transactions may allow the row count to grow without bounds, making transforms performance increasingly worse at the point where a state resolution stage is reached (or a standalone snapshot required).

Keeping the row count under control for such pipelines is a little trickier. Snapshotting is possible, and may help a lot in intermediate transforms when partial states are present (e.g. [API Reference ↗](#) go away in a full rebuild). But to fully benefit from snapshots, the logic must be able to update to their latest state, which is not always desirable (in some cases we may want to figure out the state of rows at any given point in time, not just the current state). Projections can only help so much. And retention policies may not have the desired effect if

[Send feedback](#)

rows aren't atomic units (for example, we may end up with an end event without a matching start event). So special care must be taken when designing such pipelines.

PREVIOUS
← [Creating incremental syncs](#)

NEXT
[Streaming pipelines / Overview](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Building pipelines / Streaming pipelines / Overview

Streaming pipelines

Streaming pipelines provide the ability to make immediate critical decisions based on real-time data. By processing data as a stream with dedicated compute, streaming pipelines are able to process records with [very low latency](#). On average, streaming data can be accessible in the Ontology and available for analysis in time series applications, such as [Quiver](#) or [Foundry Rules](#), in under 15 seconds. To achieve this low-latency, streams are built on top of [compute that runs continuously](#) and require different architecture and maintenance consideration compared to batch pipelines.

Best practices

When building out streaming pipelines, consider these factors:

- Streams often power highly operational workflows and require careful planning around downtime, maintenance, and logic changes to ensure high uptime and availability.
- Compute for streaming runs continuously. This can result in higher compute costs than a periodic batch job. Similarly to batch pipelines, consider starting with the smallest profile available and adjust that if the scale of your data requires it.
- Streams operate on a per-row basis and have constraints on the maximum row size to ensure low latency data transfers. The constraint is set to 1mb per individual row.
- Streams using state (windows or aggregations, for example) require design consideration to ensure the state is not broken when changing the stream logic.

API Reference ↗



Send feedback

To start using streaming pipelines in Foundry, review how to [create a simple streaming pipeline](#), and learn about streaming transforms in [Pipeline Builder](#). If you want to learn about connecting your data sources to Foundry, review [how to push data into a stream](#), or [how to setup a streaming sync](#).

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

API Reference ↗

Send feedback

Comparison: Streaming vs batch

When deciding whether to process data in Foundry as a stream or as a batch dataset, it is important to understand the requirements of your particular use case. We recommend considering the feature, performance, and technical expectations of both streaming and batch pipelines to ensure you choose the tool that works best for your use case.

Generally, streaming is used for workflows that require low end-to-end latency. For uses cases that can tolerate more than ten minutes of latency, [incremental](#) or standard batch [datasets](#) may also be suitable solutions.

Feature considerations

Streaming and batch pipelines share many common features. The primary differences to consider are latency, compute cost, and supported transform languages. The table below shows the features available for streaming and batch pipelines.

Feature	Streaming	Batch
Higher latency data usage for all Foundry products	Yes	Yes
Transactional	Yes	Yes
Atomic	Yes	Yes
Branching	Yes	Yes
Security markings and classifications	Yes	Yes
API Reference	Yes	Yes
Time series support	Yes	Yes
Time series support	Yes	Yes

Send feedback

Feature	Streaming	Batch
Java supported transforms	Yes	Yes
Pipeline Builder support	Yes	Yes
Python supported transforms	No	Yes
Low latency data access	Yes	No

Front-end tools

Some Foundry front-end tools can consume a streaming dataset in "real time", such as live updating a graph. The tools that can natively consume streams are [Ontology](#), [Pipeline Builder](#), [Quiver](#), [Dataset Preview](#), and [Foundry Rules](#). Other apps in Foundry, like [Contour](#), can consume the stream's archival dataset. This is a standard batch dataset that updates with the stream's newest data every few minutes. In practice, you can select the streaming dataset in any app you want, and Foundry will automatically know which mode to use.

Performance considerations

Along with feature considerations, consider the different performance expectations between streaming and batch pipelines. These factors include latency and throughput. Review [streaming latency and throughput considerations](#) for additional details.

Technical considerations

In addition to the feature and performance factors that define streaming and batch processing, consider technical factors including state management, downtime impact, and consumption layer latency.

State management

If [API Reference ↗](#) transforms, where the size of inputs is bounded and known ahead of time, [stateful streaming ↗](#) applications may have unbounded state that can grow over time and result in an out of memory error at an unknown point in the future. Performing an aggregation over one or more keys is generally a dangerous operation if the

Send feedback

size of the key space is unbounded. Unlike batch compute, it is usually difficult to anticipate and provision sufficient resources. For this reason, and due to the temporal nature of most streaming applications, ensure state growth is not unbounded when designing streaming transforms.

Low latency

Low latency is usually an important requirement of a streaming pipeline, but achieving low latency is not always a clear process. Since streaming pipelines typically have multiple steps, latency is bounded by the highest latency across all layers. Therefore, achieving low latency may involve fine-tuning multiple applications and consumption layers, from the source system to the streaming application and downstream consumers.

Downtime impact

Since batch pipelines are not typically used for operational workflows with strict low latency requirements, they tend to have a higher tolerance to downtime compared with streaming pipelines. Batch pipelines usually run on a fixed schedule; an individual run failure is not typically a major issue and the build can be retried. In contrast, streaming pipelines are continuously running processes with no defined endpoint, and failures are often more impactful. A 10 minute outage of a data store holding real time data has a much higher impact than a 10 minute outage of a data store with daily updates.

Consumption layer latency

Because batch pipelines are usually held up by transforms rather than reading or writing data, almost any consumption layer can be used. For streaming pipelines, consumption layers that natively support fast, frequent writes are required to maintain low latency. This means that each application in the end-to-end pipeline must be able to process data with low latencies to ensure low end-to-end latency. Most Foundry products support this natively, including Pipeline Builder, time series, Foundry Rules, and the Ontology. Data connectors also exist to support low latency writes to external systems.

[API Reference ↗](#)

Cost considerations

[Send feedback](#)

Due to the high-throughput, low-latency computation, the average cost of a streaming pipeline is often higher than the average cost of a batch pipeline. However, for a streaming-shaped pipeline, which requires low-latency processing of data, streaming could be the most cost-effective solution compared to a continuously running batch pipeline.

Review [Compute usage with Foundry Streaming](#) for more information on how streaming costs are computed.

PREVIOUS
← Overview

NEXT
Performance considerations →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Building pipelines / Streaming pipelines / Performance considerations

Performance considerations

As you prepare to create a stream in Foundry, it is important to consider the latency and throughput expectations that define your stream. This page will present some questions to consider regarding both latency and throughput performance for your stream use case.

Latency

Latency is the speed at which stream records are processed. Latency is a core performance component that defines realtime streams, and the speed expectations from when your records process through a stream and arrive at their destination can have real-world impact. For example, stream latency determines how quickly alerts are triggered for airline flight delays or supply chain issues that require immediate action. The factors that impact latency are multi-faceted, but some of the most significant considerations are listed below.

Latency factors

- How fast is the source producing data?
 - Foundry can only consume data as fast as it is produced, so you should ensure the data source is able to produce data quickly.
- How long does it take for the data to cross network boundaries?
 - When ingesting data into Foundry, the data often must pass across network boundaries which can introduce latency depending on network configuration, firewalls, and other factors.

API Reference ↗ are in your end-to-end pipeline?



Foundry streaming will co-locate pipeline transformations def
sitory or Pipeline Builder graph onto the same physical h

Send feedback

automatically optimize latencies. When more stages are added to the pipeline (for example, multiple repositories or Builder pipelines are chained together) we are unable to perform the same optimizations, incurring additional latency.

- Is data being sent to external systems?
 - For a record to be accessible in a low latency manner, the destination system must be able to process data in a low latency manner. Foundry offers optimized destinations, such as time series and the Ontology, but if data is traveling to external systems that system must support the latency requirements.
- What is the consistency model of your data?
 - Data consistency plays a significant role in end-to-end latency. Data that requires exactly-once guarantees (e.g. your record is guaranteed that it will be processed exactly one time) adds overhead and latency to ensure atomicity of your pipeline. If, however, your pipeline can run with at-least-once guarantees (e.g. each record is guaranteed to be processed at least one time, but may also be processed more than once), the system will automatically optimize your pipelines to make them run faster.

End-to-end latency of a streaming pipeline

A standard streaming pipeline can run through the following stages in under 15 seconds:

- **Ingestion:** ~1-2 seconds
- **Transformation:** ~5 seconds if exactly-once is enabled (default), or 1 second if at-least-once is enabled
- **Syncing into a backing datastore (object storage sync or time series sync):** ~5 seconds if exactly once is enabled (default), or 1 second if at-least-once is enabled

As indicated above, there are three major factors that influence the end-to-end latency of a streaming pipeline:

1. The complexity of the pipeline based on the number of transforms.
2. The consistency model based on whether the pipeline is running at-least-once or exactly-once mode.
3. Time-based windows and aggregations in transforms. For example, if you specify that you want to aggregate over a 30-second window, then the data will implicitly have 30 API Reference ↗ latency for the aggregation.

Send feedback

Throughput

Throughput is the amount of records that can be processed over a period of time.

Throughput is often equally as important as latency for measuring the performance of a low latency pipeline, and some of the most significant considerations are listed below:

- How many records is your source producing per time interval?
 - Foundry's streaming capabilities come with high throughput levels out of the box. However, if your source stream is producing at an exceptionally high rate, you can:
 - Increase the number of [partitions](#) your stream uses to support those higher volumes.
 - Set an existing stream's stream type to "HIGH THROUGHPUT". This configuration increases the number of records your source sends in one batch and is recommended if you notice that your stream's "Total Lag" metric is greater than 0. Note that this setting directly trades off latency for throughput. Before proceeding, check your stream's "Total Throughput" metric to confirm that applying "HIGH THROUGHPUT" is the right choice for your stream.
- How CPU intensive is the processing portion of your pipeline?
 - Throughput can often be limited by delays in processing. There are many ways to increase throughput in processing, most of which can be solved by scaling the size of your processing cluster.
- How fast can your destination system receive records?
 - The stream destination system can also cause a delay if it cannot receive records as fast as they are produced. This can lead to [backpressure](#), which decreases throughput and increases latency. Foundry's streaming products are optimized and designed to keep up with extremely high throughputs. However, if you set a streaming pipeline to write to your own destination, you should ensure the destination can keep up with the volume of records produced.

Advanced

- For users with an in-depth understanding of their data pipeline, another potential bottleneck for stream performance is network bandwidth. Symptoms of suboptimal network bandwidth include non-zero lag, lower than expected throughput, and records being dropped. To alleviate these symptoms, you can apply data compression to your stream. However, before doing so, keep in mind that:

API Reference ↗

Send feedback

- Data compression works best on high volume streams with repetitive strings.
- For lower volume streams whose primary concern is reducing latency, enabling data compression will further increase latency due to time spent compressing data in a suboptimal manner (for example, if there is a low volume of unique strings).

PREVIOUS

← [Comparison: Streaming vs. batch](#)

NEXT

[Streaming compute usage](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Building pipelines / Streaming pipelines / Streaming compute usage

Compute usage with Foundry Streaming

Foundry Streaming is a high throughput, low latency form of compute that consistently listens for new incoming messages, applies user-defined logic, and pushes them to the next stage in the pipeline.

Foundry streams rely on a distributed parallel worker-based architecture, with parallel workers each having their own resources that are dedicated to their particular streaming job. Stream resource requirements scale with the max throughput of the active stream along with the total number of messages it processes.

How compute is measured

Streaming compute usage is divided into two types:

- **Live processing compute:** The process of running user-defined transforms on live data. This source type is called “streaming”.
- **Archiving compute:** The process of moving data from the streaming layer to Foundry’s storage layer. Archive compute is a batch process and will appear as a “transform”.

i Note that when paying for Foundry usage, the default `streaming_usage_rate` is `0.5`. This is the rate at which user-defined transforms run on live data. If you have a contract with Palantir, contact your Palantir representative before proceeding with compute usage calculations.

API Reference ↗

Send feedback

The live processing compute of a stream in Foundry is measured by the number of compute-seconds it uses over its full duration in wall-clock time. Therefore, using more computational resources (e.g. vCPUs, memory, parallelization) in a streaming job increases the cost of the job. The longer a job runs, the more compute it uses. Since streams are designed to run persistently to continuously process data, a stream will continue using compute until it is ended by the user.

Streams are statically allocated; they will use a constant number of compute-seconds per wall-clock second while the stream is running. Streams are also often tuned to meet peak demand, meaning compute usage from the stream is unaffected by variable data volume. Streams use compute-seconds even if no data is moving through the stream.

Stream usage can be calculated as the sum of total seconds used by a single job manager and many task managers. Note that each parallel worker will have identical computational resources.

Job manager compute seconds are calculated in the following way:

- $\text{max}(\text{num_vCPU}, \text{gib_ram} / 7.5) * \text{streaming_usage_rate} * \text{stream_duration_seconds}$

Task manager compute seconds are calculated in the following way:

- $\text{max}(\text{num_vCPU}, \text{gib_ram} / 7.5) * \text{streaming_usage_rate} * \text{stream_duration_seconds} * (\text{num_parallel_task_managers} + 1)$
- $\text{compute_seconds} = \text{job_manager_compute_seconds} + \text{task_manager_compute_seconds}$

Learn more about values used to calculate compute usage, including [memory-to-core ratio](#).

Archiving jobs are batch processing jobs that run alongside each stream. Archive jobs periodically read from the hot storage layer of the stream and move the data into Foundry storage for robust persistence and historical tracking. As archive jobs do not have the same low latency requirements as the streams themselves, they run on a schedule and only use compute when there is data to be archived.

Archiving job usage is based on a single Spark driver and can be calculated with the

for [API Reference ↗](#)

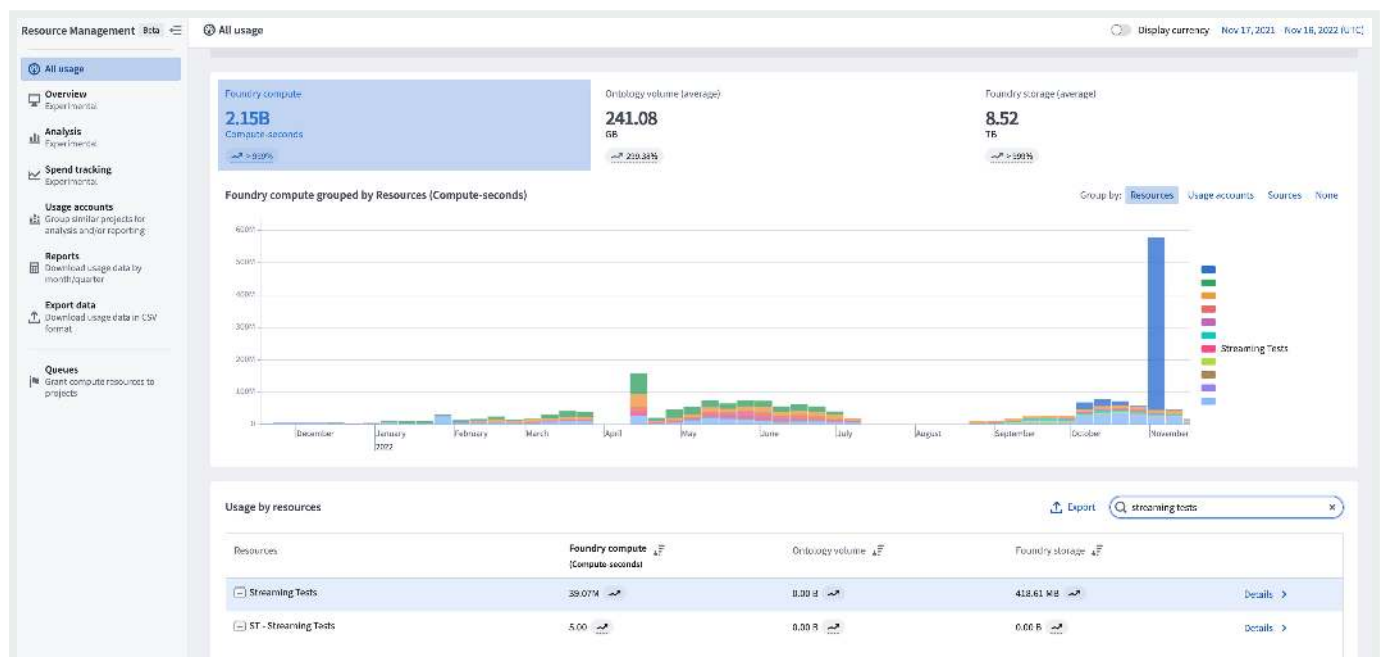
- $\text{compute_seconds} = \text{max}(\text{num_vcpu}, \text{gib_ram} / 7.5) * \text{num_seconds}$

[Send feedback](#)

- Archive jobs are small. The archiver always runs with a minimal profile of 1 vCPU and 4 GiB of RAM.

Investigate Foundry Streaming usage

To view the total usage of streams, first navigate to the [Resource Management application](#). Then, find your stream under the **Usage by resource** section and select **Details** to view usage by individual dataset.



The cost of a stream is attributed to the checkpoint dataset that the stream produces. This dataset serves as the permanent usage record of the processing of that stream. The streaming usage on this dataset falls under the “streaming” category in the Resource Management application.

Each time a stream is ran it will run continuously until stopped by a user. When a user stops a stream, that run will appear under the **History** tab of the dataset. You can investigate the profile of each individual stream to understand the performance and compute usage of historical stream runs.

Each time a historical archive is ran, it publishes its compute metrics in the [Builds](#)

[API Reference](#) Builds application to investigate the resource allocation for each archive that was ran.

Send feedback

Understand drivers of usage

The main driver of usage for a stream is the computational resource footprint of the stream itself. In this case, the compute resources include the number of vCPUs per task manager, the GiB of RAM per partition, and the number of partitions in the stream. These resources are set in the profile of the stream and persist for the duration of the stream.

- Stream resources should be allocated to meet the peak throughput of the incoming stream. If the volume of incoming messages is too high for the computational resources to effectively service, the stream will fall behind.
- To change the resources used by a stream, you must change the resource profile and restart the stream.
- Archive job compute usage scales with the amount of data coming through the stream. Archive jobs read all data since the last archive. If no data has been streamed, then the archive jobs will use zero compute. Archive jobs run every 10 minutes while a stream is active.

Manage usage with Foundry Streaming

It is important to understand when to choose streaming and when to choose batch for specific workflows. Streaming is designed for workflows that require second-level latency and constant compute. If data can run every few minutes, consider a small micro-batch job which can push the same amount of data as the stream but with a reduced compute-second cost and a significantly higher data latency.

- The size of a stream significantly affects the total compute-seconds used per run. Streams should be configured with enough resources to handle the maximum simultaneous load expected for that stream.
- It is important to choose the size of the stream to ensure that peak load can be served while ensuring it is not over-provisioned. This involves carefully configuring the size of each job (vCPUs and memory) and the total number of task managers for the stream.
- Streams will run until they are stopped. Carefully monitor sources of streaming compute to ensure that streams are only running when needed.

[API Reference ↗](#)

Calculating usage

[Send feedback](#)

The following example shows how compute usage is calculated for a hypothetical stream that runs for 10 minutes. Note that most production streams run continuously.

Stream profile

- Job manager vCPUs: 0.5
- Job manager gib_ram: 1
- Task manager vCPUs: 0.5
- Task manager gib_ram: 2g
- Parallelism: 2
- gib_ram: 4
- Duration of stream: 10 minutes (600 seconds)
- streaming_usage_rate: 0.5

Calculation

- Job manager compute seconds = $\max(\text{vCPUs}, \text{gib_ram} / 7.5 \text{ gib_ram}) * \text{streaming_usage_rate} * 600\text{s}$ = $\max(0.5, 0.133) * 0.5 * 600\text{s}$ = 150 compute-seconds
 - Alternatively, 0.25 compute-seconds per second or 900 compute-seconds per hour
- Task manager compute seconds = $\max(\text{vCPUs}, \text{gib_ram} / 7.5 \text{ gib_ram}) * (\text{parallelism} + 1) * \text{streaming_usage_rate} * 600\text{s}$ = $\max(0.5, 0.267) * 3 * 0.5 * 600\text{s}$ = 450 compute-seconds
 - Alternatively, 0.75 compute-seconds per second or 2700 compute-seconds per hour

The total compute usage for this stream running for 10 minutes is 150 job manager compute-seconds and 450 task manager compute-seconds. Learn more about the factors that impact [compute usage in Foundry](#).

Streaming keys

Foundry streams allow you to specify one or more columns as key columns, as well as a primary key to identify a resolved record. The sections below explain how to set up and maintain both partitions and primary keys for your streaming pipeline.

Partition keys

Generally, [primary keys](#) in the Palantir platform are used to uniquely identify a database record. However, *partition keys* in Foundry streams group *records* with the same **key**, such as all *readings* for a particular **device**, or all *transactions* for a particular **customer**. In contrast to primary keys in batch pipelines, partition keys in streams do not deduplicate records since they do not uniquely identify records.

Keys in Foundry allow users to ensure all records for a particular key will maintain order. When the record enters Foundry, it is stored internally in Kafka. Kafka guarantees that records written to the same partition will maintain order on read and write. If a record is sent to Foundry with no key, the record will be written to any partition in the internal Kafka topic in a round-robin style. However, if the user specifies a key for the record, the record will be written to the dedicated partition for that key, thus maintaining order when consumed downstream.

Similarly, Flink streaming jobs automatically maintain ordering based on the partition key column(s) set on input sources. Flink transform jobs may run with one or more parallel partitions per transform operation. For partition keyed input streams, Flink jobs will automatically send all records with the same values for all key columns to the same parallel partition. Unless specifically re-keyed (as with the [Key by transform](#)), partitioning and ordering for the entire length of the pipeline is determined by the key columns and row values. If a transform drops columns, the key values were on the source, even if transform logic drops columns.

API Reference ↗

Send feedback

Primary keys: Change data capture (CDC) mode

Primary keys in Foundry streams work similarly to those used in relational databases and in batch datasets. However, streaming primary keys consist of a set of deduplication columns that uniquely identify a resolved record and do not specify ordering columns.

The primary key is a piece of metadata that lives in the schema of a stream in Foundry. The key does not impact the contents of the stored data or how streaming data pipelines and transforms are applied. Primary keys control how some Foundry consumers read the data; the full *data* can be thought of as a **changelog**, while the *primary key* tells consumers how to compute a deduplicated **current view** of the data after applying all changes.

The current view is a filtered view of the data that only contains the most recently streamed record for each key. The full most recent record for a key will always be retained, even if it contains `nulls`. If a deleted column is specified, and the most recently streamed record for a key has this value set to `true`, the record will be filtered.

The following example shows a primary key and data stream, where the most recently streamed rows are higher in the table.

Primary key: `{Deduplication column(s): [Key], isDeletedColumn: Optional[isDeleted]}`

Key	Value	isDeleted
Key2	<code>null</code>	<code>false</code>
Key1	thirdVal	<code>true</code>
Key2	secondVal	<code>false</code>
Key1	firstVal	<code>false</code>

This stream of changelogs, when read by a CDC-aware consumer, will be read as the following current view:

Key	Value	isDeleted
Key2	<code>null</code>	<code>false</code>

If [API Reference ↗](#) primary key for your streaming data, you must also set the same columns as partition keys to maintain ordering and correctly resolve

Send feedback

views. Once you set a primary key in the streaming setup interface, the same columns will be automatically added as partition key columns.

The following two CDC-aware jobs will automatically read primary keyed streaming data as a current view:

1. **Hydrate the Ontology with a primary keyed stream:** New update records will cause a new object to be created, updated, or deleted as necessary.
2. **A stream's archive dataset is read by any Spark transformation job:** Before any transformations occur, the source will be deduplicated. Notably, almost all interactions with the archive dataset run a Spark job, including Dataset Previews and Contour analyses. When viewing the archive dataset, it will always appear as a deduplicated current view, even though the data itself contains the full changelog (if you download the data as a file). The full changelog will be processed by any streaming job that does not run Spark, including replays.

Key propagation

Pipeline Builder tracks the evolution of both partition key and primary key columns through pipeline transforms and writes any valid keys to the schema of the output stream. If your transforms do not invalidate any key columns, the same partitioning and deduplication instructions will be automatically maintained in a sequence of any number of downstream pipelines.

If you rename a key column, the key will update to contain the new column name. Similarly, if you apply a transform that removes or overwrites a key column, that column will be removed from the key. Since any overwriting of key column contents may represent a new ordering guarantee or a new deduplication strategy, Pipeline Builder drops the column from the key entirely. You must apply the `Key by` transform again if you want to keep a key column that was previously overwritten. Even when partition key columns are dropped due to removals or overwrites, the same ordering guarantee will persist for the remainder of the pipeline unless you re-key.

API Reference ↗

When keys or deduplication CDC keys are removed or overwritten, the key will be dropped entirely. Be mindful when deleting or overwriting the contents of key

Send feedback

columns; doing so may lead to unexpected results, including losing ordering guarantees or deduplication strategies.

Currently, keys **never** propagate for user-defined functions (UDFs). Since the function is user-defined, there is no way to infer which key propagation strategy, if any, the user intends. If you intend to propagate keys, be sure to apply the `Key by` transform after your UDF.

Additionally, primary keys do not propagate for stateful transforms (most transforms are not stateful).

Use a streaming key

In a streaming pipeline in the Pipeline Builder application, add the `Key by` transform to your graph. Notably, any keys you set here will override and replace any keys that were on the input data.

If you only want to set a partition key, toggle off the `CDC mode` option and only supply the `Key by columns` list. The other parameters are not required or allowed unless you are in CDC mode.

To set both a primary and partition key, toggle on the **CDC mode** option. If you have an `isDeleted` column, optionally specify it in the **Primary key is deleted** field. For streaming use cases, we highly recommend leaving the optional **Primary key ordering columns** parameter blank. The transform will set both a partition and primary key, where the partition key columns are the same as primary key deduplication columns. The **Primary key ordering columns** parameter matters only when consuming the archive dataset in a batch job and will never impact how streaming thinks about deduplication. The option to specify ordering columns is available for backwards compatibility, users of batch transforms, or users who intend to consume the stream archive in a specific batch-only way.

Check current keys

The [API Reference](#) describes how to find and verify streaming key logic in your Foundry stream.

[Send feedback](#)

Check partition keys

Open your streaming dataset, navigate to the **Details** tab, then open the **Schema** tab to view the data schema in JSON.

Search for `includeInPartitioning`, which will appear in an element of the `fieldSchemaList` for each column that is part of the primary key:

```
"customMetadata": {  
  "includeInPartitioning": true  
}
```

If you do not see any schema fields with `includeInPartitioning`, your stream is not keyed and ordering is not guaranteed for how your data will be stored or processed. To manually add keys, edit the schema as JSON text and insert the custom metadata (as described above) to each schema field (column) that you want to set as a partition key column.

i If a stream already has one or more partition keys, adding a new partition key column will cause a *weaker* ordering guarantee since there will be more partitions; order is only guaranteed to be maintained for rows that share the same value for all partition key columns.

Before deployment, if partition key columns are set on an input stream to a pipeline, ordering will be guaranteed for that source and all its transforms for the entire pipeline unless you intentionally re-key. Partition key columns may appear in a data preview with a key symbol.

Check primary keys

Open your streaming dataset, navigate to the **Details** tab, then open the **Schema** tab to view the data schema in JSON.

[API Reference ↗](#)

You will see a JSON property named `primaryKey`. If your stream has deduplication columns called `uniqueCol1` and `uniqueCol2`, and the `isDeleted` column is `boolean`, the schema should appear as follows:

[Send feedback](#)

```

"primaryKey": {
  "columns": [
    "uniqueCol1",
    "uniqueCol2"
  ],
  "resolution": {
    "type": "duplicate",
    "duplicate": {
      "resolutionStrategy": {
        "type": "customStrategy",
        "customStrategy": {}
      },
      "deletionColumn": "isDeletedCol"
    }
  }
}

```

If no primary key is set, the schema will show `null`:

```

"primaryKey": null

```

To manually set or remove the primary key, you may edit the schema JSON to specify a key in the above format, or use `null` to remove the key. If you manually set primary keys, we strongly recommend setting the same columns as partition key columns.

i The only way to guarantee ordering is to set partition key columns on your entire streaming lineage. Once set, the partition key columns will automatically propagate downstream. Even if a stream is configured to have only one partition, ordering is not necessarily guaranteed due to the way Flink applications scale and process records non-deterministically.

[API Reference ↗](#) **Key best practices**

Send feedback

Choose partition keys carefully, as keys that result in inefficiently distributed records can artificially increase load and limit throughput. If ordering is important to your use case, then set a partition key to a generic grouping identifier on which you want to maintain order, such as email ID, customer ID, or organization ID. If ordering is not important for your use case, you can choose to use a unique ID as a key or to not use a key at all for your stream.

The ordering guarantee of your final streaming output will be as strong as the *weakest* guarantee in your streaming series (backed by Kafka topics) and transforms (Flink jobs) that lead to your output. Therefore, make sure your desired ordering is maintained and that correct partition keys are set for your entire data lineage, starting from an initial streaming extract that pulls records into Foundry. The ordering guarantee will be no stronger than the system from which you are extracting to Foundry. For example, if you are using a Kafka connector to extract from Kafka, set partition key columns equal to the Kafka key column to allow Foundry to maintain an equivalent ordering guarantee on your system.

Additionally, problems can arise if you completely change the partition columns (and the ordering guarantee) during a series of data transforms. If a different ordering was guaranteed before applying a new **Key by** transform, the transform will receive records that are out of order from the newly added key columns; these records will remain in the wrong order during the transform series.

PREVIOUS

← [Streaming compute usage](#)

NEXT

[Streaming stateful transforms](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Building pipelines / Streaming pipelines / Streaming stateful transforms

Streaming stateful transformations

Foundry pipelines provide stateful data streaming transformations that enable complex data transform behavior at fast streaming speeds.

What is state?

In data streaming, a data transformation is **stateful** when each output row may depend on information contained in previously processed rows. The information that persists between rows is called **state**. The state may be accessed and mutated by each row.

An example of a stateful transform is a *sum aggregate transform*. Consider the following sample table, where a *sum aggregate transform* can be used to calculate the number of sensor readings with the value `hot` on each day. More recently streamed rows are higher in the table.

Day	SensorReading	Timestamp
Monday	hot	3
Monday	cold	2
Monday	hot	1

A stateful sum aggregate transform that computes the live running number of `hot` readings for each day may have output that looks like the following:

API Reference ↗

	SensorReading	Timestamp	State
☰ 🔍	hot	3	2
Monday	cold	2	1

Send feedback

Day	SensorReading	Timestamp	State
Monday	hot	1	1

Stateful transforms may store state of any serializable type including entire rows, which can enable them to achieve complex behavior. State transforms in Pipeline Builder are pre-built and automatically handle the state type and how the state type evolves.

The content of the state is not always accessible to the user. State can be used to enable backend processing behavior. For example, reading a stream input source in a data pipeline Flink job will store the state of the last offset for each partition; this enables the job to recover from the last successful point after a failure or restart.

Why is state powerful in data streaming?

Foundry data streaming uses the Flink architecture to provide low-latency data pipelining; each row is computed and passed downstream to the next operation immediately after processing. Unlike batch transformations, streaming transformations determine the next output one row at a time, without a full view of the data.

For non-stateful (also known as *stateless*) streaming transformations, this architecture means that transform logic can only depend on one row at a time. For example, a stateless transform could always add 5 to an integer column.

By contrast, stateful streaming transforms have access to persistent data about previous rows while continuing to be able to process rows one at a time and immediately as they arrive.

Foundry stateful streaming provides the option for [exactly-once guarantees](#), which is the default pipeline configuration. When selected, rows that cause the state to mutate are guaranteed to mutate the state exactly once, and in-order by key. This enables precise and complex data streaming behaviors.

For instance, if you intend to sum, your sum will always be accurate even if your stream restarts. If using a sort, the sort will always produce exactly one output per input. [API Reference ↗](#) Start the job mid-sort after the job has produced a partial sorted output that is not live.

Send feedback

Keyed state

All Pipeline Builder stateful streaming transformations use **keyed state** and require the user to specify [partition key columns](#). Stateful transforms are processed separately for rows with different values for key columns. This allows the backend to parallelize processing and scale to large data volumes.

For example, consider the stateful sum aggregate example, computing the live running count of `hot` readings each day. Note that in this example, the `Day` column is used as the partition key.

Day	SensorReading	Timestamp	State
Tuesday	hot	5	1
Monday	hot	4	2
Tuesday	cold	3	0
Monday	cold	2	1
Monday	hot	1	1

Notice how state is computed independently for rows with `Day` value `Monday` and those with value `Tuesday`. The occurrence of rows with key value `Tuesday` do not affect what is stored for `Monday`, and the state for these keys would be unaffected if more rows arrived with a different value for the `Day` column, like `Wednesday`.

Keys should be chosen carefully as keys that result in inefficiently distributed records can artificially increase load and limit throughput. See [streaming keys best practices](#).

Event time and watermarks

Stateful streaming transforms often rely on timing information. Because streams are ongoing and may at any moment receive a new real-time row, it often makes sense to group together temporally near rows. For instance, the [Outer Caching Join Transform](#) joins rows together only when they share values for join columns and when the rows are within an expiry limit. Pipeline Builder streaming uses Flink event time to achieve stateful transformations in a close-to-deterministic way that produces the same or very similar output upon replay.

API Reference ↗

Send feedback

Stateful transforms in Pipeline Builder that perform a time-based operation require the Assign Timestamps And Watermarks transform upstream and will produce a validation error if it is missing in your pipeline graph. Assign timestamps and watermarks assigns each row an "event time," usually a timestamp column contained in the row. The watermark is each transform operation's mostly deterministic sense of "what is the current time," and is a monotonically increasing value that closely follows behind the maximum event time value seen in any input row to that operator. For example, when an Outer Caching Join determines if entry in cache has expired, it checks if the watermark is greater than or equal to the expiry time, which will be true only when the join has received an input row with at least that event time.

For a transform operator with a single stream input, the watermark is the minimum of the maximum event times row seen by each parallel instance. For a transform with more than one stream input, the watermark is the minimum of the watermarks of the inputs.

Replay will result in similar, but sometimes slightly different, outputs. This is because upon replay, different partition keys may be assigned different Flink parallel instances, and operators with multiple inputs might have be processed upstream at different rates. Flink processing time is unsupported and not recommended since it can produce significantly different and potentially unintuitive results upon replay.

If a parallel instance is not receiving records, it will *not* produce a watermark. This will keep the overall watermark of the transform operator from advancing and is often a symptom of [poorly distributed keys](#), which has important implications for [state expiry](#) and [windows](#). To resolve this, [configure stream idleness in Pipeline Builder](#).

If a parallel instance does not receive records for the configured amount of processing time, then its watermark will no longer be considered for the computation of the overall transform operator watermark. While this configuration can resolve the issue of stalled watermarks, setting this timeout too short can cause slower instances to be marked as idle erroneously. Faster instances will then progress the overall transform operator watermark, which can lead to more dropped records.

State expiry

API Reference ↗

Storing large state can be lead to performance bottlenecks that can impact negatively throughput and latency, so Pipeline Builder requires the user to limit

Send feedback

Typically, state is limited through user-provided cache time expiry. For stateful transforms that require a cache time parameter, state is usually stored in state `cache` for each key until the watermark is beyond the last event time seen for that key, plus the expiry.

In the case of a stalled watermark, stateful transforms will not evict state promptly. This can lead to unexpected outputs and unbounded state growth.

Windows and triggers

The Aggregate Over Window transform allows user to set [windows](#), which are strategies for grouping rows and their state together, as well as [triggers](#), which are strategies for when the aggregate should produce output.

Windows

The currently supported windows are:

- **Tumbling event time:** Divides time into fixed-length, non-overlapping, continuous intervals. Rows with the same key and an event time that falls within the same interval are grouped together. For example, you can group together all rows on the same date with event time in the same hour of the day.
- **Count:** Given a user-specified count n , for each key, groups together the most recent n rows with that key.
- **Session:** Groups together rows that are part of the same *session*. Rows are in the same session if they share a key and there is no break of rows with that key in event time for more than the user-specified session gap. For example, in a dataset containing data about streaming platform user actions, you can group together all rows for one user workflow until the user takes a break.

Windows that depend on time (such as the tumbling event time window and session window) will eventually close once the watermark advances far enough.

- If the allowed lateness is not set or is zero, the windows stay open until the watermark [API Reference ↗](#) time of the window, at which point the window closes, may produce output, and deletes its state.
- if allowed lateness is specified, the window will stay open until the [Send feedback](#) end of the window plus allowed lateness. This allows late arriving or out of order records

to still be part of a window even if the watermark is past the end of the window.

- Rows that arrive when the watermark is past the end of the window plus the allowed lateness will always be dropped because the window has already been closed and its state deleted.

Windows that depend on time also allow specification of a custom trigger.

In the case of a stalled watermark, windows will stay open for much longer. This can lead to rows not being emitted, unbounded state growth, or missing trigger fires.

Triggers

The currently supported triggers are:

- **After watermark trigger:** Causes the window to output when the watermark passes the end of the window. Allows specification of other custom triggers for when the watermark is before the end of the window, and when the watermark is after the end of the window (when the window is still alive due to its allowed lateness). For example, a user may want no outputs until the window closes, but want to see every output for late arrive records in the allowed lateness period.
- **Count trigger:** Given a user specified count n , causes the window to output for each key after it receives a multiple of n rows.
- **Window close trigger:** Fires an output only as the window closes and deletes state. Will fire only exactly once per window and only at the end of the window.

Stateful streaming best practices

Large state can have negative performance implications, so when designing stateful pipelines it is recommended to use as "tight" of a state expiry policy as possible. This usually means not setting cache time expiry to be larger than necessary, nor setting a count larger than necessary for count windows.

For pipelines that require large state, performance (including throughput, checkpoint d [API Reference ↗](#)) scales with the parallelism of the Flink job. Parallelism can be edited in streaming pipeline settings, where larger parallelism allows for increased capacity and increased state read and write speed.

[Send feedback](#)

Appropriate keys should be chosen for stateful transforms, because too many values for key columns or imbalances in row distribution can lead to bottlenecks or trouble scaling.

PREVIOUS
← Keys

NEXT
Scheduling / Overview →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Scheduling

In Foundry's data integration layer, a [schedule](#) is a key concept enabling data to stay up to date for end users who rely on data in a pipeline. Foundry's scheduling capabilities are designed to support all types of data pipelines, and provide considerable flexibility when creating scheduled builds.

Scheduled builds can be configured to run:

- At certain times
- When data has been updated
- When logic has been updated
- *Any combination of the above conditions*

Scheduled builds can be configured to build:

- A single dataset
- A single dataset and all its dependencies
- All datasets that depend on a dataset
- All datasets that connect two datasets
- *Any combination of the above configurations*

PREVIOUS

← [Streaming pipelines / Streaming stateful transforms](#)

NEXT

[Create a schedule](#) →

[API Reference ↗](#)

© 2025 Palantir Technologies Inc. All rights reserved.



[Privacy Statement ↗](#)

[Send feedback](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Building pipelines / Scheduling / Create a schedule

Create a schedule

The [Data Lineage](#) schedule editor is where you can create new schedules and edit existing schedules. You can also view metrics on existing schedules and see how the schedule will build your datasets.

Navigate to the schedule editor

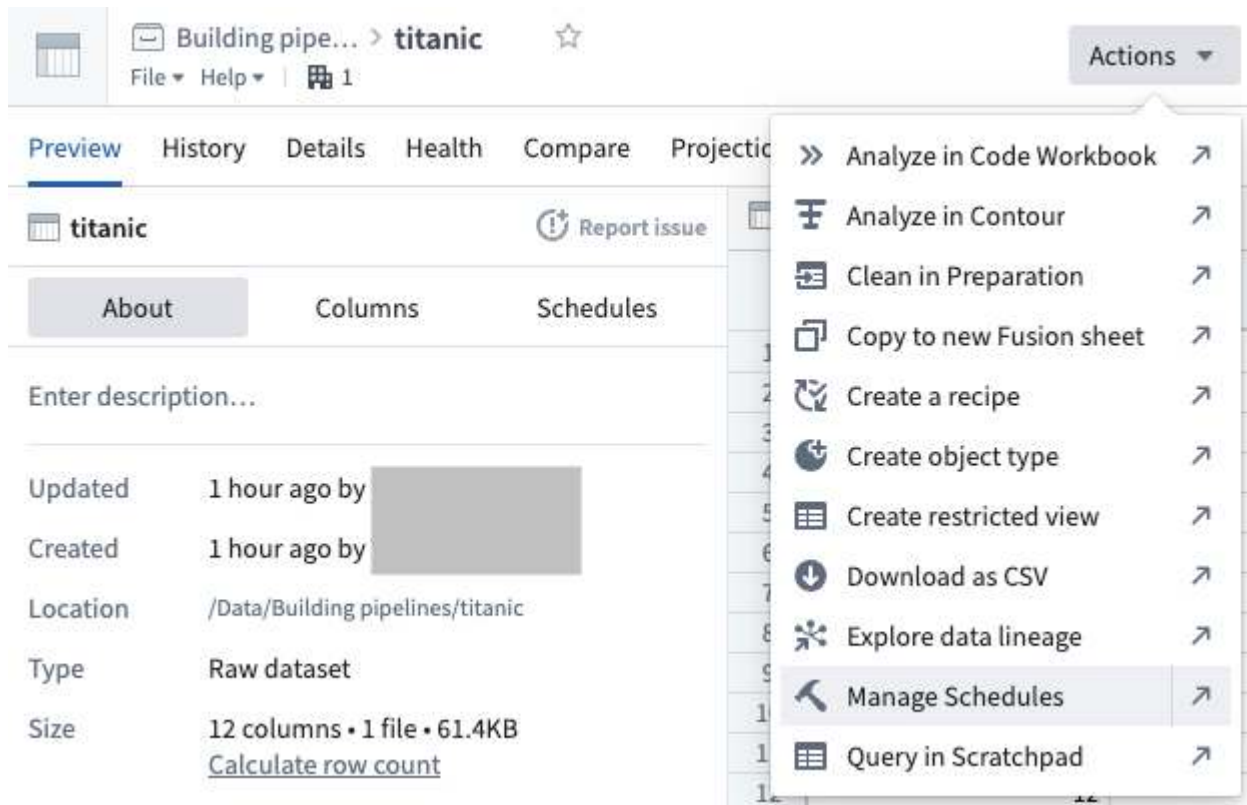
The following steps guide you to the Data Lineage schedule editor, where you can create a new schedule:

1. Navigate to the [dataset preview](#) of the dataset for which you would like to schedule a build, open the **Actions** menu, and select **Manage Schedules**.

API Reference ↗

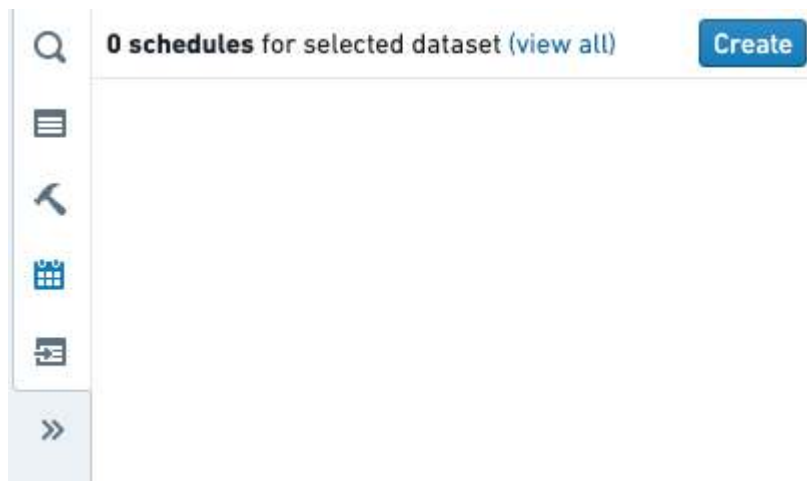


Send feedback



2. This will bring you to a [Data Lineage](#) graph, with the schedules panel to the right listing all current schedules that affect the dataset.

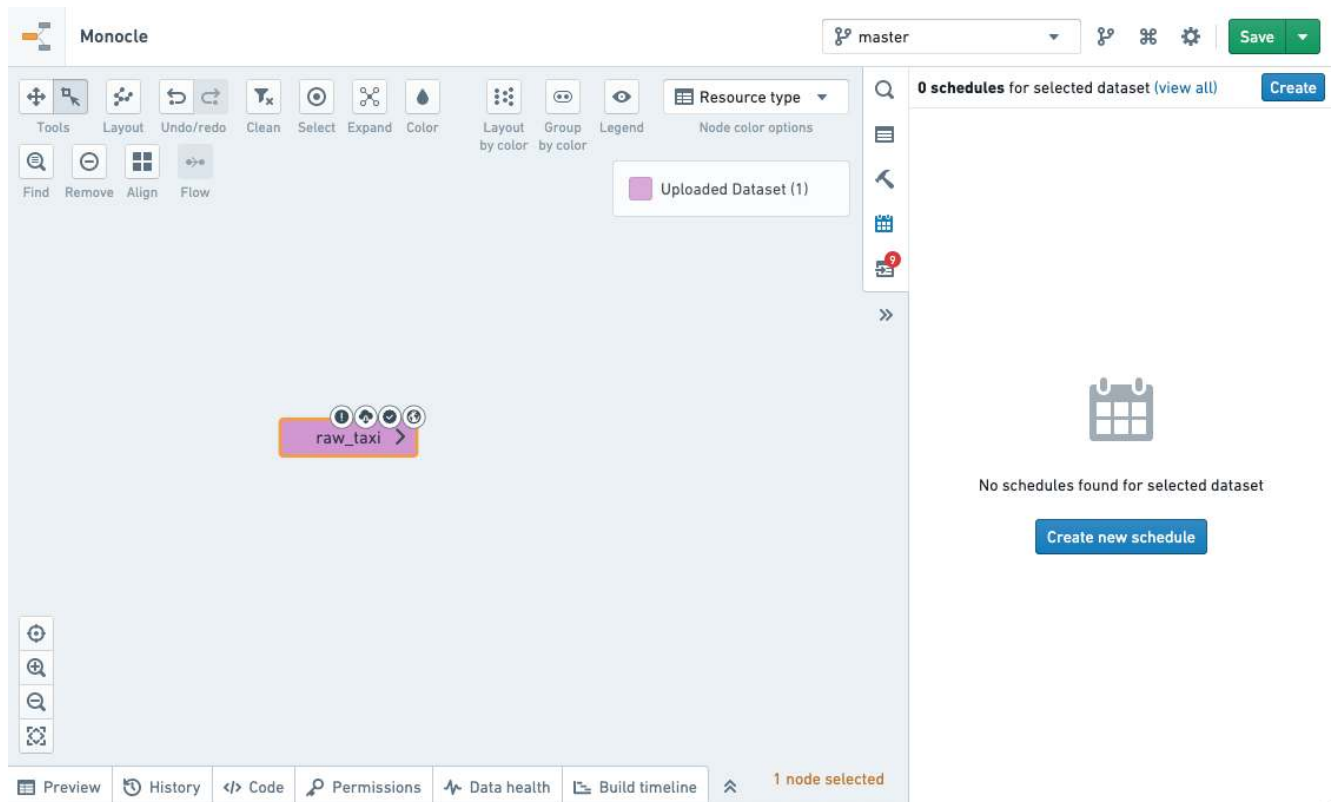
To create a schedule, click the **Create schedule** button.



3. This will bring you to the schedule editor.

[API Reference](#)

[Send feedback](#)



Define the schedule

To define a schedule, you will need to complete the following sections.

i Branch selection

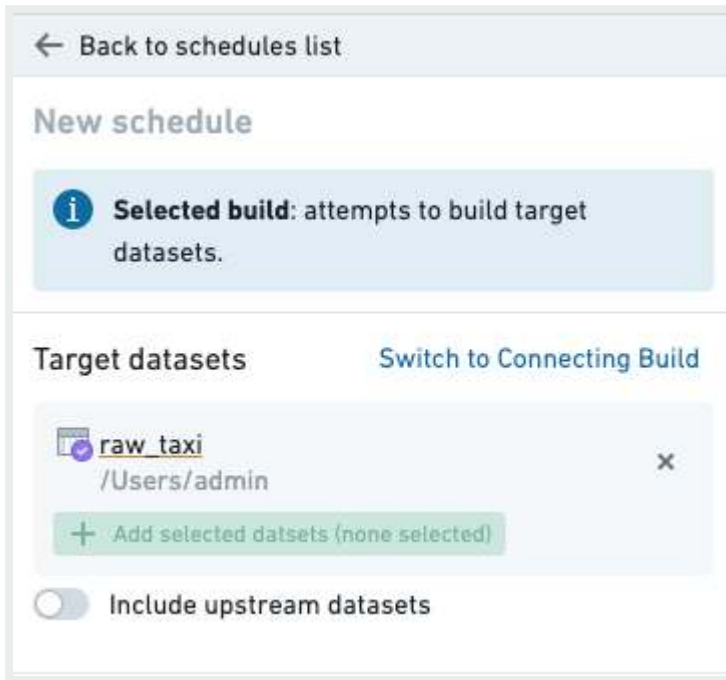
When creating schedules in Data Lineage, the schedules apply to the branches (including fallback branches) configured in the graph.

Target dataset(s)

The target datasets specify the datasets at the end of the build. By default, only these datasets will be built. In other configurations, these datasets may be the last in a line of datasets being built.

[API Reference ↗](#)

[Send feedback](#)



Target datasets can be selected by adding them to the graph and marking them as a target.



The datasets will be built on the branch set in the top right of the Data Lineage window. Datasets on the graph will be colored according to how they will affect or be affected by the schedule.

Excluded datasets

Excluded datasets specify the datasets that will be ignored during graph traversal when determining which datasets to build. All datasets upstream of ignored datasets will not be built.



Send feedback

When to build

The trigger specifies when the build should run. The build will be run when the condition defined in the trigger is satisfied.

There are two basic types of triggers:

- **Time:** A time trigger is satisfied at specified instants in time.
[Learn more about time triggers.](#)
- **When dataset(s) update:** Satisfied once a specified datasets update has occurred in the platform.
[Learn more about event triggers.](#)

Triggers may be combined to create a more complex compound trigger. The method used to combine triggers can be selected using the menu in the trigger editor:

- **Any of these triggers:** The trigger will be satisfied when any of the component triggers are satisfied.
- **All of these triggers:** The trigger will be satisfied when all of the component triggers are satisfied.
- **Advanced configuration:** Explicitly configure how to combine the component triggers. When advanced configuration is selected, the component triggers are combined by inserting the keywords **AND** and **OR** between two triggers. A compound trigger can be further combined by putting it inside parenthesis **(** and **)**.
 - **AND** indicates that the compound trigger will be satisfied when both of the component triggers are satisfied.
 - **OR** indicates that the compound trigger will be satisfied when either of the component triggers are satisfied.

[Learn more about compound triggers.](#)

Build type

The build type specifies how to build the selected datasets:

API Reference ↗

Build the target datasets (no other datasets will be built).

- **Full build (include upstream):** Build all target datasets and all upstream dependencies, except excluded datasets. Datasets built may change if upstream dependencies change.

Send feedback

change.

- **Connecting build:** Build all datasets between the input datasets (excluded) and target datasets (included). If a dataset is between an input dataset and a target dataset but is explicitly added as an input, it will still be built by the schedule. If a dataset is chosen as a target dataset, but there is no build path from an input dataset to the target dataset, the target dataset will not be built.

For a connecting build, all of the datasets connecting the target datasets and upstream datasets must be connected by a job spec path on the branch on which the schedule is set.

We illustrate the above with an example lineage below with datasets `D1, ..., D6`, where two different branches of the dataset `D2` are used as inputs to `D3` and `D4`. A connecting build schedule on the master branch with input `D1` and targets `D5` and `D6` will attempt to build only `D2`, `D4` and `D6`.

This behavior occurs because a connecting build selection between input `D1` to target `D6` on the master branch will select datasets `D2`, `D4`, and `D6`, since a job spec path exists between datasets `D1` and `D6`. However, a connecting build selection between input `D1` to target `D5` will not exist, as the job spec path along the master branch is broken.

```
D1 (master) --> D2 (master) --> D4 (master) --> D6 (master)
      |
      | D2 (develop)
      |
      --> D3 (master) --> D5 (master)
```

To include dataset `D3` in the scheduled build for this example, the build should be changed to a full build, which would include `D3` as well as the input `D1` as it is upstream of `D5`.

Alternatively, to remain as a connecting build selection, the transform which has `D2 (develop)` as an input and `D3` as an output can be edited to include an additional dummy input `D7` on the master branch, as below. The connecting scheduled build on the master branch with input `D1` and targets `D5` and `D6` will now build `D2`, `D3`, `D4`, `D5`, and `D6`.

API Reference ↗

```
D2 (master) --> D4 (master) --> D6 (master)
      |
      | D2 (develop)
      |
      --> D3 (master) --> D5 (master)
```

Send feedback

D7 (master) -----> D3 (master) --> D5 (master)

i Connecting build branch selection

For a connecting build schedule, if you are using datasets from different branches than the branch containing the schedule, then a connecting job spec path on the same branch must exist between the input and target builds in order for these datasets to be included in the build.

Build scope

To account for the dynamic nature of data pipelines, the set of datasets to be included in the build is evaluated every time the schedule is triggered. The **build scope** defines the boundaries for the schedule build, these boundaries will remain fixed even as the content of the pipeline changes. There are two scoping options available:

1. **By Projects:** The schedule will be allowed to build only eligible datasets within the selected Projects.
2. **By user account:** The schedule will be allowed to build only datasets that the schedule creator/editor is allowed to build.

Project scoping

Project scoping assures the schedule runs only on datasets within the selected Projects. It allows the schedule to run unaffected by changing user permissions. This option is most reliable when the build content is unlikely to change to the extent that it can no longer run within the selected Projects (as described below).

A build cannot be Project scoped in the following cases:

- You do not have permissions to create schedules on the Project.

API Reference ↗

ufficient permissions to build the datasets.

The schedule settings cover datasets that [remove markings](#) (you can see this in [Data Lineage](#) when the graph is colored by Permissions).

Send feedback

- The schedule settings cover datasets that must build with a user account.

- The schedule content cannot be resolved to a distinct set of Projects.

User scoping

With user scoping, the schedule build will be triggered on behalf of the user who last edited (or created) the schedule. Therefore, the build will only include datasets that the user is permitted to build. This option is most reliable when the scheduled pipeline is likely to change and render Project scoping impossible (e.g. datasets are moved to another Project, new datasets require to run with a user account, etc.).

Warning

If the user is deactivated or loses permission to datasets that are essential to the build, the schedule build will fail to start. When scoping by user account, make sure the account has reliable permissions and remember to change the ownership if the user is about to be deactivated.

Advanced settings

The advanced settings specify additional build options:

▼ Advanced options

- ☐ Abort build on failure
- ☐ Customize the number of attempts for failed jobs
- ☐ Force build
- ☐ Re-trigger upon successful build
- ☐ Allow overlapping runs
- ☐ Customize behavior on job failure

- **Abort build on failure:** If any job in the build is unsuccessful, immediately finish the build by canceling all other jobs.

API Reference ↗

Number of attempts for failed jobs: The number of run attempts for failed jobs. Retried jobs are run as part of the same build. A job is retried until all retries have been attempted or an error occurs indicating

Send feedback

performed. Setting this value to 1 will prevent retries, as the job will only be attempted once. Be aware that not all types of failures can be retried. The number of retries when the schedule runs will be capped to a maximum configured by your administrator.

- **Force build:** Ignore staleness information when running the build. All datasets will be built whether or not they are stale. This option is almost never required. The rare instances in which this option is necessary are:
 1. **Data Connection syncs:** Data Connection syncs are always marked as up to date because they have externally originating inputs; Foundry does not know whether there have been updates to the external data or not. We recommend scheduling Data Connection syncs separately from the rest of a build to ensure that only the syncs get force-built.
 2. **Object Storage v1 (Phonograph) syncs**

⚠ External transform schedules

Transforms that make API calls behave differently from Data Connection syncs. These transforms are always considered stale, and any build schedule will trigger them without requiring the **Force build** option.

- **Re-trigger upon successful build:** Repeatedly triggers the schedule to keep building until all the inputs have been processed and the targets of the schedule are no longer stale. For this setting to have an effect, the schedule must:
 1. Contain at least one target output that uses either [incremental transaction limits](#) for datasets or [incremental batch limits](#) for media sets. If there are no such outputs in the schedule, a warning with the text `No resources in this schedule require re-triggering` will be displayed.
 2. Have the **Force build** setting disabled.
- **Allow overlapping runs:** By default, a schedule does not start a new run while another run of the same schedule is in progress. Enable this setting to allow runs to overlap. Use this setting to:
 - **Reduce latency in a pipeline with a long sequence of jobs:** A new run can begin processing new input data at the start of the pipeline before an earlier run finishes processing data through the entire pipeline.

Use one schedule to keep multiple datasets up to date: A single schedule starts

API Reference ↗

one dataset as needed, without requiring a separate schedule for each dataset.

Send feedback

- **Customize behavior on job failure:** By default, when a job fails, the build cancels its dependent jobs. Enable this setting to specify datasets for which a job failure should not cancel dependent jobs. If a job for one of the specified datasets fails, the build continues to run its dependent jobs.

Save the schedule

When you have finished defining your schedule, click the **Save schedule** button.

This will bring you back to the schedules page where you should see the saved schedule.

PREVIOUS
← Overview

NEXT
View and modify schedules →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—Cookies Settings

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Building pipelines / Scheduling / View and modify schedules

View and modify schedules

View metrics

To view metrics and run history of an existing schedule:

1. Navigate to the dataset view, open the **Actions** menu, and select **Manage Schedules**.
2. This will bring you to the Data Lineage graph for that dataset, with the schedule panel open on the right.

To view metrics and build history for a schedule, select the **Metrics** button.



[API Reference ↗](#)

☰ 🔍 schedule edit history

[Send feedback](#)

A new version of a schedule is created whenever the schedule is edited. The schedule versions page allows you to:

- See previous versions of the schedule.
- Compare two schedule versions to see what has changed.

To view the previous versions of a schedule:

1. Navigate to the schedule's metrics page.
2. Select the versions tab:

The screenshot shows the 'My Schedule' interface with the 'VERSIONS' tab selected. On the left, a sidebar lists 'DATASETS TO BUILD' (my_output), 'WHEN TO BUILD' (At 12:41 AM, on day 1 of the month...), and 'Last modified a few seconds ago by Jane Doe'. The main content area is divided into two columns. The left column shows a list of schedule versions, with the current version 'Today at 10:15 AM (Current Version)' highlighted. The right column shows a comparison between the current version and a previous version 'Apr 14 at 1:59 PM'. The comparison highlights changes in the 'SCHEDULE TYPE' (from 'Connecting build' to 'Full build'), 'INPUT DATASETS' (from 'dataset_b' to 'Inputs not configurable'), 'TARGET DATASETS' (from 'my_output' to 'my_output'), 'EXCLUDED DATASETS' (No datasets), and 'WHEN TO BUILD' (from 'At 12:41 AM, on day 1 of the month (Europe/London)' to 'At 12:41 AM, on day 1 of the month (Europe/London)').

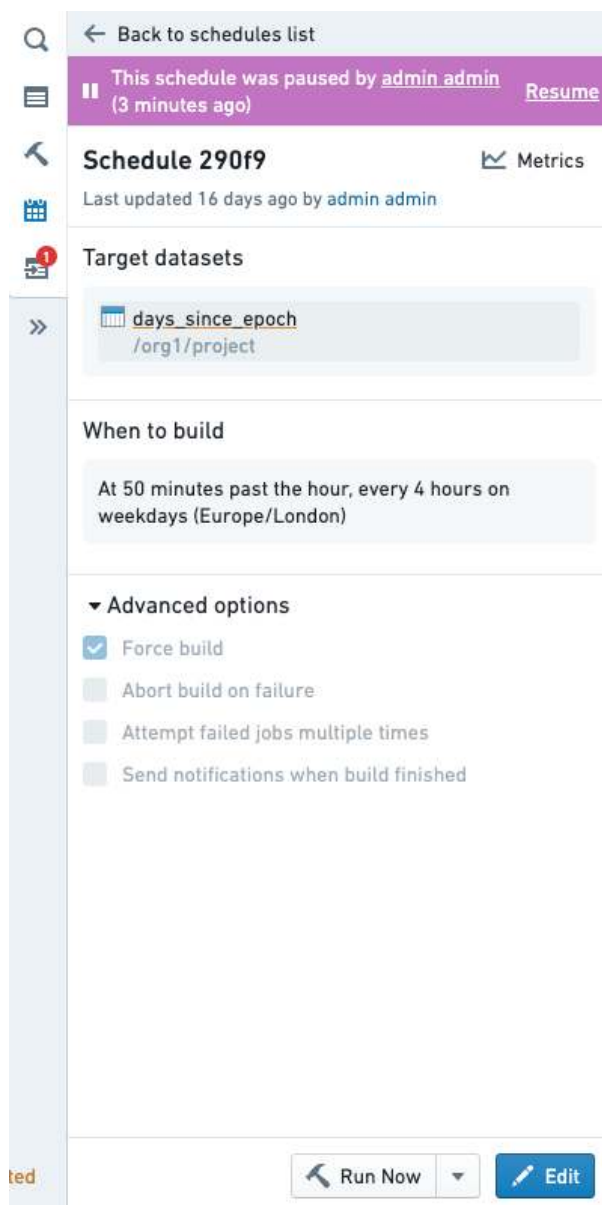
Only the versions up until the date displayed will be shown. If you need to see older versions, edit the date picker in the top right corner of the page.

Edit a schedule

To edit an existing schedule:

1. In the dataset view, open the **Actions** menu, and select **Manage Schedules**.
2. This will bring you to the Data Lineage graph for that dataset, with **Send feedback** open on the right.

3. To edit a schedule, click on the entry for that schedule.

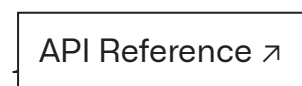


4. To edit the schedule, select the edit button. [Learn how to edit a schedule using the schedule editor.](#)

5. When you have finished editing your schedule, click the **Save** button.

Pause a schedule

To pause a schedule:

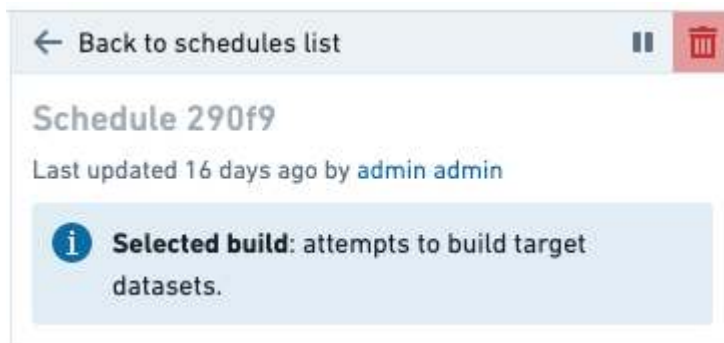


1. Click on the dataset in Data Lineage, and open the schedule sidebar, selecting the schedule from the sidebar.

Send feedback

2. This will bring you to the schedules page for the dataset.

3. To pause a schedule, select the pause icon in the top right.



Resume a schedule

To resume a schedule:

1. Navigate to the dataset in Data Lineage, and open the schedule sidebar, selecting the schedule from the sidebar.
2. This will bring you to the schedules page for the dataset.
3. To resume a schedule, select **Resume** in the top right.



Automatically paused schedules

Foundry automatically pauses schedules that fail all jobs consecutively and over multiple times. Once the schedule is run successfully, the schedule will automatically unpause and the failure counter will be reset. If your schedule was paused and you would like to resume it

[API Reference ↗](#)

1. Navigate to the schedule by clicking the link in the email sent to you

[Send feedback](#)

2. Debug the schedule. Refer to the [troubleshooting guide](#) for reference.

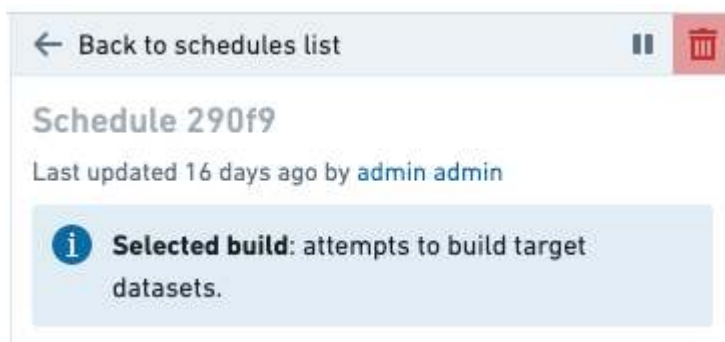
3. Run the schedule after the fix is applied. After a job initiated by the schedule is completed successfully, the schedule will be automatically unpaused.

If you have a specific schedule that you want to exempt from being paused, contact your Palantir representative and provide the schedule RID.

Delete a schedule

To delete a schedule:

1. Navigate to the schedule in Data Lineage, and choose the **Edit** button on the schedule.
2. To delete a schedule, select the trash icon in the top right.



← PREVIOUS
[Create a schedule](#)

NEXT →
[Find and manage schedules](#)

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

[API Reference](#) ↗

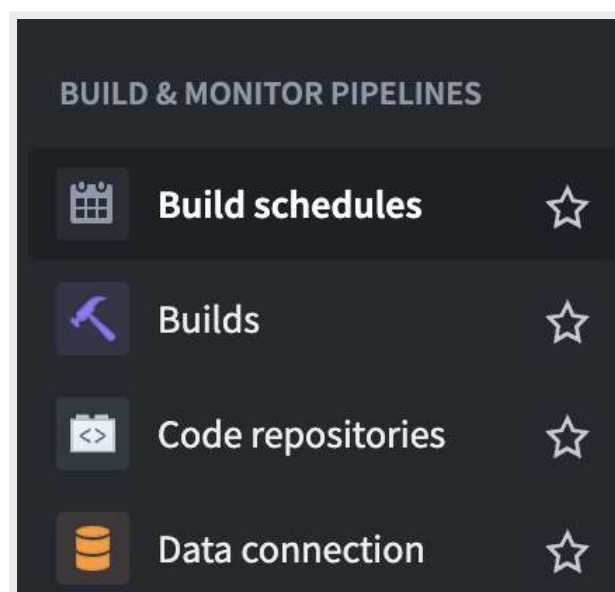
[Send feedback](#)

Data connectivity & integration / Building pipelines / Scheduling / Find and manage schedules

Find and manage schedules

[Schedules](#) can be searched for and managed using the **Build Schedules** application, allowing you to apply criteria and filters to your searches. For example, you can search for paused schedules by a given user or schedules scoped to a certain Project, filtered by name and sorted by latest run.

To access the Build Schedules application, look for **Build schedules** in the Foundry navigation sidebar to the left of your browser.



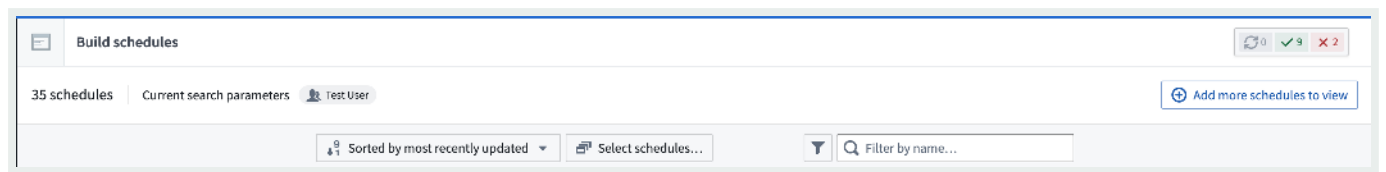
From the Build Schedules landing page, you will first see the schedules you created. Search parameters are stored in the page link, allowing you to bookmark a page or send the link to

o [API Reference ↗](#)



Send feedback

Adding search criteria



To find other schedules, select **Add more schedules to view**. Schedules will be found if they match any search criteria. The available search criteria are:

- **Files:** Find schedules by the datasets or other files in Foundry that they build. If no branch is specified, the default branch will be used.
- **User:** Find schedules last updated by the user(s) selected.
- **Projects:** Find schedules scoped to a given Project. User-scoped schedules are currently not supported for this parameter.

Further filtering and options

The list of schedules can be further refined by schedule name (if one is present on the schedule) and pause status. It can also be sorted by name, creation date, last run date, or last update date of the schedule.

Schedule information

For each schedule, the configuration of the schedule is shown as well as details on recent runs. The following details are shown over the 10 most recent runs (or fewer if the schedule has run fewer than 10 times):

- **Total compute:** Total CPU time for the recent runs of the schedule.
- **Median duration:** The median build duration over the recent runs of the schedule where the build succeeded.
- **Mean frequency:** How often the schedule has recently run, on average.

API Reference ↗

represent the recent runs of the schedule, with the most recent on the right. Hover over them to see more details or, for schedules that ran a view the build report for that build.

Send feedback

Schedule actions

Actions on found schedules can be performed individually or in bulk. The menu on each schedule card contains links to edit the schedule in [Data Lineage](#) and a link to the **Metrics and History** page, where you can view more details on the schedule’s performance and history and set up health checks. There are also controls to toggle pause, run, or delete the schedule.

It is also possible to pause, enable, or delete schedules in bulk:

- 1. Choose **Select schedules...** in the header.
- 2. Select the schedules on which you want to perform the action.
- 3. Select an action from the header.

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

API Reference ↗

Send feedback

Data connectivity & integration / Building pipelines / Scheduling / Common scheduling configurations

Common scheduling configurations

Get started with some examples of common schedules:

- [Build datasets regularly](#)
- [Build datasets when new data is available](#)
- [Advanced \(multiple\) trigger configurations](#)
- [Update a dataset at a specific time *only* if its parent has been updated](#)

Build datasets regularly

For this example, we want **raw_taxi (cleaned)** to update every weekday at 9 AM, and we want to build not just **raw_taxi (cleaned)** but also all of its upstream dependencies. We should configure our schedule as follows:

API Reference ↗



Send feedback

Monocle

master

Showing schedule configuration coloring. Click to go back.

Tools Layout Undo/redo Clean Select Layout Group Legend Node color options

Expand Color Find Remove Align Flow

Will attempt build (0) Target dataset (1)

Excluded dataset (0) Trigger dataset (0)

Unbuilt dataset (1)

raw_taxi → raw_taxi (cleaned)

Back to schedules list

Target datasets Switch to Connecting Build

raw_taxi (cleaned) /Users/admin

+ Add selected datasets (none selected)

Include upstream datasets

Exclude some datasets

When to build

When specific dataset(s) update

At a specific time

When multiple time or event conditions are met

At 09:00am on weekdays (Europe/London)

By week

On Su M Tu W Th F Sa

At 9 : 00 AM

Europe/London

Advanced options

Force build Build datasets that are up-to-date

Abort build on failure

Attempt failed jobs multiple times

Send notifications when build finished

Cancel Save

Preview History Code Permissions Data health Build timeline

Build datasets when new data is available

For this example, we want the schedule to run whenever another dataset has been updated. We can use the same configuration as in the previous section, with one small modification. An [event trigger](#) should be chosen, selecting which dataset(s) on the graph you wish to trigger the update.

[API Reference ↗](#)

[Send feedback](#)

When to build

☒ When specific dataset(s) update

☐ At a specific time

☐ When multiple time or event conditions are met

No datasets selected. Start by selecting a dataset on the graph.

+ Add selected datasets (none selected)

For more details on event-based schedules, see the [event triggers](#) documentation.

Advanced (multiple) trigger configurations

Trigger

Add trigger:

Time

Event

Any of these triggers

At 09:00 AM (UTC)

Any change on Parent A

Trigger

Add trigger:

Time

Event

Advanced configuration

At 09:00 AM (UTC)

OR

Any change on Parent A

For this example, we want **Dataset D** to update at 9 AM daily, but also whenever the dataset it depends on, **Parent A**, sees a change. According to [the table of combinations for compound triggers](#), if we combine a time-based trigger with an event-based trigger through an OR, the dataset will build at time T, as well as when event E occurs. Therefore, we will set the dataset we want to schedule the build for to **Dataset D**, and add a time-based trigger for 9 AM with an event-based trigger that watches for any update on **Parent A**. Choosing "Any of these triggers", or an advanced configuration and adding an OR between the conditions, are equivalent in this case.

[API Reference ↗](#)

[Send feedback](#)

Update a dataset at a specific time *only* if its parent has been updated

Trigger

Add trigger: 🕒 Time 📅 Event All of these triggers ▾ ↕

🕒 At 09:00 AM (UTC) × 📅 Any change on 📅 Parent A ×

Trigger

Add trigger: 🕒 Time 📅 Event Advanced configuration ▾ ↕

🕒 At 09:00 AM (UTC) × AND 📅 Any change on 📅 Parent A ×

For this example, we want **Dataset D** to update at 9 AM daily, but only if the dataset it depends on, **Parent A**, has seen a change. According to [the table of combinations for compound triggers](#), if we combine a time-based trigger with an event-based trigger through an AND, the dataset will build at time T *if* event E has previously occurred. Therefore, we will set the dataset we want to schedule the build for to **Dataset D**, and add a time-based trigger for 9 AM with an event-based trigger that watches for any update on **Parent A**. Choosing "All of these triggers" or an advanced configuration and adding an AND between the conditions, are equivalent in this case.

- i** This configuration does not limit the time window in which **Parent A** has been updated. Whether it was updated at 8:55 AM on the same day or at 9:10 AM the day before, the event-based trigger will evaluate to TRUE at 9 AM, causing all criteria to be met and the schedule to run. This means that if **Parent A** is consistently updating *after* 9 AM, e.g. at 9:10 AM every day, then **Dataset D** will be built daily at 9 AM, with data from **Parent A** that is 23 hours and 50 minutes old.

API Reference ↗

← Find and manage schedules

NEXT →

Tri

Send feedback

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Building pipelines / Scheduling / Trigger types reference

Trigger types reference

Foundry supports arbitrary nesting of `AND` and `OR` triggers, enabling the creation of specific customized triggers.

Time trigger

A time trigger is a trigger that is satisfied at specified instants in time.

i A time trigger is only satisfied at the specified instants. After a specified instant has passed the time trigger is no longer satisfied.

Cron expression

A time trigger is defined using a [cron expression ↗](#) and a time zone.

i Tip

The schedule editor provides an easy-to-use interface to define simple time triggers without having to write a cron expression.

More complex time triggers may require writing a custom cron expression.

The [API Reference ↗](#) uses the standard Unix cron expression format with five fields:



Send feedback

```

+----- Minute
| +----- Hour
| | +----- Day of Month
| | | +--- Month
| | | | +- Day of Week
| | | | |
* * * * *

```

i Data Health uses the [Quartz cron format ↗](#) (six-parameter), not the [Unix cron format ↗](#) (five-parameter). Quartz implements an additional field for seconds, which is prepended to the standard Unix cron format.

The schedule editor allows the following values for each field:

Field	Allowed Values	Allowed Special Characters	Notes
Minute	0-59	* - / ,	
Hour	0-23	* - / ,	
Day of Month	1-31	* - / , L	
Month	1-12 or JAN-DEC	* - / ,	
Day of Week	0-6 or SUN-SAT	* - / , L #	7 is also Sunday

The meaning of the special characters is detailed below:

- **Asterisk** (*): Specifies **all** values.

Examples:

Minute	Meaning
--------	---------

*	Every minute
---	--------------

- **Hyphen** (-): Specifies a **range** of values.

Examples:

Minute	Meaning
--------	---------

API Reference ↗	minutes between 10 and 20, inclusive
-----------------	--------------------------------------

- **Slash** (/): Specifies a **stepped range** of values.

A stepped range may be defined using either a value or a range:

Send feedback

- When used with a value, the range is from the specified value to the maximum value for that field
- When used with a range, the range is the specified range

Examples:

Minute	Meaning
25/10	Every tenth minute beginning from 25 (25, 35, 45, and 55)
25–45/10	Every tenth minute between 25 and 45, inclusive (25, 35, and 45)

- **Comma (,)**: Specifies a **list** of values, ranges, and/or stepped ranges.

Examples:

Minute	Meaning
10, 20, 30	Minutes 10, 20, and 30
10, 20–30	Minute 10 and the minutes between 20 and 30, inclusive
10, 25–45/10	Minute 10 and every tenth minute between 25 and 45, inclusive

- **L (L)**: Specifies the **last** value.

The meaning of **L** is dependent on how it is used:

- When used in the Day of Month field, it specifies the last day of the month
- When used in the Day of Week field by itself, it specifies the last day of the week (Saturday)
- When used in the Day of Week field with a value, it specifies the latest day of the month with the specified day of week

Examples:

Day of Month	Meaning
L	Last day of the month
Day of Week	Meaning
L	Last day of the week (Saturday)
2L	Last Tuesday of the month

- **Hash (#)**: Specifies the **nth day of week of the month**.

Examples:

Day of Week	Meaning
2#4	Fourth Tuesday of the month

API Reference ↗

The time trigger will be satisfied when the cron expression matches the current date and time.

Send feedback

i If both the Day of Month and Day of Week fields are not `*`, the trigger will be satisfied if either matches the current date and time.

Examples:

Cron Expression	Meaning
30 9 * * 1	9:30am on Monday
30 17 * 2 1	5:30pm on Monday in February
0 9-17 10 * *	Every hour from 9:00am to 5:00pm on the 10 th of the month
0 9-17/2 10 * *	Every two hours from 9:00am to 5:00pm on the 10 th of the month
0 9,17 10 * *	9:00am and 5:00pm on the 10 th of the month
0/5 9-17 15 3 *	Every five minutes from 9:00am to 5:55pm on the 15 th of March
0/5 9,17 15 3 *	Every five minutes from 9:00am to 9:55pm and from 5:00pm to 5:55pm on the 15 th of March
0 9 L * *	9:00am on the last of the month
0 9 L 2 *	9:00am on the last of February
0 9 * * L	9:00am on Saturday
0 9 * * 2L	9:00am on the last Tuesday of the month
0 9 * 4 3#1	9:00am on the first Wednesday of April
0 9 20 * 4	9:00am on the 20 th of the month and on Thursday

Time changes

All time triggers are evaluated using the wall-clock time in the specified time zone. A time trigger will be satisfied each time a wall-clock time occurs that satisfies the cron expression. Time changes are handled in the following way:

[API Reference ↗](#)

forward, a time trigger satisfied in between the time change will not be satisfied

[Send feedback](#)

- If the time moves backward, a time trigger satisfied in between the time change will be satisfied twice

Examples:

- If the time moves forward from 1:00am to 2:00am, a time trigger satisfied at 1:30am will not be satisfied because 1:30am never occurred
- If the time moves backward from 2:00am to 1:00am, a time trigger satisfied at 1:30am will be satisfied twice because 1:30am occurred twice

Event trigger

An event trigger is a trigger that is satisfied once a specified event has occurred.

-  An event trigger remains satisfied after the event has occurred until the entire trigger is satisfied and the schedule is run.

Event types

The schedule editor currently supports the following event types:

- **New logic:** Occurs when the logic to compute a dataset is updated.
- **Data updated:** Occurs when a transaction is committed that updates a dataset.
- **Job succeeded:** Occurs when a job on a dataset is completed, regardless of whether a transaction was committed.
- **Schedule ran successfully:** Occurs when a scheduled build is completed successfully.

Compound trigger

A compound trigger is created by combining multiple component triggers using **AND** or **OR**.
[API Reference ↗](#)

- An **AND** trigger creates a trigger that is the conjunction of its components.

Send feedback

- An **OR** trigger creates a trigger that is the disjunction of its component triggers.

Examples:

In the following examples, **T1**, **T2** are time triggers and **E1**, **E2** are event triggers.

Trigger	Meaning
AND(T1, E1)	Satisfied at T1 if E1 has occurred
OR(T1, E1)	Satisfied at T1 or when E1 occurs
AND(T1, T2) [1]	Satisfied at times that satisfy both T1 and T2
OR(T1, T2)	Satisfied at either T1 or T2
AND(E1, E2)	Satisfied when both E1 and E2 have occurred
OR(E1, E2)	Satisfied when either E1 and E2 have occurred
AND(T1, OR(E1, E2))	Satisfied at T1 if either E1 or E2 has occurred
OR(T1, AND(E1, E2))	Satisfied at T1 or when both E1 and E2 have occurred

[1] Creating an **AND** trigger with multiple time triggers will only be satisfied when all time triggers coincide. For example, an **AND** trigger with a daily trigger and an hourly trigger will only be satisfied once per day. Instead, a more specific single time trigger should be used. In the previous example, the **AND** trigger should be replaced with just the daily trigger.

← PREVIOUS Common scheduling configurations Troubleshooting reference NEXT →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Building pipelines / Scheduling / Troubleshooting reference

Troubleshooting reference

This page describes common scheduling issues with steps to debug.

Scheduled build issues

Schedule metrics page

One of the best ways to begin troubleshooting a schedule issue is by looking at the [schedule metrics page](#). The metrics page can tell you the source of your failure. Common failures include:

- The [scheduled builds are failing](#). You will see evidence of failed builds in the **Run History** tab. Selecting these builds will navigate to the Build Report in the [Builds application](#) for full logs.
- The [scheduled builds were ignored](#). The **Run History** tab will show **Ignored** in the Status column for any builds that were triggered, but not built.
- The [schedule was not triggered](#) at the expected time or cadence. For this case, the **Run History** tab may not show a build was triggered when you expected it to have.

Another useful tab is the **Versions** tab that shows past schedule versions and edits. If your schedule suddenly begins behaving differently than expected, check for any changes to the schedule version that align with this change, and consider reverting your schedule to a [previously working state](#).

API Reference ↗

Scheduled builds are failing

Send feedback

You can verify if a schedule was triggered at the expected time by checking the **Run History** tab on the [schedule metrics page](#).

If the schedule was triggered, but the build subsequently failed, you can debug this build in the [Builds application](#) similar to any other build.

A schedule will also fail to build if the appropriate permissions are not set. The permissions of a schedule depends on the token mode the schedule is in. Learn more about [Project-scoped schedules](#).

Scheduled builds were ignored

You can verify if a schedule was triggered at the expected time by checking the **Run History** tab on the [schedule metrics page](#). This information will usually give the reason that the schedule was ignored.

All datasets are up to date

A schedule run will be ignored if all the target datasets are up-to-date (i.e. if their inputs have not updated since the last build on that dataset). If this is the case, you will see this reason in the **Run History** tab. In the [schedule editor](#), navigate to the schedules list. You will then have the option to color the Data Lineage graph by `Out-of-date`, which will give you an overview of which job specs are considered stale.

This behavior can be overridden in special circumstances using the **Force Build** option in [advanced settings](#), though this is computationally wasteful outside these circumstances. If any of the target datasets build by Object Storage v1 (Phonograph) syncs, transforms with API calls, or Data Connection syncs, they may not show up as stale and may require the **Force Build** option to be enabled for the schedule to run.

Schedule builds a subset of job specs

If the schedule only triggers a subset of job specs, you will see evidence of this in the **Run**

H [API Reference](#) [Schedule metrics page](#).

One cause of this behavior is that only a subset of the job specs were [Send feedback](#) will only build the stale job specs, and those that are up-to-date will be ignored during the

build. See [all datasets are up-to-date](#) for more troubleshooting details. The case where the build is **Ignored** happens if all of these job specs are up to date.

Another cause could be that the job spec was not included in the job spec graph of the build. When a given schedule is selected in the [schedule editor](#), the job specs to be built are highlighted in the Data Lineage graph. The job spec selection depends on the [build type](#). If using a **Connecting build**, be sure to verify if a connecting job spec is present for schedules using the same datasets on multiple branches.

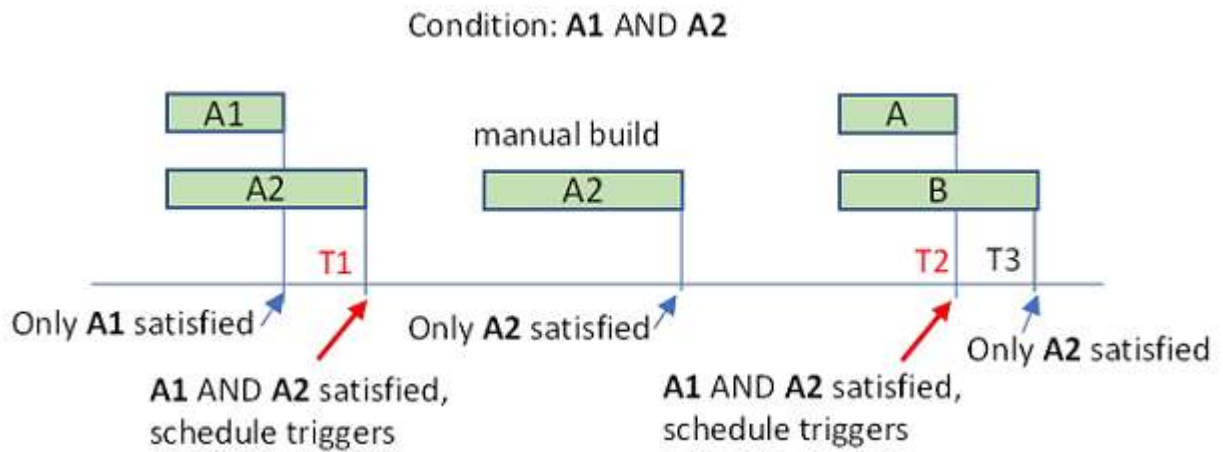
Schedule was not triggered

You can verify if a schedule was triggered at the expected time by checking the **Run History** tab on the [schedule metrics page](#). Some common debugging steps are:

- Check that the [schedule is not paused](#). Paused schedules will not trigger until they are unpaused.
- Check the [schedule trigger configuration](#). If the schedule was previously triggered successfully, check the schedule history to see if there was a recent change to the trigger.
- If the schedule is using an **Event** trigger, verify that the expected event actually happened.
 - For example, if the build should be triggered when the input updates, check that the last build on the input ran successfully and transactions on this build were successfully committed in the [Dataset Preview history view](#).
 - If the build should be triggered after multiple inputs update, check builds and timing on *all* inputs. For example, consider a schedule with input triggers A1 and A2, and "Wait until all these datasets update" is turned on. Say this schedule was previously ran at time T1. For this schedule to run again at time T2, A1 and A2 would need to both be updated in the time period between (T1, T2).

[API Reference ↗](#)

[Send feedback](#)



Schedule retries differ from configured

Not all types of failure are able to be retried. The number of retries when the schedule runs will be capped to a maximum configured by your administrator. Learn more about [advanced schedule settings](#).

The retrieable error codes are:

- INTERNAL
- TIMEOUT
- CUSTOM_SERVER
- FAILED_PRECONDITION

Schedule is failing with JobSpecInputsTrashed or JobSpecOutputsTrashed, or Data Lineage warns that some datasets are trashed

This error means that the schedule contains or reads from a trashed resource. You can do one of the following to resolve:

1. Remove the trashed dataset from trash.

2. Remove the trashed dataset from the schedule. If this dataset is used as an input to another downstream dataset in the schedule, you will also need to remove the dataset from the downstream dataset. For more information, see the following:

[API Reference](#) ↗

[Send feedback](#)

- Exclude the downstream dataset along with the trashed one.
- Modify the logic of the downstream dataset so that it no longer takes the trashed dataset as input.

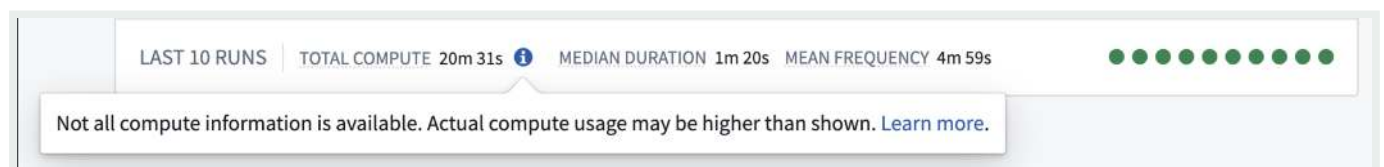
Schedule editor issues

Schedule permissions

To edit a schedule in [Project-scoped mode](#), you must have **Editor** permissions on the target datasets, **Viewer** permissions on the trigger datasets, and **Editor** permissions on the Project to which the schedule is scoped. If you lost permissions for one dataset, remove this dataset from the schedule before you save your changes.

To edit, delete, or pause a schedule, you need to have **Editor** permissions on the target dataset and **Editor** permissions on the Project to which the schedule is scoped. To view a schedule, you need to have **Viewer** permissions on the target dataset.

Schedule compute information is unavailable



A message stating "Not all compute information is available. Actual compute usage may be higher than shown." may appear when viewing a schedule for either of the following reasons:

- One or more scheduled builds are still in progress, and the total compute usage is not yet finalized.
- Information is missing about the resources used by at least one scheduled build.

When this message is displayed, no user action is required. The purpose of this message is to inform you that the compute usage shown may be inaccurate at a given time.

[API Reference ↗](#)

[Send feedback](#)

[PREVIOUS](#)



© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Building pipelines / Scheduling / Add schedule to a Marketplace product

Add schedule to a Marketplace product

Use [Foundry DevOps](#) to include your schedules in [Marketplace products](#) for other users to install and reuse. [Learn how to create your first product.](#)

Supported features

We support including schedules in Marketplace products that satisfy the following:

- The schedule is not user-scoped.
- The schedule does not have [fallback branches](#).
- All [triggers](#) should be defined for the same branch.
- No [triggers](#) or [target](#) datasets are [Restricted Views](#).

We strongly recommended that all packaged datasets (excluding static datasets) have a corresponding schedule that builds the dataset as a target. If datasets are included without a schedule, the dataset and anything downstream will become stale.

Adding schedules to products

To add a schedule to a product, first [create a product](#), then [add outputs](#). Choose the

S

API Reference ↗

 e.

If



 not yet added any [pipelines](#) or [dataset transformations](#), you can click

Send feedback

 to add schedules to select. Given this, we typically recommend adding these resource types first.

Once you have added a pipeline or dataset transformation, view the **Datasets** content type to review which datasets will not be built, and schedules you could add to your product to remedy this.

← All products

Data Governance Pipeline

Pipeline product

Creating new version Draft

General information

Content

Datasets

62

⊕ Add content

Inputs

Documentation

Installation mode

Folder structure Beta

Changelogs

Review and publish

Schedules

View

✖ 0 out of 1 supported schedules are packaged

✖ 62 datasets will not be built by one of your schedules

☐

Datasets (62)

NAME	DEPENDENCIES
☐ actions Produced by transforms	2 dependencies
☐ agent-source Produced by transforms	4 dependencies
☐ agents Produced by transforms	4 dependencies
☐ agents Produced by transforms	4 dependencies
☐ applications Produced by transforms	4 dependencies
☐ applications Produced by transforms	6 dependencies
☐ applications Produced by transforms	4 dependencies

Select any relevant schedules to include with your product. If you don't see any schedules, you should [create one with your source datasets](#) and then [create a new version of your product](#).

Add schedules

×

☒

Schedules (1)

SCHEDULE	ENABLED	TARGET DATASETS
☒ Data Governance Schedule	▶ Enabled	62 datasets

Cancel

Add

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Building pipelines / Scheduling / Parameterization

Parameterization [Beta]

Beta

Schedule parameterization is in the [beta](#) phase of development and may not be available on your enrollment. The feature is currently only available for [Python transforms](#), and functionality may change during active development.

Parameterization allows transform logic to run with different parameter values.

Define parameters in Python transforms

To use parameterization, you must first define the parameters in your transform logic and verify that your [repository](#) meets the following version requirements:

- `transformsVersion` must be `10.96.0` or later.
- `transformsLangPythonPluginVersion` must be `1.1137.0` or later.

If it does not, follow the [repository upgrade guide](#) to ensure your repository meets the version requirements above.

Each parameter has a name, type, and default value.

- **Parameter name:** Declared by the Python variable name.

API Reference ↗

One of [StringParam](#), [IntegerParam](#), [FloatParam](#), or [BooleanParam](#).



Parameter default value: The value used when the parameter is not specified, meaning form was not built through a parameterized schedule.

Send feedback

The following example Python transform has two parameters named `town_name` and `risk_factor` with default values of `"Seattle"` and `5`.

```
1 @transform.using(  
2     processed_towns=Output('/path/to/output_towns'),  
3     towns=Input('/path/to/input_towns'),  
4     town_name=StringParam("Seattle"),  
5     risk_factor=IntegerParam(5),  
6 )  
7 def compute(processed_towns, towns, town_name, risk_factor):  
8     ...  
9     selected_town_name = town_name.value  
10    selected_risk_factor = risk_factor.value  
11    ...
```

Enable parameterization in the schedule editor

Once you have your transform logic in place, [create a new schedule in Data Lineage](#), and enable the **Parameterization** option. The parameters defined on your target datasets' transforms will be available here.

The screenshot displays the Data Lineage schedule editor. The main workspace shows a workflow with a source node 'towns' pointing to a target node 'processed_towns'. A legend indicates the status of the build: 'Will attempt build (0)', 'Target (1)', 'Excluded (0)', 'Trigger (0)', and 'Unbuilt (1)'. The right-hand panel contains the 'New schedule' configuration. The 'Parameterization' option is highlighted with a red box and is currently disabled. Below it, the 'When to build' section offers four options: 'When specific resource(s) update' (selected), 'At a specific time', 'When triggered manually', and 'When multiple time or event conditions are met'. The 'Target resources' section lists 'processed_towns' as the selected resource. A 'Send feedback' button is located at the bottom right.

API Reference ↗

Send feedback

Standard mode

In standard mode, you provide the parameter values on the schedule, and these values are passed to your transforms when the schedule runs.

To use the default value defined on the transform for a parameter, hide that parameter in the schedule.

The following example shows the `risk_factor` parameter hidden, and the `town_name` set to "New York" for this schedule.

The screenshot displays the Data Lineage interface. The main workspace shows a workflow with two nodes: 'towns' and 'processed_towns'. The 'processed_towns' node is highlighted in green, indicating it is the target of the build. A legend on the right side of the workspace shows the following status counts: Will attempt build (0), Target (1), Excluded (0), Trigger (0), and Unbuilt (1). The right-hand configuration panel is open, showing the schedule configuration for 'My schedule'. The 'Parameterization' section is active, showing the 'risk_factor' parameter hidden (indicated by a crossed-out icon) and the 'town_name' parameter set to 'New York'. The 'When to build' section is configured to 'At a specific time' with a frequency of '1 Hour' and a time of '50 Minutes'. The 'Build scope' section is set to 'Projects'.

API Reference ↗

Send feedback

Example use case: Periodic snapshot jobs for an incremental transform

Suppose you have an incremental pipeline where you need to run hourly incremental builds, but also run a daily non-incremental build that performs data compaction. You can achieve this using a parameterized schedule by following the steps below:

1. Create a transform with a boolean parameter that indicates the runtime type for the job.

```
1 @incremental()
2 @transform.using(
3     processed_towns=Output('/path/to/output_towns'),
4     towns=Input('/path/to/input_towns'),
5     is_compaction_job=BooleanParam(False),
6 )
7 def compute(processed_towns, towns, is_compaction_job):
8     towns_df = towns.polars()
9
10     if is_compaction_job.value:
11         processed_towns.set_mode("replace")
12
13     processed_towns.write_table(towns_df)
```

2. Create two schedules that build the transform:

- An hourly schedule for your incremental builds. You can skip the parameterization and rely on the default value.

[API Reference ↗](#)

[Send feedback](#)

The screenshot displays the Data Lineage tool interface. The main workspace shows a workflow with two nodes: 'towns' and 'processed_towns', connected by a right-pointing arrow. The 'processed_towns' node is highlighted in green. A legend box above the workflow defines node colors: blue for 'Will attempt build (0)', green for 'Target (1)', black for 'Excluded (0)', orange for 'Trigger (0)', and white for 'Unbuilt (1)'. The right-hand sidebar is titled 'Hourly incremental job' and contains several configuration sections:

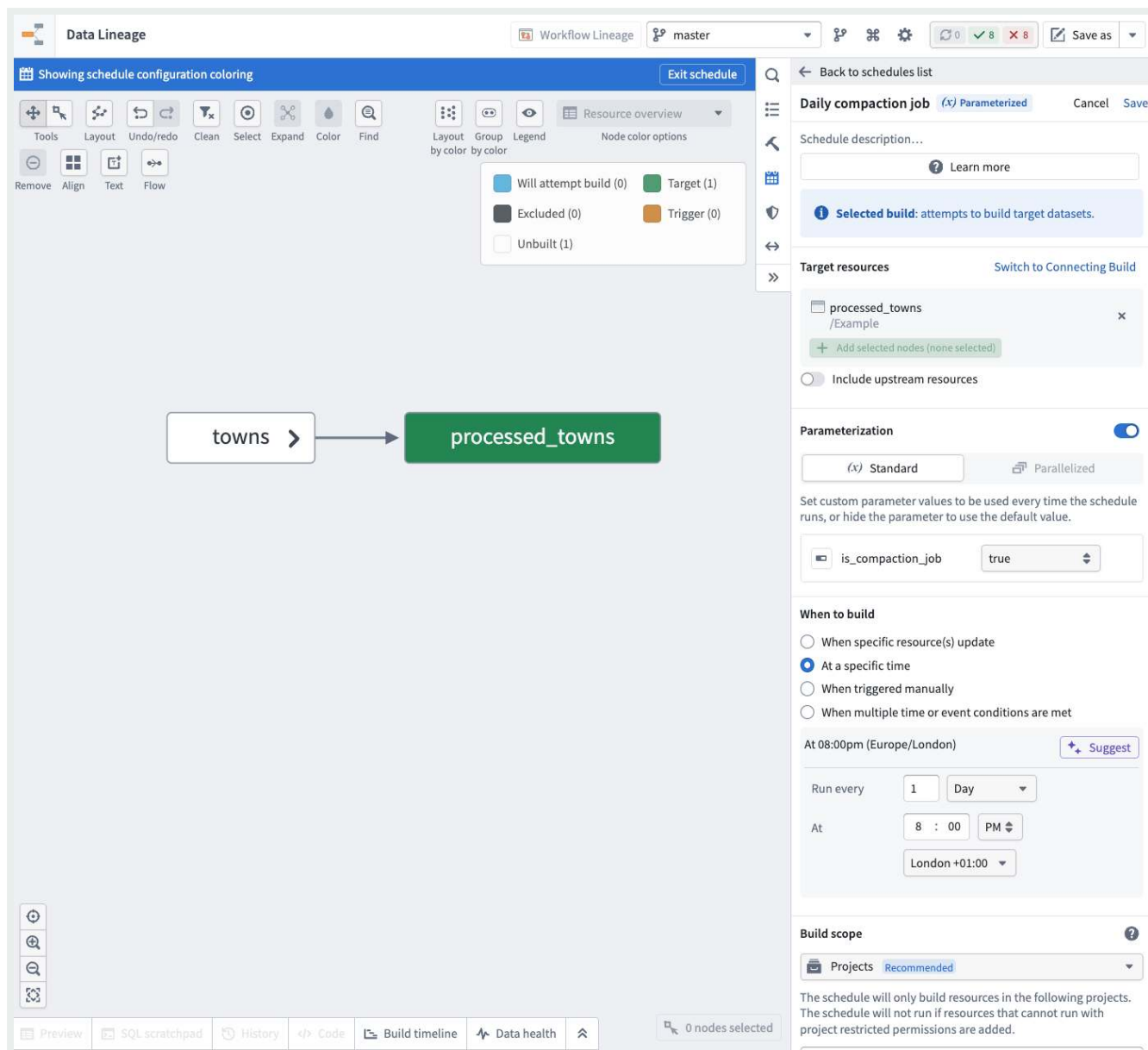
- Schedule description...**: Includes a 'Learn more' link and a message: 'Selected build: attempts to build target datasets.'
- Target resources**: Shows 'processed_towns /Example' with an 'Add selected nodes (none selected)' button and an 'Include upstream resources' checkbox.
- Parameterization**: A toggle switch is currently turned off.
- When to build**: Radio buttons for scheduling options. 'At a specific time' is selected. Below, it shows 'At 50 minutes past the hour (Europe/London)' with a 'Suggest' button. Further down, 'Run every' is set to '1 Hour' and 'At' is set to '50 Minutes'. There is also a 'Snooze' checkbox for 'On weekends'.
- Build scope**: A dropdown menu is set to 'Projects Recommended'. Below, it states: 'The schedule will only build resources in the following projects. The schedule will not run if resources that cannot run with project restricted permissions are added.' It includes a 'Filter projects...' search bar and lists 'INCLUDED' projects (with 'Example' expanded) and 'EXCLUDED' projects (showing 'No excluded projects').

The bottom of the interface features a toolbar with icons for 'Preview', 'SQL scratchpad', 'History', 'Code', 'Build timeline', and 'Data health', along with a status bar indicating '0 nodes selected'.

- A daily schedule for your non-incremental archive builds. Enable the parameterization and set the boolean parameter `is_compaction_job` to `true`.

API Reference ↗

Send feedback



These schedules allow you to run hourly incremental builds and daily non-incremental builds while keeping the same transform logic deployed.

Parallelized mode (advanced)

Parallelized mode allows you to create a schedule that can run independent parallel builds with different parameter values and aggregate their results.

F [API Reference ↗](#)

In parallelized mode, parameter values are not directly provided on the schedule, they are defined by run setups of the schedule.

[Send feedback](#)

A run setup is a set of parameter values, including a unique value for the special **run setup key** parameter. When you create a parallelized schedule, you designate one of the parameters as the run setup key, which is used to identify that run setup.

You can build run setups independently in parallel without waiting for other run setups to finish building. Foundry stores the output data of each successful build in the configured [union view outputs](#).

To manage the run setups of a schedule, select **Metrics & History** on an existing parallelized schedule, and navigate to the **Run setups** tab.

Build schedules

My schedule

My schedule

Parameterized

DATASETS TO BUILD

Builds the following only:

processed_towns

WHEN TO BUILD

This schedule will run:

When triggered manually

Parameter schema

2

123 risk_factor

55 town_name

Outputs

1

Backing target

Union view output

processed_t.../Example

processed_towns.../Example

Last modified a few seconds ago by John Doe

OVERVIEW

RUN SETUPS

RUN HISTORY

VERSIONS

HEALTH

Show runs from

Today at 3:58 PM

Show ignored runs

1

Run setups

Create run setup

Parameter values	Status	Schedule version	Recent run	
Denver + 1	---	---	---	Build now ...
Miami + 1	Running 0 of 1 jobs succeeded	bd62335f5c28	Today at 4:01 PM	Build now ...
Los Angeles + 1	Canceled 0 of 1 jobs succeeded	bd62335f5c28	Today at 4:01 PM	Build now ...
New Jersey + 1	Succeeded 1 of 1 jobs succeeded	bd62335f5c28	Today at 3:59 PM	Build now ...
Washington + 1	Succeeded 1 of 1 jobs succeeded	bd62335f5c28	Today at 4:00 PM	Build now ...

1. Choose **Create run setup**.
2. Fill in the parameter values for the run setup and provide a unique value for the run setup key parameter.
3. Choose **Save** to create the run setup.

Build a run setup

To build an existing run setup:

1. Locate the run setup in the **Run setups** list.
2. Choose **Build now** on the corresponding row.

If the build succeeds, the rows will be available in the configured [union view outputs](#).

Edit a run setup

To change the parameter values of an existing run setup:

1. Locate the run setup in the **Run setups** list.
2. Choose the (•••) menu icon on the corresponding row and choose **Edit**.
3. Update the parameter values as needed and choose **Save**.

The next time the run setup is built, it uses the updated parameter values. The old rows from the previous builds will remain in the configured union view outputs until the deployment is rebuilt.

Delete a run setup and its output data

To remove a run setup along with the rows it contributed to the union view outputs:

1. Locate the run setup in the **Run setups** list.
2. Choose the (•••) menu icon on the corresponding row and choose **Delete**.
3. Confirm the deletion.

When a run setup is deleted, its corresponding rows are removed from the configured [union view outputs](#).

API Reference ↗

Send feedback

Union view outputs

When the build of a parallel run setup succeeds, the contents of the output datasets will not be updated on the main schedule branch. Instead, the contents of the rows are collected inside an output union [view](#) that you configure on the schedule.

You can configure the union outputs while creating or editing the schedule. For each target dataset in the schedule, you can select a union view that will be used to store the resulting rows.

The screenshot shows the Data Lineage interface for configuring a parallelized schedule. The main canvas displays a workflow with two nodes: 'towns' and 'processed_towns'. The 'processed_towns' node is highlighted in green, indicating it is the target of the build. The right sidebar contains several configuration sections:

- My schedule:** Shows the schedule is 'Parallelized' and provides options to 'Cancel' or 'Save'.
- Schedule description:** A text area for describing the schedule.
- Target resources:** Lists the target resources, including 'processed_towns /Example'. It includes a 'Switch to Connecting Build' button and an 'Include upstream resources' toggle.
- Parameterization:** A section for configuring parameters, including a 'risk_factor' parameter and a 'town_name' parameter.
- Union view outputs:** A section for configuring union view outputs. It includes a table with columns 'Backing target' and 'Union view output'. The table shows 'processed_towns /Example' as the backing target and 'processed_towns_union /Example' as the union view output.
- When to build:** A section for configuring when the schedule should build. It includes options for 'When specific resource(s) update', 'At a specific time', 'When triggered manually' (selected), and 'When multiple time or event conditions are met'.

[API Reference ↗](#)

Use action types for parallelized schedules

[Send feedback](#)

Parallelized schedules are most useful when the run setups are built by an [action type](#).
[Learn more about creating an action type that runs a schedule.](#)

When you run a parallelized schedule through an action, Foundry automatically creates a new run setup with the parameter values provided by the action and invokes a build right away.

If there is already an existing run setup with the same value for the run setup key, then the parameter values are updated for this run setup before the build is invoked.

Limitations

The following limitations apply to all parameterized schedules:

- Parameter types and names defined in a schedule cannot be edited after the schedule has been created. You must create a new schedule to use new parameters.
- Parameterized schedules cannot be included as part of [Marketplace](#) products.

The following additional limitations apply to parallelized mode schedules:

- A schedule can have at most 1,000 run setups at a time.
- Collecting the results of parallel runs in an output union view is only supported for tabular Foundry datasets.
- Schedules in parallelized mode do not support defining automated triggers; they can only be run manually using [run setups](#) or executing [actions](#).

PREVIOUS

← [Add schedule to a Marketplace product](#)

NEXT

[Logic Flows \[Sunset\]](#) / [Overview](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[API Reference](#) ↗ ↗

[Terms of Use](#) ↗

[Send feedback](#)

Data connectivity & integration / Building pipelines / Logic Flows [Sunset] / Overview

Logic Flows [Sunset]

Logic Flows allow you to automate common workflows in Foundry. Logic Flows integrate with Foundry build and scheduling infrastructure to incorporate them into your data pipelines.

⚠ Sunset

As of August 2024, Logic Flows is in a [sunset](#) stage and should not be used for new development.

⚠ Sunset

Logic Flows is in the [sunset](#) phase of development and will be deprecated at a future date. Full support remains available. We recommend migrating your workflows to other applications and tools to serve your use case purposes:

- **AIP Logic:** Use the power of LLMs to automate and support critical tasks, including connecting key information from unstructured inputs to your Ontology, resolving scheduling conflicts, optimizing asset performance by finding the best allocation, reacting to disruptions in your supply chain, and more.
- **Actions:** Build automated transactions to change object properties based on the logic and criteria you define.

An **automation** is a script interacting with Foundry service APIs using Builds intended to replace manual repeatable actions in the platform. It acts like other jobs in Foundry, except it doesn't use datasets as inputs or outputs. Palantir maintains a curated library of automations.

Automations take resources as *parameters* and a JSON *configuration*.

Specific instances of automations are called **connected flows**. A connected flow is created with a project, parameters, and configuration.

Connected flows:

- Are defined and executed on a single [project](#)
- Run according to parameters and configuration, validated when a connected flow is created
- Can be run as part of a [schedule](#)
- Can be run manually through Logic Flows UI, Data Lineage or Builds application
- Do not store information between runs, therefore creating a new connected flow will not change the outcome of upcoming runs
- Cannot be edited
- Can be archived

Available automations

- [Compass Files Lister](#)

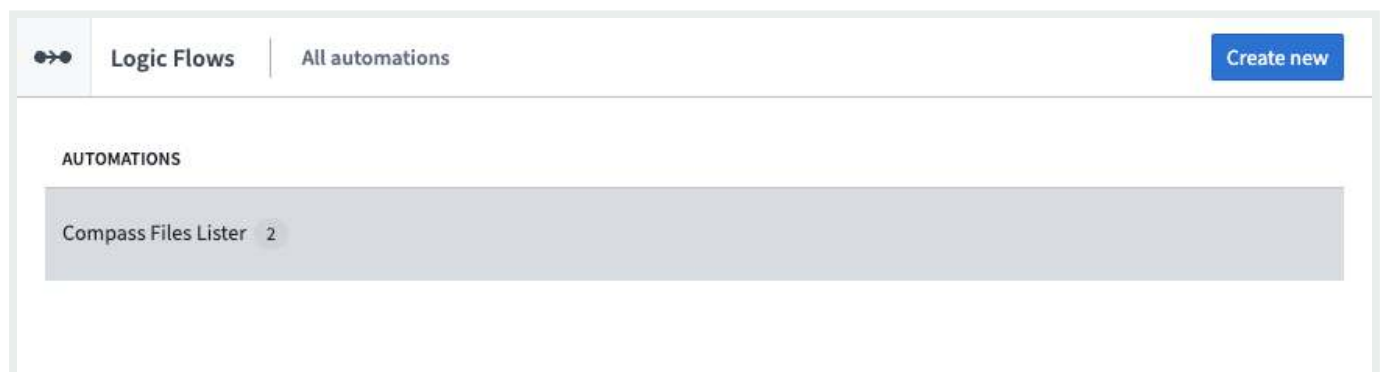
Data connectivity & integration / Building pipelines / Logic Flows [Sunset] /
Create a connected flow

Create a connected flow

In the Logic Flows App, you can:

- **Create new** connected flows (Top right corner)
- View existing connected flows, grouped by automation

To open the app, go to `/workspace/logic-flows`.



To create a connected flow in the [Compass Files Lister](#), follow these steps:

1. When creating a new connected flow, you will be presented with a dialog that allows you to set a name, project & required parameters and configuration. These are validated as it is being typed in or manually by clicking **Validate**.

For Compass Files lister automation, you are required to set input folder and output repository.

[API Reference ↗](#)



[Send feedback](#)

Connect a flow
✕

Compass Files Lister

Flow to list rids of resources in a folder into a file in a Stemma repository. Upon running, a new pull request will be opened on the repo where it either creates the file for the first time, or overrides the existing one. The generated file will be stored by default at the following path: `compass-listers/rids.json`. The base path can be overridden by setting the `generated\_file\_path` key in the configuration block. Example would be to set it to `transforms-python/generated` which will result in all runs of the connected flow to write & overwrite `transforms-python/generated/rids.json`. The resulting PR can be merged when ready if setting the `merge\_when\_ready` key to `true` in the configuration block.

Automation Name (required)

Project (required)

Foundry Training and Resources

ri.compass.main.folder.7454e939-1250-4d7e-acd2-595738005378

/Public/Foundry Training and Resources

Required resources (required)

Input folder

Folder to enumerate all rids underneath

datasets

ri.compass.main.folder.7bc638b7-2bfc-4e4f-ab7b-6980b3e8ce42

/Public/Foundry Training and Resources/Foundry Reference Project/Transform Project: Aviation Metrics/datasets

Output repository

Repository to produce list of rids within

logic

ri.stemma.main.repository.f0f18580-18a9-424d-a6d6-b8391abaa78d

/Public/Foundry Training and Resources/Foundry Reference Project/Logic Flows/logic

↓ Configuration

```

1 {
2   "generated_file_path": "transforms-python/generated/folder_clean",
3   "merge_when_ready": true
4 }

```

Ln: 3 Col: 27

i Logic flows are *project-scoped*, meaning you must specify the project where the connected flow resource will be created. Connected flows should be saved in the same project as any parameters used to connect the flow.

2. After saving the connected flow, it will appear in the list of connected flows.

API Reference ↗

Send feedback

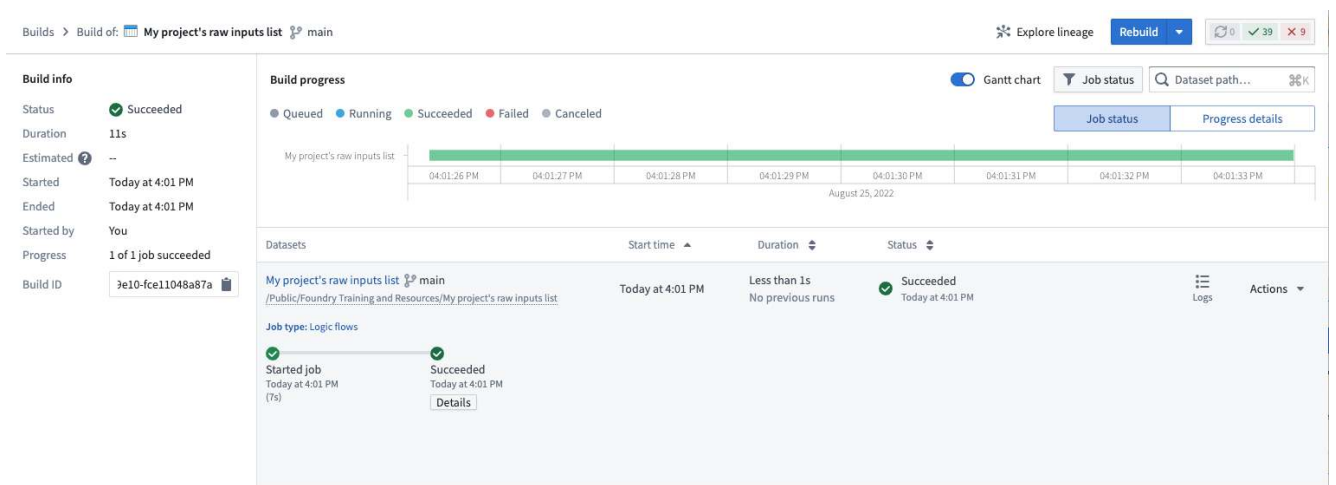
Logic Flows

All automations > Compass Files Lister

Create new

CONNECTED FLOWS	LAST RUN AT	STATUS	ACTIONS
My Compass Files Lister /demo/Sandbox/My Compass Files Lister	Fri, Jul 29, 2022, 3:55 PM	CONNECTED	Build
My project's raw inputs list /Public/Foundry Training and Resources/My project's raw inputs list	Never run before	CONNECTED	Build

3. To trigger the job, click **Build** in-line with the specific connected flow. A message will point you to the Builds application view where you can monitor the status of the job.



4. To check the results of your job, verify the pull request in the target repository view.

Foundry Trai... > Foundry Referen... > ... > **logic**

Excavator: List children rids of compass folder. **Merged**

Foundry created pull request into master from cfi/from-refs/heads/master/6431.383288301448-16614... 2 minutes ago

Foundry merged the pull request. 46 seconds ago

Overview **Files changed 1** Impact analysis Pipeline review Security changes Commits 1 Conversation

Showing 1 file change

NEW transforms-python/generated/folder_clean/rids.json

```

1 {
2   "childrenRids": [
3     "ri.foundry.main.dataset.8501383b-b4ab-42d3-9489-6fac675e0600",
4     "ri.foundry.main.dataset.7268fba8-824b-4ad4-867f-9156ad18b7b9",
5     "ri.foundry.main.dataset.bcf96fb-c769-4b45-ba90-7532f18a963c",
6     "ri.foundry.main.dataset.8be87be9-caed-44fd-90bc-b20e0594ede9"
7   ]
8 }

```

5. By heading back to Logic Flows application, you can see your connected flow, its **API Reference** configuration, and create a schedule from there.

Send feedback

Logic Flows

All automations > Compass Files Lister > My project's raw inputs list

Create new

Connected flow properties

FLOW

My project's raw inputs list

/Public/Foundry Training and Resources/My project's raw inputs list

STATUS

CONNECTED

FLOW ID

ri.logic-flows.main.connected-flow.7974bfb8-4523-483d-b4e8-586f

PROJECT

Foundry Training and Resources

/Public/Foundry Training and Resources

Parameters

Input folder

Folder to enumerate all rids underneath

datasets

ri.compass.main.folder.7bc638b7-2bfc-4e4f-ab7b-6980b3e8ce42

/Public/Foundry Training and Resources/Foundry Reference Project/Transform Project: Aviation Metrics/datasets

Output repository

Repository to produce list of rids within

logic

ri.stemma.main.repository.f0f18580-18a9-424d-a6d6-b8391abaa78d

/Public/Foundry Training and Resources/Foundry Reference Project/Logic Flows/logic

Configuration

```
{
  "generated_file_path": "transforms-python/generated/folder_clean",
  "merge_when_ready": true
}
```

Schedule

No schedules

Data connectivity & integration / Building pipelines / Logic Flows [Sunset] /
Compass File Lister

Compass File Lister

Compass file lister is an automation that lists rids (resource identifiers) of resources in a given **input folder** into a Code repository. When run, a new pull request is opened on the **output repository** where it will either create the file for the first time, or override the existing one. The generated file will be stored by default at the following path: `compass-lister/rids.json`. Review [Create a connected flow](#) for a step-by-step guide.

Configuration options

- You can override the base path in output repository by setting the `generated_file_path` in the configuration block. If you set it to `transforms-python/generated`, the output will be written to `transforms-python/generated/rids.json`.
- The resulting PR can be allowed to merge automatically if `merge_when_ready` is set to `true` in the configuration block. See your output repository settings to review the conditions that allow the PR to be merged.

Configuration example

```
{  
  "generated_file_path": "transforms-python/generated",  
  "merge_when_ready": true  
}
```

[API Reference ↗](#)

PREVIOUS

[Send feedback](#)

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Building pipelines / Best practices / Recommended Project and team structure

Recommended project and team structure

This page outlines the recommended ways to structure a [data pipeline](#) in Foundry using [projects](#) to enable:

- Well-structured permissions across the pipeline
- Effective collaboration across roles
- A legible and maintainable pipeline structure

Additionally, we cover the range of [roles and responsibilities](#) that need to be fulfilled for successful pipeline management in production environments.

This provides a conceptual overview of the various pipeline stages and what each is for. For a step-by-step guide for how to set up this structure in Foundry, refer to [Securing a data foundation](#) in the Platform Security documentation.

Pipeline stages

These stages define the logical separation of projects that compose a well-ordered pipeline. We'll go into each in more detail below.

1. [Data Connection](#): Syncs land raw data from source systems into a **Datasource project**.

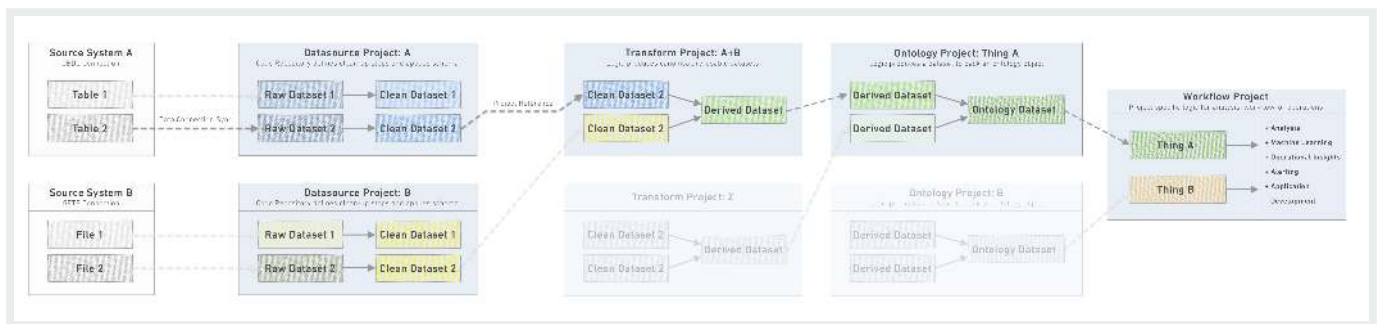
2. [Data Ingestion](#): One **Datasource project** per logical datasource defines the basic structure of each raw dataset and applies a consistent schema.

3. [Data Refinement](#): Datasets are imported from one or more **Datasource projects** and transformed to produce canonical, re-usable datasets.

API Reference ↗

Send feedback

4. Ontology project: Datasets are imported from one or more **Transform projects** and transformed to produce the canonical tables representing discrete operational objects. A single Ontology project often groups related sets of objects for a given operational domain for ease of management.
5. Workflow project: Workflow projects import data from Ontology projects to pursue a specific outcome. Frequently this is an operational workflow, data science investigation, business intelligence analysis and report, or application development project.



Each pipeline stage is a discrete unit, and the outputs are the datasets made available for downstream projects to import and use for other use cases, pipeline development, analysis, and so on. Within each project, the responsible team should, in addition to the implementation of the transformation steps, also manage the stability and integrity of their outputs. This involves managing the build schedules, configuring and monitoring data health checks, and, where relevant writing unit tests or additional data integrity tests.

The sections below will walk through each downstream project type in more detail to follow the flow of data through the pipeline. In the process of designing a pipeline, instead work backwards from the Ontology layer to determine the necessary source systems to connect and pipeline transformations to implement.

i Projects provide a way of organizing data and code within Foundry. They are the primary unit to manage access and permissions for individuals and teams collaborating around a specific scope of work. In addition, projects expose functionality for capturing and sharing metadata such as an event log of project activity, a space for project documentation, and project resource consumption and computation metrics.

When thinking about what makes a good project, consider as an analogy what

API Reference ↗

microservice: well defined purpose, clear output datasets/API used by downstream dependencies, ownership by a set of people small enough that they can effectively coordinate with each other and set their own standards for management.

Send feedback

When deciding whether to create a new project or build in an existing one, consider:

- Projects should be 1:1 with Code Repositories. Larger scope enables deeper collaboration, but introduces more challenging operational requirements to ensure frictionless version control.
- Projects are the level at which you want to impose a file structure and permissions for all the related resources. If a single conceptual "project" has sub-components that will require individual permissions, consider creating multiple projects in Foundry.

1. Data Connection

The data flowing through a pipeline in Foundry normally originates from an external source system. The [Data Connection](#) service provides a management interface where source systems are registered. For each source system, one or more individual syncs are configured and data is loaded into **raw** datasets in a **Datasource project**.

In order to configure a source system or sync in the first place, an **Agent Administrator** typically needs to configure a Data Connection [agent](#). Each agent should be stored in a dedicated project only accessible to **Agent Administrators**.

Managing the new connection of a new datasource is a multi-disciplinary effort requiring the collaboration of the **Datasource Owner**, the **Agent Administrator**, and the **Datasource Developer**, though it is often necessary for a **Compliance/Legal Officer** to sign off on the movement of data out of the source system into the new environment.

Once a source system is configured, however, the Datasource Developer is able to work independently to configure individual syncs. This greatly reduces the back-and-forth between teams and ensures connecting a new system is a low, one-off effort.

A critical consideration for each connection is choosing between a **[SNAPSHOT](#)**, which replaces the current data with each sync, and an **[APPEND](#)**, which adds cumulatively to the existing data with each sync. To understand these concepts more fully and the implications for building efficient and performant pipelines in the [Incremental pipelines](#) section.

[API Reference](#) ↗

Further reading

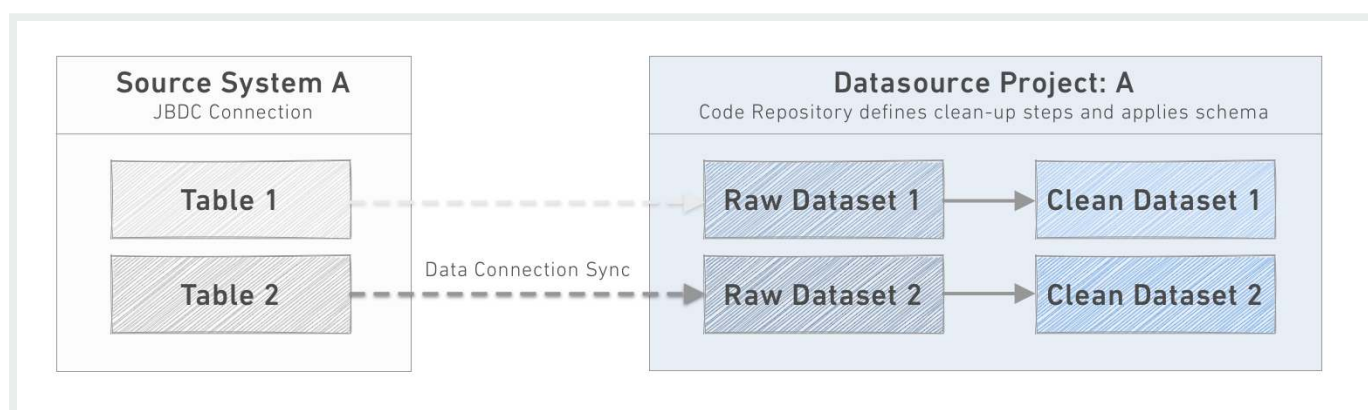
Send feedback

For more on principles, architecture, and configuration of datasources and syncs, visit the [Data Connection documentation](#).

For more thoughts on monitoring your connections and ensuring the data flowing into your pipelines is accurate and within expectations, review the [Maintaining pipelines](#) section.

2. Datasource project

Each source system should land data into a matching project inside the platform. The normal pattern involves landing data from each sync in as 'raw' a format as possible. The transformation step to a “clean” dataset is defined in the project *Code Repository*.



Establishing this project-per-datasource model has a number of convenient benefits:

- Encourages exploration of new data sources in a consistent location.
- Provides a single location to specify Access Controls to a data source.
- Minimizes the chance of duplicated effort syncing the same data by different teams.
- Code Repositories are small and purpose-built to handle the type and format of source system data.
- Allows anonymization, clean up, and standardization of data before it is accessed by the pipeline layer.

Datasource project goals

Inside each datasource project, the goal is to prepare the synced data for consumption by a wide range of use cases, and pipelines across the organization.

[API Reference ↗](#)

The output datasets, and their schema, should be thought of as an API. It should be for them to be logically organized and stable over time, as the output datasets from a

[Send feedback](#)

data source project are the backbone of all downstream work. The outputs should map 1:1 with the source tables or files; each row in the dataset should have a direct matching row in the source system.

To this end, some typical objectives of the raw → clean transformation include:

- Imposing a consistent column naming scheme.
- Ensuring appropriate types are applied to columns.
- Handling missing, malformed, or mis-typed values.
- Establishing primary [Health checks](#).
- Removing [PII \(Personally Identifiable Information\)](#) or other sensitive data unsuitable for general consumption.

Even where the source system provide column data type information, it is sometimes necessary to bring in values as a `STRING` type. Pay special attention to `DATE`, `DATETIME`, and `TIMESTAMP` types, which are often represented in source systems in non-standard formats and numeric types, which are occasionally troublesome. If these types do prove to be unreliable in format from the source system, importing as a `STRING` type provides the option to implement more robust parsing and define logic for handling malformed values or dropping unusable or duplicated rows.

In addition to these programmatic steps, clean datasets should be rigorously documented. A qualitative description of the dataset, the source system provenance, the appropriate contacts and management protocol and, where relevant, per-column metadata describing features of the data will all ensure future developers use the data appropriately.

Implementing a Datasource project

1. A system administrator creates a new project for the target datasource and adds the appropriate **Datasource Developers** to a new [user group](#) to manage the project.
 - Only members of the datasource developers group for the specific datasource should have permissions to build datasets within the project.
2. The **Datasource Owner** and **Agent Administrator** collaborate to [deploy a Data Connection agent](#) on the source system and configure the datasource connection in the

API Reference ↗

Appropriate permissions are configured so that tables and files in the source system are able to be previewed by the datasource developers

Send feedback

3. The **Datasource Developers** [configure the syncs](#) for one or more raw datasets in the project, considering the correct sync cadence and sync type, as well as adding appropriate Data Health checks for each raw dataset.
4. The **Datasource Developers** implement the cleanup transformations to produce “clean” datasets ready for use in other projects.
5. Downstream developers request new datasets from the source system, enhancements to the clean datasets, or data quality issues by filing an **Issue** with the project or a specific clean dataset.

While every datasource will be somewhat unique, the steps of cleaning and preparing a source system often have shared steps, such as parsing raw string values to a given type and handling errors. When datasources have a number of similar cleanup transforms, it's best practice to define a library in [Python](#) to provide a set of consistent tooling and reduce duplicated code.

Recommended folder structure

- `/raw` : For datasets from a data connection sync.
- `/processed` (optional): File-level transformations to create tabular data from non-tabular files.
- `/clean` : For datasets that are 1:1 with datasets in `/raw` but with cleaning steps applied.
- `/analysis` : For Resources created to test or document the cleanup transforms and show the shape of the data.
- `/scratchpad` : Any temporary resources created in the process of building or testing the cleanup transforms.
- `/documentation` : The [Data Lineage](#) graphs that show the pipeline steps for cleanup as well as additional documentation (beyond the top-level project README) written in [Foundry Notepad](#).

3. Transform project

The goal of Transform projects is to encapsulate a shared, semantically meaningful grouping of data and produce canonical views to feed into the Ontology layer.

Transform projects use the cleaned datasets from one or more Datasource projects, join them with lookup datasets to expand values, normalize or de-normalize data.

[API Reference ↗](#)

[Send feedback](#)

create object-centric or time-centric datasets, or aggregate data to create standard, shared metrics.

Recommended folder structure

- `/data`
 - `/transformed` (optional): These datasets are output from intermediate steps in the transform project.
 - `/output` : These datasets are the "output" of the transform project and are the only datasets that should be relied on in downstream Ontology projects.
- `/analysis` : Resources created to test or document the cleanup transforms and show the shape of the data.
- `/scratchpad` : Any temporary resources created in the process of building or testing the cleanup transforms.
- `/documentation` : The [Data Lineage](#) graphs that show the pipeline steps for cleanup as well as additional documentation (beyond the top-level project README) written in Notepad.

4. Ontology project

The Ontology enforces a shared communication layer across Foundry, and the curation of a clean, organized, and stable Ontology is the highest order objective to ensure a wide-range of valuable projects can move forward simultaneously while enriching the common operating picture. For more on the concepts and practicalities of designing and managing your Ontology, reference the [Ontology concepts documentation](#).

An Ontology project is the center-point of any pipeline and represents the final transformation necessary to produce datasets that conform to the definition of a single or related-group of objects defined in the Ontology. These projects also (and separately) produce datasets that are synced to back the Object Explorer views.

Since the Ontology datasets represent the canonical truth about the operational object they represent, they form the starting point for all “consuming” projects. While the provenance and transformation logic of upstream cleaning and pipeline steps are visible, conceptually

the intermediate datasets could be a black box for the project developers, analysts, data scientists, and operational users who consume data on these projects. In this sense, the Ontology layer serves as an API for operational

[API Reference](#) ↗

[Send feedback](#)

Implementing an Ontology project

Similar to Datasource projects, essential factors in maintaining an Ontology project are:

- Robust documentation
- Meaningful health checks
- Regular schedules
- Curation in the [Data Catalog](#) and appropriate tags

These are discussed in more detail in the [Maintaining pipelines](#) and in the callout on documenting projects below.

In addition, as your Ontology grows more robust and more teams contribute to the Ontology layer, maintaining the integrity of the dataset "APIs" becomes more critical. To this end, consider implementing additional checks to ensure that proposed changes preserve the integrity of the output dataset schema. This will be discussed in more depth in additional documentation available soon.

Recommended folder structure

- `/data`
 - `/transformed` (optional): These datasets are output from intermediate steps in the Ontology project.
 - `/ontology` : These datasets are the "output" of the Ontology project and are the only datasets that should be relied on in downstream use case or workflow projects.
- `/analysis` : Resources created to test or document the cleanup transforms and show the shape of the data.
- `/scratchpad` : Any temporary resources created in the process of building or testing the cleanup transforms.
- `/documentation` : The Data Lineage graphs that show the pipeline steps for cleanup as well as additional documentation (beyond the top-level project README) written in reports.

5 Workflow project

API Reference ↗

Workflow projects, also known as **use case projects**, should be flexible in the context at hand, but usually are built around a single project or a team to an effective unit of

Send feedback

collaboration and delineate the boundaries of responsibility and access.

In general, workflow projects should reference data from the Ontology projects, to ensure that operational workflows, business intelligence analysis, and application projects all share a common view of the world. If, in the course of developing a workflow project, the datasources available in the Ontology layer aren't sufficient as sources, it's an indication that the Ontology should be enriched and expanded. Avoid referencing data from earlier (or later) in the pipeline as this can fragment the source of truth for a particular type of data.

i Document projects

Each project should be documented thoroughly throughout the development process. Here are a few common patterns and best practices:

- Save one or more Data Lineage graph resources at the root of each project to capture the most important datasets and pipeline relationships. Different graphs can use different color schemes to highlight relevant features, such as the regularity of data refresh, the grouping of related datasets, or the dataset characteristics, such as size.
- Add a short description to the project - available directly below the project title. This will be used in the list view of all projects. Focus on putting the project into context for users who may be unfamiliar with it.
- Use the **Add Documentation** button in the right sidebar of the top-level project folder to write [Markdown ↗](#) syntax. This is the best place to explain the purpose of the project, the primary outputs, and any other relevant information for project consumers.
- Place further global project documentation in Markdown (.md) files created locally and uploaded to a **Documentation** folder or in [Notepad](#) and link to these from the top level documentation.
- Add a README.md file to the Code Repository for your project and ensure that each transform file has a description and additional inline code comments.

Recommended folder structure

T

API Reference ↗

 Workflow projects will be more varied than other project types and should focus on making the primary resource(s) immediately accessible and well-documented.

Send feedback

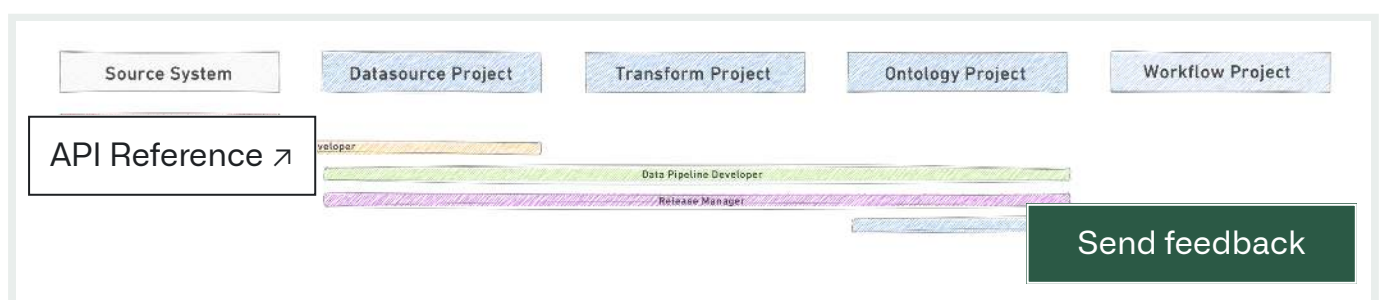
- /data

- `/transformed` (optional): These datasets are output from intermediate steps in the workflow project.
- `/analysis` (optional): These datasets are output from analysis workflows.
- `/model_output` (optional): These datasets are outputs from model workflows and can then be analyzed to determine model fit.
- `/user_data` (optional): If your workflow enables users to create their own "slice" of data, store them here in user-specific folders.
- `/analysis` : Resources and reports created to drive decisions and feedback loops.
- `/models` : Any models created should be stored here.
 - `/templates` : Any [Code Workbook templates](#) shared for project-specific usage should be stored.
- `/applications` : Additional workflow applications or sub-applications are stored here. A primary application should be stored at the project root for prominent access.
 - `/develop` : Applications in development, new features, and templates are stored here.
- `/scratchpad` : Any temporary resources created in the process of building or testing the cleanup transforms.
- `/documentation` : The Data Lineage graphs that show the pipeline steps for cleanup as well as additional documentation (beyond the top-level project README) written in reports.

Pipeline management roles

The roles below are examples of the profiles commonly involved in the scoping, design, implementation, and management of pipelines in Foundry. Not all roles may be necessary in all circumstances, especially in the early stages of Platform development, and individuals might fill more than one role at a time. However, generally considering these role descriptions and how they interact will help create well ordered, effective teams.

The diagram below relates the primary roles to the segments of the pipeline where they're most commonly active.



Primary roles

- **Agent Administrator:** The Agent Administrator is responsible for creation and configuration of a Data Connection *agent* and sharing it with the relevant **datasource owner**. This centralizes control of agent installation and configuration for better security and control.
- **Datasource Developer:** Data source developers are engineers who own the sourcing process for each of the data sources relevant to the data lake. This can be done as a single team or broken out by source systems.
 - Each datasource developer owns (1) a Data Connection data source with the technical connection to the data source and (2) a Foundry project to which the synced data is sourced.
 - The output of the datasource project is a cleaned and prepared dataset, ready for consumption by one or more downstream use case projects. Datasource developers also define the refresh frequency of each data source they own and work with the **Release Manager** or **Operation Manager** to schedule a *pipeline build policy*.
- **Data Pipeline Developer:** Data pipeline developers are data engineers responsible for building a common Ontology layer, which provide data assets for use by all use cases. The data pipeline developers are responsible for defining this Ontology layer and building the transformations needed from the source structure to the Ontology structure.
- **Release Manager:** Release managers own the production pipeline and manage the release process to ensure the production pipeline adheres to the organization's standards and executes reliably.
 - For code changes to the pipeline, this is usually done through the approval process of pull-requests into master and approval to run the pipeline build on master. For ongoing reliability, the Release Manager monitors the *Health Checks* defined in conjunction with the pipeline and datasource developers for key datasets.
- **Compliance Officer / Legal Officer:** A compliance officer is responsible for approving use cases before they are developed. Such approval should take into account: (1) the data that needs to be used, (2) the purpose of the project, and (3) the users who will have access to the data in light of the data protection regime for their jurisdictions and the organization's own policies.
- **Use-Case Developer:** Use-case developers are the engineers - software developers and/or data scientists - who own the development of the use case. They use data which

API Reference ↗

the Ontology layer and approved to them by the compliance officer.

Send feedback

Other roles

- **Operations Manager:** This role owns the monitoring of the healthy execution of the production pipeline. They monitor data refresh of the different sources, execution of the build of different pipeline and help triage support requests. This role supports the **Release Manager**.
- **Permissions Owner / Identity Manager:** In most cases the assignment of users into groups is managed in the customers identity management system and the information is passed into Foundry through SAML integration. In such cases the permission manager owns that process.
- **Data Officer:** Responsible for modeling the Ontology layer into a user-centric view of the world, owning data quality, and hunting for new data sources which should be integrated to expand the Ontology layer. The data officer is also responsible for monitoring enrichments done in the use cases and generalizing them into the Ontology layer for better reusability.
- **Project Manager:** A project manager works with the engineers contributing to a project to ensure the alignment of technical and business goals, the adherence to best practice standards for development on the platform, and coordination between other project teams. The project manager is also responsible for documenting the project.
- **Datasource Owner:** This is the primary point of contact for the particular source system. They will need to work closely with the **Agent Administrator** during the configuration of a new data connection, and afterwards assume some level of responsibility for “upstream” issues - either with the agent service running on their server or file system or with the integrity of the data itself.

PREVIOUS

← [Logic Flows \[Sunset\] / Compass File Lister](#)

NEXT

[Development best practices](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Building pipelines / Best practices /
Development best practices

Development best practices

This guide is meant to provide guidance for pipeline developers who are developing transformations. The **General** recommendations may also be of interest to project managers or platform administrators as they focus on higher-order principles of clean pipelines in general.

General best practices

Pipeline development is software development

Many of the best practices from general software development apply equally to defining the transformations that make up your data pipeline. Below are a few common practices that exemplify this approach:

- **Reduce cognitive load:** Wherever possible, reduce the amount of thinking necessary. If you need to use complex functions or API calls that are non-obvious, make sure you have clear documentation.
- **Be nice to future you:** Pipelines are the foundation of your data enterprise. Think long-term and high vision while planning and implementing.
- **Don't Repeat Yourself (DRY):** Duplicated code or concepts require more maintenance and lead to subtle errors. Instead, refactor frequently at all levels, including building and publishing your own libraries or packages for cross-repository use, to ensure minimum duplication.
- **Avoid tech debt:** A corollary to long-term thinking, tech debt is easy to accrue in the face of object-specific demands, but always comes back to bite.

API Reference ↗



Consistency matters: Set a precedent and stick with it; this reduces the friction and inconsistency between Code Repositories. For instance, column names should be consistently written in `snake_case` - following this convention means anyone else who

Send feedback

wants to reference your awesome dataset knows that it's `awesome_dataset`, not `AwesomeDataset` or `Awesome_Dataset`.

- **Less is more:** Systems with many smaller units are easier to maintain and reason about than systems with few large units; therefore bias towards:
 - Tightly-scoped Foundry Projects
 - Smaller transforms chained together
 - Short functions - and helper functions in transforms

Anti-patterns

- **Overwrite vs. Delete:** Unlike other file systems, Foundry resources and datasets are connected to other artifacts rather than just the file itself. Hence, if you are iterating on a pipeline or data, deletion should occur only if you fundamentally want that type of data gone. For example, if you've written a transaction with incorrect data to a dataset, don't delete the dataset; instead, write a new `SNAPSHOT` transaction to overwrite the previous one. More challenges are likely to arise from trying to delete a dataset than from creating a new dataset in the same location.
- **Don't introduce circular dependencies:** If you are looking to use the dataset output of your transform as an input into other transforms, you should ensure you are not introducing any circular dependencies in your code. Foundry's build orchestration layer will attempt to prevent you from configuring any loops on the branch you are currently on and your Code Repository branch will fail checks if a loop is detected

i Foundry will check for circular dependencies on the branch being developed on, but will not run the check across all branches while writing code, such as if there are ontology writeback datasets that only exist on the master branch. Foundry will still fail checks if circular dependencies are detected on other branches when attempting to merge the feature branch with the branch that contains the circular dependencies.

Project folder structure

API Reference ↗

See [Recommended Project structure](#) for a description of an overarching model to organize the entire flow of data through multiple Projects.

Send feedback

Best practices

- **Manage permissions at the Project level:** If you anticipate needing to further partition permissions within a project, consider if the project should be further decomposed into smaller pieces. [Learn more about the concepts and practices of applying and managing permissions.](#)
- **Keep Projects tightly scoped:** Avoid “feature creep” and adding tangentially-related resources or use-case specific logic within your Project
- **Use meaningful folder names:** Remember that you're designing a Project for consumption outside your team as well as for daily iteration and development. Consider that at minimum you should have a `/Documentation` folder and a `/Output` folder. Different use-case or workflow Projects will have needs for more specific names, but always consider that your Project structure and naming scheme are signposts for visitors.

Anti-patterns

- **Working in your home folder:** Your personal folder is not a place for development – the access controls are strict and sharing is difficult. Consider creating a `/scratch` folder in your Project for experiments or throw-away work rather than building in your home folder.

Naming

Best practices

- **Follow convention:** Name columns, datasets, repositories, files, and other resources following the common patterns for your organization or the convention for your Foundry instance.
- **Choose descriptive names:** Take the time to come up with two or three word names that orient a reader to the purpose or content of the resource being named. Consider putting the distinctive portion of the name first; this can help with long names where the full text is cut off in a dropdown and helps immediately distinguish references when scanning a

API Reference ↗

 of files, for instance. Avoid using abbreviations.

Send feedback

Anti-patterns

- **Cryptic names:** Choosing names that simply increment a number (i.e. `dataset1`, `dataset2`, and so on) or names that are a single letter will make it more difficult to read and refactor your code and much more difficult for a new developer to approach your work.
- **Names distinguished only by path:** In some cases this is acceptable, for instance when you have `/raw/my_important_dataset` and then `/clean/my_important_dataset`, however in many cases this naming pattern can create confusion in views where only the dataset name itself is prominently displayed. Remember, provenance is tracked and easily visible, so you don't need to embed this kind of "state" into the names of your datasets.

Data types

Best practices

Explicitly cast column types: If you are working in a **Datasource Project**, explicitly cast the column types in the `raw` → `clean` transform, even if the schema inference from the data connection has chosen correct values. This will help catch breaking changes from the source system if a column type changes or an invalid value creates an incorrect inference during the sync.

Patterns

Use Timestamps for 'Time Only' data types: Spark doesn't have a time-only data type for fields with values like "10:59:00". In order to leverage the time functions that come with Spark's timestamp type, cast the values to seconds and then add `-2208988800` to it before casting it to a timestamp in order to put it in year 0. Alternatively, leave it as a string and let the users parse it as they need to.

Anti-patterns

API Reference ↗

Cast string fields to numeric datatypes: Consider a column of aircraft IDs, like `545`, `972`, `314`. It can be tempting to cast these to an integer column.

Send feedback

like integers and may even be integers in the source system). However, this has significant drawbacks:

- If leading zeros are meaningful, e.g. 123 and 0123 are both legal IDs that should be differentiated, this will result in errors.
- The IDs may be formatted in an undesirable way in the UI (e.g. 545.0, 1,234, right justified).
- If the IDs are too long, you can run into MAX_INT issues.
- Numeric functions won't ever be applied to these values (e.g. you won't ever add two aircraft IDs together). *The rule of thumb should be to only cast numeric fields to numeric datatypes if it would be appropriate to perform arithmetic on them, otherwise cast them as strings.*
- **Storing timestamps in different timezones:** — Spark timestamps are timezone agnostic. They are stored internally in UTC (aka Zulu, GMT); displaying a timezone is expected to be done by the front end.
 - If you need a particular timezone for display purposes, use [PySpark's from_unixtime\(\) function ↗](#) to store a string in the appropriate timezone.
 - If timestamps in a given datasource are in a timezone other than UTC, use [PySpark's to_utc_timestamp\(\) function ↗](#) to normalize them to UTC.

Code comments

Anti-patterns

- **Commenting out code in commits:** When making changes, it can be tempting to comment out old code and leave it for reference or in case you need to revert later. You can use comments while iterating, but do not commit code with statements commented out. Doing so builds cruft and reduces legibility. Old code is easy to find in previous commits.
- **Comments with authorship details and dates:** These are automatically tracked in the commits/git repository. Manually commenting them is prone to not being updated and creates cruft.
- **Over-verbose Commenting:** Comments should share the *rationale behind decisions* in the logic itself. Strive to write “self-documenting” code; if a set of statements is difficult to understand, that is a clear sign to refactor and simplify.

API Reference ↗

Send feedback

Repository practices

Best practices

- **Protect the master branch:** If you're developing with a team, or even just working on a long-lived individual project, protect the master branch and practice [GitFlow ↗](#) or your preferred development workflow. The key concepts are simply to ensure that code moving to master is reviewed and tested.
- **Write commit messages:** Commit messages are the log of all activity in the repository. Take the time to write a useful description of your change:
 - Code reviewers can look at your commits to get a sense of your workflow and what changes were made.
 - If you want to revert a change, it's much easier if you can find the commit you were looking for at a glance.
 - Note that clicking the build button will autogenerate a commit message with a timestamp; avoid using this. Click commit first, write a commit message, and then click build.
- **Prune your branches:** In long-lived repositories, branches can accumulate. If development of a branch is abandoned, especially if a branch is merged into another, keep things tidy by *deleting the branch*. This helps with legibility of which branches are actively developed.
- **Upgrade your Repository:** When prompted, follow the steps to upgrade the language bundles in your repository. This process will open a pull request to the active branch containing the upgrades. You should feel free to run a build of your pipeline on the upgrade branch to ensure that none of the version bumps impact your code. However, staying up-to-date with these upgrades often ensures you do not encounter edge cases seen elsewhere, which are patched in the upgraded versions.
- **Practice Code Reviews:** As you collaborate with teammates to develop transformations, implement some practice of code reviews during the pull request process. We've shared our thoughts on [Code Review Best Practices ↗](#) and many of the concepts will apply equally to reviewing data transformation code.

F API Reference ↗

Share code between repositories: Repositories in Foundry operate for a variety of reasons, but often there is logic that could be reused across pipelines, which has

Send feedback

several advantages:

- General code reuse in accordance with DRY.
- Avoiding forked/inconsistent logic across different areas of the data foundation.
- There may be pipelines which would ideally use foundational pipelines but which have much stricter SLAs or performance requirements. In this case, the solution is often to share the logic but not the transforms/datasets so that the critical pipeline can rely on pre-filtered datasets, fewer transforms, have different build schedules, and so on.
- Code Repositories are an excellent way to accomplish this. For example, when working with Python transforms, libraries can publish themselves to a Conda channel and allow other repositories to consume them. See the documentation on [Sharing Python Libraries](#).
- An additional advantage of shared repositories is *semantic versioning*. The shared repository can tag its commits with versions (e.g. 1.0.0, 1.0.1, 2.0.0) and the consuming libraries can choose how they pick up new versions. For example, a repository might choose to take the latest version (2.0.0 above) or to only take a specific version (say, 1.0.1) and defer picking up new versions until they manually decide to. The latter case is particularly valuable when the pipeline is critical and the pipeline owners would like a chance to opt into/approve changes to the shared repository.
- **Releases:** Along the same lines, if a team wants to have an explicit release schedule for pipelines, one option (which avoids staging instances or long-lived develop branches) is to factor the logic out into functions in a shared repository and use the semantic version to keep the consuming repository at major releases, such as 1.0.0, 2.0.0, or even .0.0. That way, the developers can continue to iterate on the logic and tag intermediate releases without them going live on master. Moreover, on a branch on the consuming repository, the developers can always pick up intermediate versions as long as they do not merge them to master before the release date.

Unit testing

Unit testing is a popular way of improving and maintaining code quality. In [unit testing 1](#), small and discrete components ("units") of software are tested in an individual, independent, and automated fashion.

API Reference ↗

: You can enable `pytest` unit tests as part of the CI checks for your Python transforms repository by following the [Python unit test ins](#)

Send feedback

- **Java unit tests:** The steps for configuring unit tests for Java transforms can be found [in the Java unit tests documentation](#).

Best practices

Unit tests should:

- Only test one piece of logic at a time;
- Be lightweight (see the language-specific instructions for tips on how to reduce memory usage);
- Not rely on extra inputs to function; and
- Not rely on or call each other.

Health checks

Best practices

Check your data health: Often, once a portion of a pipeline is completed, it is easy to set the schedule and put it out of mind. However, even if your logic is sound, the incoming data can change in ways that affect your build, leading to slower performance, increased data scale, or outright build failure. Configuring basic checks on dataset size and build time, even if you do not configure alerts, will provide a view over time of these key metrics so you can observe, for instance, the rate of increase in dataset size or the average build time for the dataset. Read more about the specific [health checks](#) available and how to configure them.

Patterns

Extend health checks: In most cases, the default health check configurations should be sufficient. If you need further flexibility, however, consider adding one or more derived `health check` datasets to your pipeline. The transform for this dataset can perform a `validate` operation to determine the validity of its input dataset (the dataset you are validating), and the output can be formatted so that a simple health check, like `Allowed Value`, can report if the dataset is valid.

API Reference ↗

Send feedback

Set a schedule for this dataset to build whenever there is an update on the input dataset and you will have an extra set of comprehensive health checks.

Scheduling

Best practices

Review the [scheduling best practices](#).

PREVIOUS

← Recommended Project and team structure

Branching and release process

NEXT →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

API Reference ↗

Send feedback

Data connectivity & integration / Building pipelines / Best practices / Branching and release process

Branching and release process

The purpose of this document is to outline both the recommended best practices for branch management and the release process of data pipelines managed through Projects within Code Repositories.

The high level goal is to design a process to strike a balance between quick iteration on new data features / data quality fixes and having a stable and process-compliant change management process.

Scope of release

Before we jump into the internals of the release management process, we need to better define exactly "what is the product" that we are releasing? For our purposes we'll define the **product** to be *the set of assets released as a conceptual unit*.

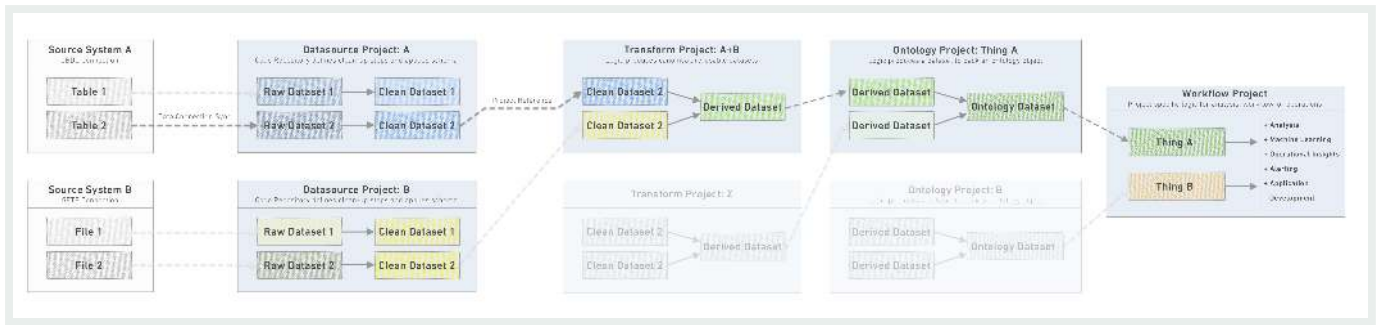
Taking [Recommended Project structure](#) as a reference for how we build pipelines in Foundry, we see that the pipeline is defined by multiple projects of different types - datasource, transforms, Ontology and workflow. In most cases, however, we will not manage each project on its own as a product, but instead, each project will define a subset of its resources as one of the following product types:

- The Ontology product: this product is the collection of output datasets created out of the datasource, pipeline and Ontology projects.
- A Use-case product: every use case project represents a given use case that stands on its own. The use cases is defined as a product.

API Reference ↗



Send feedback



i The definition we give to a product here can cover code which is managed in multiple repositories and other Foundry resources.

Product branch management

The following section refers to the branching strategy we want to deploy to manage the release of every product. [Learn more about branching in general and how branches work between Code Repositories and datasets.](#)

⚠ Branch naming

It is highly recommended to have the same branch name across all repositories within the product. This will ensure downstream branches read from the correct upstream branch.

The general branching strategy we recommend following is based on a common practice named [GitFlow](#) with some modification. The main change is a recommendation to skip the `release` branch. Due to the way artifacts are deployed in Foundry, we can merge changes from `dev` into `master` directly after testing.

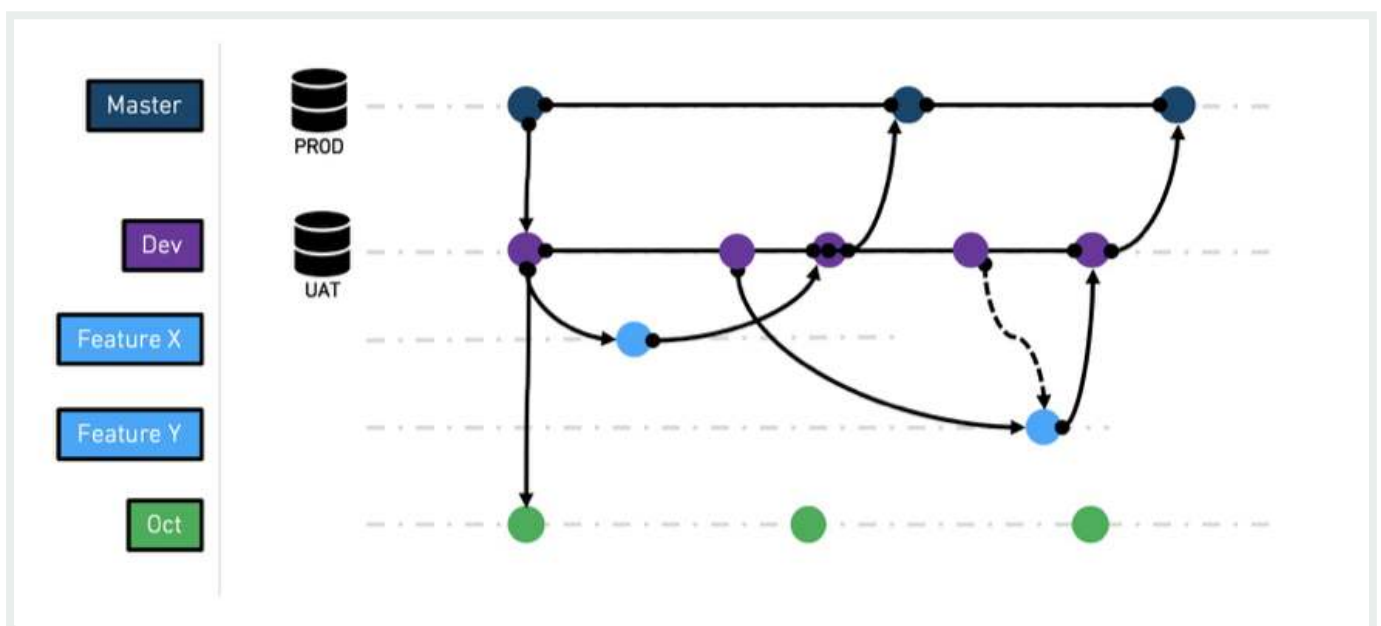
Common branches

`master` - this is the production branch. As such, it is a protected branch and only the `API Reference` can approve merging a PR to this branch. It is assumed that the `master` branch is sourced with production data.

Send feedback

dev - the **dev** (aka development) branch is a staging branch, derived from the **master** branch. It is mostly relevant when you want to test a complete feature using testing data. The **dev** branch can be sourced from **master** (this happens automatically through fallback branches) or from a UAT data source.

feature-[X] - feature branch is where a given feature is being developed and tested. This branch is derived from **dev** branch. This is a short-lived branch, meaning it can be deleted once the branch is being merged into master. Since the branch is derived from **dev** it will be sourced with the same data **dev** branch has. Note that this remains true as long as the fallback branches don't get reconfigured, and as long as the input datasets don't exist on the feature branch.



Other special branches

major-release-[X] - special branch which integrates all required changes due to changes in the source system's schema (occurring at a specific cadence).

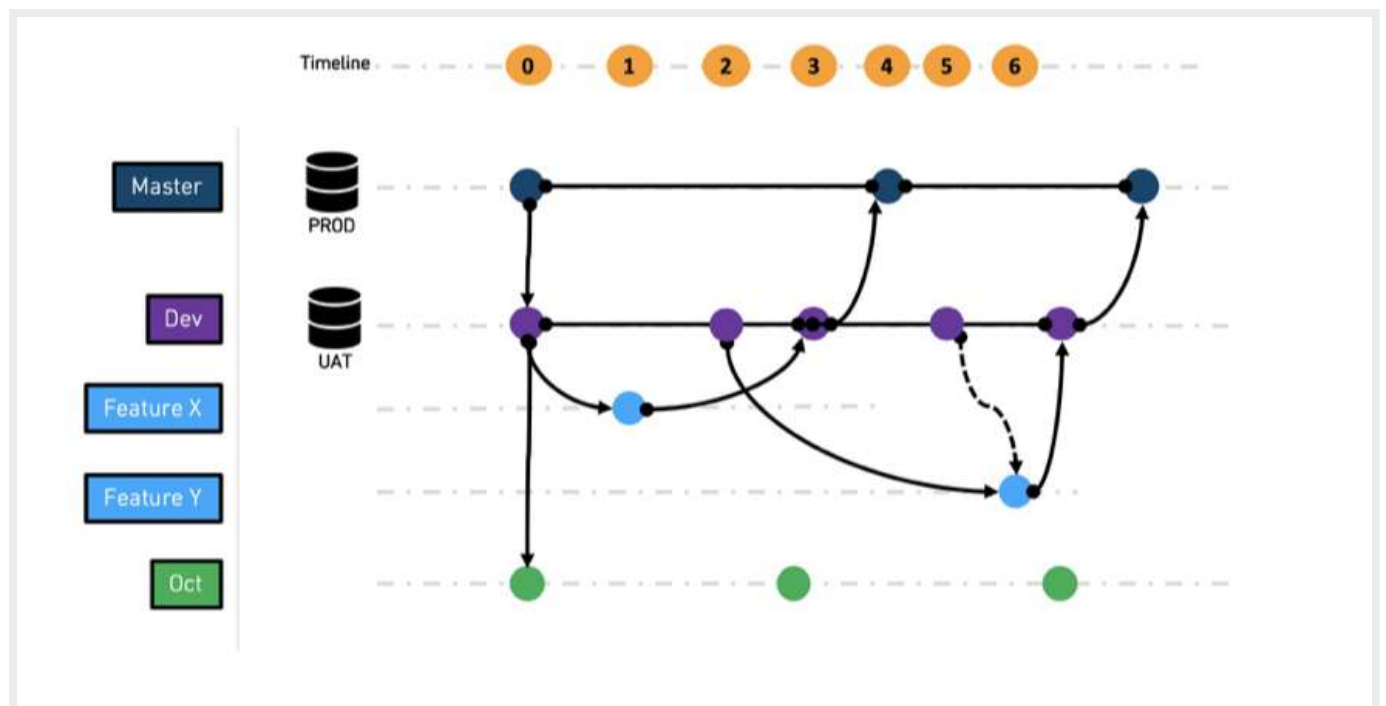
hot-fix-[X] - special branch used to fix an issue discovered in production while the code base has moved to a new version. Since we assume dev and master are in sync (unless there is a feature which is being actively tested) it is unlikely we need such a branch.

[API Reference ↗](#)

Branch management workflow

[Send feedback](#)

As described above, some branches are permanent while others are short-lived. The following section describes the recommended workflow between branches.



- At t0, master and dev branches are created.
- At t1, feature x branch is created from dev
- At t2, feature y branch is created from dev
- At t3, feature x is merged into dev
- At t4, dev is merged into master, and feature x branch is deleted
- At t5, dev is merged into feature y to pick up the latest state of dev
- At t6, feature y is merged into dev and later from dev to master as before

In addition to features developed and released to dev and master, a long lived branch for a major release like major-release-[X] is being developed.

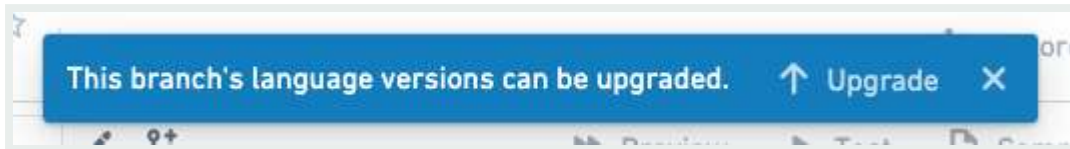
It is recommended to keep merging dev onto major-release-[X], so the latest features are in sync. We recommend keeping the Ontology schema as stable as possible and to hide the backend schema changes through the transformation in the datasource and transform projects.

API Reference [Upgrades](#)

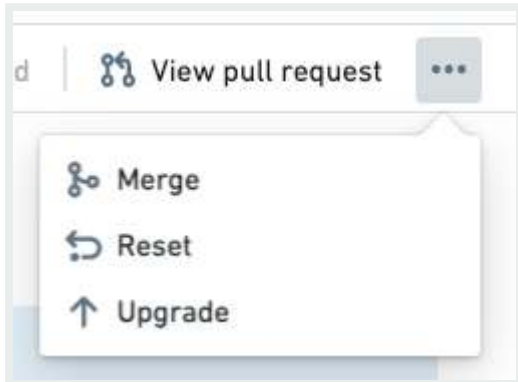
Repository upgrades are periodic prompts to update the configuration Repository. These updates apply fixes or bump versions of language-bundle dependencies

[Send feedback](#)

and best practice is to stay on top of applying these recommended upgrades.



Note that if you miss clicking the "Upgrade" button on the prompt, the button is also available for any branch under the "..." options menu in the upper right corner:



The upgrade process should be treated the same way as a feature development cycle. This means:

1. `master` should be upgraded through `dev`.
2. When prompted to upgrade `dev`, an automated feature branch will be created. You can test the configuration changes by running a build on this branch once CI checks are complete. After confirming the transforms still execute after the upgrade, merge the configuration changes into `dev`. Once `dev` is upgraded, a PR should be used to push these changes to master.
3. To upgrade all open feature branches, merge `dev` back into each branch to include these configuration upgrades.

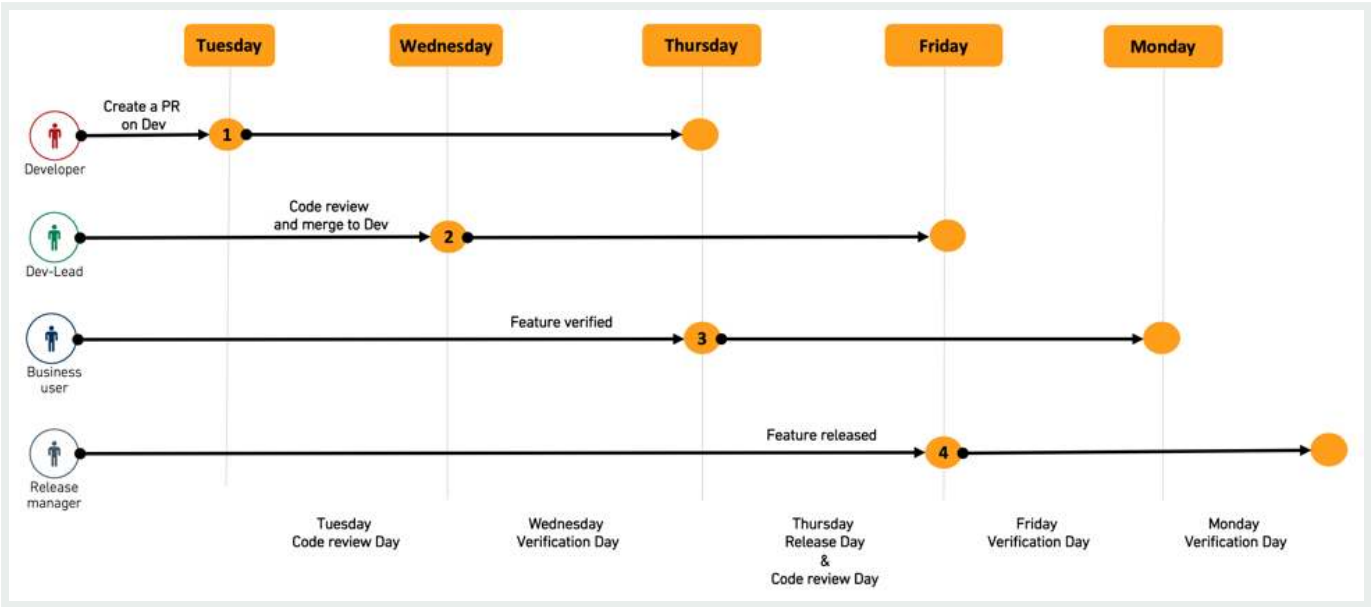
Maintenance windows

A fast iteration loop from idea to production is common in development practices and critical to the success of projects that depend on end-user feedback to determine the next iteration loop. In order to support such a process, we recommend establishing a pre-defined API Reference window.

For example, once all feature PRs are merged, all datasets get built on `dev` and the results can be verified on Wednesday and Friday. During these days, only fixes to

Send feedback

problems discovered during verification get merged into `dev` , until `dev` finally gets merged into `master` on Thursday and Monday, the "release days"



It is recommended to agree on such cadence with all parties involved to make sure they devote enough time during these days to push new features forward.

Feature testing and end user verification

Once feature development is complete it is critical to test and validate both the correctness of the feature outcome and the impact on any dependent resource. In Code Repositories we consider the output datasets as the feature outcome which should be tested.

Based on the branch structure we defined above, when we build the output dataset it will use either data from the input datasets on the same branch of the feature, or through the fallback logic will read from the source branch, master in the simplified version and dev in the other.

The developer-level testing can be done using dataset [Health Checks](#), dependent test datasets, or manually using tools such as [Contour](#).

API Reference ↗ Methodology

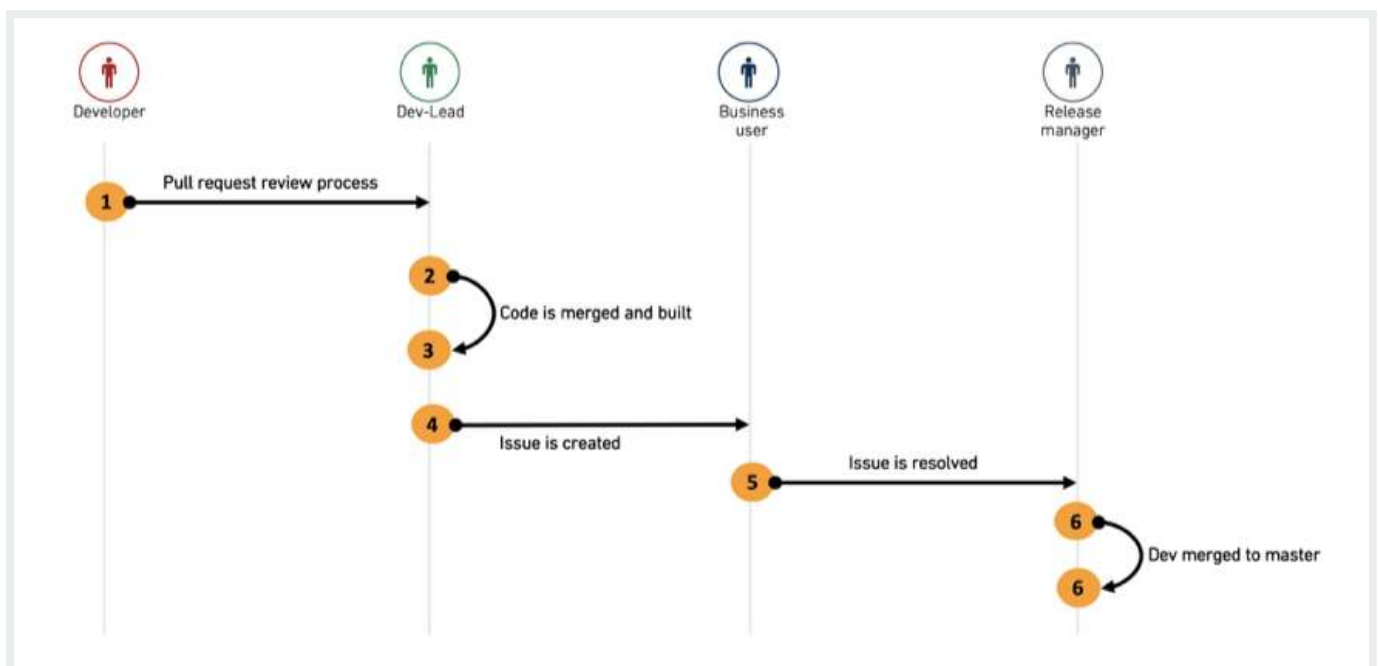
It is highly discouraged to do this testing via the preview or SQL helper. The preview helper uses only a sample of the data when executing code, which may not cover all cases - especially in code that involves one or more join operations. The SQL

Send feedback

helper tests the correctness of the code but not the output of the build which in some cases could be different.

Upon completion of feature development, the new feature code needs to be reviewed before being merged into the `dev` branch and the feature outcomes need to be verified by the end user for correctness and completeness.

We recommend using a combination of Pull Request reviews for the first and Foundry Issues for the second as explained in the diagram below.



1. The developer creates a pull request from his feature branch to the `dev` branch and adds implementation comments.
2. The dev-lead reviews the PR, collaborates with the developer on required changes and finally merges the PR into `dev`.

i During the pull request review, make sure that you check the changes to the schemas of any affected datasets. Schemas should be treated as your data API, so any changes to column names or columns types is considered a "breaking" change for any consumers. Limit breaks where possible by creating new columns rather than deleting or modifying old ones while announcing a deprecation of the old

API Reference ↗

instructions for data consumers (identified through Data Connection) to update to the new column. At the next major release, column previous major release can be removed.

Send feedback

To review schema changes in a pull request, use schema diff view in the [Affected Datasets](#) tab.

3. The dev-lead runs the build on the `dev` branch to create the feature output datasets.
4. The dev creates a Foundry Issue on the output dataset, documents any testing and verification requests and assigns the issue to the relevant end user for verification.
5. The end user receives the issue, with full context and view of the relevant dataset and branch to be tested. They collaborate with the developer on the required changes until the issue is resolved and marked as ready to merge. The issue is then assigned to the release manager.
6. The release manager closes the issue and merges `dev` into `master`.
7. The release manager deletes the feature branch.

The above workflow allows for a relatively fast collaboration cycle between the parties, without losing context. Closed issues are kept on the platform and can act as audit trail for the change-management process.

Foundry Issues feature review workflow

Below is an example of how a team can use Foundry Issues to track requests for review from other stakeholders, carry out conversations around new features, and control and document the process of merging code to `master`, so all interested parties remain informed and up-to-date. Consider this directional, rather than proscriptive, and adapt it to existing operational and technical processes as needed.

Example workflow

The first time a developer creates a feature review request issue, the issue is assigned with status `Feature Review: Request Review`. The issue can be attached to the dataset or to a specific column if the feature is specific.

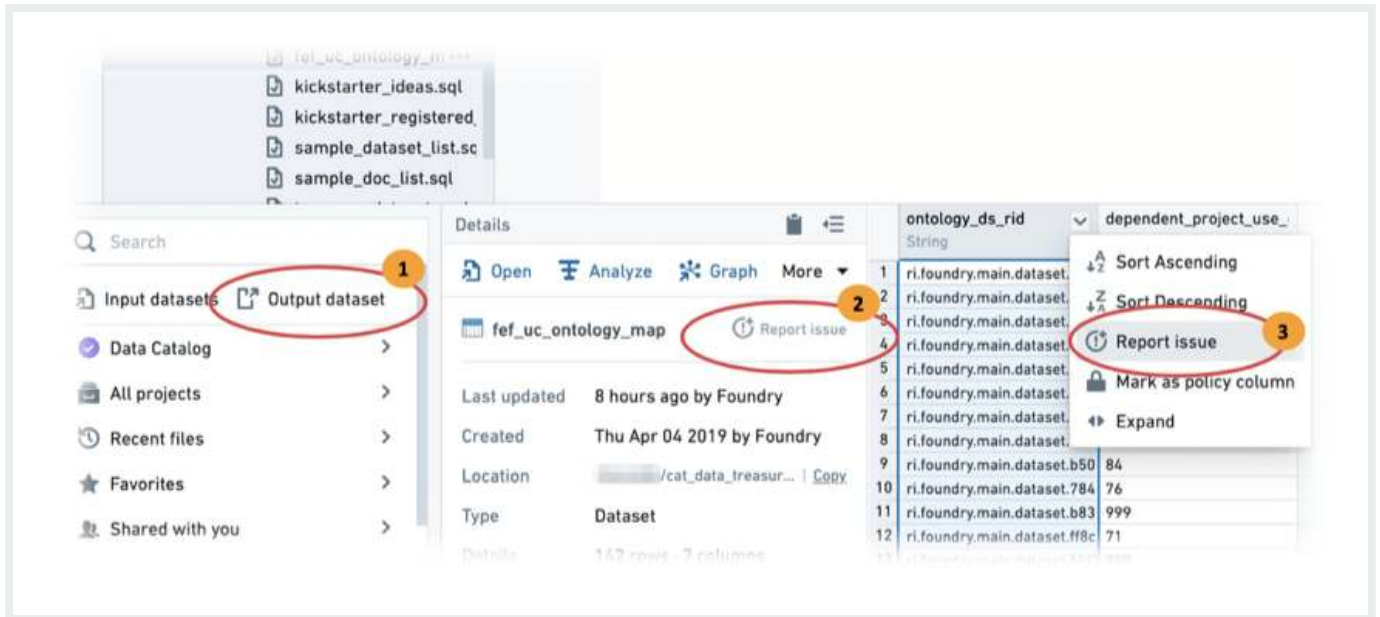
Collaboration with the end user is done through the comment section and by changing the status between `Request Change` or `Ready to Merge` and changing the assignee of the

is [API Reference ↗](#)

Issues is a great place to store the feature review workflow as end users complete their verification process (for audit purposes) and attach links to other assets.

[Send feedback](#)

The diagram below shows how an issue is created on the build output dataset.



1. In Foundry Explorer, select Output dataset.
2. Report the issue on the whole dataset by clicking **Report Issue**.
3. If the feature has to do with a specific column, you can use the column level report issue to provide the reviewer better context of the change you made.
4. Complete the issue fields as prompted by the form.
5. Once the issue is created, make sure to change the label to **Feature Review: Request Review**. (If you do not have these labels available, contact your Palantir representative for assistance.)

Ontology_map is ready for review

0 comments • Reported a few seconds ago

1

reported on

cat_data_treasury_ontology_repo

Edit • Thu, Jun 20, 2019 at 4:44 PM

I created the required mapping between use case and ontology datasets. ready for review.

File reported from: [fef_uc_ontology_map](#)

Link reported from: [workspace/code/repos/ri.stemma.main.repository.5099b2e...](#)

Add a comment

B I

Add a comment. Type @ to mention someone.

Close

Comment

PRIORITY

Medium

DUE DATE

+ Add due date

ASSIGNED TO

LABELS

+ Add label

Filter...

Feature Review: Ready to Merge

Feature Review: Request Change

Feature Review: Request Review

Inquiry Type: Access Request

Inquiry Type: Analysis Help

Inquiry Type: API

Inquiry Type: Build Error

Inquiry Type: Build Error-Spark OOM

Inquiry Type: CI Error

Inquiry Type: Data Quality

As the review process continues, the collaboration thread including all status changes and verification evidence are stored in the issue.

I created the required mapping between use case and ontology datasets. ready for review.

File reported from: [fef_uc_ontology_map](#)

Link reported from: [workspace/code/repos/ri.stemma.main.repository.5099b2e...](#)

added label

Feature Review: Request Review

Thu, Jun 20, 2019 at 4:45 PM

removed label

Feature Review: Request Review

Thu, Jun 20, 2019 at 4:50 PM

added label

Feature Review: Ready to Merge

Thu, Jun 20, 2019 at 4:50 PM

commented

Edit • Thu, Jun 20, 2019 at 4:50 PM

This looks great, when can we get it?

commented

Edit • Thu, Jun 20, 2019 at 4:52 PM

API Reference ↗ [ares/links/](#)

PRIORITY

Medium

DUE DATE

+ Add due date

ASSIGNED TO

LABELS

Feature Review: Ready to Merge

Send feedback

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Building pipelines / Best practices / Scheduling best practices

Scheduling best practices

To ensure schedules of production pipelines can easily be managed, we have developed a set of scheduling guidelines. Use this page to manage the schedules for which you are responsible.

- Each dataset should only be built by one schedule.
 - You can see the number of schedules on each dataset in a Data Lineage graph by selecting **Schedule count** in the node color dropdown.
- You must have one **force build** schedule for each raw Data Connection sync.
 - There should be no force builds on other schedules. Force builds should only be used with Data Connection syncs.
- Try to avoid **full builds** and use **connecting builds** instead.
- Your schedule should only build datasets that you own; your schedule should not build datasets that you do not own.
- Trashed datasets should not be built by your schedule. Use the [Upgrade Assistant](#) to investigate and remedy your schedules that contain trashed resources.
- Add retries to your schedule (we recommend three retries with an interval between one and three minutes).
- Avoid having multiple schedules on each step of the pipeline; rather, combine datasets into logical schedules.

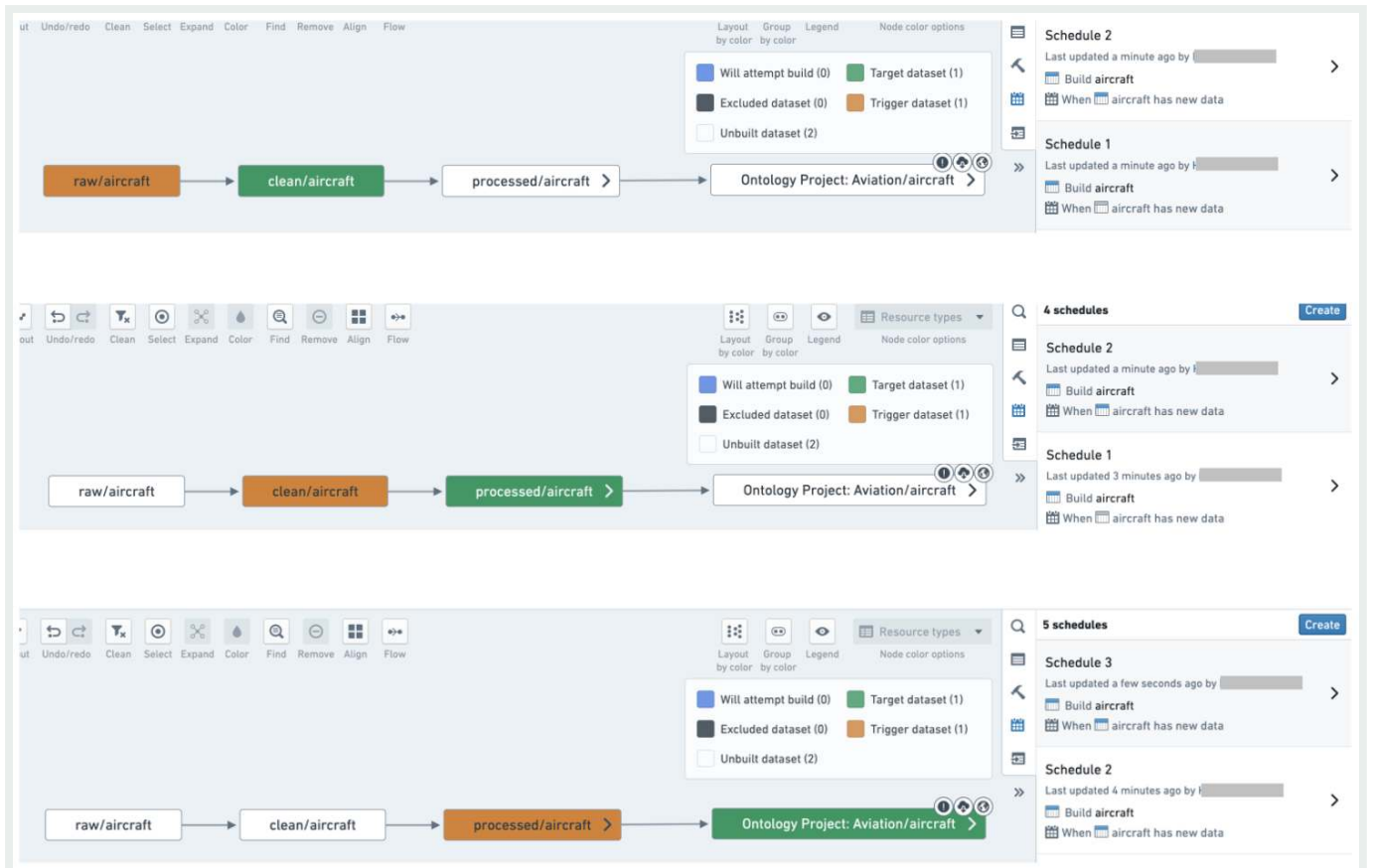
Examples

As an example, here are some unnecessary schedules that should be combined into one schedule:

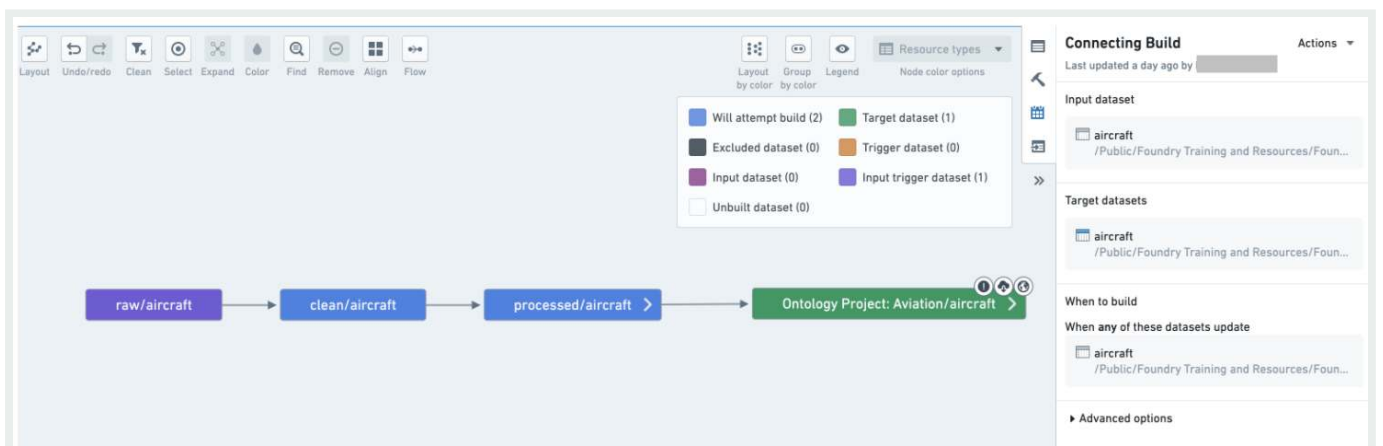
API Reference ↗



Send feedback



Combine datasets into a logical schedule using a connecting build:



Use the guidance below to set up an effective system of schedules for your pipelines.

Best practices

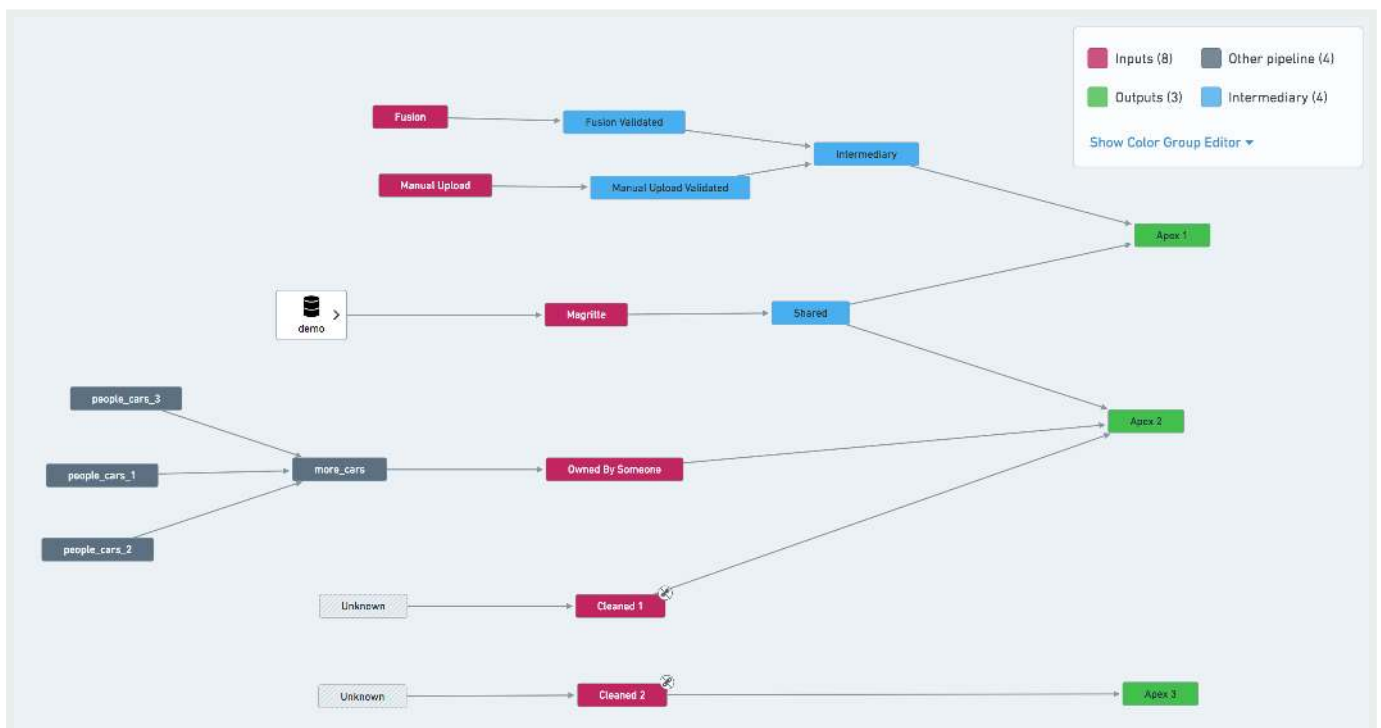
API Reference ↗ per pipeline

Send feedback

In most cases, you should only have one schedule per pipeline. In fact, each dataset in your pipeline should only have one scheduled build associated with it. Having a dataset built by two different schedules can lead to queuing and slow down both schedules. Keep your set of schedules simple and uncluttered to clarify the responsibilities of each schedule and make debugging easier if a build fails on a given dataset. It may also help with pipeline maintenance; you only have one place to check or modify when you want to configure the pipeline.

An exception could be made for pipelines that run in multiple sequential steps; some of these examples are described below:

1. If a pipeline has multiple terminal nodes, you may consider separate schedules for each node. In the example pipeline shown below, the most natural approach is to set up three schedules: defined on `Apex 1`, `Apex 2` and `Apex 3`. However, it may also be beneficial to create a schedule on `Shared` and treat this dataset as an input to other pipelines.
2. If your pipeline makes use of validation tables, you may need separate schedules for them.
3. Refer to [stability recommendations](#) for more information on how to effectively handle complex pipelines like these.



[API Reference ↗](#)

Project-scoped schedules

[Send feedback](#)

All schedules should be Project-scoped when possible so that a schedule's ability to run successfully does not depend on the permissions of a single user (the schedule owner). Otherwise, the schedule owner's permissions could change and prevent the schedule's datasets from building. This is particularly important because ownership of the schedule could change after a schedule edit, since the last user to modify a schedule becomes the schedule's owner.

There are some instances where it is not possible to Project scope schedules:

- Transforms where permissions are severed
- Contour jobs
- Fusion syncs

One owner per schedule for user-scoped schedules

For schedules that cannot be Project-scoped, it is preferable that a single user owns all schedules on a given pipeline. This user is often a “service user” (a special user account not tied to a single person). Alternatively, the single owner could be a team lead with responsibility for all modifications to schedules in the Project.

Having a single user owning the schedules on a pipeline reduces complexity and protects a schedule from being adversely affected by changes in user permissions or team composition.

As mentioned above, it is important to note that the last user to modify a schedule becomes the schedule's owner. If multiple user profiles are managing a schedule and access rights across your team are not uniform, you may end up with permission errors that are difficult to trace if you do not account for ownership changes.

i In addition to helping with permissions consistency, a single owner also makes it easier to track relevant builds in the Builds application, where you can filter all builds on “Started by”.

[API Reference ↗](#)

Using retries

[Send feedback](#)

Use retries when configuring your schedule. Retries are part of a job; if a job fails with an exception that can be retried, the build orchestration layer will relaunch a new attempt of the same job without failing the build.

We recommend configuring schedules for at least three retries, with at least one minute between retries. This allows the platform to automatically intercept jobs that would be failing and re-trigger them within the same build. The additional gap of one minute gives the platform a chance to recover from transient problems that caused the job to fail the first time.

There are some cases where you might not want to enable retries on your job. For example, pipelines with very tight Service Level Agreements (SLAs) where you want to be alerted as soon as there's a failure, or transforms that have a non-idempotent side effect. However, note that even if you do not enable retries in your schedule configuration, jobs may still be retried as part of [adjudication](#).

No multi-dataset force builds

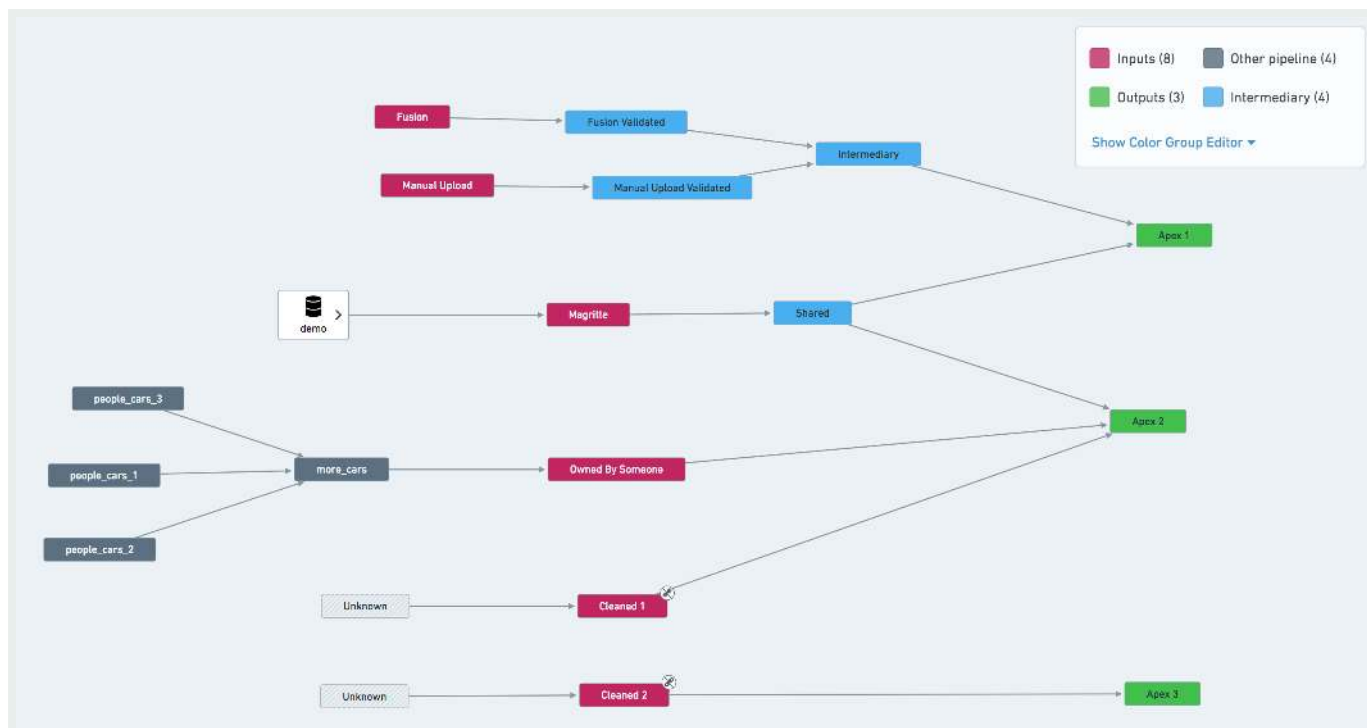
Schedules can enable a “force-build” option, where all datasets in the schedule will be built, regardless of their staleness. This option is very useful for Data Connection syncs, since those are always up to date in the platform and thus will never build as part of a scheduled build unless force build is used. However, we discourage having a multi-dataset schedule with the force build option for any derived datasets, since this can be costly in terms of resource usage. Instead, split your schedules so that datasets that must be force-built are in their own schedules.

What to include and what to exclude in a schedule

Be explicit about what is included in your schedule. Explicitly ignore datasets that sit outside your pipeline. Build resolution can be complex, but being explicit about what is included provides more transparency than being implicit.

- In our example pipeline screenshot below, `Owned By Someone` should be ignored from a [API Reference ↗](#) on `Apex 2`. The same applies for `Cleaned 1` and `Cleaned 2` — triggering a job on either of them would immediately cause the build to fail due to lack of permissions.

Send feedback



In addition, schedules should not depend on or try to build datasets that are:

- **Currently in the trash:** To remediate existing schedules with trashed resources, use the [Upgrade Assistant](#) to investigate the schedules you own to either un-trash or exclude the resource(s) in question from the schedule.
- **Produced from Contour:** For schedules to be considered “production ready”, we need to be able to review changes to the pipeline and revert to a previous version if something goes wrong. Contour does not support such workflows, and we discourage users from using them in production pipelines.
- **Produced from Code Workbook and Code Workspaces:** This is less of a strict rule than Contour output datasets, however we discourage the use of Code Workbook and Code Workspaces in production pipelines. Code Workbook and Code Workspaces are excellent for iterating on a solution but are less robust in regard to change management and reverting than Code Repositories. [Learn more about differences between Code Workbook, Code Workspaces, and Code Repositories.](#)

All outputs of the pipeline should be tagged as “targets” in your schedule. Schedules can have one or multiple targets, which are the datasets on which the schedule is installed. Schedules can also have one or multiple outputs, which are the final datasets built by the schedule and not used elsewhere in the build. In most cases, target datasets and output datasets have the same name, but in some cases (especially with multi-output builds), a build may have different target and output datasets. As a general rule, all outputs of a pipeline should be defined as targets.

Send feedback

Fail fast

Configure the build to fail immediately once a job fails by ticking the **Abort build on failure** checkbox in **Advanced options**). Failing fast will provide you with signal sooner.

Name schedules

All schedules should have a descriptive name. Schedule names can be very helpful as your pipeline grows in complexity and users over time.

Descriptive schedule names provide information about a schedule's intent to those who interact with a schedule; for instance, those with responsibility for fixing problems involving your schedule in the future.

Naming schedules is particularly important if you have multiple ingestion streams, if you expect other users to fork off any paths in your pipeline, or if you anticipate maintenance responsibility to change over time.

← PREVIOUS
Branching and release process

NEXT →
Building a production pipeline

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Building pipelines / Best practices / Building a production pipeline

Building a production pipeline

Production workflows need reliable pipelines to back them. Following the principles laid out in this document when building a pipeline will result in easier maintenance, allowing you to catch problems before they cause SLA breaches. Some guidance here will also make it easier to share knowledge about what is important in your pipeline. This is important in all stages of the pipeline’s lifecycle, from development all the way through to long-term maintenance.

This document is useful for both pipeline developers and pipeline maintainers. For developers, it is useful before you start building a pipeline if it will be going straight into production. Equally, this document can be used when a proof-of-concept pipeline is being converted into a production pipeline. For pipeline maintainers, the following elements should be prerequisites to entering maintenance mode.

Pipeline definition and expectations

While it may not always be possible to have definitive answers around expectations before you start building a production pipeline, it is valuable to be mindful about them early as possible. It’s highly recommended to document the pipeline definition and expectations as you establish them.

The expectations influence several aspects of design and set-up of a production pipeline, including:

- API Reference ↗

is built).

Send feedback
- What validations and monitoring are required.

- How to prioritize issues when the pipeline is in maintenance mode.

The important questions that should be addressed by your team, include:

- **What exactly is the scope of the pipeline?** Where does it start, where does it end?
Where should it feed into other pipelines?
 - If parts of your pipeline overlap with another pipeline or use case, consider treating the overlapping section as its own pipeline.
- **What is the requirement on pipeline refresh rate?**
 - Or perhaps there is a specific time of day when data needs to be refreshed by?
 - Should the pipeline run over weekends?
 - When is data considered critically out of date?
 - What will the expectation in terms of refresh rate and support?
 - *Be careful* — while this sounds easy, this area is where pipeline maintenance teams often face the most difficulty. Without clear definitions here, it is difficult to prioritize work as a pipeline maintainer or make the right fix.
- **What is the expectation for end-to-end propagation delay?** In other words, how long should it take for data to flow through the pipeline from the moment it lands in Foundry, to the point where the outputs of the pipeline are updated?
- **Who's your contact point on each and every external source that you pull from?**
External sources can be Data Connection syncs or a separate upstream Foundry pipeline that feeds into your pipeline.
- **What are the functional guarantees for correctness in your data?** Are there critical columns or key validations that must be true for your pipeline?
 - *Note:* it's important to determine what the outcome should be if a guarantee is broken. Should a failing validation prevent data from reaching your end user or should it fire an alert without preventing your pipeline from updating? You may want to do the latter to allow other up-to-date data to flow through your pipeline? The implementation for these two different situations would differ. Documentation is a good place to start tracking guarantees early on.
- **Are the expectations determined compatible?** As complex systems grow, they become more prone to failures. You generally want to allow sufficient time after an alert is fired to address the underlying issue - whether that is an unexpected pipeline failure or missing data from an upstream source.

API Reference ↗

important to think about the propagation delay in relation to the
expected pipeline refresh rate. If the expected refresh rate is every 2 hours, and the
pipeline takes 1.5 hours to build, it may become difficult to add

Send feedback

- *Example 2:* if a workflow requires an hourly refresh rate, but the upstream data source only provides data twice a day. This means that you will not be able to achieve hourly refreshes.

Principles for production pipelines

The key principle for setting up a successful production pipeline can be summarized as: **“build with the idea that you won’t be around to maintain it”**.

Some concrete tips that can help you achieve this:

- **Version your code, and write meaningful commit messages:** this makes it easier to keep track of what changes were made, when they were committed and by whom.
 - In production pipelines, [Java](#) and [Python](#) are recommended if possible. There are a lot of available resources and a strong developer community for both these languages. Whilst SQL is very accessible, it can get convoluted very quickly and hence can be more difficult to debug.
 - **Optimize for legibility above all else:** Simple pipelines are simpler to maintain. Opting for standard or existing solutions in transform code will reduce the maintenance burden and development complexity down the line. If you *must* do something unconventional, be sure to document it thoroughly.
 - **Linear pipelines where possible:** In terms of general architecture, the pipelines that are easier to maintain are those that are fairly straightforward and linear, but it’s difficult to give recommendations on how to get there. Be mindful of the general structure of the pipeline, and optimize for readability and simplicity as much as possible given your constraints. Once you are thinking about moving the pipeline to production, taking some time to untangle the pipeline is usually time well spent.
 - **Production pipelines are often long-lived.** As a result there’s a good chance that those who write the pipelines are not those who maintain or manage the pipelines in the long-term. Keep in mind that the next person who takes over the pipeline development or maintenance needs to be able to read the code and make sense of the pipeline setup.
 - **Keep concise documentation:**
 - *Logic documentation:* It is generally advised that documentation about the logic itself
- [API Reference ↗](#)
- ; comments.
e documentation (including documenting recurring issues in a
pipeline): should also be kept close to the pipeline, in an intuit

Send feedback

example place for this would be in the Project that the pipeline belongs to. It should be easy to find for pipeline developers and maintainers.

- Bear in mind: over-documenting can also make it hard to read and less useful. Be concise and document key information. If you need to capture very long and detailed pipeline information that will not be referred to as often, consider keeping it in a separate document.

Development process and infrastructure setup

Development

Once a pipeline goes into production, it's important to ensure that the development processes around making further contributions to the pipeline are established and effectively communicated to pipeline developers. This ensures that no unexpected outages on the pipeline occur.

As a result, we recommend reading:

- **Development Best Practices**: At the very least, production pipelines should be easy-to-read, easy-to-maintain, and with locked master branches. However to ensure your development setup is more robust, we recommend reading through the more detailed advice in this documentation.
- **Branching and Pipeline release process**: if you haven't already been working with a branching and pipeline release process during the development phase of your pipeline, we recommend this when moving your pipeline into production. The documentation will provide an example process that you can use.

Infrastructure (schedules)

On a related note, setting up schedules early will allow you to develop without having to think about manually triggering builds of different parts of your pipeline. If your scheduling is messy, you may find that this hinders your development and slows you down as changes are not propagating automatically through your pipeline.

When moving your pipeline into production, it is also recommended to review and re-structure your schedules as the schedules used during development may no longer make sense or may contain antipatterns. This step should be done before moving in.

[API Reference ↗](#)

[Send feedback](#)

Setting up or reviewing schedules according to best practices can be achieved by following the [scheduling best practices documentation](#).

Start monitoring early

As soon as the pipelines are running regularly, you want to start monitoring its behavior. This prevents tech debt from accumulating and allows you to track if the expectations for the pipeline are realistic or not. It may not always be feasible to start monitoring right away, given deadlines and the capacities of your team, however it is encouraged where possible.

See the documentation on [Pipeline Monitoring Best Practices](#) for more information on how to set up monitoring.

Data connectivity & integration / Building pipelines / Pipelines on unstructured data / Overview

Pipelines on unstructured data

As discussed in the overview for [datasets](#), unstructured data in Foundry is stored as a collection of files in a dataset just like tabular data.

These are some features that work identically between pipelines on structured and unstructured data:

- Pipelines can be made incremental to optimize compute performance.
- You can write unit tests against your pipelines.
- Computing output datasets is done using [builds](#) and [schedules](#).
- Foundry's [pipeline security](#) features enable robust, end-to-end security guarantees.

Some differences from pipelines on tabular data include:

- Most guidance and example code in documentation focuses on processing dataframes, which are not the input types used for unstructured data.
- You must use the lower-level file system APIs to read and write files in unstructured datasets.
- Because unstructured datasets have no schema, some features focused on validating rows and columns of tabular datasets are unavailable.
- It is possible to use Spark to process unstructured files in parallel, but the APIs are lower-level and more complex than for dataframe processing.

To get started with pipelines on unstructured data, refer to the relevant parts of documentation for Python and Java transforms:

API Reference ↗

Python transforms: [Read and write unstructured files](#)

Java transforms: [Read and write unstructured files](#)

Send feedback

Once unstructured data has been cleaned and normalized, you can use [Code Workbook](#) to analyze unstructured datasets and train machine learning models in Python and R. [Learn more about unstructured data access in Code Workbook.](#)

PREVIOUS

← [Best practices / Building a production pipeline](#)

NEXT

[Infer a schema for CSV or JSON files](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Building pipelines / Pipelines on unstructured data / Infer a schema for CSV or JSON files

Infer a schema for CSV or JSON files

It's easiest to work with datasets in Foundry if they have a schema. Foundry allows you to manually add a schema to datasets containing CSV or JSON files by selecting the **Apply a schema** button in the dataset. The **Apply a schema** button will automatically infer the schema based on a subset of the data. Once a schema is applied, select **Edit schema** in the dataset view to modify column types or apply additional parsing options to drop jagged rows, change encoding, or add additional columns like file path, byte offset for row, import timestamp, or row number.

Schemas applied statically based on the initial dataset's files can become out of date if data changes. Thus, it can be helpful to have Spark dynamically infer a schema as the first step of a transforms pipeline on semi-structured data.

Note that inferring a schema dynamically on each pipeline build has a performance cost, so this technique should only be used sparingly (for instance, when the schema may change).

Below are examples for CSV and JSON inputs.

i Parquet, the default output file format for Transforms, does not allow certain special characters that may be present in an automatically-inferred schema. Therefore, we recommend that you use `sanitize_schema_for_parquet` as in the examples below to prevent potential issues.

API Reference ↗

For dynamic schema inference, there are many other use-cases for reading all or a subset of dataset files with [SparkSession.read](#) as in the examples below. If your use-case is one that does not actually need the dynamic schema inference behavior, you should disable it either by setting `inferSchema` to `False` (which will

Send feedback

result in all columns being strings) or by leaving that option out and [explicitly passing a schema ↗](#). Disabling automatic schema inference will result in significantly better performance and consistency especially for incremental pipelines where different schema inference results between incremental batches can be problematic.

CSV

```
1 from transforms.api import transform, Input, Output
2 from transforms.verbs.dataframes import sanitize_schema_for_parquet
3
4 @transform(
5     output=Output("/Company/sourceA/parsed/data"),
6     raw=Input("/Company/sourceA/raw/data_csv"),
7 )
8 def read_csv(ctx, raw, output):
9     filesystem = raw.filesystem()
10    hadoop_path = filesystem.hadoop_path
11    files = [f"{hadoop_path}/{f.path}" for f in filesystem.ls()]
12    df = (
13        ctx
14        .spark_session
15        .read
16        .option("encoding", "UTF-8") # UTF-8 is the default
17        .option("header", True)
18        .option("inferSchema", True)
19        .csv(files)
20    )
21    output.write_dataframe(sanitize_schema_for_parquet(df))
```

JSON

```
1 from transforms.api import transform, Input, Output
2 from transforms.verbs.dataframes import sanitize_schema_for_parquet
3
4 @transform(
5     output=Output("/Company/sourceA/parsed/data"),
6     raw=Input("/Company/sourceA/raw/data_json"),
```

API Reference ↗

Send feedback

```
7 )
8 def read_json(ctx, raw, output):
9     filesystem = raw.filesystem()
10    hadoop_path = filesystem.hadoop_path
11    files = [f"{hadoop_path}/{f.path}" for f in filesystem.ls()]
12    df = (
13        ctx
14        .spark_session
15        .read
16        .option("multiline", False) # False is the default; use True if
17        .json(files)
18    )
19    output.write_dataframe(sanitize_schema_for_parquet(df))
```

PREVIOUS

← Overview

NEXT

Pipeline security / Overview →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Pipeline security

Foundry supports robust platform security features that enable you to ensure that the security of organizational data sources is maintained while you create data pipelines to enable organizational outcomes.

We recommend referring to the following resources to learn more about setting up data pipeline security:

- [Recommended Project structure](#)
- [Securing a data foundation](#)
- [Protecting sensitive data](#)

Additionally, this section includes guidance on advanced workflows related to Marking removal. Only follow these guides once you have a thorough understanding of the Markings concept introduced in the pages linked above.

- [Guidance on Marking removal](#)
- [Remove inherited Markings](#)

PREVIOUS

← Pipelines on unstructured data / Infer a schema for CSV or JSON files

NEXT

Guidance on removing markings →

© 2026 Palantir Technologies Inc. All rights reserved.

Data connectivity & integration / Building pipelines / Pipeline security / Guidance on removing markings

Guidance on removing markings

Access requirements for platform resources are controlled by [Markings](#). Markings restrict access in an **all-or-nothing** fashion: in order to access a resource, a user must be a member of all Markings applied to the resource. In addition, Markings are **inherited** along both the file hierarchy and direct dependencies. If you have the correct permissions, you can remove Markings directly from resources and [along direct dependencies](#).

Markings are frequently used because they are legible throughout the platform and propagate along direct dependencies, protecting sensitive data. In some circumstances, a Marking may be applied early in a pipeline and need to be removed later in a pipeline. This page provides more information on how to remove Markings depending on your pipeline structure.

- If you are starting to apply Markings to your pipelines, we suggest reading through the [scenarios](#) section below to make sure you don't accidentally lock out any of your end users.
- Additionally, we recommend following the [best practices](#) when applying and removing Markings, and consulting the [documentation on how to remove inherited Markings and Organizations](#) for more advanced information.

You can also [remove markings or organizations with Pipeline Builder](#).

Scenarios

Below are scenarios related to applying a Marking early in a pipeline and removing it later in a pipeline.

API Reference ↗



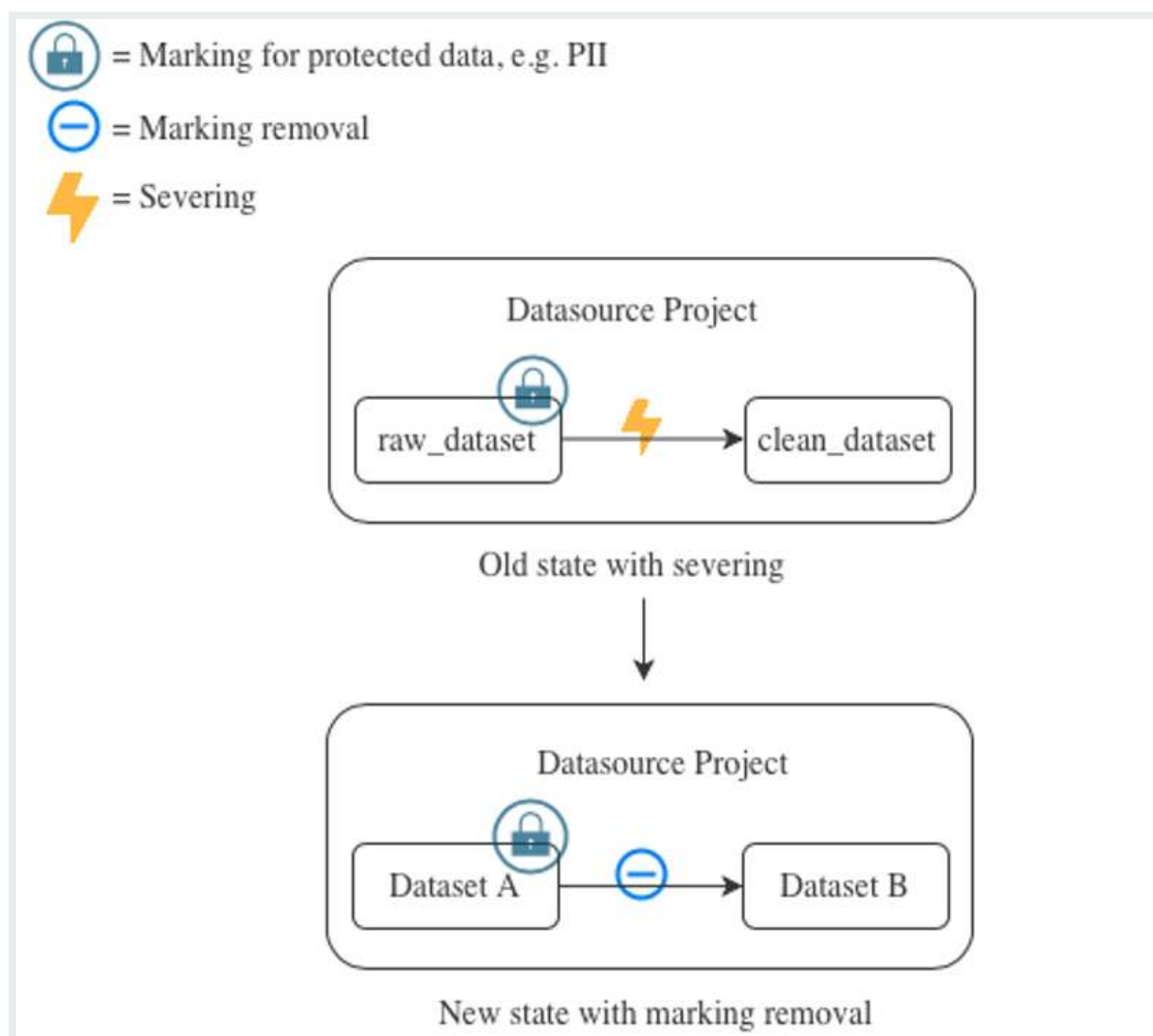
Send feedback

Scenario 1: Replacing severing by Marking removal

This scenario is for when the pipeline:

- already has a Marking applied,
- the Marking is removed by severing, and
- the Project-level propagate view requirements permissions are not turned on.

Therefore, you are migrating from using severing (a deprecated feature) to removing an inherited Marking.



[API Reference ↗](#)

In the example above, severing has been used to prevent the Marking from propagating. Assuming that severing is only being used to remove a marking, we strongly recommend that you replace severing with Marking removal, as in the new state in

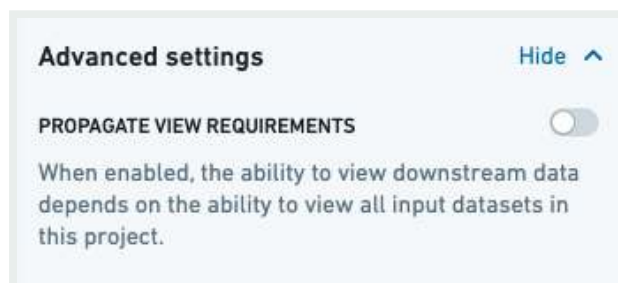
[Send feedback](#)

the example above. When removing the Marking, it is useful to think about the [approval mode](#) configuration of the repository which contains the Marking removal transform.

In the case that propagate view requirements is enabled, read scenario 2 below.

Scenario 2: Apply a Marking followed by Marking removal to disable "Propagate View Requirements"

This scenario involves applying Markings to a dataset in your pipeline in order to disable the project-level propagate view requirements settings.



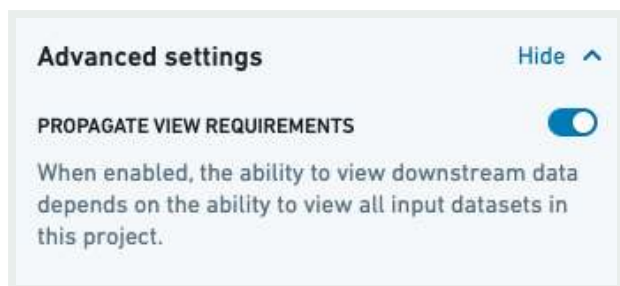
New Projects have the **Propagate View Requirements** option disabled by default, as seen above. For these new Projects, view requirements will not be enforced for downstream derived datasets. Specifically, this means that users accessing a downstream version of the data in a separate Project would not also require access to the upstream data in the Projects where this configuration is disabled.

i Reminder

Markings always propagate. If data in a new Project has a Marking, that Marking will still propagate to all downstream datasets, regardless of the "propagate view requirements" setting.

[API Reference ↗](#)

[Send feedback](#)



If you have Projects with the **Propagate View Requirements** option enabled as in the image above, then view requirements have propagated for datasets in these Projects. This means that users accessing a downstream version of the data in a separate Project would additionally require access to the upstream Project(s) with this config enabled.

We highly recommend **disabling** *view requirement propagation* in favor of using *Markings*.

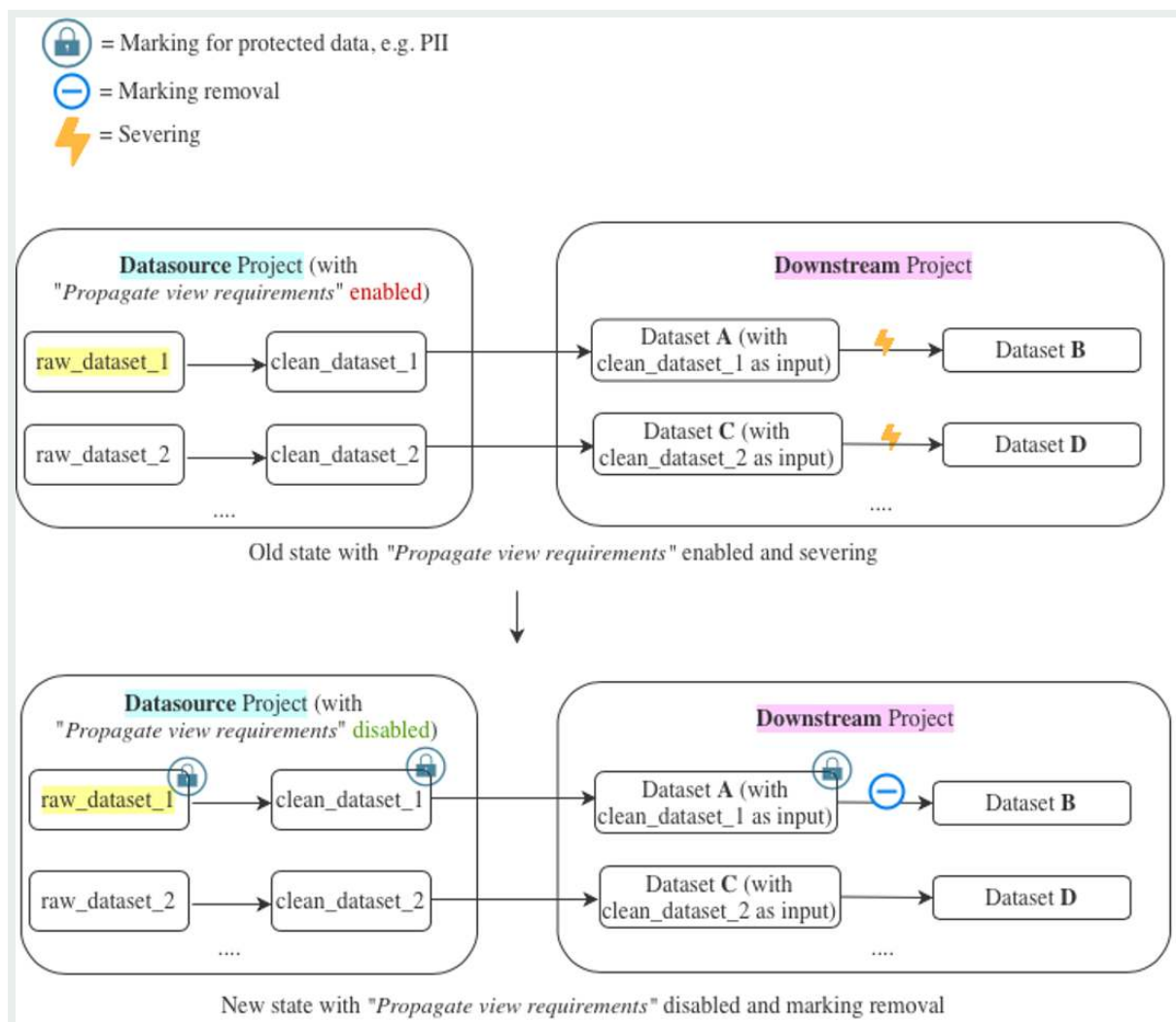
Before disabling view requirement propagation and introducing Markings to your pipeline, it is worth considering the original purpose of enabling propagating view requirements:

1. Perhaps there is no compelling reason for having "propagate view requirements" enabled. In that case, you might be able to simply disable this setting, and then ensure that users are only granted access to the datasets to which they explicitly require access. Users will no longer also require access to the upstream datasets which were previously propagating view requirements.
2. There are a few sensitive datasets in the Project. In this case, it's best to isolate the sensitive datasets from non-sensitive ones and follow the steps outlined below. Those steps demonstrate how to replace view requirement propagation with Markings, which are the appropriate security primitive meant for these scenarios.
3. If the steps below do not meet your needs, contact your Palantir representative.

In the example below, our goal is to disable "propagate view requirements" on the **Datasource** Project. After following the steps above, we learned that the reason "propagate view requirements" is enabled on the project was to protect the `raw_dataset_1` dataset because it has sensitive data.

[API Reference ↗](#)

[Send feedback](#)



In the old state, viewing the contents of Dataset **A** would require at least “viewer” access on both the **Datasource** Project and the **Downstream** Project. Subsequently, severing has been used to remove the view requirement propagation on Dataset **B**.

In the new state, viewing contents of Dataset **A** requires at least “viewer” access on the **Downstream** Project and access to the Marking. Note that with "propagate view requirements" disabled, requiring access to Datasets **C** & **D** only requires “viewer” access on the **Downstream** Project.

This change allows disabling of "propagate view requirements" by using a security Marking.

In the proposed solution below, we'll apply a Marking on `raw_dataset_1` which is then *immediately propagated* to all downstream datasets which have any non-severed

tr API Reference `dataset_1` as an input. There is an assumption that **severing was already in place**, and severing is only being replaced by Marking removal. In your situation, see [Scenario 3](#), where we discuss implications of applying a marking to a dataset in detail.

Send feedback

The following steps are recommended for introducing this change so that users don't lose access to Dataset **B** when the Marking is added after disabling "propagate view requirements" in the **Datasource** Project:

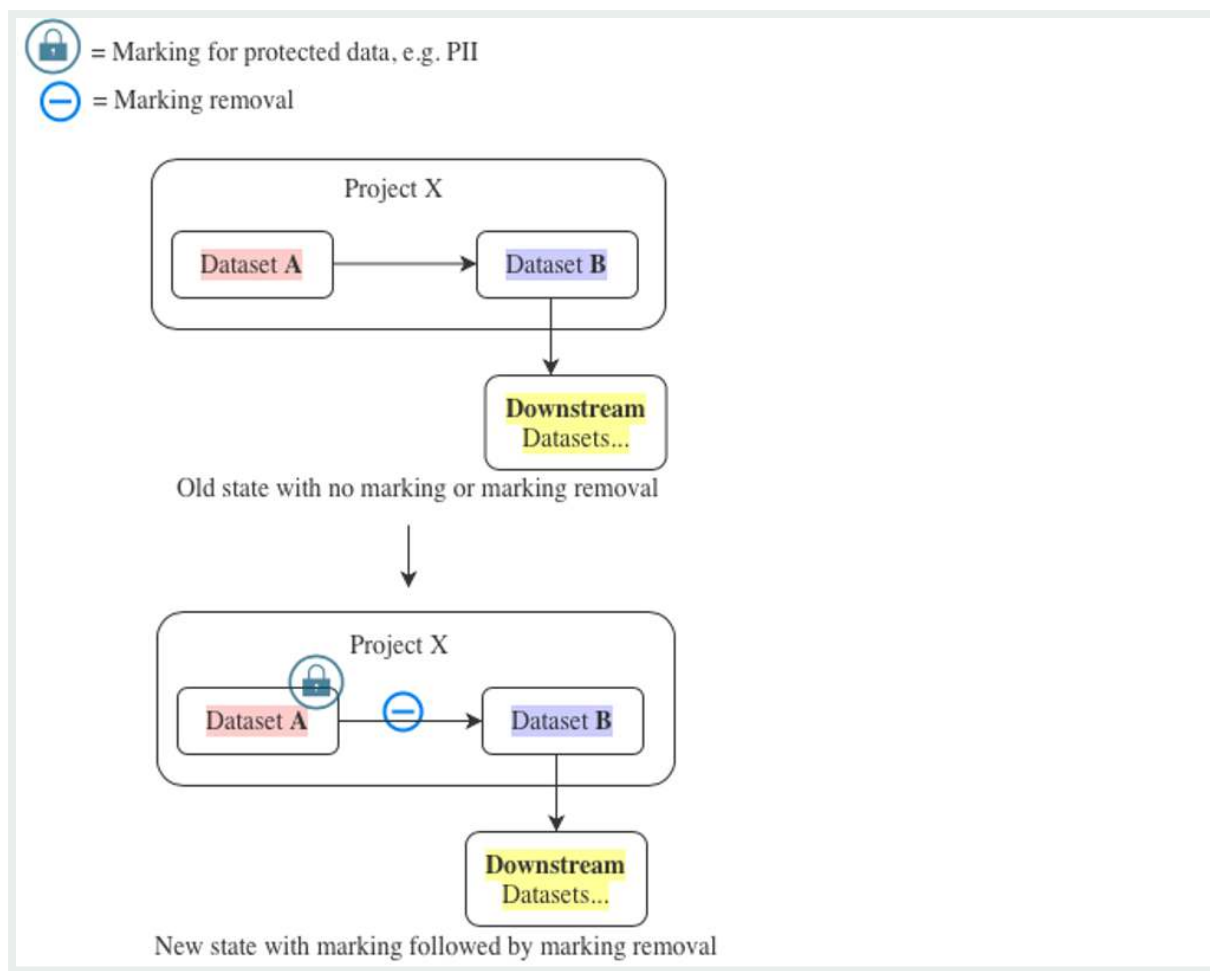
1. Create a new Marking and give relevant users access to the Marking.
2. Apply the Marking on `raw_dataset_1`.
3. Now that the Marking has been applied, you can safely disable "propagate view requirements" in the **Datasource** Project.
4. Replace severing with Marking removal in the transform, and ensure that unsevering and Marking removal happens at the same time.
5. Ensure all datasets in the pipeline are built.
6. Remove users from the **Datasource** Project who no longer require access to it.

Scenario 3: Applying a new Marking followed by Marking removal at the dataset level

This potentially complex scenario involves introducing a new Marking early in an existing pipeline without accidentally locking out users later in the pipeline.

[API Reference ↗](#)

[Send feedback](#)



It is critical to note that the Marking introduced on Dataset **A** will immediately propagate to all resources that are downstream of that dataset along the transaction lineage. Users will require the marking to access anything derived from the marked dataset.

To understand this better, let's extend the example above with a pipeline as follows: Dataset **A** → Dataset **B** → **Downstream** Datasets:

- Dataset **A** is a raw dataset (about to be marked).
- Dataset **B** is derived from Dataset **A** with sensitive data removed (so the Marking can be removed).
- **Downstream** Datasets are datasets derived from Dataset **B** for which users need access.

Our goal in this example is to ensure that **Downstream** Datasets never inherit the Marking from

[API Reference ↗](#)

The first thing we need to understand is that marking Dataset **A** (or a Dataset **A**) effectively marks all transactions in the entire history of Dataset **A**. As a

[Send feedback](#)

consequence, Dataset **B** and **Downstream** Datasets inherit the Marking immediately.

If we perform the following steps:

- Add Marking removal in the Dataset **A** → Dataset **B** transform,
- Update Dataset **B**'s code, and
- Rebuild Dataset **B** ...

... then the latest snapshot transaction on Dataset **B** will be Marking-free, but all older transactions on Dataset **B** will still be marked.

i Note that while marking a dataset will mark all of its transactions, removing a Marking in transforms will only remove the Marking for new output transactions. Markings will not be removed from existing transactions. This behavior is **not** symmetrical.

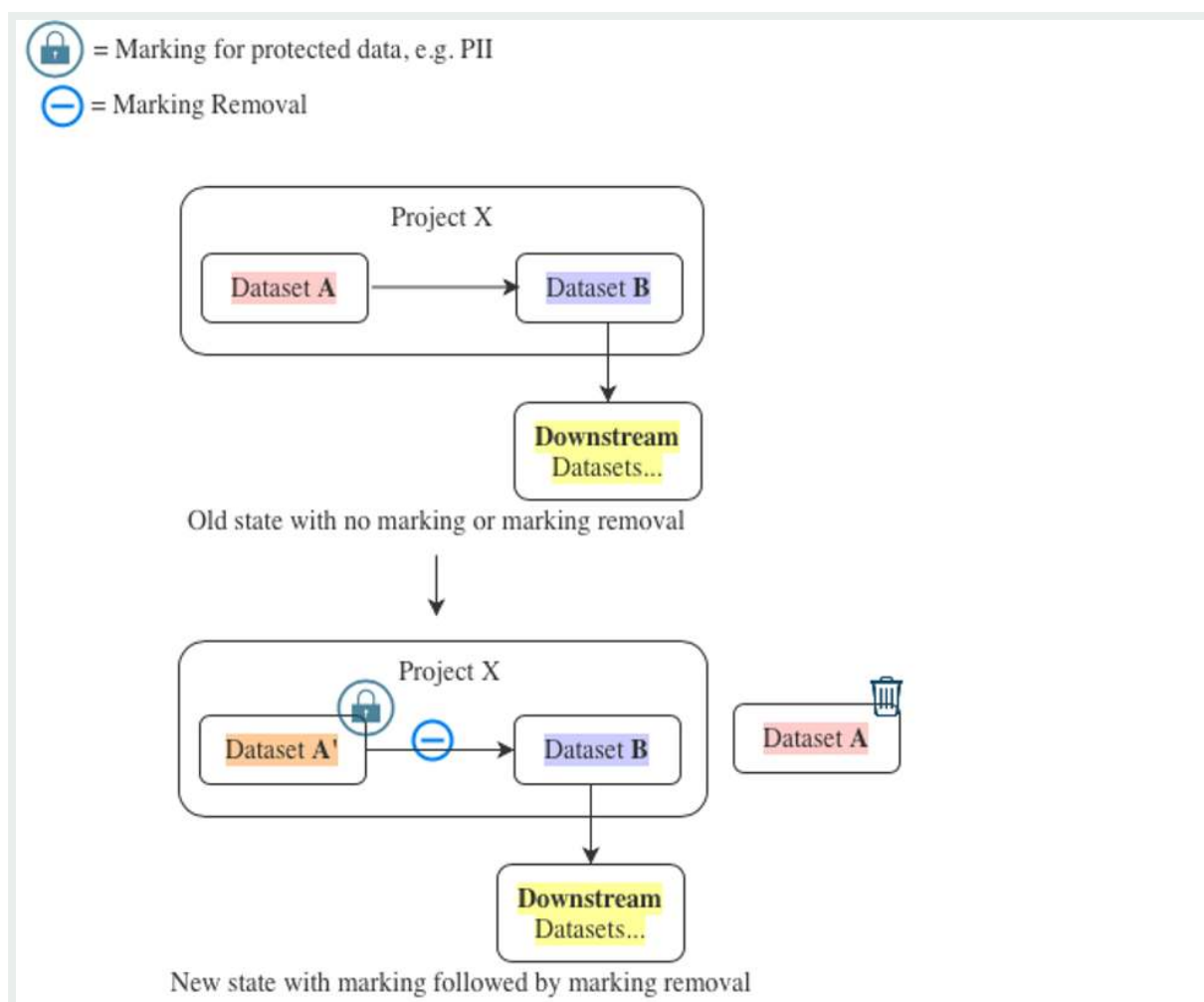
This means that any data in the **Downstream** Datasets derived from an older transaction on Dataset **B**, such as **Downstream** datasets that are built incrementally, will still inherit the Marking. However, Foundry checks only the most recent transaction for each input when verifying permissions on a dataset view. For incremental datasets, this means a regular build is sufficient to remove the marking.

To ensure each incremental **Downstream** dataset is unmarked, everything between the unmarking transform on Dataset **B** and the incremental **Downstream** dataset **must be rebuilt** after the unmarking transform is applied on Dataset **B**. This will ensure that each **Downstream** dataset depends only on the latest (unmarked) transaction on Dataset **B**, rather than earlier (marked) transactions.

If the number of **Downstream** datasets is infeasible for manually triggering a rebuild, we suggest the following steps:

[API Reference ↗](#)

[Send feedback](#)



1. Create a Dataset **A'** and make sure its contents are identical to Dataset **A**.
2. Rewrite the code for Dataset **B** to use Dataset **A'** as an input in place of Dataset **A**. At the same time, make sure to add appropriate Marking removal to the transform and then build Dataset **B** immediately afterward. There is no need to snapshot build Dataset **B**.

i Consider the performance effects of doing this swap as it could trigger a **SNAPSHOT** build of Dataset **B**.

1. Mark Dataset **A'**.
2. Move Dataset **A** into trash.
3. No further rebuilds are required at this point.

[API Reference ↗](#)

If you are making these changes in an important pipeline or are unsure about any of these steps, contact your Palantir representative for assistance.

[Send feedback](#)

Best practices

- **Do not** set a repo to “Don’t require re-approval” mode unless absolutely necessary.
 - If you must enable this mode, **do** minimize the set of editors of that repository and **do** ensure that you get a sign-off (if required) for this setting.
- **Do** have clear, well-defined criteria for what makes a resource require a Marking.
- **Do** write Marking removal transforms to actively select data that does not require a Marking
- **Do not** write Marking removal transforms that filter out data which requires a Marking. This is because new data which merits a Marking can commonly be introduced upstream, and you need to protect against it accidentally flowing through your Marking removal transform.
 - Examples showing “filter in” approach (recommended):

```
1 # column-based
2 df.select("salary","title","department")
3
4 # row-based
5 states_to_keep = ["OH","CA","DE"]
6 df.filter(df.state.isin(states_to_keep))
```

- Examples showing “filter out” approach (**not** recommended):

```
1 # column-based
2 df.drop("firstname","lastname")
3
4 # row-based
5 states_to_drop = ["FL","TX","IL"]
6 df.filter(~df.state.isin(states_to_drop))
```

- **Do** perform the Marking removal in the transform where sensitive data removal logic is implemented.
 - **Do not** abstract or hide this logic in a different repository.

[API Reference ↗](#) **Do not** create a separate Markings removal repository (with a suite of transforms). The marking removal logic **should be** explicitly approved during the Marking removal PR review.

Send feedback

- **Do** remove Marking from the data either as far upstream (when a marked dataset is added as an import) or as far downstream (in the last step before creating a Project export) as possible while performing Marking removal *within* a Project.

FAQs

Do we need an approval for the Marking removal every time there is any logic changes in the code?

This depends on whether the **require re-approval** or **don't require re-approval** approval mode is set on the repository which has your Marking removal transform. [Learn more about approval modes.](#)

The Marking removal workflow and the “one repo per Project” recommendations don’t go very well together. How many repositories should we set up per Project for the Marking removal workflow?

Ideally, every time the *logic* of a Marking removal transform changes, it should undergo a security approval. To balance between excessive friction in the approval process and good security posture, we recommend that, if you can, move all transforms with Marking removal logic (such as obfuscating data, removing columns, and so on) to a separate repo and set the separate repo to “Require re-approvals”.

Can I add the Marking removal properties (*stop_propagating* and *stop_requiring*) on an output in my transforms?

No, these are **input** properties and cannot be added on outputs. If your goal is to remove a Marking from an output, you need to identify all inputs that carry a Marking and add *stop_propagating* statements to them respectively. For more details, refer to the [input transform property](#) documentation.

API Reference ↗

Send feedback

Which languages support Marking removal?

The following languages support Marking removal:

- [Python](#)
- [SQL](#)
- [Java](#)

Why is Marking removal preferred over severing?

Declassification should be carried out carefully and not scattered around Projects and repositories. Marking removal features in the platform provide granular control over permission propagation changes and ensure that such changes are appropriately reviewed.

We are transitioning repos to use Marking removal instead of severing. How can we disable severing for new transforms?

Contact your Palantir representative to disallow adding severing on datasets that have not had severing enabled before.

PREVIOUS
← [Overview](#)

NEXT
[Remove inherited markings and organizations](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

[API Reference](#) ↗

[Send feedback](#)

Data connectivity & integration / Building pipelines / Pipeline security / Remove inherited markings and organizations

Remove inherited Markings and Organizations

[Markings](#) and [Organizations](#) restrict access to resources based on a user's eligibility.

When restricted content is removed or obfuscated while deriving a dependent resource, users may wish to remove the Marking and/or Organization on that derived resource. This process of removing inherited Markings and Organizations can be done by using the `stop_propagating` and `stop_requiring` input transform properties.

- `stop_propagating` is used to remove inherited [Markings](#) (for example, PII).
- `stop_requiring` is used to remove inherited [Organizations](#) (e.g. Palantir).

Terminology

- [Organizations](#) are an access requirement applied to Projects that guarantees strict silos between groups of users and resources. In order to meet access requirements, users must be a member or guest member of at least one Organization applied to a Project.
- [Markings](#) are an access requirement applied to resources that restricts access in an all-or-nothing fashion. In order to meet access requirements, a user must be a member of **all** Markings applied on the resource.
- [Roles](#) are a collection of permissions that define the specific workflows a user can perform on a given resource (e.g. Viewer, Editor, etc.).

API Reference ↗

I   ant to know

Send feedback

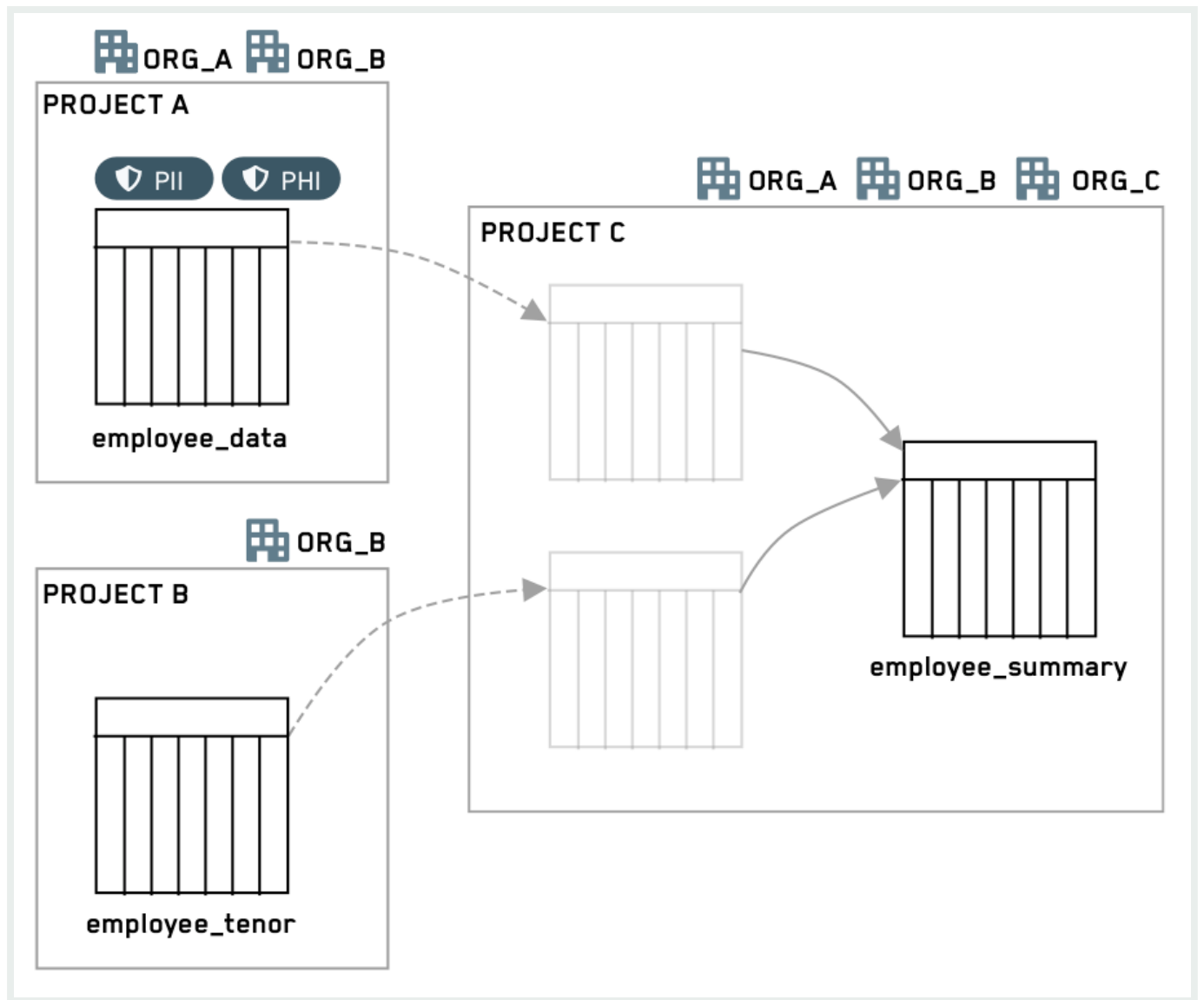
- The `stop propagating` and `stop requiring` key phrases only apply to Organizations and Markings and **NOT** Roles.
- The repository needs to have at least one protected branch (for example, **main**). This branch must also [enforce at least one required approver](#).
- You can only remove Organizations and Markings on protected branches. Mentioning an un-protected branch (e.g. `on_branches=[..., "not-protected-branch"]`) will cause the build to fail.
- Removal of organizations and Markings is supported in Python, Java, and SQL.
- Special user permissions are necessary to approve the removal of inherited Organizations and Markings. A user will need `Remove marking` permissions to remove Markings and `Expand access` permissions to remove Organizations.

Basic workflow

i The gray dataset boxes in Project C below highlight the fact that Project References must be added for all the inputs in the destination Project.

[API Reference ↗](#)

[Send feedback](#)



Steps

1. Create a new branch off of a protected branch (for example, *main*).
2. Add one or both of the `stop_propagating` and `stop_requiring` properties to the input transform. For example:

[API Reference ↗](#)

[Send feedback](#)

```

from transforms.api import transform, Input, Output, Markings, OrgMarkings

@transform(
    input_1=Input(
        "/ProjectA/employee_data",
        stop_propagating=Markings(
            ["8cbb369b-1f80748e2", "ea2be354"], mock_category: PII, on_branches=["master"]),
        stop_requiring=OrgMarkings(
            ["9128ed70-011a-4b5b4cb47bab59", "98ce6e6a-36d5-4b3-f9320aebf63d"], on_branches=["master"])
    ),
    input_2=Input(
        "/ProjectB/employee_tenor",
        stop_requiring=OrgMarkings(["98ce6e6a-36d5-4b3-f9320aebf63d"], on_branches=["master"])
    ),
    output=Output("/ProjectC/employee_summary")
)

def my_compute_function(input_1, input_2, output):
    # Logic that filters out PII and PHI sensitive data and joins the 2 datasets

```

3. Create a pull request to merge this code into a protected branch.
4. A user with either `Remove marking` permissions for Markings or `Expand access` for Organizations can approve or reject the proposed changes. If multiple reviewers are added, rejection by any reviewer will result in rejection of the entire pull request.
5. If approved, the code editor merges the PR and builds the output dataset. After the output dataset is built, it will no longer have the propagated Markings and/or Organizations.

i Internally, Organizations are represented as a slightly different kind of Marking, hence the transforms keyword following `stop_requiring` is called `OrgMarkings`.

Input transform property

To remove inherited Markings (e.g. PII), use the `stop_propagating` keyphrase.

To remove inherited Organizations (e.g. Palantir), use the `stop_requiring` keyphrase.

Each of these keyphrases must be specified on **every** input that requires removal of Markings or Organizations. For every removal, you must also specify the protected branches to which the removal should apply. Marking IDs, Organization IDs, and branches should

all be specified as quoted strings.

[API Reference](#) ↗

Send feedback

- i** You need to provide at least one upstream Organization, since users only need to satisfy at least one Organization. Approvals will be required for each listed Organization. The [detailed workflow](#) below provides an example illustrating this point.

Python

In Python, Marking removal is specified in the input constructor.

```
1 @transform(  
2     input_1=Input("<input_id>",  
3         stop_propagating=Markings([markingId1, ...], [branch1, ...]),  
4         stop_requiring=OrgMarkings([orgMarking1, ...], [branch2, ...])),  
5     output=Output("<output_id>")  
6 )
```

The `Markings` class takes a list of Marking IDs and a list of protected branches on which to apply the marking removal. Marking IDs can be found in the `Markings` list on the `Settings` page.

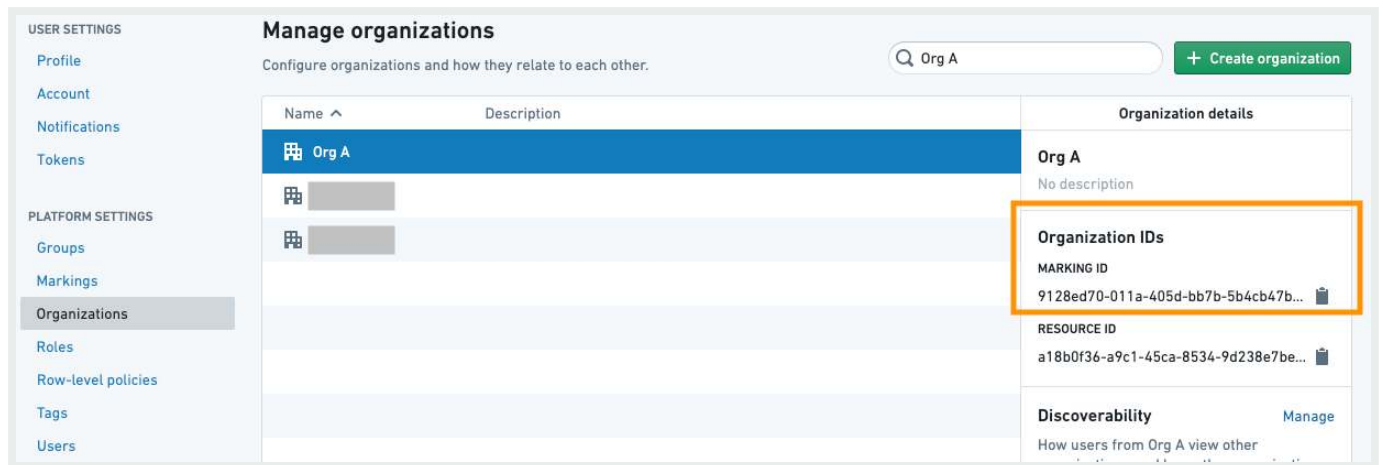
Manage markings
Create, edit, and organize security markings.

Categories	Marking	Category	Marking details
AT Markings	mock_category: MPii	mock_category	PII
	mock_category: PII	mock_category	Personally identifiable information
			Marking ID 8cbb369b-2107-4d1d-8750-082df007...
Category Display Name			

The `OrgMarking` class takes a list of Organization IDs and a list of protected branches on which to apply the Marking removal. Organization IDs can be found in the `Organizations` list on the `Settings` page.

[API Reference ↗](#)

[Send feedback](#)



Java

Java automatic registration

In Java, Marking removal is specified via annotations on the inputs for automatically registered transforms.

Syntax:

```
1 @Compute
2 public void myComputation(
3     @StopPropagating(markings = {markingId1, ...}, onBranches = {branch1, ..
4     @StopRequiring(orgMarkings = {orgId1, ...}, onBranches = {branch2, ...})
5     @Input("<input_id>")
6     FoundryInput input,
7     @Output("<output_id>")
8     FoundryOutput output)
```

The `@StopPropagating` and `@StopRequiring` annotations take a set of Marking IDs and a set of protected branches on which to apply the Marking removal. When only one Marking or branch is specified you do not need to wrap it in `{}` (e.g. `@StopPropagating(markings = marking1, onBranches = "my-branch")`).

J API Reference ↗ stration

For manually registered Java transforms, we use the following syntax during registration in the `MyPipelineDefiner.java` file.

Send feedback

```

1  @Override
2      public void define(Pipeline pipeline) {
3          HighLevelTransform highLevelManualTransform = HighLevelTransform.
4              .computeFunctionInstance(new HighLevelManualFunction())
5              .putParameterToInputAlias("myInput", "/path/to/input/data")
6              .returnedAlias("/path/to/output/dataset")
7              .desiredUnmarkings(Set.of(
8                  Unmarking.builder()
9                      .branch("branch1")
10                     .input(alias("/input1"))
11                     .output(alias("/output"))
12                     .markingId(MarkingId.valueOf("mar
13                     .build(),
14                  Unmarking.builder()
15                      .branch("branch1")
16                      .input(alias("/input1"))
17                      .output(alias("/output"))
18                      .markingId(MarkingId.valueOf("org
19                      .build()
20                  ))
21              .build();
22      pipeline.register(highLevelManualTransform);
23  }

```

SQL

In SQL, Marking removal is specified by using SparkSQL hint statements:

```

1  CREATE TABLE <output_id> AS
2      SELECT /*+ foundry_stop_propagating(markingId1, ...) foundry_stop_require
3      FROM <input_id>

```

Marking and Organization removal in SQL can be added to any `SELECT` statement. For example:

API Reference ↗

```

1  CREATE TABLE <output_id> AS SELECT * FROM <input_1_id>
2      CROSS JOIN (SELECT /*+ foundry_stop_propagating(markingId1, ...) foundry_stop_require

```

Send feedback

Removal permissions

To be able to view code and approve pull requests in general, the approver must pass any Organizations and Markings on the Project and the repository itself, as well as having a [Role](#) that includes the basic Stemma `View Repository` workflow (by default, included in the Viewer role). Users must also have permissions on each Organization and Marking to set approval modes or approve pull requests removing those Organizations and Markings. Users do not necessarily need to be members of the Organization or Marking.

For a Marking approval, the user approving needs to have the `Remove marking` role on the Marking.

USER SETTINGS

Profile

Account

Authorized applications

Notifications

Tokens

PLATFORM SETTINGS

Groups

Markings

Organizations

Roles

Row-level policies

Tags

Users

Manage markings

Create, edit, and organize security markings.

Categories

Details

Marking

Category

mock_category

Add description...

Created

Feb 15, 2019, 5:31 PM by admin admin

Category type

Conjunctive And

All applied markings will be required.

Category visibility

Visible

Visible to all users.

Organization

This category and its markings can be seen by users from all organizations.

Category permissions

Manage

People with administrative permissions to manage aspects of this category.

AA

MPII

PII

mock_category

mock_category

Marking details

Manage permissions

Q PII

Q Add a user or group...

admin admin...

Manage permis... +1

John Smith jsmith

Expand access

1 role selected

Remove

☐ Manage permissions

People who can grant permissions to manage this marking, its members, and edit its metadata.

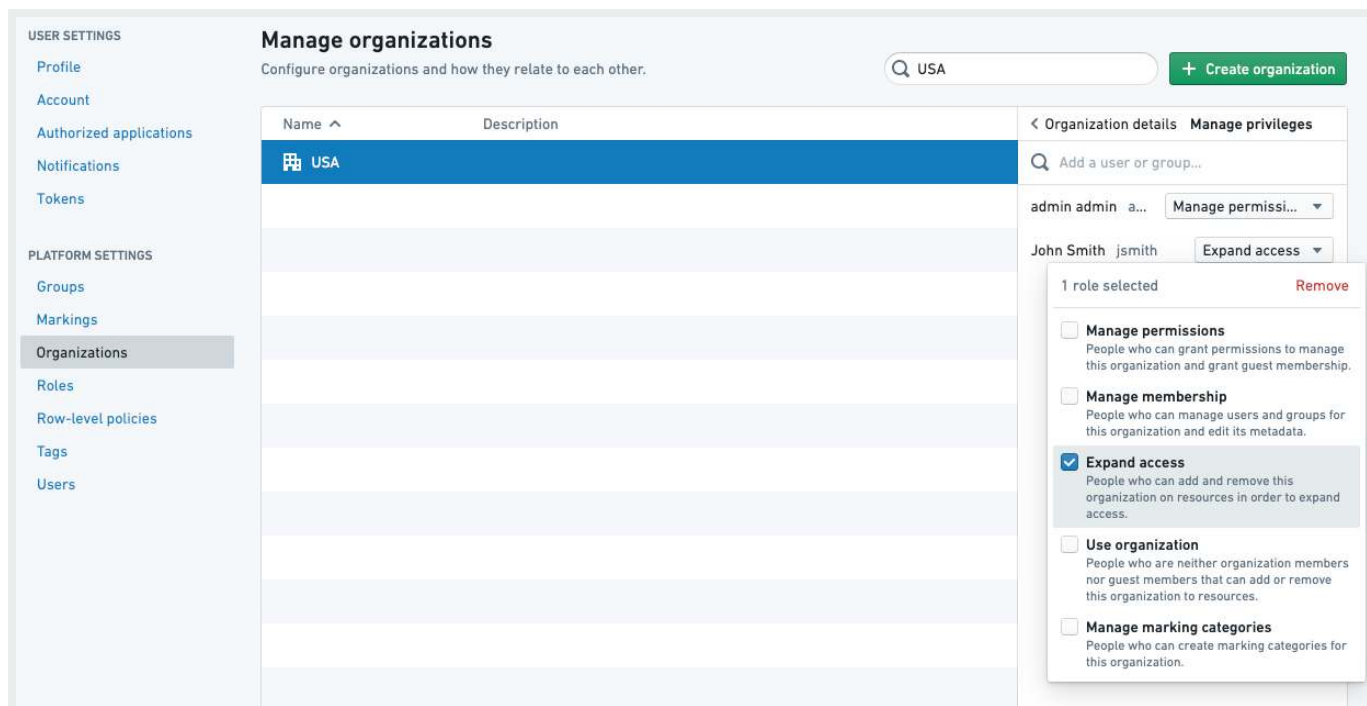
☒ Expand access

People who can remove this marking from projects and files.

For an Organization approval, the user approving needs to have the `Expand access` role on the Marking.

API Reference ↗

Send feedback



Approval modes

For each repository and each Organization and Marking, a data governance user can define which mode should be used to trigger a new approval:

- **Require re-approval:** This is the default mode for every Organization and Marking. The repo will always require security approvals for any pull request made to a branch where this Organization and Marking has been removed. This mode guards against changes to the logic, thereby ensuring that Organizations and Markings are safely removed.
- **Don't require re-approval:** When this Organization and/or Marking is removed on a transform for the first time for a given input, approval is required. Subsequent changes to the logic will not be blocked on security approvals.

Example 1

API Reference ↗

Send feedback

Settings

Preferences

Branches

Repository Settings

Webhooks

Spark

Artifacts

Project scope

Security Approvals

Security Approvals

All pull requests to protected branches in this repository will need to comply with security policies which may require additional mandatory reviews.

☒ Allow changes to security markings in this repository

Markings

The following security markings are changed within this repository.

Marking

Require re-approval ?

☒

☒ mock_category: PHI

```
input=Input(  
  "/ProjectA/patient_data", stop_propagating=Markings(  
    ["ea2be354" ☒ mock_category: PHI "bc5cca753"], on_branches=["master"])  
  ),
```

Above is a transform in a repository with one Marking `PHI` that requires re-approval.

Given the above setup, the following will happen:

- When the user creates their first PR to stop propagating the PHI Marking, they will be required to get approval from a user with the `Remove` role on the PHI Marking.
- If the user later modifies the transform above in their next PR, they will again be asked to get approval.
- If the user modifies anything in the repository – in this file or any other file – they will again be asked to get approval for PHI Marking.

Example 2

Settings

Preferences

Branches

Repository Settings

Webhooks

Spark

Artifacts

API Reference ↗

Security Approvals

All pull requests to protected branches in this repository will need to comply with security policies which may require additional mandatory reviews.

☒ Allow changes to security markings in this repository

Markings

The following security markings are changed within this repository.

Marking

Require re-approval ?

☐

☒ Palantir

Send feedback


```
input=Input("/ProjectA/metrics_data", stop_requiring=OrgMarkings(
| ["d30ec727-d03e-4b2b-8b7f-17e5a5d3da88"], on_branches=["master"])
),
```

Above is a transform in a repository with one organization, **PALANTIR**, that does **NOT** require re-approval.

Given the above setup, the following will happen:

- When the user creates their first pull request to stop requiring the **PALANTIR** Organization, they will be required to get approval from someone with the **Expand access** role on the **PALANTIR** Organization.
- If the user later modifies the transform above in their next pull request, they will **NOT** be asked to get approval.
- If the user subsequently modifies anything in this repository, they will **NOT** be asked to get approval.

Example 3

Settings

- Preferences
- Branches
- Repository Settings
- Webhooks
- Spark
- Artifacts
- Project scope
- Security Approvals**

Security Approvals

All pull requests to protected branches in this repository will need to comply with security policies which may require additional mandatory reviews.

☒ Allow changes to security markings in this repository

Markings

The following security markings are changed within this repository.

Marking	Require re-approval ?
mock_category: PII	☒
mock_category: USA	☐
Palantir	☐

```
input=Input("/ProjectA/employee_data", stop_propagating=Markings(
| ["8cbb369b- mock_category: PII df00748e2"], on_branches=["master"]),
stop_requiring=OrgMarkings(["d30ec727-d03e- Palantir f-17e5a5d3da88"], on_branches=["master"])
),
```

The API Reference is a transform with one Marking, **PII**, and one Organization, **PALANTIR**. The **PII** Marking requires re-approval and the **PALANTIR** Organization does not require re-approval.

Send feedback

```
input=Input("/ProjectB/usa_deployments_data", stop_propagating=Markings(  
| | ["027ec8e4" mock_category: USA f6e302f0"], on_branches=["master"]),  
)
```

Transform 2: Above is a transform with one Marking, `USA`, that does **NOT** require re-approval.

Given the above setup, here's what will happen:

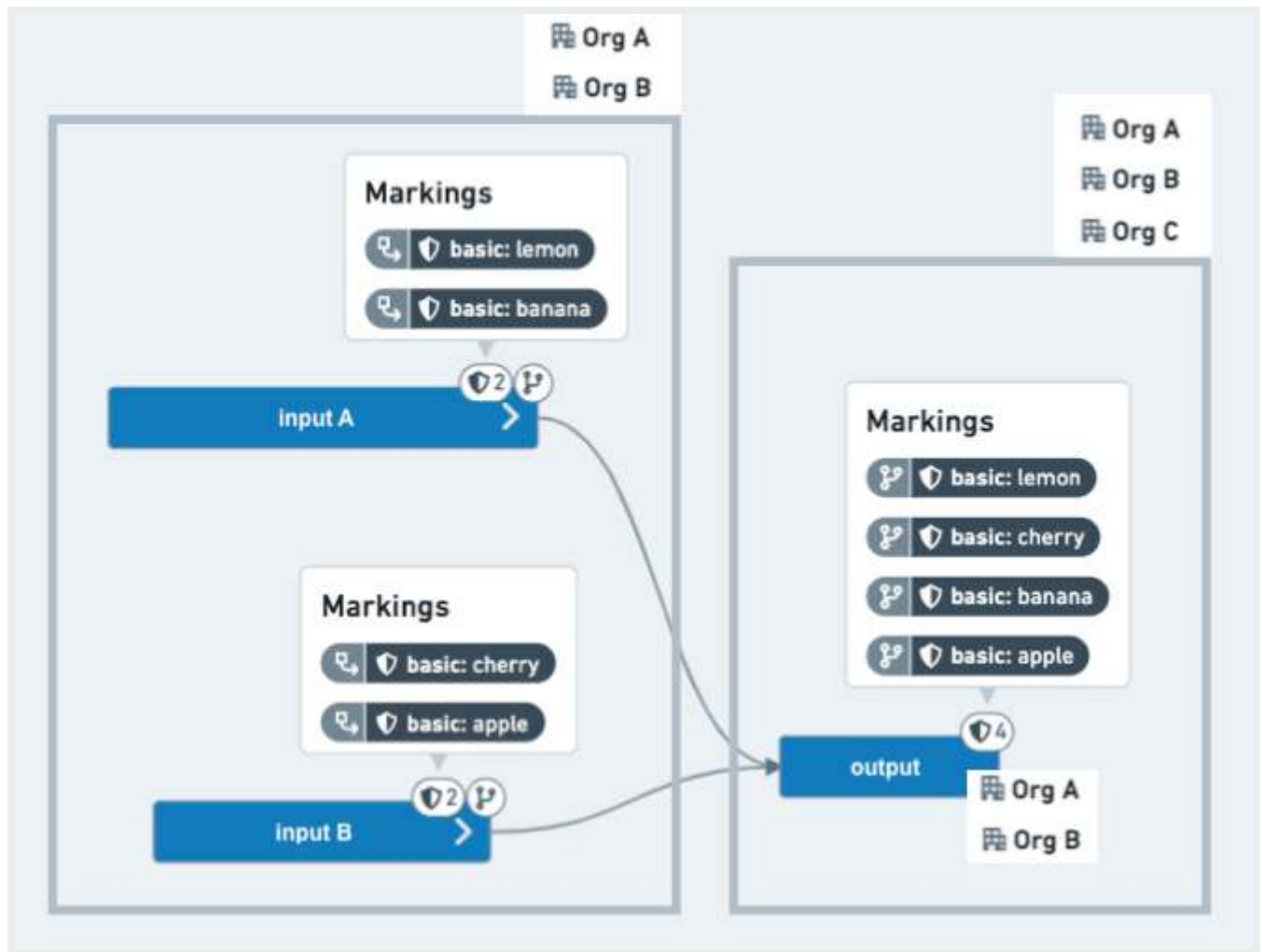
- When the user creates their first pull request in Transform 1, they will be required to get approval from a user with `Remove marking` role on the `PII` Marking **AND** a user with `Expand access` role on the `PALANTIR` Organization.
- If the user later modifies Transform 1 in their next pull request, they will be asked to get approval from **ONLY** a user with `Remove marking` role on the `PII` Marking.
- When the user creates their first pull request for Transform 2, they will be required to get approval from a user with `Remove marking` role on the `USA` Marking **AND** a user with `Remove marking` role on the `PII` Marking.
- If the user later modifies Transform 2 in their next pull request, they will be asked to get approval from **ONLY** a user with `Remove marking` role on the `PII` Marking.
- If the user subsequently modifies anything in this repository, they will be asked to get approval from **ONLY** a user with `Remove marking` role on the `PII` Marking.

Detailed workflow

In this example scenario, a code editor wants to use two datasets from a sensitive upstream project, remove certain information, and allow a wider audience to access the resulting dataset. The two datasets, which have two Markings each, have been added as references in the downstream Project. The code editor wants three of the four Markings to stop propagating, such that they do not appear on the output dataset. In addition, the upstream Project is restricted to users from OrgA or OrgB, and the intent is to distribute the downstream data to users from OrgC.

[API Reference ↗](#)

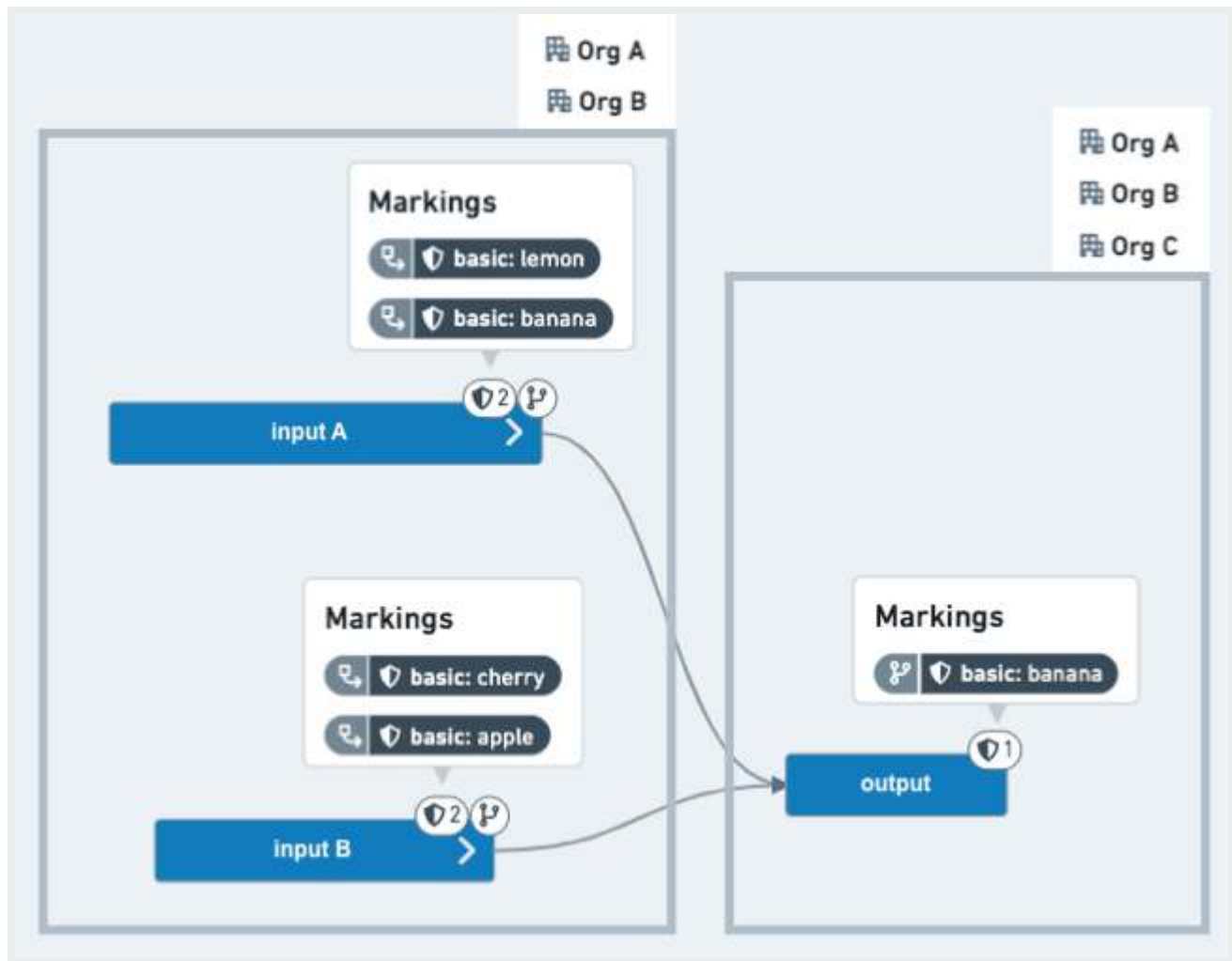
[Send feedback](#)



Before: The **output** dataset on the code editors branch has inherited all four Markings and is still restricted to users from OrgA or OrgB.

[API Reference ↗](#)

[Send feedback](#)



After: The **output** dataset, once merged into a protected branch (for example, **main**), now has only one inherited Marking and does not require users to be members of **OrgA** or **OrgB**.

Steps

1. The code editor writes a new transform on the **feature/clean-data** branch of the repository of the downstream Project.

[API Reference ↗](#)

[Send feedback](#)

```

from transforms.api import transform, Input, Output, Markings, OrgMarkings

@transform(
    input_A = Input(
        "/upstream/ input A",
        stop_propagating=Markings(["4c25d748-b1 basic: lemon f8ba25ffda5"], on_branches=["master"]),
        stop_requiring=OrgMarkings(["9128ed70-811a- Org A 7b-5b4cb47bab59"], on_branches=["master"])
    ),
    input_B = Input(
        "/upstream/ input B",
        stop_propagating=Markings(
            ["93927a0d-36 basic: apple 296d8ae05cf", "fff32d2f-dd basic: cherry e16e0303b04"], on_branches=["master"]),
        stop_requiring=OrgMarkings(["9128ed70-811a- Org A 7b-5b4cb47bab59"], on_branches=["master"])
    ),
    output=Output("/downstream/ output ")
)

def my_compute_function(input_A, input_B, output):
    # Logic that joins and cleans the data

```

2. Since all the marking changes are being requested for the `master` branch, no approvals are needed to work on `feature/clean-data`. In other words, when the output dataset is built on the `feature/clean-data` branch, all the upstream Markings will still be inherited.
3. The code editor creates a pull request to the main branch and requests approvals from the data governance users, who manage data restricted by the `lemon`, `apple`, and `cherry` Markings. The code editor also requests an approval from an `Expand` access Organization administrator from OrgA, who can approve when OrgA data needs to be shared with other Organizations.

i Expanding access by removing an inherited Organization has the effect of removing all inherited Organizations, but approval is only required from users with the appropriate permissions on the Organizations listed in the transform. If you want to require approval from all of the Organizations, then you need to list all of the IDs in the ``stop_requiring`` component. In this example, OrgA is primarily responsible for the data in the upstream Project, so the editor chose OrgA for the cross-Organization approval process. As such, the editor only needs approval from an OrgA admin to remove inherited Organizations. Depending on which Organization approves the editor wants to request, the editor can choose to ``stop_requiring`` either: (1) OrgA (with approval by an OrgA admin), (2) OrgB (with approval by an OrgB admin), or (3) both OrgA and OrgB (with approval from admins from both Organizations).

The end result is that the output dataset will not inherit any Organizations from the [API Reference](#) only respect the Organizations from the Project in which it is located.

[Send feedback](#)

4. The data governance users and Organization administrators receive Foundry notifications that their approval has been requested.
5. Assuming the PR is approved, the code editor merges it and builds the output dataset as shown in the *After* image above.
6. The following week, another code editor makes a change to a different code file and opens a PR to merge to `master`.

i If all the Markings do not require re-approval, the PR can be approved without going through a security review. If any of the Markings do require approval, this new PR will require a security review by the data governance or organization administrator who manages that Marking.

PREVIOUS
← [Guidance on removing markings](#)

NEXT
[Optimizing and debugging pipelines / Overview](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)